

LENGUAJE ASSEMBLER

MICROPROCESADOR INTEL 8088-8086

INDICE GENERAL

CAPITULO 1 CONCEPTOS BASICOS

- 1.1. SISTEMAS DE NUMERACION
 - 1.1.1. Cambio de base
 - 1.1.2. Complementación
- 1.2. REPRESENTACION DE LA INFORMACION
 - 1.2.1. Unidades de almacenamiento
 - 1.2.2. Códigos (ASCII, EBCDIC)
 - 1.2.3. Tipos de datos
 - 1.2.4. Generalidades de los datos en el 8088-8086
- 1.3. CONCEPTOS GENERALES DE HARDWARE
 - 1.3.1. Unidad central de proceso
 - 1.3.2. Comunicación entre las distintas. partes de un computador.
 - 1.3.3. El microprocesador INTEL 8088-8086

CAPITULO 2 REGISTROS Y BANDERAS

- 2.1. REGISTROS
 - 2.1.1. Segmentación y registros de segmentos
 - 2.1.2. Registros generales
 - 2.1.2.1. Registros de datos
 - 2.1.2.2. Registros punteros e índices
 - 2.1.3. Puntero de instrucción
- 2.2. BANDERAS
 - 2.2.1. Banderas de estado
 - 2.2.2. Banderas de control
- 2.3. EJEMPLO DE REGISTROS Y BANDERAS

CAPITULO 3 MEMORIA

- 3.1. ALMACENAMIENTO
- 3.2. DIRECCIONAMIENTO
- 3.3. STACK
- 3.4. ESPACIO DE E/S
- 3.5. MAPA DEL ESPACIO DIRECCIONABLE

CAPITULO 4 INTRODUCCION AL LENGUAJE ASSEMBLER

- 4.1. LENGUAJE DE MAQUINA
- 4.2. DESARROLLO DE UN PROGRAMA EN ASSEMBLER
 - 4.2.1. Edición
 - 4.2.2. Compilación
 - 4.2.3. Linkedición
 - 4.2.4. Depuración

CAPITULO 5 ESTRUCTURA DE UN PROGRAMA EN ASSEMBLER

- 5.1. GENERALIDADES
- 5.2. SINTAXIS DE INSTRUCCIONES
 - 5.2.1. Rotulo
 - 5.2.2. Código de operación
 - 5.2.3. Operando
 - 5.2.4. Comentario
- 5.3. SINTAXIS DE PSEUDO-OPS
- 5.4. MODOS DE DIRECCIONAMIENTO

- 5.4.1. Modo registro
- 5.4.2. Modo inmediato
- 5.4.3. Modo directo
- 5.4.4. Modo registro indirecto
- 5.4.5. Modo relativo a base
- 5.4.6. Modo indexado directo
- 5.4.7. Modo indexado a base
- 5.4.8. Prefijos de segmento
- 5.5. CONSTANTES
- 5.6. OPERADORES
 - 5.6.1. Operadores aritméticos
 - 5.6.2. Operadores lógicos
 - 5.6.3. Operadores relacionales
 - 5.6.4. Operadores de retorno de valores
 - 5.6.5. Operadores de atributos
 - 5.6.6. Operadores de macros
- 5.7. UN PROGRAMA EJEMPLO

CAPITULO 6 CLASIFICACION DE LAS INSTRUCCIONES

- 6.1. INSTRUCCIONES DE TRANSFERENCIA DE DATOS
- 6.2. INSTRUCCIONES ARITMETICAS
- 6.3. INSTRUCCIONES DE MANIPULACION DE BITS
- 6.4. INSTRUCCIONES DE TRANSFERENCIA DE CONTROL
- 6.5. INSTRUCCIONES DE MANEJO DE STRINGS
- 6.6. INSTRUCCIONES DE STACK
- 6.7. INSTRUCCIONES DE CONTROL DEL PROCESADOR
- 6.8. INSTRUCCIONES DE INTERRUPCION

CAPITULO 7 CLASIFICACION DE LAS PSEUDO-OPS

- 7.1. PSEUDO-OPS DE DATOS
- 7.2. PSEUDO-OPS CONDICIONALES
- 7.3. PSEUDO-OPS DE LISTADO
- 7.4. PSEUDO-OPS DE MACROS

CAPITULO 8 SET DE PSEUDO-OPERACIONES

CAPITULO 9 SET DE INSTRUCCIONES

CAPITULO 10 SUBROUTINAS

- 10.1. DEFINICION DE SUBROUTINAS
- 10.2. MECANISMO DE LLAMADA Y RETORNO
- 10.3. USO DE SUBROUTINAS
- 10.4. PASAJE DE PARAMETROS
 - 10.4.1. Interfaz con subrutinas de tipo NEAR
 - 10.4.2. Interfaz con subrutinas de tipo FAR
 - 10.4.3. Uso de la directiva STRUC

CAPITULO 11 MACRO-INSTRUCCIONES

- 11.1. INTRODUCCION
- 11.2. DEFINICION DE MACROS
- 11.3. USO DE MACROS
 - 11.3.1. Escritura de macros en el programa fuente
 - 11.3.2. Bibliotecas de macros
 - 11.3.2.1. Creación de bibliotecas de macros
 - 11.3.2.2. Invocación de bibliotecas de macros

CAPITULO 12 INTERRUPCIONES

- 12.1. INTRODUCCION

- 12.2. TIPOS DE INTERRUPTACIONES
- 12.3. MECANISMO DE INTERRUPTACIONES. ENTORNO DE UN PROGRAMA
- 12.4. RUTINAS DE ATENCION DE INTERRUPTACIONES
- 12.5. EJEMPLO DEL MECANISMO DE INTERRUPTACIONES
- 12.6. INTERRUPTACIONES DEL BIOS
 - 12.6.1. Interrupciones del microprocesador 8088-8086
 - 12.6.2. Interrupciones del controlador 8259
 - 12.6.3. Puntos de entrada al BIOS
 - 12.6.4. Rutinas del usuario
 - 12.6.5. Punteros a tablas de datos
- 12.7. INTERRUPTACIONES DEL DOS
 - 12.7.1. Interrupción 21H --- Llamada a función
- 12.8. CAMBIO DE UN VECTOR DE INTERRUPTACION
 - 12.8.1. Con dos MOV
 - 12.8.2. Con un MOV encadenado
 - 12.8.3. Con la interrupción 21H

CAPITULO 13 COMIENZO Y TERMINACION DE PROGRAMAS

- 13.1. EL PREFIJO DE SEGMENTO DE PROGRAMA (PSP)
- 13.2. COMIENZO DE PROGRAMAS
- 13.3. TERMINACION DE PROGRAMAS
 - 13.3.1. Terminar y devolver recursos
 - 13.3.2. Terminar y dejar residente

CAPITULO 14 FECHA Y HORA DEL SISTEMA

- 14.1. RELOJ DE INTERVALOS
- 14.2. INTERRUPTACIONES DEL BIOS
- 14.3. INTERRUPTACIONES DEL DOS

CAPITULO 15 LA PANTALLA

- 15.1. GENERALIDADES DE HARDWARE
 - 15.1.1. El adaptador de pantalla
 - 15.1.2. Generación de caracteres
 - 15.1.3. Memoria de pantalla
- 15.2. LA PANTALLA EN MODO TEXTO
 - 15.2.1. La memoria de pantalla en modo texto
 - 15.2.2. Localización de cada caracteres en la M de P .
- 15.3. LA PANTALLA EN MODO GRAFICO
 - 15.3.1. La memoria de pantalla en modo gráfico
 - 15.3.2. Localización de cada pixel en la M de P .
 - 15.3.3. Generación de caracteres en modo gráfico
- 15.4. INTERRUPTACIONES PARA EL MANEJO DE LA PANTALLA
 - 15.4.1. Interrupción 10H del BIOS
 - 15.4.1.1. Interface CRT
 - 15.4.1.2. Manejo de caracteres
 - 15.4.1.3. Gráficos
 - 15.4.1.4. Teletipo (TTY)
 - 15.4.1.5. Estado de la pantalla
 - 15.4.2. Interrupción 21H del DOS

CAPITULO 16 EL TECLADO

- 16.1. GENERALIDADES DE HARDWARE
 - 16.1.1. La interrupción 09H
- 16.2. INTERRUPTACIONES PARA EL MANEJO DEL TECLADO
 - 16.2.1. Interrupciones BIOS
 - 16.2.2. Interrupciones DOS
 - 16.2.2.1. Ejemplos de aplicación

CAPITULO 17 DISCOS

- 17.1. ESTRUCTURA DE UN DISCO
- 17.2. E/S SOBRE DISCO
- 17.3. MAPA DE UN DISCO

- 17.3.1. Area de bootstrap o registro de autoarranque
- 17.3.2. Area de tabla de alocación de archivos (FAT)
- 17.3.3. Area de directorio
- 17.3.4. Area de datos
 - 17.3.4.1. Subdirectorios
- 17.4. INTERRUPCIONES PARA EL ACCESO A DISCO
 - 17.4.1. Interrupción 13H del BIOS
 - 17.4.2. Interrupciones DOS de E/S absoluta

CAPITULO 18 ARCHIVOS

- 18.1. CONCEPTOS GENERALES
 - 18.1.1. Archivos, registros y campos
 - 18.1.1.1. Registro lógico y registro físico
 - 18.1.1.2. Factor de bloqueo
 - 18.1.2. Organización y acceso a archivos
 - 18.1.2.1. Organización
 - 18.1.2.2. Acceso
 - 18.1.3. Almacenamiento de archivos
- 18.2. ARCHIVOS BAJO MS-DOS
 - 18.2.1. Registros y bloques
 - 18.2.2. Areas referenciadas por el MS-DOS
 - 18.2.2.1. Bloque de control de archivo (FCB)
 - 18.2.2.2. Area de transferencia a disco (DTA)
 - 18.2.3. Acceso a los registros de un archivo
 - 18.2.3.1. Acceso secuencial
 - 18.2.3.2. Acceso al azar
 - 18.2.3.3. Acceso al azar por bloques
 - 18.2.4. Procedimiento estándar con un archivo bajo MS-DOS
- 18.3. FILE HANDLE
 - 18.3.1. Acceso a archivos con las file handle functions
- 18.4. ALMACENAMIENTO DE ARCHIVOS BAJO MS-DOS
 - 18.4.1. Algoritmo para interpretar cada entrada de la FAT
 - 18.4.2. Relación entre unidades de alocación y sectores
- 18.5. FUNCIONES DEL DOS PARA EL MANEJO DE ARCHIVOS
 - 18.5.1. Funciones generales
 - 18.5.2. Funciones con FCB
 - 18.5.2.1. De búsqueda en directorios
 - 18.5.2.2. De archivos en general
 - 18.5.2.3. De E/S de archivos
 - 18.5.3. Funciones con file handle
 - 18.5.3.1. De directorios
 - 18.5.3.2. De búsqueda en directorios
 - 18.5.3.3. De archivos en general
 - 18.5.3.4. De E/S de archivos

APENDICES

APENDICE A Almacenamiento de datos numéricos

APENDICE B Comandos del EDLIN

APENDICE C Comandos del DEBUG

APENDICE D Operadores

APENDICE E Códigos de las teclas

CAPITULO 1 **CONCEPTOS BASICOS**

El propósito de este capítulo es el de repasar un conjunto de definiciones y conceptos básicos, como introducción a los temas específicos del microprocesador INTEL 8088/8086.

1.1. SISTEMAS DE NUMERACION

Un sistema de numeración queda unívocamente definido a través de su base, la cual indica la cantidad de dígitos distintos que posee el sistema que se pueden utilizar para escribir cualquier número dentro de ese sistema.

Ejemplos de sistemas de numeración son:

- Decimal, base 10 (dígitos: 0,1,2,3,4,5,6,7,8,9)
- Binario, base 2 (dígitos: 0,1)
- Octal, base 8 (dígitos: 0,1,2,3,4,5,6,7)
- Hexadecimal, base 16 (dígitos: números 0 a 9 y letras A,B,C,D,E,F)

Para indicar, explícitamente, que un número es expresado en una determinada base, lo representaremos con la base a la que pertenece como subíndice a la izquierda. Por ejemplo:

Base 10: 123410

Base 16: 23AF16

Base 2: 11012

En el transcurso del texto, adoptaremos como convención el uso de un sufijo para referirnos a la base a la que pertenece un número: binario B

decimal D (valor por default)

Octal Q

Hexadecimal H

Una manera de representar un número en una determinada base es mediante la notación polinómica, de modo que cada dígito, en el sentido de izquierda a derecha, está multiplicado por las sucesivas potencias de la base. Así, con por ejemplo, el número 1234 en base decimal se puede escribir como:

$$1 \times 10^3 + 2 \times 10^2 + 3 \times 10^1 + 4 \times 10^0$$

Los sistemas de numeración más usuales son el binario, el decimal, y el hexadecimal; también, y en menor medida, el sistema octal.

Las computadoras trabajan internamente con el sistema binario, aunque pueden mostrar la información en cualquier otro sistema.

En este texto, utilizaremos los sistemas binario, decimal y hexadecimal.

Existe una notación formal para expresar cualquier sistema de numeración. Esta notación consiste en indicar, entre paréntesis, la base y la cantidad de dígitos. Así, por ejemplo, un sistema (2,8) hace referencia a un sistema binario, base 2, con hasta 8 dígitos de tamaño.

En general, diremos que un número pertenece al sistema de numeración (p,q) si está expresado en la base p con q dígitos.

Por ejemplo: 1100 pertenece al sistema (2,4)

AB12 pertenece al sistema (16,4)

093 pertenece al sistema (10,3)

Cuando la cantidad de dígitos significativos de un número que pertenece a un cierto sistema de numeración es inferior a la cantidad q de dígitos del sistema, entonces se deben completar los lugares restantes con ceros a la izquierda.

Por ejemplo: 00101 pertenece al sistema (2,5)

0101 pertenece al sistema (2,4)

000AB pertenece al sistema (16,5)

1.1.1. Cambio de base.

Cualquier número expresado en una base puede, también, ser representado en cualquier otra base. Para poder hacer esto se debe proceder a cambiar la base del número.

En general, para expresar un número en otra base se deben realizar los siguientes pasos:

- Dividir el número a convertir por la nueva base; el resto de la división representa el dígito menos significativo del número en su nueva base.

- Dividir el cociente por la nueva base; el resto de la división representa el próximo dígito menos significativo del número en su nueva base.
- Seguir así sucesivamente hasta que el resto de la última división sea menor que la nueva base; en este punto, el resto de la última división corresponde al dígito más significativo del número expresado en la nueva base.

Las operaciones que se indican en los pasos descritos, se deben hacer en la aritmética de la base vieja.

Por ejemplo, convertir 3410 a base 2:

- a) $34 / 2 = 17$ ---> resto = 0
- b) $17 / 2 = 8$ ---> resto = 1
- c) $8 / 2 = 4$ ---> resto = 0
- d) $4 / 2 = 2$ ---> resto = 0
- e) $2 / 2 = 1$ ---> resto = 0 Entonces 3410 = 1000102

Los cambios de base más utilizados son: de decimal a binario y viceversa y de binario a hexadecimal y viceversa.

Para pasar de binario a decimal, se utiliza la forma polinómica; esto es, expresar al número binario descompuesto en potencias de 2 y efectuar las operaciones en aritmética decimal.

Por ejemplo, para convertir el número binario 100010 a base 10:

$$1 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 32 + 2 = 34_{10}$$

Para pasar de binario a hexadecimal, se deben tomar grupos de 4 dígitos binarios justificando, si es necesario, con ceros a la izquierda y luego convertir cada uno de los grupos a un dígito hexadecimal.

Por ejemplo, para convertir el número binario 11110100101 a base 16:

$$11110100101 \text{ ---> } 0111 \ 1010 \ 0101 \text{ ---> } 7A5_{16}$$

Para pasar de hexadecimal a binario se convierte cada dígito hexadecimal a su representación binaria, usando siempre cuatro dígitos binarios.

Por ejemplo, la inversa del ejemplo anterior:

$$\begin{aligned} 7A5_{16} &\text{ ---> } 7_{16} = 0111 \\ A_{16} &= 1010 \\ 5_{16} &= 0101 \quad \text{---> } 01110100101 \end{aligned}$$

1.1.2. Complementación.

Cuando se trabaja con números en bases diferentes a la decimal, surge el inconveniente de cómo representar a los números negativos.

Esto se resuelve mediante el concepto de complementación, de modo que siempre se representen cantidades positivas, pero su significado puede ser negativo.

Para implementar este concepto, se define a un número negativo como aquel que, sumándole su positivo, el resultado es 0.

En términos prácticos, trabajaremos con la notación complemento en números binarios, tomando siempre al primer dígito como dígito de signo.

Para complementar un número binario se deben hacer los siguientes pasos:

- Convertir cada dígito a su inverso (los unos a cero y los ceros a uno).
- Si se utiliza la notación complemento a 2, sumar un 1 al número resultante de la inversión de los dígitos.

Por ejemplo, el complemento de 0101 es $1010 + 1 = 1011$.

Para expresar en forma polifónica un número en notación complemento a 2, se debe tener en cuenta que el dígito más significativo representa el signo.

En el ejemplo: $1011 = -1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = -8 + 2 + 1 = -5_{10}$

Notar que 1011 (-5) es el complemento de 0101 (5).

1.2. REPRESENTACION DE LA INFORMACION.

Ya hemos visto que existen distintos sistemas de numeración, que permiten expresar números en diferentes bases, y que la computadora está diseñada para realizar operaciones aritméticas y lógicas con cantidades binarias.

La computadora, para poder efectuar esas operaciones debe, en primera instancia, poder almacenar esas cantidades numéricas.

De ahora en adelante llamaremos dato a toda cantidad almacenada en la computadora.

1.2.1. Unidades de almacenamiento.

Los datos se almacenan en la memoria de la computadora, en distintas unidades de almacenamiento. Las unidades de almacenamiento más comunes son las siguientes:

-BIT Abreviatura de la expresión inglesa Binary Digit. Es la mínima unidad de almacenamiento, y permite representar un único dígito binario; es decir, almacena un uno o un cero. Toda la electrónica digital se basa en el concepto de dígito binario.

-BYTE Es un conjunto de bits, que conforman la mínima unidad direccionable, es decir que cada byte puede ser referenciado mediante un número de orden o dirección dentro de la memoria. En general el tamaño de un byte es de 8 bits, aunque existen computadoras diseñadas con bytes de otros tamaños. Desde el punto de vista de sistemas de numeración, un byte pertenece al sistema (2,8).

-NIBBLE Ocupa medio byte; en nuestro caso un nibble ocupa 4 bits. Es útil para la representación de la información en notación hexadecimal, dado que 4 dígitos binarios equivalen a 1 dígito hexadecimal.

-PALABRA Es un conjunto de bits que puede procesarse en forma paralela. El tamaño de una palabra (o WORD) es dependiente de cada máquina, aunque en general equivalen a 2 bytes; en nuestro caso, una palabra tiene 16 bits de tamaño.

Desde el punto de vista de sistemas de numeración, una palabra pertenece al sistema (2,16).

Además de las unidades de almacenamiento indicadas, existen otras, menos frecuentemente usadas:

DOBLE PALABRA, CUADRUPLE PALABRA, etc.

1.2.2. Códigos.

Para normatizar y facilitar la representación de diferentes símbolos a través de la combinación de dígitos binarios en un byte, se crearon diferentes códigos estándar.

Estos códigos tienen como fundamento la necesidad de poder estandarizar la representación de la información, para permitir que se puedan intercambiar datos entre distintas computadoras.

Los códigos más difundidos son el EBCDIC (Extended Binary Coded Decimal Interchange Code) y el ASCII (American Standard Code Information Interchange).

- EBCDIC Este código, especialmente difundido en computadoras grandes (Main Frames), se basa en la codificación de 256 símbolos distintos, entre los que se incluyen todas las letras mayúsculas y minúsculas, los dígitos del 0 al 9, caracteres especiales etc. Para representar 256 códigos se utilizan 8 bits, de modo que cada símbolo ocupa un byte.

- ASCII Este código es el más ampliamente difundido. De un modo similar al EBCDIC, el ASCII permite representar símbolos. Sin embargo, a diferencia de aquel, sólo admite la codificación de 128 símbolos distintos, pues cada símbolo ocupa los 7 bits menos significativos (7 bits de la izquierda) de un byte, quedando el bit 8 como bit de control. No obstante esto, actualmente es muy común encontrar computadoras que incluyen un código ASCII extendido, que se basa en la utilización del bit 8 para poder representar, en total, 256 símbolos distintos (los 128 símbolos originales del ASCII más símbolos definidos con la utilización del bit 8).

Los primeros 32 símbolos del código ASCII son los denominados caracteres de control o caracteres no imprimibles.

Los códigos de control son especialmente usados en comunicaciones.

1.2.3. Tipos de datos.

Así como hemos visto las diferentes unidades de almacenamiento (bit, byte, palabra, etc.), es necesario definir los tipos de datos que pueden ser representados en esas unidades de almacenamiento.

Cualquier computadora puede trabajar con datos alfanuméricos y con datos numéricos.

Los datos alfanuméricos son cadenas de caracteres (o string) ASCII o EBCDIC, donde cada carácter ocupa un byte. Así, por ejemplo, la representación de HOLA en código ASCII sería, visto en hexadecimal, 48 4F 4C 41.

Los datos numéricos pueden ser binarios, decimales desempaquetados, decimales empaquetados y punto flotante.

Binarios:

Son cantidades de 1 " más bytes con o sin signo. Para representar binarios con signo, se utiliza la notación complemento a 2. (En el apéndice A se incluyen algunas especificaciones para los números enteros representables en el microprocesador 8088-8086).

Decimales desempaquetados:

Se representan a razón de un dígito BCD (Binary-Coded Decimal) por byte, que ocupa el nibble más bajo (4 bits menos significativos). Para el 8088-8086 el nibble más alto es siempre cero. Para otras arquitecturas, el nibble más alto está ocupado por un dígito de zona, que en general es 3 " F.

Cada byte puede almacenar un valor entre 0 y 9.

Los dígitos BCD tienen la siguiente codificación:

DIGITO	BCD	DIGITO	BCD
0	0000	1	0001
2	0010	3	0011
4	0100	5	0101
6	0110	7	0111
8	1000	9	1001

El tamaño de un dato decimal desempaquetado está dado por la cantidad de dígitos que tiene. Por ejemplo, el numero 1234 ocupa 4 bytes, y su configuración ser 01 02 03 04.

El microprocesador 8088-8086 opera sólo con números decimales desempaquetados positivos.

Decimales empaquetados:

Cada byte contiene dos dígitos BCD: el dígito más significativo se almacena en el nibble más alto y el dígito menos significativo se almacena en el nibble más bajo. Cada byte puede almacenar un valor entre 00 y 99.

El tamaño de un dato decimal empaquetado está dado por la mitad de la cantidad de dígitos que tiene (si un dato tiene una cantidad impar de dígito, entonces el nibble más alto del dígito más significativo se pone en cero). Por ejemplo, el numero 2345 ocupa 3 bytes, y su configuración ser 01 23 45.

El microprocesador 8088-8086 opera sólo con números decimales empaquetados positivos.

Punto flotante:

Los números en punto flotante son representados en una cadena de 4 bytes para simple precisión o de 8 bytes para doble precisión.

Un numero en punto flotante tiene 3 campos: signo de la mantisa, exponente y mantisa.

(En el apéndice A se incluye información acerca de los números en punto flotante, en simple y en doble precisión.)

1.2.4. Generalidades de los datos en el 8088-8086.

Para poder interpretar cualquier dato en el entorno del microprocesador 8088-8086, se debe tener en cuenta lo siguiente:

- Siempre la parte más significativa de una cantidad se escribe en la dirección más alta de la memoria y la parte menos significativa en la dirección más baja, considerando que las posiciones se numeran, en forma creciente, de derecha a izquierda. Así por ejemplo, un dato de una palabra de tamaño tiene el byte más significativo a la izquierda y el byte menos significativo a la derecha.

Por ejemplo: 09FA se ve como FA09

120555AA se ve como AA550512

- Siempre debe saberse de que modo se va a interpretar un determinado vuelco de la memoria, ya que un mismo vuelco puede significar cosas distintas según el tipo de dato que se "vea". Por ejemplo:

09 -----> 9 (decimal desempaquetado)

137 (binario sin signo)

-119 (binario con signo)

4140 -----> 4140 (decimal empaquetado)

16704 (binario sin signo)
16704 (binario con signo)
A@ (cadena de caracteres ASCII)

1.3. CONCEPTOS GENERALES DE HARDWARE

Daremos una visión general de los elementos de hardware de un sistema de computación. Los mismos serán tratados posteriormente, en particular y más en detalle, para el microprocesador 8088-8086.

Una computadora es una máquina de tratamiento automático de la información que manipula señales eléctricas digitales binarias.

Sus componentes principales son la Unidad Central de Proceso (CPU), la memoria, los controladores y los dispositivos de entrada y salida (E/S).

La memoria es el medio principal de almacenamiento de la información, ya sean estos datos o instrucciones. Las posiciones de memoria se identifican por medio de direcciones.

Los programas, durante su ejecución, leen y/o escriben datos desde o hacia dispositivos externos tales como discos, cintas, pantallas, impresoras, etc. Estos dispositivos de E/S, denominados periféricos, son manejados o controlados a través de mecanismos especiales llamados controladores.

1.3.1. Unidad Central de Proceso.

La **CPU** o microprocesador es el "cerebro" de una computadora. Su función consiste en realizar tareas de cálculo y control, a través de la ejecución de instrucciones del actual programa que se está procesando en ese momento.

Todo microprocesador está conectado a un oscilador o reloj, que genera una cantidad determinada de impulsos en el tiempo.

La frecuencia del oscilador se mide en megahertz (millones de ciclos por segundo). Esta frecuencia, dividida por una constante d lo que se denomina un ciclo de máquina. Cada instrucción que ejecuta el microprocesador tiene una determinada duración medida en cantidad de ciclos de máquina.

Las partes de una CPU son las siguientes:

- **UAL:** La Unidad Aritmética y Lógica es la encargada de realizar todas las operaciones aritméticas y lógicas.
- **UC:** La Unidad de Control es la encargada de interpretar las instrucciones y generar las órdenes para que se ejecuten.

Además de la UAL y de la UC, la CPU incluye a los registros.

Los registros son circuitos electrónicos muy veloces, usados para operaciones y/o almacenamiento de datos, que pueden ser accedidos en forma más rápida que las posiciones de la memoria central. Existen registros de uso general, para el usuario, y registro de uso especial, propios del procesador.

1.3.2. Comunicación entre las distintas partes de una computadora.

Como se indica más arriba, una computadora está formada por la CPU, la memoria, los controladores y los dispositivos de entrada y salida .

Estos componentes están comunicados entre si para poder intercambiar información. Así, por ejemplo, la CPU debe ejecutar instrucciones de un programa, pero ese programa debe ser leído de disco a memoria, para que la CPU pueda leer las instrucciones y luego ejecutarlas.

El canal de comunicación entre todas las unidades funcionales de un sistema de computación es el bus. Una unidad queda interconectada a las demás cuando se "cuelga" al bus.

Un bus se compone de una determinada cantidad de líneas, equivalentes a la cantidad de bits que transmite. Por las que circula un cierto tipo de información que depende del tipo de bus que sea. En función de esto, existen 3 tipos de buses:

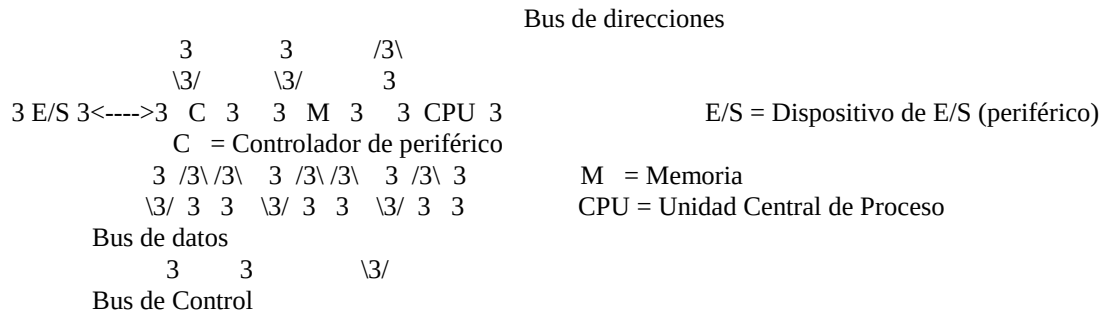
el bus de datos, el bus de direcciones y el bus de control.

* El bus de datos funciona en conjunción con el bus de direcciones para la transmisión de datos.

* El bus de direcciones transmite direcciones de las posiciones de memoria y de los dispositivos conectados al bus.

* El bus de control sirve, básicamente, para indicar que tipo de información es la que circula.

El siguiente esquema muestra como es la interconexión entre los diversos componentes de un sistema de computación:



La información que se transmite de un componente a otro dentro de un sistema de computación, viaja a través del bus desde el componente de origen hasta el componente destino, almacenándose temporariamente en puertos y registros.

Un puerto de E/S es una vía de comunicación entre los diversos componentes de un sistema de computación, y, al igual que la memoria, cada puerto se identifica mediante una dirección. De este modo, es posible leer datos de un puerto o escribir datos sobre él.

Cuando un dato está listo para ser transmitido, en primer lugar se envía su dirección de destino por el bus de direcciones, y a continuación se envía el propio dato por el bus de datos.

1.3.3. El microprocesador INTEL 8088-8086.

El microprocesador INTEL 8088 tiene las siguientes características generales:

- Bus de datos de 8 bits
- Bus de direcciones de 16 bits
- Oscilador de 4,77 # 8 MHz (millones de ciclos por segundo)
- Hasta 64 K puertos de E/S direccionables
- 92 tipos de instrucciones
- Las instrucciones más rápidas requieren 2 ciclos de máquina
- Las instrucciones más lentas requieren 206 ciclos de máquina

El microprocesador INTEL 8086 es idéntico al 8088, con la única diferencia que tiene el bus de datos de 16 bits.

Esto lo hace más rápido en la transferencia de información, pues donde el 8086 requiere de una sola operación, el 8088 necesita dos operaciones para transferir el mismo dato.

El 8088-8086 se comunica con el exterior de tres maneras:

- Mediante el acceso directo o indirecto a memoria.

El acceso directo se hace a través del controlador DMA (Direct Memory Access); las unidades de disco y los puertos de comunicaciones seriales (RS-232) acceden directamente a la memoria a través de este controlador.

El acceso indirecto se logra a través de registros internos, que se usan para transferir datos a/desde memoria.

- A través de los puertos de E/S. Este es el medio normal de comunicación del 8088-8086 con otros procesadores.
- A través de interrupciones.

La transferencia de datos entre la memoria y la CPU se efectúa a través de la interface de memoria.

Las instrucciones, que va a ejecutar el procesador, se almacenan en la cola de instrucciones; el sistema de control de la unidad de ejecución toma las instrucciones desde la cola de instrucciones, y las envía hacia la unidad de ejecución, quien es la parte responsable de interpretarlas y ejecutarlas.

Cada vez que se produce este proceso, la unidad de interface del bus va rellorando la cola de instrucciones, de modo que la unidad de ejecución nunca tenga que esperar para interpretar y ejecutar la próxima instrucción.

Este modo de funcionamiento se denomina pipelined (entubado), y consiste en asegurar que una vez que la unidad de ejecución finaliza la ejecución de una instrucción, siempre haya otra instrucción esperando para ser interpretada y ejecutada.

La mayoría de los microprocesadores deben esperar, antes de poder ejecutar una instrucción, que primero sea leuda desde la memoria.

El esquema pipelined le permite al 8088-8086 ejecutar instrucciones mucho más rápido.

CAPITULO 2

REGISTROS Y BANDERAS

Dentro de la CPU existen los registros, que habíamos definido como circuitos electrónicos muy veloces usados para realizar operaciones o almacenar datos temporariamente.

En este capítulo veremos en detalle que registros existen y como se utilizan.

2.1. REGISTROS

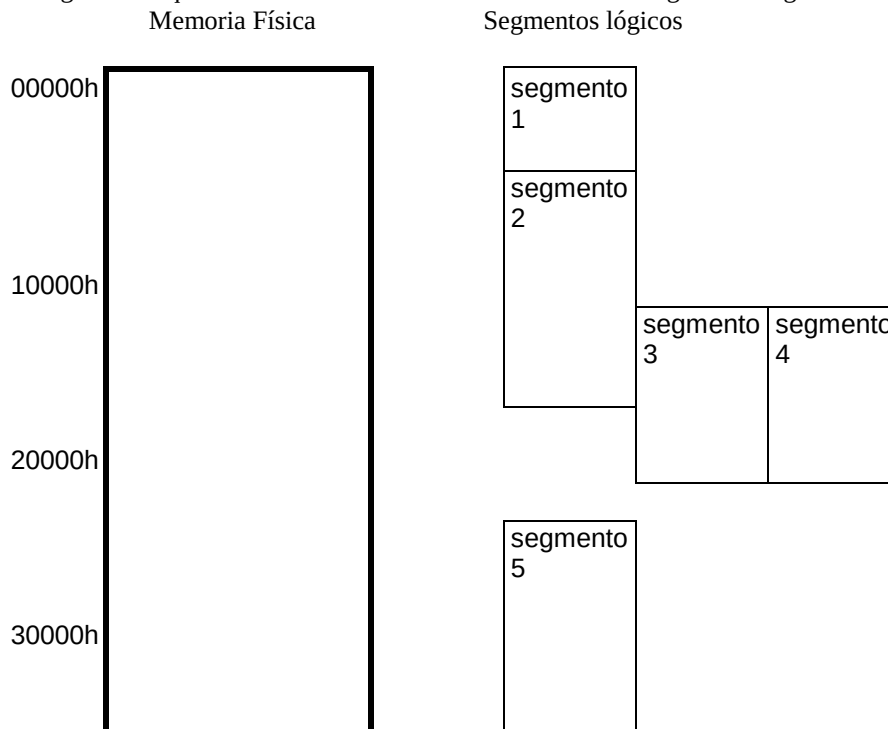
El microprocesador 8088-8086 utiliza 13 registros de 16 bits. De ellos, 8 son registros generales (o también, de uso general), 4 son registros de segmentos y 1 es el puntero de instrucción (IP).

2.1.1. Segmentación y registros de segmentos.

El microprocesador 8088-8086 tiene un espacio de dirección física de memoria de 1 Megabyte. Los programas ven ese megabyte de memoria como una serie de segmentos lógicos, cada uno de hasta 64 Kbytes de tamaño. Cada segmento consiste de una serie de ubicaciones contiguas de memoria, y es una unidad direccionable separadamente. La ubicación de un segmento en memoria está dada por una dirección base que corresponde a la dirección de la primera posición del segmento en la memoria física. Los segmentos siempre comienzan en una dirección que es múltiplo de 16.

Los segmentos pueden estar adyacentes, disjuntos, parcialmente superpuestos o totalmente superpuestos, es decir que más de un segmento lógico puede contener la misma posición física de memoria.

En el siguiente esquema se observa como es la relación de los segmentos lógicos con la memoria física:



Los segmentos 1 y 2 son adyacentes; los segmentos 2 y 3 están parcialmente superpuestos; los segmentos 3 y 4 están totalmente superpuestos; los segmentos 4 y 5 son disjuntos.

De todos los posibles, el microprocesador 8088/8086 sólo puede tener activos 4 segmentos a un tiempo. Esos segmentos son, respectivamente, de código (code segment), de stack (stack segment), de datos (data segment) y auxiliar (extra segment).

El segmento de código contiene el código del programa; el segmento de stack es el área reservada para el stack; el segmento de datos contiene las variables que utiliza el programa; y el segmento auxiliar contiene, también, variables.

La ubicación de esos segmentos de memoria activos, queda identificada por el contenido de los 4 registros de segmentos incluidos en el 8088. Esos registros son: CS (Code Segment), que apunta al segmento de código; SS (Stack Segment), que apunta al segmento de stack; DS (Data Segment), que apunta al segmento de datos; y ES (Extra Segment), que apunta al segmento auxiliar.

Cada uno de los registros de segmentos contiene una cantidad de 16 bits que representa la dirección base de los respectivos segmentos; en otras palabras, la dirección base es la dirección correspondiente al primer byte de cada segmento.

Los registros de segmento son accesibles al usuario, es decir que sus contenidos pueden ser modificados a través de software, lo que permite ganar acceso a distintas direcciones de memoria con sólo cambiar los valores de los registros apropiados.

2.1.2. Registros Generales.

Hay dos tipos de registros generales: 4 registros de datos y 4 registros punteros e índices.

2.1.2.1. Registros de datos.

Los registros de datos (Data Registers) son utilizados, durante la ejecución de un programa, para almacenamiento temporario de resultados intermedios. La ventaja de almacenar datos en los registros en vez de la memoria, es que el acceso a los registros es mucho más rápido.

Los registros de datos son respectivamente A (Acumulador), B (Base), C (Contador) y D (Dato). El uso de esos nombres implican ciertas funciones especiales que son hechas por cada registro.

Los registros de datos pueden ser accedidos de dos modos: en su totalidad o en sus partes baja y alta separadamente; es decir que cada registro de datos puede funcionar ya sea como un registro de 16 bits o como 2 registros de 8 bits.

La referencia a un registro entero se hace agregando la letra X a continuación de la letra de identificación del registro; así, por ejemplo, el acumulador es referenciado como AX.

La referencia a la parte baja de un registro se hace mediante la letra L (low) siguiendo a la identificación del registro; así, por ejemplo, la parte baja del acumulador se identifica como AL. Del mismo modo, la parte alta de un registro se referencia con la letra H (high).

Las siguientes son las funciones específicas de los registros de datos durante la ejecución de un programa:

- **AX:** multiplicaciones y divisiones con operandos de longitud de palabra; operaciones de E/S.
- **AL:** multiplicaciones y divisiones con operandos de longitud de byte; operaciones de E/S; conversiones; operaciones aritméticas decimales.
- **AH:** multiplicaciones y divisiones con operandos de longitud de byte.
- **BX:** direccionamiento de datos en memoria; conversiones.
- **CX:** contador de elementos para operaciones de string; contador de repeticiones para loops.
- **CL:** contador para operaciones de cambio de bits y de rotación.
- **DX:** multiplicaciones y divisiones con operandos de longitud de palabra; direccionamiento de la puerta de E/S.

El siguiente es un esquema de los registros de datos **AX, BX, CX y DX:**

15	8	7	0	
AH		AL		AX
BH		BL		BX
CH		CL		CX
DH		DL		DX

2.1.2.2. Registros punteros e índices.

El otro grupo de registros de uso general son los registros punteros e índices, los cuáles sólo pueden ser referenciados como unidades de 16 bits.

Estos registros son utilizados particularmente para almacenar valores de desplazamiento de direcciones de memoria relativos a los registros de segmentos.

El contenido de los registros punteros e índices puede ser leído, cargado o modificado a través del software. Esto permite, por ejemplo, la ejecución de instrucciones que referencian a un registro en vez de un desplazamiento; de este modo, la instrucción debe especificar cual registro contiene el desplazamiento deseado.

De los 4 registros mencionados, 2 son punteros y 2 son índices.

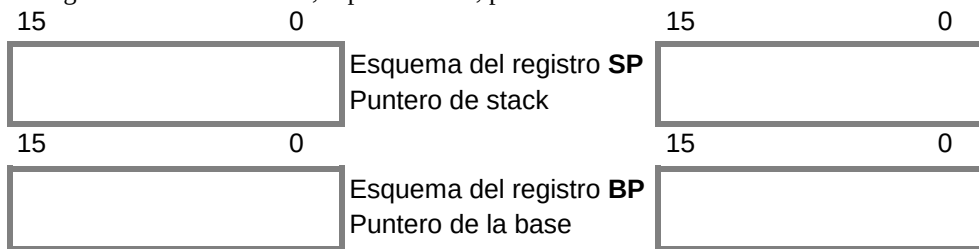
Los registros punteros, SP (Puntero del Stack) y BP (Puntero de la Base), contienen desplazamientos respecto a la dirección base contenida en el registro SS (Segment Stack), y son utilizados durante la ejecución de

instrucciones que involucran el uso del segmento de la memoria definido como stack. De este modo, ellos permiten acceder fácilmente a posiciones de memoria dentro del stack.

El contenido de SP siempre representa el desplazamiento de la primera ubicación libre del stack; es decir que, en combinación con el SS, se obtiene una dirección de 20 bits que apunta al tope del stack.

BP, en cambio, es utilizado para acceder a datos dentro del stack. Su contenido es, también, un desplazamiento que debe ser combinado con el valor base de SS.

Los registros índices, SI (Índice del Fuente) y DI (Índice del Destino), son usados para contener desplazamientos en instrucciones que acceden a datos almacenados en el segmento de datos o en el segmento auxiliar. Es por este motivo que los registros índices siempre deben ser combinados con el registro DS o con el registro ES. Se los utiliza, especialmente, para acceder a datos en forma indexada.



2.1.3. Puntero de Instrucción.

El IP (Puntero de Instrucción) es un registro de 16 bits que contiene la dirección de la próxima instrucción a ser ejecutada del actual segmento de código. En realidad, el contenido de IP es un desplazamiento, que debe ser combinado con el contenido del registro CS para obtener la dirección real de la próxima instrucción a ejecutarse.

Cuando la unidad de ejecución toma una instrucción desde la memoria, automáticamente actualiza el valor de IP en la cantidad precisa de bytes, de modo que IP queda apuntando al primer byte de la siguiente instrucción. En el caso de instrucciones de bifurcación, se modifica directamente el contenido de IP colocando en él la dirección (desplazamiento) de la posición de memoria a la que se bifurca.

2.2. BANDERAS

Además de los registros antes mencionados, el microprocesador 8088-8086 incluye un registro especial de 16 bits, que contiene 9 flags o banderas de 1 bit cada una (los restantes 7 bits no se usan).

Las posiciones de las banderas dentro del registro se indican en el siguiente esquema:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				OF	DF	IF	TF	SF	ZF		AF		PF		CF

Las banderas son utilizadas para fijar algunas condiciones que se van produciendo cuando se ejecuta un programa.

Existen 2 tipos de banderas: las banderas de estado y las banderas de control.

2.2.1. Banderas de estado.

Las banderas de estado reflejan propiedades de los resultados de operaciones aritméticas o lógicas.

Las siguientes son las banderas de estado y las condiciones que reflejan cuando están en ON (encendidas) y en OFF (apagadas):

* AF (acarreo auxiliar) Ocupa el bit 4 del registro de banderas.

Es usada por instrucciones aritméticas. Se pone en ON cuando se produce un acarreo o "préstamo" desde la parte baja a la parte alta. Las partes alta y baja son de 4 bits en una cantidad de 8 bits (byte) y de 8 bits en una cantidad de 16 bits (palabra).

Ejemplos:

0011 1001 39H

0111 0111 77H

+ 0010 1100 2CH
0110 0101 65H

Acarreo desde la parte baja a la parte
alta en una cantidad de 8 bits.

- 0001 1111 1FH
0101 1000 58H

"Préstamo" desde la parte alta a la parte
baja en una cantidad de 8 bits.

* CF (acarreo) Ocupa el bit 0 del registro de banderas.

Es utilizada por las instrucciones de suma y resta, y también por instrucciones de rotación que pueden aislar un bit en memoria o de un registro ubicándolo en la bandera de acarreo. La bandera de acarreo se pone en ON cuando el bit más alto del resultado produce un acarreo, y se pone en OFF cuando no hay acarreo.

* OF (overflow) Ocupa el bit 11 del registro de banderas.

Se coloca en ON cuando se produce un overflow, es decir cuando se pierde un dígito significativo pues el tamaño del resultado excede la capacidad del campo de destino. Existe, también, una instrucción que genera una interrupción por overflow. Se coloca en OFF cuando no se produce overflow.

* SF (signo) Ocupa el bit 7 del registro de banderas.

Indica siempre el signo del resultado de una operación, pues almacena siempre el bit más alto del campo donde se guarda el resultado. Si está en ON, el resultado es negativo; si está en OFF, el resultado es positivo.

* PF (paridad) Ocupa el bit 2 del registro de banderas.

Indica la paridad del resultado. Si está ON, significa que el resultado tiene paridad par (número par de bits encendidos). Si está OFF, significa que el resultado tiene paridad impar (número impar de bits encendidos)

* ZF (cero) Ocupa el bit 6 del registro de banderas.

Se pone en ON si el resultado de una operación es cero; se pone en OFF si el resultado de una operación es distinto de cero (positivo o negativo).

2.2.2. Banderas de control.

Las 3 banderas de control permiten alterar las operaciones del procesador. Las banderas de control y sus efectos son las siguientes :

* DF (dirección) Ocupa el bit 10 del registro de banderas.

Esta bandera maneja el modo de procesar cadenas de caracteres. Si está en ON los string serán procesados desde la dirección más baja a la más alta, es decir de derecha a izquierda. En cambio, si está en OFF los string serán procesados en sentido contrario, es decir desde la dirección más alta a la más baja o, lo que es lo mismo, de izquierda a derecha.

* IF (interrupciones) Ocupa el bit 9 del registro de banderas.

Esta bandera maneja el modo en que el procesador actuar ante un requerimiento de interrupción enmascarable (interrupción externa). Si está en ON la CPU reconocer los pedidos de interrupciones enmascarables. Si está en OFF se deshabilitan estas interrupciones. Esta bandera no tiene efecto sobre las interrupciones externas no enmascarables y sobre las interrupciones internas.

* TF (trampa) Ocupa el bit 8 del registro de banderas.

Esta bandera pone al procesador en modo de paso simple para depurar el programa. Cuando está en ON genera una interrupción interna después de cada instrucción para permitir la inspección del programa que se está ejecutando (modo debugger). Cuando está en OFF, queda deshabilitado el modo de debugger.

2.3. EJEMPLO DE REGISTROS Y BANDERAS.

El siguiente ejemplo fue desarrollado a partir de un vuelco de memoria, realizado con el utilitario DEBUG.COM, de un programa ejecutable.

El vuelco, en un determinado momento, arroja información del contenido de todos los registros y banderas y también de la actual dirección y contenido de la próxima instrucción a ejecutarse.

El vuelco en cuestión es el siguiente:

AX=0000 BX=0000 CX=0380 DX=0000 SP=0080 BP=0000 SI=0000 DI=0000
DS=1121 ES=1121 SS=1166 CS=1131 IP=0000 NV UP DI PL NZ NA PO NC
1131:0000 BE8100 MOV SI,0081

El significado de cada ítem es el siguiente:

AX=0000 Registro A (acumulador)

BX=0000 Registro B (base)

CX=0380 Registro C (contador)

DX=0000 Registro D (datos)

SP=0080 Registro puntero del stack

BP=0000 Registro puntero de la base

SI=0000 Registro índice del origen

DI=0000 Registro índice del destino

DS=1121 Registro base del segmento de datos

ES=1121 Registro base del segmento extra

SS=1166 Registro base del segmento de stack

CS=1131 Registro base del segmento de código

IP=0000 Registro de próxima instrucción

NV Bandera OF en OFF; no hay overflow

UP Bandera DF en OFF; manejo de string de izquierda a derecha

DI Bandera IF en OFF; interrupciones enmascarables deshabilitadas

PL Bandera SF en OFF; signo positivo

NZ Bandera ZF en OFF; no es cero

NA Bandera AF en OFF; no hay acarreo auxiliar

PO Bandera PF en OFF; paridad impar

NC Bandera CF en OFF; no hay acarreo

1131:0000 BE8100

Esto significa que en la dirección física generada por la dirección lógica CS:0000 hay un código BE8100.

La dirección física que se obtiene es $11310 + 0000 = 11310$.

Entonces, en la dirección física 11310H está la instrucción MOV SI,0081.

CAPITULO 3

MEMORIA

La memoria está constituida por una serie de chips individuales que sirven para almacenar la información.

Las computadoras que trabajan con el microprocesador 8088-8086 vienen con 2 tipos de memoria: la memoria **ROM** (Read Only Memory) y la memoria **RAM** (Random Access Memory).

La memoria **ROM** contiene una serie de programas de control y sincronización, y es de uso exclusivo del sistema. La memoria **RAM**, en cambio, es accesible a los usuarios.

El microprocesador 8088-8086 trabaja sobre un esquema de memoria organizada por segmentos. Como fue visto anteriormente, estos segmentos de memoria pueden tener hasta 64 Kbytes de tamaño y, físicamente, pueden ser adyacentes, parcialmente superpuestos, totalmente superpuestos o disjuntos, según sea el modo de direccionarlos mediante los registros de segmentos. Existen algunas áreas especiales de la memoria, tales como el stack y el espacio de E/S, que serán tratados en forma particular.

3.1. ALMACENAMIENTO

La memoria está compuesta de bytes de 8 bits, siendo un byte la mínima unidad direccionable.

El sistema siempre almacena información de a palabra (2 bytes), con el byte más significativo en la más alta ubicación de memoria. Otra estructura utilizada es la doble palabra (4 bytes), que sirve para establecer direcciones fuera del segmento actualmente apuntado por los registros de segmentos.

En una doble palabra usada para direccionamiento, la palabra de la más alta dirección de memoria contiene la dirección base de un segmento y la palabra de la dirección más baja contiene el desplazamiento. Asimismo, el byte más significativo de cada palabra está ubicado en la más alta dirección de la palabra.

Por ejemplo: dirección 0500H dirección 0501H

0	2	A	F	El contenido de la palabra que está en la dirección 0500H de memoria es AF02 en hexa (en binario es 1010 1111 0000 0010). Se puede ver que en la dirección más alta (0501H) se ubica el byte más significativo (AF hexa).
0000	0010	1010	1111	
Menos signif.		Más signif.		

dirección 0016H dirección 0017H dirección 0018H dirección 0019H

4	F	0	1	4	4	1	B	El contenido de la doble palabra, usada para direccionamiento, que está en la dirección 0016H es 1B44014FH, donde la palabra que está en la dirección más alta (1B44H) contiene la dirección base del segmento (1B44H) y la palabra que está en la dirección más baja (0016H) contiene el desplazamiento (014FH).
0100	1111	0000	0001	0100	0100	0001	1011	
Desplazamiento				Segmento				

3.2. DIRECCIONAMIENTO

Cada posición de memoria puede ser referenciada por una dirección física y por una dirección lógica.

Una dirección física es un valor representado en 20 bits y es único para cada posición de la memoria; con 20 bits se abarca desde 00000H hasta FFFFFH, es decir un rango de 1 megabyte de espacio de dirección.

Una dirección lógica consiste de un valor base de segmento y un desplazamiento. El valor base representa la dirección de memoria donde comienza el segmento, y el desplazamiento señala la cantidad de bytes contando desde el comienzo del segmento.

De este modo, el byte más bajo de un segmento es aquel que está ubicado a un desplazamiento 0000H, y el byte más alto de un segmento es el que está ubicado a un desplazamiento FFFFH.

Para obtener una dirección física a partir de una dirección lógica, deben sumarse la dirección base y el desplazamiento.

Tanto la dirección base de segmento como el desplazamiento son cantidades de 16 bits sin signo; esto abarcaría un rango de 64 Kbytes de espacio de dirección, pero para representar una dirección física que abarque 1 megabytes de espacio de dirección son necesarios 20 bits.

Para implementar, esto se desplaza la dirección base de segmento en un nibble (4 bits) a la izquierda; estos 4 bits son siempre cero. Así, entonces, cada dirección base puede representarse en 20 bits con ceros en los 4 bits de más bajo orden.

Por lo tanto, cuando se hace referencia a una dirección base mediante una palabra de 16 bits, el valor de la palabra representa los 16 bits más altos de una dirección base de 20 bits. Es decir, la dirección base es igual al valor de un registro de base de segmento desplazado a la izquierda 4 posiciones binarias.

Por ejemplo, si el registro ES contiene el valor 005AH, entonces ES apunta al segmento que comienza en la dirección física 005A0H.

El cálculo de una dirección física se realiza, entonces, sumando el desplazamiento a la dirección apuntada por un registro de base de segmento.

En el siguiente ejemplo se muestra como es la generación de una dirección física a partir de una dirección lógica:

Registro de base de segmento	<table border="1"><tr><td>0</td><td>5</td><td>A</td><td>9</td></tr></table>	0	5	A	9	<table border="1"><tr><td>0</td><td>5</td><td>A</td><td>9</td><td>0</td></tr></table>	0	5	A	9	0
0	5	A	9								
0	5	A	9	0							
	15	0	19	0							
			+								
Desplazamiento	<table border="1"><tr><td>0</td><td>0</td><td>1</td><td>4</td></tr></table>	0	0	1	4	<table border="1"><tr><td></td><td>0</td><td>0</td><td>1</td><td>4</td></tr></table>		0	0	1	4
0	0	1	4								
	0	0	1	4							

Dirección física

0	5	A	A	4
---	---	---	---	---

El uso de direcciones lógicas le permite al sistema manejar dinámicamente la ubicación de los programas en memoria. Todos los desplazamientos dentro de un programa son siempre relativos a los valores fijos contenidos en los registros de segmento.

En general, el valor de los registros de segmento es determinado por el sistema al tiempo de inicialización.

3.3. STACK

Podemos definir un stack como un arreglo de ítems de datos ubicados en direcciones consecutivas de memoria y mantenido en forma LIFO (Last In First Out - último en entrar primero en salir). Esto significa que los datos del stack son colocados (operación PUSH) en el tope y son extraídos (operación POP) desde el tope. Como hemos visto antes, el stack es un segmento específico de hasta 64 Kbytes de tamaño o, en otras palabras, de hasta 32 Kwords, cuya dirección base está indicada por el registro SS (stack segment).

El manejo del stack se realiza a través de los registros SP (stack pointer) y BP (base pointer).

El 8088-8086 guarda datos y direcciones al tope del stack de a una palabra por vez.

Cuando se va a colocar algún dato al tope del stack, primero el stack pointer se decrementa automáticamente en 2 (una palabra), y luego el dato es copiado en el stack. Es decir que el stack "crece" (se llena) hacia direcciones bajas de memoria. El caso inverso ocurre cuando se extrae información desde el tope del stack; primero se extrae del stack el dato apuntado por el stack pointer y, luego, éste se incrementa automáticamente en 2. Es decir que el stack "decrece" (se vacía) hacia direcciones altas de memoria.

Un ejemplo de aplicación de uso del stack se da con las subrutinas. Cuando se produce una llamada a una subrutina, el contenido de los registros y otros parámetros son colocados al tope del stack. Allí se conservan temporariamente hasta que finalice la ejecución de la subrutina; en ese momento, se extraen del stack todos los valores antes almacenados, de modo que la ejecución del programa continúe normalmente, y bajo las mismas condiciones, que antes de la llamada a la subrutina.

Todo este procedimiento descrito se realiza en forma automática, lo cual no significa que, a través de instrucciones de PUSH y POP, se pueda seguir manejando el stack mientras dura la ejecución de la subrutina.

3.4. ESPACIO DE E/S

El microprocesador INTEL 8088-8086 incluye un área especial de memoria, separada del resto, que se utiliza exclusivamente para operaciones de E/S. Es en esa zona donde existen las interfaces de E/S para periféricos tales como impresoras y terminales.

3.5. MAPA DEL ESPACIO DIRECCIONABLE

El siguiente cuadro muestra de qué manera está distribuida la totalidad de la memoria direccionable por el 8088-8086:

DIRECCIONES (Hexa)	LONGITUD (Kb)	CONTENIDO
00000-003FF	1	Vectores de interrupciones
00400-004FF	0.25	Area de datos del BIOS
00500-005FF	0.25	Area de datos del BASIC y del DOS
00600-0FFFF	62.5	Memoria del usuario
10000-9FFFF	576	Area de expansión de la memoria
A0000-AFFFF	64	Area de expansión de memoria de pantalla
B0000-BFFFF	64	Memoria de pantalla
C0000-FFFFF	256	Memoria ROM

- Los primeros 640 Kbytes (00000-9FFFF) corresponden a la máxima cantidad de memoria RAM

- soportada.
- El área de memoria ROM está ubicada en las posiciones más altas del espacio direccionable.
- El área de datos del BIOS (Basic Input Output System) es una zona reservada para uso del BIOS, pero se puede explorar y modificar con un programa de usuario. Este área se inicializa en el arranque del sistema.
- Dentro del área de datos del BIOS (00400-004FF) existe lo que se llama el área de comunicaciones entre aplicaciones; son, en total, 16 bytes, ubicados en las direcciones 004F0 a 004FF, que se usan para almacenar información que comparten varios programas.
- El área de memoria de pantalla (B0000-BFFFF) es usada por el adaptador de monitor monocromático y por el adaptador de monitor color/gráficos.
- El área de expansión de memoria de pantalla (A0000-AFFFF) es usada por el EGA (Enhanced Graphics Adapter) y por el Adaptador Profesional de Gráficos.

CAPITULO 4

INTRODUCCION AL LENGUAJE ASSEMBLER

El procesador o CPU está diseñado para ejecutar instrucciones, y puede, en un pequeño lapso de tiempo, "reconocer" y ejecutar miles de ellas.

Cada instrucción es una secuencia de unos y ceros residente en la memoria, que la CPU puede interpretar a través de sus circuitos.

Cuando se escribe un programa en cualquier lenguaje, siempre el producto final que el procesador puede entender es ese mismo programa traducido a lenguaje de máquina.

Se llega al programa en lenguaje de máquina después de una serie de "traducciones" que va sufriendo el programa fuente.

De este modo, cada instrucción de un programa se transformará, finalmente, en una o más instrucciones en lenguaje de máquina.

En particular, cada instrucción de un programa escrito en un lenguaje de alto nivel (FORTRAN, COBOL, PASCAL, BASIC, etc.), equivale a varias instrucciones en lenguaje de máquina.

Distinto es lo que ocurre con el lenguaje assembler. El assembler es un lenguaje de bajo nivel que representa, en realidad, el código nemotécnico del lenguaje de máquina; es decir que cada instrucción de assembler equivale a una instrucción en código de máquina.

4.1. LENGUAJE DE MAQUINA

Para convertir un programa en lenguaje assembler a código de máquina, se debe convertir cada instrucción del programa en assembler a su equivalente en código de máquina. Esta es una de las tareas efectuada por el compilador.

El formato de una instrucción en código de máquina incluye la suficiente información para identificar qué operación es hecha, que operando u operandos son usados, si la operación que se efectúa es sobre byte o sobre palabra, si la operación incluye a operandos que están ubicados en registros o en un registro y en una posición de memoria, y, si uno de los operandos es una posición de memoria, cómo es la generación de su dirección. Toda esta información está codificada en los bits de cada instrucción en código de máquina.

Las instrucciones en código de máquina del 8088-8086 varían en el número de bytes que necesitan para ser codificadas.

Algunas instrucciones pueden ser codificadas en un sólo byte, como ser el caso de operaciones simples con un registro; otras, en cambio, requieren dos o más bytes. El máximo número de bytes que puede ocupar una instrucción es de 6.

Cada instrucción de máquina es, en realidad, un conjunto de microinstrucciones. Una microinstrucción es, entonces, la unidad más pequeña que puede ser ejecutada, y se corresponde directamente a los circuitos electrónicos de la CPU.

4.2. DESARROLLO DE UN PROGRAMA EN ASSEMBLER

El desarrollo de un programa en assembler requiere de una serie de pasos, desde que se escribe hasta que está en condiciones de ser ejecutado. Esos pasos son, en general, los mismos que deben efectuarse para cualquier programa en cualquier lenguaje, es decir, diseñarlo, escribirlo, compilarlo, linkeditarlo, depurarlo, y ejecutarlo.

La escritura de un programa fuente se hace a través de un programa editor, que permite escribir las líneas de sentencias fuente. Para ello, se utiliza cualquier editor de líneas o procesador de la palabra disponible para los PC, tales como Wordstar, Microsoft Word, MultiMate, IBM Personal Editor, etc. En particular, veremos el EDLIN, que es el editor incluido con el sistema operativo DOS.

El procedimiento de compilación (generación del programa objeto), se efectúa utilizando el compilador del assembler, llamado ASM, o bien el compilador del macroassembler, denominado MASM.

El procedimiento de linkedición o vinculación (generación del programa ejecutable), se efectúa con el utilitario LINK.

La depuración de un programa es un procedimiento que consiste en seguirlo durante su ejecución para detectar y corregir todos los errores que se produzcan. En particular, usaremos el utilitario DEBUG para ejecutar un programa compilado y linkeditado, de modo de observar los cambios que se producen en las distintas posiciones de memoria y para, eventualmente, efectuar las correcciones que sean necesarias.

Un programa, ya compilado, linkeditado y depurado, se puede ejecutar a través de su invocación directa desde el DOS.

4.2.1. Edición.

El proceso de edición de un programa fuente consiste en escribir las líneas de sentencias en el lenguaje fuente. Para la escritura de programas en assembler, utilizaremos el editor de línea incluido con el sistema operativo DOS, conocido como EDLIN .

El EDLIN se puede usar para crear, modificar y visualizar archivos de texto, entre ellos los programas fuente. Consta de una serie de comandos que facilitan las distintas operaciones de manejo de textos, tales como borrar, editar y visualizar líneas enteras, y buscar, editar y sustituir texto dentro de una o más líneas.

El texto de los archivos editados con EDLIN se divide en líneas (registros) de longitud variable, de hasta 253 caracteres.

Cada línea se identifica mediante un número de línea, que es dinámicamente generado y visualizado por EDLIN (la generación y visualización dinámica significa que cuando se insertan y/o borran líneas, automáticamente se corre la numeración de las líneas posteriores a las líneas insertadas y/o borradas).

El número de línea es la única identificación de una línea de texto, y es a través de dicho número que se puede direccionar a una línea.

Cuando se finaliza la edición de un archivo que ya existía, EDLIN genera una copia de seguridad del archivo editado, denominándolo con el mismo nombre y extensión .BAK.

Para correr el EDLIN, se debe introducir, desde el DOS, el siguiente comando:

A>EDLIN archivo

Si el archivo ya existe, se cargará en memoria para poder ser editado, visualizado y/o modificado. En cambio, si el archivo no existe, EDLIN indica que se trata de un archivo nuevo y, entonces, se puede comenzar a editarlo con el comando Insert.

Para programas assembler, se utiliza la extensión .ASM.

En el apéndice B puede verse un sumario de los comandos de EDLIN más usados.

4.2.2. Compilación.

El procedimiento de compilación o ensamblado de un programa es un primer paso en la conversión del lenguaje simbólico en lenguaje de máquina.

El producto de la compilación es el módulo objeto, es decir el programa original convertido en lenguaje de máquina.

Para programas assembler, usaremos el compilador MASM, correspondiente al macro-assembler del microprocesador INTEL 8088-8086.

Se invoca al compilador con el siguiente comando:

A>MASM archivo

El nombre de archivo es un parámetro opcional; si no se especifica, el compilador lo pedirá en primer lugar.

El archivo debe ser un programa fuente en assembler, con extensión .ASM.

Como respuesta al comando introducido, el compilador pide que se especifiquen los restantes parámetros:

Object filename [archivo.OBJ]:

Source listing [NUL.LST]:

Cross reference [NUL.CRF]:

Lo indicado entre corchetes es la opción default que brinda el compilador. Es decir, da la posibilidad de aceptar esos parámetros con sólo oprimir la tecla ENTER.

El archivo objeto, es el nombre del módulo objeto resultante de la compilación. MASM, por default, toma el mismo nombre de archivo que el programa fuente, pero con extensión .OBJ.

El listado del fuente es un archivo de texto que crea el compilador, donde incluye las instrucciones simbólicas y sus respectivos códigos absolutos, como así también los desplazamientos de cada elemento del programa dentro de cada segmento. El nombre sugerido por el compilador, NUL.LST, indica que, por default, no se generará ningún listado del fuente; para obtener un listado del fuente, basta con indicar una unidad, con lo que automáticamente se generará en dicha unidad un archivo con el mismo nombre que el programa fuente y extensión .LST. Si se desea otro nombre de archivo, entonces deberá ser especificado.

La impresión por pantalla del listado del fuente se hace mediante el comando TYPE:

A>TYPE nombre

donde nombre es el nombre del listado del fuente creado por el compilador.

Para obtener una copia por impresora, se deben presionar simultáneamente las teclas CTRL y PRINT SCREEN, antes de introducir el comando TYPE.

El archivo de referencias incluye una lista de todos los rótulos definidos en el programa, con indicación de la línea donde está definido cada uno, y de las líneas donde se los invoca. Al igual que el listado del fuente, el nombre sugerido por el compilador es NUL, es decir que por default no obtiene ningún archivo de referencias. Para indicar alguno, debe especificarse una unidad, en cuyo caso se creará en dicha unidad un archivo con el mismo nombre que el programa fuente y extensión CRF. Si se desea otro nombre de archivo, entonces deberá ser especificado.

Para obtener una copia del archivo de referencias, primero debe ejecutarse el comando:

A>CREF nombre;

y luego se debe seguir el mismo procedimiento que para imprimir el listado del fuente.

4.2.3. Linkediación.

La linkedición o vinculación obtiene el programa ejecutable a partir de un programa objeto. Su tarea consiste en completar la conversión a lenguaje de máquina.

La linkedición hace que un programa sea reubicable; esto significa que pueda ser ejecutado a partir de cualquier dirección física de memoria. Para lograr que un programa sea reubicable, las direcciones del mismo deben ser referencias relativas a una dirección base.

Otra función del linkeditor es la de "unir" varios módulos objeto en un sólo programa ejecutable. Esto es muy usado cuando un programa utiliza subprogramas residentes en bibliotecas o otros módulos objeto. De aquí deriva el nombre de vinculación, ya que el linkeditor se encarga de vincular esos distintos módulos objeto, resolviendo las direcciones que quedaron sin definir en el proceso de compilación, por ser referencias externas.

Para linkeditar un programa se debe introducir el siguiente comando:

A>LINK archivo

El nombre de archivo es un parámetro opcional; si no se especifica, el linkeditor lo pedirá en primer lugar. El archivo debe ser un programa objeto, con extensión .OBJ. Puede incluirse más de un archivo, en cuyo caso se los debe especificar uno a continuación del otro separados por el signo +.

Como respuesta al comando introducido, el linkeditor pide que se especifiquen los restantes parámetros:

Run file [archivo.EXE]:

List File [NUL.MAP]:

Libraries [LIB]:

Lo indicado entre corchetes es la opción default que brinda el linkeditor. Es decir, da la posibilidad de aceptar esos parámetros con sólo oprimir la tecla ENTER.

El archivo ejecutable es el programa que finalmente se ejecutará. El nombre sugerido por el linkeditor es el mismo que el del archivo objeto, pero con extensión .EXE. En caso de existir más de un archivo objeto, el nombre del archivo ejecutable será el mismo que el del primero de la lista de archivos objeto.

El archivo de listado es un mapa de la memoria, que contiene información referida a cada uno de los segmentos. Para cada segmento, se indica la dirección de inicio, la dirección final, la longitud en bytes y la clase (CODE, DATA, EXTRA o STACK).

El nombre sugerido por el linkeditor es NUL.MAP, es decir que, por default, no se genera ningún mapa. Si se desea crear un archivo de mapa de memoria, basta con indicar una unidad, en cuyo caso se creará un mapa con nombre igual al nombre del programa y extensión .MAP. Si se desea crear un mapa con otro nombre, se deberá especificar.

Para imprimir el mapa, se utiliza el comando TYPE.

Las bibliotecas son archivos objeto que contienen rutinas usadas implícitamente por los programas. Por ejemplo, toda operación de E/S, que en lenguaje fuente se invoca a través de una instrucción, está asociada a una rutina que deberá estar presente en el módulo ejecutable.

Entonces, la referencia a bibliotecas le sirve al linkeditor para saber cuales son las bibliotecas que debe incluir en el módulo ejecutable. Si se responde con ENTER, el linkeditor incluirá las bibliotecas estándar; si se desea incluir otras bibliotecas, estas deberán ser indicadas explícitamente. Los archivos que contienen bibliotecas, se identifican por su extensión .LIB.

4.2.4. Depuración.

Para depurar programas, usaremos el utilitario DEBUG.

El DEBUG es una poderosa herramienta que permite ejecutar programas en la modalidad "paso a paso", de modo de poder detectar errores a través de la visualización del contenido de la memoria.

La posibilidad de observar el contenido de la memoria, brinda también un interesante e instructivo método de ir viendo como se producen los cambios en las distintas posiciones de memoria después de la ejecución de cada instrucción. En esta modalidad, DEBUG despliega en la pantalla el contenido de los registros y el estado de las banderas o códigos de condición.

Para correr el DEBUG, se debe ingresar el comando:

A>DEBUG [archivo]

Si se explicita un archivo, este se cargará en la memoria y, entonces, pueden introducirse comandos para modificar, visualizar o ejecutar el contenido del archivo especificado.

Si no se explicita un archivo, el DEBUG trabajará con el contenido actual de la memoria. En este caso, para poder trabajar con un archivo, este deberá cargarse usando los comandos Name y Load del DEBUG.

En el Apéndice C se ofrece un detalle de los comandos del DEBUG más usados.

CAPITULO 5

ESTRUCTURA DE UN PROGRAMA EN ASSEMBLER

En este capítulo se ven los elementos que forman parte de un programa assembler, de modo de poder comenzar a escribir nuestros propios programas.

5.1. GENERALIDADES

Cómo fue visto anteriormente, el microprocesador INTEL 8088-8086 maneja la memoria a través de segmentos.

Entonces, una importante cuestión que se debe considerar al escribir un programa en assembler, es que deben definirse los distintos segmentos que dan la estructura básica al programa.

Estos segmentos son: de datos (data segment), de stack (stack segment), de código (code segment) y auxiliar (extra segment). Un programa debe incluir, al menos, un segmento de código.

Cada uno de los segmentos contiene una parte del programa; así, el segmento de datos incluye todos los datos y estructuras de datos que usa el programa; el segmento de código, incluye instrucciones del programa; el segmento de stack incluye un espacio de memoria para pasaje de parámetros y direcciones; y el segmento extra incluye datos adicionales.

El macro-assembler provee una serie de directivas que permiten la definición de los segmentos.

Un programa fuente es una secuencia de líneas de sentencias fuente. Cada línea de programa contiene sólo una sentencia fuente.

Un programa fuente contiene dos tipos de sentencias: instrucciones y directivas o pseudo-operaciones (pseudo-ops).

Las instrucciones son representaciones simbólicas de las instrucciones de máquina; le indican al microprocesador que operación debe ejecutar.

En cambio, las pseudo-ops son mandatos al compilador que le indican lo que debe hacer con las instrucciones y datos.

La diferencia entre instrucción y pseudo-op es un concepto fundamental para interpretar correctamente cómo es la estructura de un programa assembler: mientras que las instrucciones se aplican a tiempo de ejecución, las pseudo-ops se aplican a tiempo de compilación.

5.2. SINTAXIS DE INSTRUCCIONES.

Las instrucciones en assembler deben ser escritas cumpliendo ciertas reglas de sintaxis. Esto le permite al compilador poder interpretar cada uno de los elementos que forman parte de una línea de programa.

El formato general de una instrucción es el siguiente:

[Rótulo:] Código de Operación [Operando] [;Comentario]

De los 4 campos indicados, sólo el de código de operación es obligatorio.

Los campos de rótulo y de comentario son opcionales, y el campo de operando debe ir solamente en aquellas instrucciones que lo requieren (de lo contrario, debe ser omitido).

Todos los campos de una misma instrucción deben estar siempre en la misma línea de programa y, separados entre sí por, al menos, un blanco.

Un ejemplo de instrucción usando los 4 campos, es el siguiente:

INITCONT: MOV CX,DI ;Inicializar contador

En este ejemplo, INITCONT: es un rótulo, MOV es el código de operación, CX,DI es el campo de operando y ;Inicializar contador es el comentario.

5.2.1. Rótulo.

El rótulo o etiqueta es la asignación de un nombre a una instrucción. Esto permite que otras instrucciones hagan referencia a la instrucción rotulada a través de ese nombre.

El rótulo se utiliza, fundamentalmente, para definir variables de datos que luego el programa utilizará para operar, y para establecer puntos del programa donde se habrá de bifurcar por alguna condición.

Un rótulo puede tener hasta 31 caracteres de longitud y, opcionalmente, puede finalizar con un dos puntos (:).

Si un rótulo finaliza con el carácter :, significa que sólo puede ser referenciado por instrucciones que se ubiquen en el mismo segmento; en cambio, si no finaliza con :, significa que puede ser referenciado por cualquier otro segmento.

Un rótulo puede incluir:

- Letras mayúsculas desde A hasta Z;
- Letras minúsculas desde a hasta z;
- dígitos de 0 a 9;
- los caracteres especiales ?, @, _ y \$;
- el carácter especial ., sólo si es el primero.

Un rótulo puede comenzar con cualquier carácter de los permitidos, menos con un dígito (en este caso, debe ir precedido por un punto).

Por ejemplo, \$ROTUL1:, .1A:, FIN_CICLO:, etc.

Los símbolos reservados AH, AL, AX, BH, BL, BX, BP, CH, CL, CX, CS, DH, DL, DX, DI, DS, ES, SI, SP y ST, que corresponden a nombres de registros, no pueden ser usados como rótulos.

Otra restricción para los rótulos es que no pueden tener blancos intermedios; en su lugar, se pueden colocar caracteres de subrayado.

Por ejemplo, FIN_CICLO:.

Es conveniente tener en cuenta las siguientes recomendaciones para construir nombres de rótulos:

- Hacer los nombres lo mas cortos posible, siempre que ello sea razonable. Por ejemplo, es preferible el rótulo
KPH en vez de KILOMETROS_POR_HORA.
- Definir nombres sencillos y fáciles de tipear, para evitar errores de escritura. Esto ocurre, generalmente cuando se definen rótulos que incluyen varias veces la misma letra, como ser XXXXX, y también cuando se utilizan caracteres que se pueden confundir, como ser la letra O y el dígito 0, la letra I y el dígito 1, o la letra S y el dígito 5.
- Definir rótulos que no se confundan con otros. Por ejemplo, puede producirse confusión entre un rótulo XXXX y otro XXYX.

5.2.2. Código de Operación.

Cada una de las operaciones básicas que puede ejecutar el microprocesador 8088-8086, se identifica mediante un código simbólico de entre 2 y 6 letras, denominado código de operación.

El código de operación debe seguir al campo de rótulo, y estar separado de este por, al menos, un blanco. Si no existe rótulo, es decir que el código de operación es el primer campo de una sentencia, entonces se debe dejar, como mínimo, un blanco a la izquierda.

Por ejemplo, los códigos simbólicos para las operaciones de suma, resta, multiplicación y división son respectivamente, ADD, SUB, MUL y DIV.

El compilador utiliza una tabla interna para convertir cada código simbólico a su equivalente absoluto.

5.2.3. Operando.

El campo del operando le indica al 8088-8086 donde se encuentran los datos a ser procesados, y como se accede a ellos.

Una instrucción puede tener ninguno, uno o dos operandos, dependiendo del tipo de instrucción que sea. Cuando una operación incluye dos operandos, estos deben estar separados entre si por una coma.

En una instrucción con dos operandos, el primero es el operando destino y el segundo es el operando fuente. Por ejemplo, en la instrucción MOV AX,BX el contenido del registro BX (operando fuente), se copia en el registro AX (operando destino).

Los operandos pueden ser de los siguientes tipos:

- reg8 : registro de 8 bits
- reg16: registro de 16 bits
- reg : registro de 8 ó 16 bits
- mem8 : byte (8 bits)
- mem16: palabra (16 bits)
- mem : byte o palabra (8 ó 16 bits)
- val8 : valor inmediato que puede almacenarse en 8 bits
- val16: valor inmediato que puede almacenarse en 16 bits
- val : valor inmediato que puede almacenarse en 8 o 16 bits

La manera en que se accede a los operandos es a través de los modos de direccionamiento.

Los modos de direccionamiento son medios para permitir especificar la ubicación de los operandos; cada operando, siempre, incluye un modo de direccionamiento.

Por ejemplo, en la instrucción MOV AX,ROT1, el operando destino, AX, está direccionado con modo registro, y el operando fuente, ROT1, está direccionado con modo directo.

Más adelante se describen cada uno de los modos de direccionamiento provistos en el macro-assembler.

5.2.4. Comentario.

El campo de comentario es utilizado para describir la operación efectuada por una instrucción. Siempre debe ir precedido por un punto y coma (;).

En general, es muy útil el uso del comentario, especialmente si se describe lo que hace la instrucción en el contexto del programa. Por ejemplo:

```
MOV CX,0 ;Inicializa el registro contador
```

es mucho mas claro de entender, en el contexto de un programa, que:

```
MOV CX,0 ;Coloca 0 en CX
```

En un programa fuente, se pueden incluir líneas solamente de comentarios. Esto permite, por ejemplo, describir un bloque completo de programa o bien servir de separador entre dos bloques.

Para colocar una línea de comentarios, se debe poner un punto y coma (;) como primer carácter. Por ejemplo: ;Inicialización de contadores

```
;
MOV CX,0
.....
;
;Copiar vector fuente en vector destino
;
.....
```

El compilador ignora los comentarios cuando genera el modulo objeto; solamente en el listado del programa fuente, generado por el compilador, aparecen los comentarios.

El uso de comentarios en un programa permite tener una importante documentación del mismo, que es muy útil en caso de, por ejemplo, tener que modificarlo.

5.3. SINTAXIS DE PSEUDO-OPS

Las pseudo-ops son comandos al ensamblador, y se utilizan para fijar segmentos y procedimientos, definir símbolos, reservar memoria para almacenamiento temporario, etc.

A diferencia de las instrucciones, las pseudo-ops no generan código objeto.

La sintaxis usada para escribir sentencias con pseudo-ops es, esencialmente, la misma que la usada para escribir instrucciones.

El formato general es el siguiente:

```
[Rótulo:] Código de Pseudo-op [Operando] [;Comentario]
```

El significado de los campos es el mismo que para las instrucciones. La única diferencia de importancia es que, para una pseudo-op, el campo de operando puede incluir más de 2 operados. Por ejemplo, la sentencia:

```
DB 0FFH,0FFH,0FFH,0FFH,0FFH
```

le indica al ensamblador que cargue, en las cinco posiciones siguientes de memoria, el valor hexadecimal FF.

5.4. MODOS DE DIRECCIONAMIENTO

El acceso a los operandos se efectúa a través de los modos de direccionamiento, es decir, modos en que el 8088-8086 ubica a los datos.

Dos conceptos importantes para entender el modo de acceder a un operando son el de dirección efectiva (DE) y el de dirección real o física (DR).

La DE es la distancia en bytes desde el comienzo del segmento a la posición del operando dentro del segmento. Siempre la DE es una cantidad de 16 bits sin signo, de modo que su rango es desde 0 a 65.535 (64 K).

La DR es la dirección verdadera dentro de la memoria de un operando. La obtención de una DR se hace mediante la composición de un registro de segmento con una DE.

Existen, básicamente, 7 modos de direccionamiento, aunque se pueden obtener alrededor de 24 modos distintos al combinar los 7 modos básicos.

De esos 7 modos, los últimos 3 brindan la DE del operando.

5.4.1. Modo registro.

El operando es un registro.

Por ejemplo, en la siguiente instrucción:

`MOV AX,BX`

ambos operandos, el operando fuente (BX) y el operando destino (AX), están direccionados en modo registro. El efecto de esta instrucción es de copiar el contenido del registro BX al registro AX.

5.4.2. Modo inmediato.

El operando es un dato constante.

En el siguiente ejemplo, se observa como funciona el modo de direccionamiento inmediato:

`MOV AL,015H`

El operando fuente es una constante hexadecimal y está direccionado en modo inmediato. El operando destino, AL, está direccionado en modo registro. El efecto de esta instrucción es de colocar el valor hexadecimal 015 en la parte baja del registro AX.

5.4.3. Modo directo.

El operando es una dirección efectiva.

Por ejemplo:

`MOV CX,ALFA`

El operando fuente es un rotulo que identifica una posición de memoria dentro del segmento de datos, y está direccionado en modo directo. La unidad de ejecución obtiene la dirección real de memoria (dirección física), al combinar la DE dada con el registro base del segmento de datos DS.

El efecto de la instrucción es de copiar en el registro CX, el contenido de la DE rotulada ALFA. ALFA debe haber sido definida en el programa fuente.

5.4.4. Modo registro indirecto.

Este modo de direccionamiento es similar al modo directo en cuanto a la obtención de la dirección real, dado que la DE se combina con el registro base del segmento de datos DS. En este caso, la DE del operando está contenida en el registro BX, en el puntero de la base BP, o en uno de los registros índices (SI o DI).

Por ejemplo:

`MOV AX,[SI]`

Esta instrucción copia en el registro AX el contenido de la posición de memoria cuya DE está dada por el contenido del registro SI.

El registro que se utiliza con el modo registro indirecto, debe ir entre corchetes.

Ejemplo: antes	después
AX= 0000	AX=1BB1
SI= 0F2A	MOV AX,[SI] SI=0F2A
(0F2A)=1BB1	(0F2A)=1BB1

5.4.5. Modo relativo a base.

La DE del operando se obtiene sumando un desplazamiento al contenido de BX o de BP. La dirección física se calcula combinando la DE con el registro DS o SS, según se haya usado BX o BP.

Por ejemplo:

`MOV [BX]+8,AL`

Mueve el contenido de AL en la DE resultante de sumarle 8 al contenido del registro BX.

Ejemplo:	antes	después
	AX=12A3	AX=12A3
	BX=1111	MOV [BX]+8,AL BX=1111
	(1119)=0000	(1119)=00A3

Hay 3 maneras diferentes de especificar un operando con el modo relativo a base. Las siguientes instrucciones son equivalentes:

<code>MOV AX,[BP]+4</code>	;desplazamiento después de la base
<code>MOV AX,4[BP]</code>	;desplazamiento antes de la base
<code>MOV AX,[BP+4]</code>	;desplazamiento dentro del corchete

5.4.6. Modo indexado directo.

Este modo es idéntico al modo anterior, pero usando uno de los registros índices, SI o DI, en vez de BX o BP, en la generación de la DE. La dirección física se obtiene combinando la DE con el registro DS.

Por ejemplo:

```
MOV AX,[SI]+5
```

Se carga en AX el contenido de la DE obtenida de sumarle 5 al contenido del registro SI.

Este modo de direccionamiento es muy útil para el manejo de arreglos y tablas en memoria.

Ejemplo: antes después
 AX=0000 AX=AABB
 SI=0004 MOV AX,[SI]+VECTOR SI=0004
 dir VECTOR= 0001
 (0005)=BBAA (0005)=BBAA

5.4.7. Modo indexado a base.

Este modo es una combinación de los dos anteriores. La DE del operando se calcula como la suma de un registro base, un registro índice y, opcionalmente, un desplazamiento. La dirección física se obtiene de combinar la DE calculada con el registro DS o con el registro SS, según sea BX o BP el registro de base usado.

Por ejemplo:

```
MOV AX,[BX][SI]+ALFA
```

Mueve en AX el contenido de la DE que resulta de sumar el contenido del registro BX al contenido del registro SI, con el desplazamiento directo ALFA (lo que se suma es la dirección efectiva de ALFA, no su contenido).

Ejemplo: antes después
 AX=0000 AX=AABB
 BX=0100 BX=0100
 SI=0019 MOV AX,[BX][SI]+VALOR SI=0019
 dir VALOR=0001
 (011A)=AABB (011A)=AABB

5.4.8. Prefijos de segmento.

Para obtener la DR de un operando, el registro de segmento que se compone a la DE depende del registro usado para obtener la DE. La siguiente tabla muestra los registros de segmento que se asumen por default:

Reg. usado en DE	Reg. de seg. default
IP	CS
SP	SS
BP	SS
BX	DS
SI	DS
DI	DS

Estos valores default pueden ser modificados mediante el uso de los prefijos de segmento.

Los prefijos de segmento son DS:, ES:, SS: y CS:, y su uso hace que el registro de segmento default de un operando sea sustituido por el registro de segmento explicitado como prefijo.

Los prefijos de segmento pueden ser usados sólo con aquellos operandos cuyos modos de direccionamiento se refieren a DE (modos de direccionamiento 3 a 7).

5.5. CONSTANTES

En un programa assembler se pueden escribir distintos tipos de constantes. La forma de indicar constantes para el 8088-8086 es la siguiente:

- a) Binarias: Es una secuencia de unos y ceros, seguida por la letra B. Por ejemplo, 10111010B.
 - b) Decimales: Es una secuencia de los dígitos entre 0 y 9, con o sin la letra D. Por ejemplo, 129D o, bien, 129.
 - c) Hexadecimales: Es una secuencia de los dígitos entre 0 y 9 y de las letras A hasta F, seguida por la letra H. El primer carácter siempre debe ser un dígito entre 0 y 9, para permitir la diferenciación de números hexadecimales con nombres de variables y símbolos. Por ejemplo, 0E23H (el 0 como primer dígito le indica al 8088-8086 que el número es E23H).
 - d) Octales: Es una secuencia de los dígitos entre 0 y 7, seguida de la letra Q o la letra O. Por ejemplo, 1475Q.
 - e) De caracteres: Es una secuencia de letras, números o símbolos encerrada entre apóstrofes o comillas. El uso de ambos, apóstrofes y comillas, brinda la facilidad de poder definir como constante un mensaje encerrado entre comillas. Por ejemplo, "Dato incorrecto", o "Escriba 'FINAL' para terminar el proceso".
- Las constantes numéricas pueden ser, también, números negativos. Si el número es un valor decimal, debe simplemente colocarse el signo menos delante de la constante; en cambio, si es un valor binario, octal o hexadecimal, la constante debe ser escrita en notación complemento a 2.
- Por ejemplo, 11100000B y 0E0H son las formas binaria y hexadecimal del decimal -32.

5.6. OPERADORES

Los operadores son modificadores que se usan en el campo de operando de las sentencias assembler (instrucciones y pseudo-ops); se pueden utilizar varios operadores, y combinaciones de ellos, en una misma sentencia.

En todos los ejemplos vistos hasta aquí los operandos utilizados fueron registros, constantes, variables y rótulos. Pero, también, es posible utilizar expresiones como operandos. Por ejemplo, en la instrucción:

```
MOV AH,A+2
```

el operando fuente es una expresión aritmética.

Hay 5 diferentes tipos de operadores: aritméticos, lógicos, relacionales, de retorno de valores y de atributos. Existen, además, un conjunto de operadores para macros. En las próximas secciones, se verá cada uno de estos tipos.

En el Apéndice D se incluyen todos los operadores que incluye el assembler del microprocesador 8088-8086.

5.6.1. Operadores aritméticos.

Combinan operandos numéricos y producen un resultado numérico. Los operadores aritméticos más comunes son los de suma (+), resta (-), multiplicación (*) y división (/). Por ejemplo:

```
MOV AX,DESP+2
MOV AL,DESP-1
ADD CX,CANT/4
SUB BX,CANT*2
```

El operador MOD retorna como operando, el resto de la división. Por ejemplo, en la siguiente instrucción:

```
MOV AX,CANT MOD 10
```

el registro AX se carga con el resto de dividir por 10 el contenido de la posición de memoria rotulada CANT. Finalmente, los operadores SHL (SHift Left) y SHR (SHift Right), desplazan un operando numérico a la izquierda o a la derecha en la cantidad de bits señalada. Estos operadores son utilizados cuando se comparan máscaras de bits frente a un determinado patrón binario.

Por ejemplo, si se define una máscara:

```
MASC:      DW 110010B
```

las siguientes instrucciones:

```
MASC_IZQ_2  MOV AX,MASK SHL 2
MASC_DER_2  MOV BX,MASK SHR 2
```

dejan en AX y BX, respectivamente, 11001000B y 1100B.

Los operadores numéricos tienen el siguiente orden de precedencia: *, /, MOD, SHL, SHR, +, -. Este orden de precedencia puede, eventualmente, ser alterado mediante el uso de paréntesis. El uso de paréntesis hace que primero se resuelva lo incluido en ellos. Por ejemplo, en la expresión:

MOV BH,(A*4-2)/(B-3)

con A=8 y B=5, equivale a la instrucción MOV BH,15

5.6.2. Operadores lógicos.

Los operadores lógicos son usados para manipular valores binarios en vez de, como los operadores aritméticos SHL y SHR, valores decimales. Sin embargo, los operadores lógicos son más útiles para manejar bits individuales que grupos de bits. El operador AND es útil para filtrar, enmascarar o extraer ciertos bits. Su funcionamiento consiste en establecer el bit de resultado en 1, por cada posición en que los respectivos bits de los operandos sean 1; cualquier otra combinación de bits, AND fija el resultado en 0. Por ejemplo:

00110100B AND 11010111B produce el resultado 00010100B.

El operador OR establece el bit de resultado en 1, por cada posición de los operandos en la cual uno o ambos contienen un 1; en las posiciones en que ambos operandos tienen 0, el bit de resultado es 0. En el ejemplo anterior:

00110100B OR 11010111B

el resultado será 11110111B.

El operador XOR (OR exclusivo), establece el bit de resultado en 1 por cada posición de los operandos en la que ambos contengan bits distintos (un operando con 1, y el otro con 0); en las posiciones en que ambos operandos tienen el mismo bit (1 & 0), el bit de resultado es 0. En el mismo ejemplo:

00110100B XOR 11010111B

el resultado será 11100011.

La siguiente, es una tabla con los resultados de los operadores AND, OR y XOR para las distintas combinaciones de bits:

OP 1	OP 2	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

El último operador lógico, NOT, trabaja sólo con un operando, y su efecto es el de invertir cada uno de los bits del operando. Por ejemplo:

NOT 01101001B produce el resultado 10010110B.

El 8088-8086 tiene, también, instrucciones lógicas AND, OR, XOR y NOT, pero ellas no deben ser confundidas con los operadores lógicos. Mientras que las instrucciones lógicas actúan en tiempo de ejecución, los operadores lógicos actúan a tiempo de ensamblado o compilación.

5.6.3. Operadores relacionales.

Los operadores relacionales comparan dos valores numéricos o dos direcciones de memoria del mismo segmento, y produce un resultado numérico. Este resultado es 0 si la comparación es falsa, o FFFF si la comparación es verdadera.

En general, los operadores relacionales aparecen integrando expresiones en combinación con otros operadores. El uso de ellos en sí mismo es bastante limitado, dado que sólo producen dos posibles resultados. Por ejemplo:

MOVE AX,((OPCION LT 20) AND 5) OR ((OPCION GE 20) AND 6)

equivale a la instrucción MOV AX,5 si el símbolo OPCION es menor o igual que 20, y equivale a la instrucción MOV AX,6 si OPCION es mayor que 20.

Los operadores relacionales son EQ (Equal), NE (Not Equal), LT (Less Than), GT (Greater Than), LE (Less or Equal) y GE (Greater or Equal).

5.6.4. Operadores de retorno de valores.

Estos operadores devuelven información acerca de variables o rótulos de un programa.

El operador \$ retorna el valor de la actual posición de memoria, es decir, el desplazamiento de la actual instrucción. Es muy usado para obtener la longitud de un string. Por ejemplo:

```
MENSA      DB 'Tocar cualquier tecla para seguir'
LON_MENSA   EQU $-MENSA
```

Cuando deba imprimirse MENSA, se hará referencia a su longitud usando el rótulo LON_MENSA.

Los operadores SEG y OFFSET retornan, respectivamente, los valores de segmento y desplazamiento de una variable o rótulo.

Por ejemplo, la siguiente secuencia:

```
MOV AX,SEG TABLA
MOV BX,OFFSET TABLA
```

carga el valor del segmento y del desplazamiento donde está definido el rótulo TABLA, en los registros AX y BX respectivamente. Los valores que devuelven ambos operadores son siempre cantidades de 16 bits.

El operador TYPE devuelve un valor numérico que indica el atributo de tipo de una variable (1 para BYTE, 2 para PALABRA, 4 para DOBLE PALABRA, 8 para CUADRUPLE PALABRA, 10 para variable definida con DT), o el atributo de distancia de un rótulo (-1 para NEAR, -2 para FAR).

Los operadores LENGTH y SIZE son utilizados preferentemente para arreglos. LENGTH devuelve la cantidad de unidades(bytes o palabras) reservadas para una variable, y SIZE devuelve la cantidad de bytes que ocupa una variable. Por ejemplo:

```
TABLA  DW 100 DUP(1)
MOV CX,LENGTH TABLA
MOV DX,SIZE TABLA
```

En CX queda 100 y en DX queda 200.

El operador MASK se usa con un campo de un récord, y devuelve la configuración en bits del récord (al que pertenece el campo), con valor 1 en los bits del campo indicado y con valor cero en los restantes bits del récord. Por ejemplo:

```
REGIS  RECORD A:3,B:1,C:2,D:4,E:6
```

Con esta sentencia, se define el récord REGIS, cuya estructura se compone de 5 campos: A, de 3 bits, B de 1 bit, C de 2 bits, D de 4 bits y E de 6 bits.

La instrucción:

```
MOV     AX,MASK C
```

hace que en AX se almacene la cantidad binaria 0000110000000000.

5.6.5. Operadores de atributos.

Los operadores de atributos permiten establecer, por encima de lo ya definido, nuevos atributos para operandos y rótulos.

El operador PTR (pointer) establece el atributo de tipo (BYTE, WORD, DWORD, QWORD, TBYTE) o el atributo de distancia (NEAR o FAR) de un operando de memoria. En el siguiente ejemplo:

```
TABLA  DW 100 DUP(?)
.....
.....
BYTE_1 EQU BYTE PTR TABLA
```

se asigna el nombre BYTE_1 al primer byte del área de memoria definida como TABLA.

Notar que TABLA fue definida como un arreglo de WORD y no de BYTE, y que el uso del operador PTR redefine sólo el primer byte.

Ahora, se podría nominar cualquier otro byte de TABLA haciendo referencia a la posición BYTE_1. Por ejemplo:

```
BYTE_5 EQU BYTE_1+4
```

El operador PTR también puede ser usado para cambiar el atributo de distancia de un rótulo. Por ejemplo, en la instrucción:

```
INICIO: MOV CX,100
```

el rótulo INICIO: está definido, en forma implícita (porque finaliza con un :), como NEAR. Esto hace que cualquier bifurcación a ese rótulo deba estar en el mismo segmento que el rótulo; si se quiere bifurcar a INICIO: desde cualquier otro segmento, se debe utilizar el operador PTR con el atributo FAR:

```
INI_FAR EQU FAR PTR INICIO
```

El operador SHORT modifica el atributo NEAR de un rótulo para indicar que se encuentra entre +127 bytes y -128 bytes de la siguiente instrucción.

Se usa acompañando a una instrucción de bifurcación, colocándolo antes del nombre del rótulo al que se va a bifurcar.

Al indicar el operador SHORT, el compilador interpreta que el rótulo a bifurcar debe estar entre 127 bytes después y 128 bytes antes de la próxima instrucción. Esto produce que la instrucción de bifurcación sea codificada en 2 bytes, en lugar de los 3 bytes normales que ocupa sin este operador. Por ejemplo:

```
JMP SHORT ROTU
```

```
.....
```

```
ROTU: .....
```

El operador THIS asigna a un operando de dirección de memoria un atributo de tipo (BYTE, WORD, DWORD, QWORD, TBYTE) o un atributo de distancia (NEAR o FAR), en un desplazamiento igual al actual valor del contador de posiciones y en un segmento igual al del segmento que lo incluye. Por ejemplo, la secuencia:

```
BYTE_1 EQU THIS BYTE
```

```
TABLA DW 100 DUP(?)
```

crea el rótulo BYTE_1, con atributo de tipo BYTE, en la misma dirección que el arreglo TABLA. El mismo efecto se obtiene con la siguiente secuencia:

```
TABLA DW 100 DUP(?)
```

```
BYTE_1 EQU BYTE PTR TABLA
```

El operador THIS puede ser usado también para definir un atributo de distancia en posiciones de memoria con instrucciones. Por ejemplo, la secuencia:

```
INICIO EQU THIS FAR
```

```
MOV CA,100
```

le asigna a la instrucción MOV el atributo FAR, lo cual permite que instrucciones de bifurcación en otro segmento, diferente al actual, puedan bifurcar directamente al rótulo INICIO.

Cuando la CPU calcula una dirección de memoria, automáticamente asume que SS es el segmento a usar, si el operando emplea SP o BP para la determinación del desplazamiento; de la misma manera, asumir DS como registro de segmento, si el desplazamiento se determina con BX, SI o DI.

Los operadores de atributos de segmento (DS:, ES:, SS:, CS:), permiten especificar un segmento alternativo para un rótulo, una variable o una expresión de dirección. Por ejemplo:

```
MOV AX,ES:[BP]
```

le indica a la CPU que tome el operando destino desde el segmento extra en vez del segmento de stack.

Los atributos HIGH y LOW devuelven, respectivamente, el byte más alto y el byte más bajo de una expresión de 16 bits.

Por ejemplo:

```
CONST EQU 0ABCDH
```

```
.....
```

```
MOV AH,HIGH CONST
```

```
MOV BL,LOW CONST
```

deja en AH y BL, respectivamente, los valores 0ABH y 0CDH.

5.6.6. Operadores de macros.

El macro-assembler proporciona una serie de 4 operadores que pueden ser utilizados con macros. Más adelante se verán las macros en detalle.

Los operadores de macros serán luego tratados en detalle en el capítulo correspondiente a pseudo-ops.

El operador & concatena textos o símbolos, permitiendo, por ejemplo, crear rótulos u operandos "hechos a medida".

Por ejemplo:

```
DEF-TABLA MACRO SUFIJO, LONGI
```



```
TABLA&SUFUJO DB LONGI DUP(?)
ENDM
```

Una invocación a esta macro puede ser:

```
DEF_TABLA X,5
```

Con lo que la macro se expande como:

```
TABLAX DB 5 DUP (?)
```

El operador ;; hace que los comentarios que están a continuación se omitan en la expansión de la macro. Este operador es útil para lograr que el programa final ocupe menos memoria y, por lo tanto, sea ensamblado más rápidamente.

En general, es conveniente usar el operador ;comentario con aquellos comentarios absolutamente necesarios, y dejar ;;comentario para el resto.

El operador - le indica al compilador que el carácter que está a continuación es un valor literal en vez de ser un símbolo. Por ejemplo:

```
TABLA MACRO SUFIJO, LONGI, !CARAC
TABLA&SUFUJO DB LONGI DUP(CARAC)
ENDM
```

Una invocación a esta macro es:

```
TABLA X,5,A
```

Con lo que la macro se expande como:

```
TABLAX DB 5 DUP(A)
```

El operador % hace que el compilador, cuando expande la macro, substituya la expresión que está a continuación por un número.

5.7. UN PROGRAMA EJEMPLO

El siguiente, es un programa completo que sirve para escribir un texto por pantalla.

El desarrollo de este programa ejemplo permitir comprender los conceptos más importantes analizados en este capítulo.

```
; PROGRAMA EJEMPLO: ESCRIBIR UN TEXTO POR PANTALLA
```

```
;-----
```

```
CR EQU 13 ;retorno de carro
```

```
LF EQU 10 ;salto de línea
```

```
;-----
```

```
;SEGMENTO DE DATOS
```

```
;-----
```

```
;
```

```
DATOS SEGMENT ;comienzo del segmento DATOS
```

```
TEXTO DB 'ABCDEFGHI',CR,LF ;texto a escribir
```

```
DB '$' ;delimitador fin de texto
```

```
DATOS ENDS ;fin del segmento DATOS
```

```
;-----
```

```
;SEGMENTO DE STACK
```

```
;-----
```

```
;
```

```
PILA SEGMENT STACK ;comienzo del segmento PILA
```

```
DB 128 DUP('PILA') ;inicialización del stack
```

```
PILA ENDS ;fin del segmento PILA
```

```
;-----
```

```
;SEGMENTO DE CODIGO
```

```
;-----
```

```
;
```

```
CODIGO SEGMENT ;comienzo del segmento CODIGO
```

```
EJEMPLO PROC FAR ;comienzo del procedimiento EJEMPLO
```

```
ASSUME CS:CODIGO,DS:DATOS,SS:PILA
```

```

;
;Colocar dirección de retorno en el stack
;
    PUSH    DS    ;segmento al stack
    SUB     AX,AX  ;AX=0
    PUSH    AX     ;desplazamiento 0 al stack
;
;Direccionar segmento de datos con DS
;
    MOV     AX,DATOS    ;AX=dirección del segmento DATOS
    MOV     DS,AX       ;DS=dirección del segmento DATOS
;
;Escribir texto
;
    LEA     DX,TEXTO     ;DS:DX=dirección de TEXTO
    CALL    ESCRIBIR     ;llamada al procedimiento ESCRIBIR
    RET                     ;volver al DOS
EJEMPLO ENDP            ;fin del procedimiento EJEMPLO
ESCRIBIR    PROC        ;comienzo del procedimiento ESCRIBIR
;
;Procedimiento para escribir un texto
;Entrada: DS:DX=dirección de comienzo del texto a escribir
;
    PUSH    AX    ;resguardar registro AX en el stack
    MOV     AH,9   ;código para escribir texto por pantalla.
    INT     21H    ;llamada a función del DOS
    POP     AX     ;recuperar registro AX del stack
    RET                     ;volver
ESCRIBIR    ENDP    ;fin del procedimiento ESCRIBIR
;
CODIGO ENDS            ;fin del segmento CODIGO
;-----
;
    END     EJEMPLO ;empezar a ejecutar el procedimiento
                ;EJEMPLO

```

A continuación, comentaremos cada una de las sentencias del programa:

CR EQU 13 ;retorno de carro

LF EQU 10 ;salto de línea

Estas dos sentencias sirven para definir símbolos. Un símbolo es un nombre que se asigna a una constante, y que no variará en todo el programa. Entonces, a lo largo del programa, CR y LF serán sustituidos por los valores 13 y 10 respectivamente.

Estas sentencias no generan código ejecutable; son directivas o pseudo-ops, y sirven para darle mandatos al compilador.

DATOS SEGMENT ;comienzo del segmento DATOS

Pseudo-op que indica el comienzo de un segmento denominado DATOS.

TEXTO DB 'ABCDEFGHI',CR,LF ;texto a escribir

Pseudo-op que sirve para reservar un área de memoria para la variable TEXTO, con un cierto contenido inicial. DB significa "definir byte". La longitud de la variable TEXTO es de 11 bytes (9 letras, un byte con el valor 13 y un byte con el valor 10

DB '\$' ;delimitador fin de texto

A continuación del texto, se reserva otro byte con el contenido '\$'. Esto es necesario para la función que imprime un texto por pantalla, que necesita ese carácter como delimitador del texto a escribir. Es decir, que se imprimirán todos los caracteres hasta encontrar el delimitador '\$'.

DATOS ENDS ;fin del segmento DATOS

Pseudo-op que indica el final de un segmento denominado DATOS.

PILA SEGMENT STACK ;comienzo del segmento PILA

Pseudo-op que indica el comienzo de un segmento de stack denominado PILA. En esta directiva se especifica el tipo de segmento que se define (STACK).

Todo programa debe contener un segmento de stack.

DB 128 DUP('PILA') ;inicialización del stack

Pseudo-op que permite reservar memoria para el stack. En este caso, se reservan 128 elementos, de 4 bytes cada uno, inicializados con los caracteres 'PILA'. En total se reservan 512 bytes. Las posiciones del stack son, en realidad, palabras (2 bytes); por lo tanto, se reservan 256 palabras para el stack.

PILA ENDS ;fin del segmento PILA

Pseudo-op que indica el final de un segmento denominado PILA.

CODIGO SEGMENT ;comienzo del segmento CODIGO

Pseudo-op que indica el comienzo de un segmento denominado CODIGO.

EJEMPLO PROC FAR ;comienzo del procedimiento EJEMPLO

Pseudo-op que indica el comienzo de un procedimiento denominado EJEMPLO. Un procedimiento es un conjunto de instrucciones que puede ser invocado desde otro procedimiento o desde el DOS; un programa está compuesto por uno o más procedimientos.

El comienzo de un procedimiento se indica con la pseudo-op PROC y el final con la pseudo-op ENDP.

Un procedimiento se invoca mediante la instrucción CALL; dentro del procedimiento debe aparecer una instrucción RET, que hace que se retorne a la instrucción siguiente a la CALL. En este caso, el procedimiento EJEMPLO es invocado desde el DOS, y la RET har que retorne el control al sistema operativo.

La especificación de FAR en la definición del procedimiento hará que, cuando el control pase al mismo, se salvará en el stack la dirección de retorno completa : segmento=CS y desplazamiento=IP (recordar que CS:IP contiene siempre la dirección de la instrucción siguiente a la que se está ejecutando).

Un procedimiento NEAR (que es la opción por default si no se especifica nada), sólo salvar el desplazamiento (registro IP).

ASSUME CS:CODIGO,DS:DATOS,SS:PILA

Pseudo-op que sirve para indicar los registros de segmento default que se van a utilizar para direccionar los distintos segmentos del programa. En este caso, CS se asocia al segmento CODIGO, DS al segmento DATOS y SS al segmento PILA.

PUSH DS ;segmento al stack

Esta es la primera instrucción ejecutable del programa: almacena el registro DS al tope del stack.

Esta operación se debe hacer para guardar en el stack la dirección de retorno del procedimiento. La dirección de retorno será extraída del stack cuando se ejecute la instrucción RET.

Como se trata de un procedimiento FAR, la dirección de retorno debe ser completa (segmento y desplazamiento).

La dirección de retorno al DOS es DS:0, que corresponde al comienzo de una zona especial denominada PSP (Program Segment Prefix), contiene la codificación de la instrucción INT 20H (función del DOS que sirve para terminar un programa).

SUB AX,AX ;AX=0

Esta instrucción resta el contenido de AX a si mismo y almacena el resultado en AX. Es decir, limpia AX.

PUSH AX ;desplazamiento 0 al stack

Almacena AX (es decir, un cero), al tope del stack. Ahora ya está guardada en el stack la dirección completa de retorno (DS:0).

MOV AX,DATOS ;AX=dirección del segmento DATOS

Almacena en AX la componente segmento de la dirección del segmento DATOS.

Como el registro DS apunta inicialmente al comienzo del PSP, es necesario modificarlo para poder acceder a los datos dentro del segmento de datos con el registro DS.

Como la instrucción MOV no permite mover directamente a DS, se utiliza AX como registro intermedio.

MOV DS,AX ;ds=dirección del segmento DATOS

Copia el contenido del registro AX sobre el registro DS.

de este momento, las variables del segmento DATOS pueden ser accedidas mediante el registro DS dado que este apunta a dicho segmento.

Tanto el segmento de código como el segmento de stack son direccionados por el DOS mediante los registros CS y SS respectivamente. Este programa no tiene segmento extra.

```
LEA    DX,TEXTO    ;DS:DX=dirección de TEXTO
```

Esta instrucción almacena en el registro DX la dirección efectiva (desplazamiento dentro del segmento) de la variable TEXTO. Esto es necesario hacerlo pues el procedimiento ESCRIBIR, que se va a invocar a continuación, necesita que DS:DX apunte al texto que se quiere escribir.

```
CALL   ESCRIBIR    ;llamada al procedimiento ESCRIBIR
```

Esta instrucción llama al procedimiento ESCRIBIR. Como se trata de un procedimiento NEAR, en el stack se almacena sólo el valor del registro IP.

```
RET          ;volver al DOS
```

Cuando se ejecuta esta instrucción, el procesador toma del stack la dirección completa de retorno al DOS.

Esta dirección fue almacenada al comienzo del procedimiento.

```
EJEMPLO ENDP      ;fin del procedimiento EJEMPLO
```

Pseudo-op que indica la finalización de un procedimiento denominado EJEMPLO.

```
ESCRIBIR    PROC      ;comienzo del procedimiento ESCRIBIR
```

Pseudo-op que indica el comienzo de un procedimiento denominado ESCRIBIR, con atributo (por default) NEAR.

```
PUSH    AX    ;resguardar registro AX en el stack
```

Esta instrucción almacena al tope del stack el contenido del registro AX. Sirve para utilizar el registro dentro del procedimiento, y recuperar su valor original al final del mismo, de modo de conservar su estado inicial.

```
MOV     AH,9    ;código para escribir texto por pantalla.
```

Esta instrucción mueve un 9 al registro AH (parte alta de AX). Sirve para suministrarle un parámetro a la interrupción 21H (función del DOS), que será la próxima instrucción a ejecutar.

```
INT     21H    ;llamada a función del DOS
```

Esta instrucción produce una interrupción de código 21H, que corresponde a la invocación de una función del DOS, la que toma como parámetro el contenido del registro AH. En este caso, el registro AH contiene un 9, lo que significa "escribir un texto por pantalla". La función que se ejecuta supone que el texto a escribir está en la dirección DS:DX, y que el delimitador del texto es el carácter '\$'.

```
POP     AX    ;recuperar registro AX del stack
```

Copia en AX el contenido de la palabra ubicada al tope del stack. Con esto, se recupera el valor original del registro AX que había sido previamente resguardado con PUSH.

```
RET          ;volver
```

Esta instrucción devuelve el control a la instrucción siguiente a la llamada del actual procedimiento. Para ello, recupera del stack el valor de la dirección donde debe retornar. En este caso, tomar del stack sólo una palabra, que corresponde al desplazamiento, por ser un procedimiento de tipo NEAR.

```
ESCRIBIR    ENDP      ;fin del procedimiento ESCRIBIR
```

Pseudo-op que indica el final de un procedimiento denominado ESCRIBIR.

```
CODIGO ENDS      ;fin del segmento CODIGO
```

Pseudo-op que indica el final de un segmento denominado CODIGO.

```
END    EJEMPLO ;empezar a ejecutar el procedimiento  
        ;EJEMPLO
```

Pseudo-op que indica el final del módulo fuente, y que la primera instrucción ejecutable es el comienzo del procedimiento EJEMPLO.

CAPITULO 6

CLASIFICACION DE LAS INSTRUCCIONES

El conjunto de instrucciones de un microprocesador define las operaciones básicas que se pueden hacer en una computadora. El microprocesador INTEL 8088-8086 tiene un conjunto de 116 instrucciones, de las cuales hay 92 básicas (algunas instrucciones básicas pueden escribirse de mas de una manera).

Para aprender el funcionamiento de cada una de ellas, vamos a clasificarlas en los siguientes grupos:

1. de transferencia de datos
2. aritméticas
3. de manipulación de bits
4. de transferencia de control
5. de manejo de strings
6. de stack
7. de control del procesador
8. de interrupción

Cada uno de estos grupos tienen, a su vez, uno o mas subgrupos de instrucciones, según determinadas especificaciones.

6.1. INSTRUCCIONES DE TRANSFERENCIA DE DATOS

Este grupo de instrucciones mueven datos entre registros y posiciones de memoria o puertas de E/S. La información que se transfiere es de 8 o de 16 bits.

Se dividen en 4 categorías:

1. de propósitos generales
2. de transferencia de direcciones
3. de transferencia de banderas
4. de comunicación con periféricos

De propósitos generales (3)

MOV - Mover.

XCHG - Intercambiar dos operandos.

XLAT - Traducir.

De transferencia de direcciones (3)

LEA - Cargar dirección efectiva.

LDS - Cargar puntero usando DS.

LES - Cargar puntero usando ES.

De transferencia de banderas (2)

LAHF - Cargar AH con las banderas.

SAHF - Almacenar AH en banderas.

De comunicación con periféricos (2)

IN - Entrada de byte o palabra.

OUT - Salida de byte o palabra.

6.2. INSTRUCCIONES ARITMETICAS

Este grupo de instrucciones permite realizar operaciones aritméticas con los distintos tipos de datos numéricos soportados por el microprocesador 8088-8086.

Recordar que el 8088-8086 maneja números binarios de 8 o 16 bits, con o sin signo, y números decimales de 8 bits, empaquetados (2 dígitos BCD por byte) o desempaquetados (1 dígito BCD por byte).

Las instrucciones aritméticas afectan los códigos de condición o banderas de estado; en particular, modifican las banderas de acarreo (CF), acarreo auxiliar (AF), signo (SF), cero (ZF) y paridad (PF).

Las instrucciones aritméticas se pueden dividir en 5 subgrupos:

1. de suma
2. de resta
3. de multiplicación
4. de división
5. de extensión del signo o conversión

De suma (5)

ADD - Sumar (sin acarreo).
ADC - Sumar con acarreo.
INC - Incrementar operando en 1.
AAA - Ajuste ASCII en suma.
DAA - Ajuste decimal en suma.

De resta (7)

SUB - Restar (sin acarreo).
SBB - Restar con acarreo.
DEC - Decrementar operando en 1.
AAS - Ajuste ASCII en resta.
DAS - Ajuste decimal en resta.
NEG - Negar (complementar a 2).
CMP - Comparar operandos.

De multiplicación (3)

MUL - Multiplicar, sin signo.
IMUL - Multiplicar, con signo.
AAM - Ajuste ASCII en multiplicación.

De división (3)

DIV - División, sin signo.
IDIV - División, con signo.
AAD - Ajuste ASCII en división.

De extensión del signo o conversión (2)

CBW - Convertir byte en palabra.
CWD - Convertir palabra en doble palabra.

6.3. INSTRUCCIONES DE MANIPULACION DE BITS

Así como las instrucciones aritméticas siempre asumen que sus operandos representan información numérica, las instrucciones de manipulación de bits tratan a los operandos como simples cadenas de bits, lo que permite operar tanto con bits individuales como con mascarar o patrones de bits dentro de registros o de posiciones de memoria.

Este grupo de instrucciones se divide en los siguientes tres subgrupos:

1. Lógicas
2. De corrimiento de bits
3. De rotación de bits

Las instrucciones lógicas trabajan con mascarar de bits según las reglas de la lógica formal, y se utilizan, básicamente, para la existencia de determinados bits dentro de la mascara.

La siguiente tabla muestra la representación hexadecimal de las distintas mascarar para cada bit individual:

BIT	HEXA
0	0001
1	0002
2	0004
3	0008
4	0010
5	0020
6	0040

7	0080
8	0100
9	0200
10	0400
11	0800
12	1000
13	2000
14	4000
15	8000

Las instrucciones de corrimiento de bits efectúan el desplazamiento de bits, en cantidades de 8 bits (byte) o de 16 bits (palabra), a la izquierda o a la derecha, tantas posiciones como se indiquen. Las posiciones desplazadas se completan con bits en cero.

Las instrucciones de rotación de bits son similares a las de corrimiento de bits, pero, a diferencia de estas, preservan los bits desplazados ingresándolos al operando por el lado opuesto al del corrimiento.

Lógicas (5)

AND - Y lógico.

OR - O lógico inclusivo.

XOR - O lógico exclusivo.

NOT - No lógico.

TEST - Y lógico sin modificación de operando.

De corrimiento de bits (4)

SAL - Corrimiento aritmético a la izquierda.

SHL - Corrimiento lógico a la izquierda (equivalente a SAL).

SAR - Corrimiento aritmético a la derecha.

SHR - Corrimiento lógico a la derecha.

De rotación de bits (4)

ROL - Rotación a la izquierda.

RCL - Rotación a la izquierda a través del acarreo.

ROR - Rotación a la derecha.

RCR - Rotación a la derecha a través del acarreo.

6.4. INSTRUCCIONES DE TRANSFERENCIA DE CONTROL

Las instrucciones de transferencia de control permiten alterar la secuencia normal de ejecución de un programa. Al igual que en los demás lenguajes, las instrucciones de un programa assembler se escriben secuencialmente, es decir una a continuación de la otra. Asimismo, se ejecutan en forma secuencial: el registro de instrucción siempre apunta a la próxima instrucción, siendo esta la que está a continuación en memoria. Sin embargo, muchas veces, por una determinada condición, es necesario alterar esa secuencia normal de ejecución de instrucciones, para continuar la ejecución del programa en otra parte del mismo. Para esto es que se utilizan las instrucciones de transferencia de control o de bifurcación.

Hay tres tipos de instrucciones de transferencia de control:

1. De transferencia de control incondicional
2. De transferencia de control condicional
3. De control de ciclos

En el primer caso, la transferencia de control se hace siempre, sin testear ninguna condición.

Las instrucciones de transferencia de control condicional permiten bifurcar a una parte del programa, siempre que se verifique una condición; si la condición no se satisface, el programa continua la secuencia normal de ejecución.

La condición puede ser el resultado de una operación aritmética o lógica, pero siempre se testea en base a la presencia o ausencia de ciertas condiciones en las banderas de estado.

Las instrucciones de control de ciclos son un grupo particular de las instrucciones de transferencia de control condicionales, dado que su función es la de testear la condición de finalización de un ciclo.

Un ciclo, en un programa assembler, es una secuencia de instrucciones que finaliza con una instrucción de control de ciclos; el punto de entrada al ciclo es una instrucción que incluye un rotulo, el cual es referenciado por la instrucción de control indicada al final.

Todas las instrucciones de control de ciclos testean condiciones referidas al registro CX: cada vez que se ejecuta una pasada de un ciclo, se altera el contenido del registro CX; entonces, cada vez que se va a iniciar un ciclo, es necesario inicializar el registro CX de la manera mas conveniente.

De transferencia de control incondicional (3)

JMP - Bifurcación incondicional.

CALL - Llamada a un procedimiento.

RET - Retorno de un procedimiento.

De transferencia de control condicional (31)

JA - Bifurca si superior.

JAЕ - Bifurca si superior o igual.

JB - Bifurca si inferior.

JBE - Bifurca si inferior o igual.

JC - Bifurca si acarreo (equivalente a JB).

JCХZ - Bifurca si CX contiene cero.

JE - Bifurca si igual.

JG - Bifurca si mayor.

JGE - Bifurca si mayor o igual.

JL - Bifurca si menor.

JLE - Bifurca si menor o igual.

JNA - Bifurca si no superior (equivalente a JBE).

JNAE - Bifurca si no superior ni igual (equivalente a JB).

JNB - Bifurca si no inferior (equivalente a JAE).

JNBE - Bifurca si no inferior ni igual (equivalente a JA).

JNC - Bifurca si no acarreo (equivalente a JAE).

JNE - Bifurca si no igual.

JNG - Bifurca si no mayor (equivalente a JLE).

JNGE - Bifurca si no es mayor ni igual (equivalente a JL).

JNL - Bifurca si no menor (equivalente a JGE).

JNLE - Bifurca si no menor ni igual (equivalente a JG).

JNO - Bifurca si no overflow.

JNP - Bifurca si no paridad.

JNS - Bifurca si no signo/si positivo.

JNZ - Bifurca si no cero (equivalente a JNE).

JO - Bifurca si overflow.

JP - Bifurca si paridad.

JPE - Bifurca si paridad par (equivalente a JP).

JPO - Bifurca si paridad impar (equivalente a JNP).

JS - Bifurca si signo.

JZ - Bifurca si cero (equivalente a JE).

Las instrucciones **JG, JGE, JL, JLE, JNG, JNGE, JNL, JNLE, JNO, JNS, JO y JS** son usadas para aritmética con signo, en notación complemento a 2; las restantes, son usadas para aritmética sin signo.

De control de ciclos (5)

LOOP - Ciclo hasta que se acabe el contador.

LOOPE - Ciclo mientras igual.

LOOPZ - Ciclo mientras cero (equivalente a LOOPE).

LOOPNE - Ciclo mientras no igual.

LOOPNZ - Ciclo mientras no cero (equivalente a LOOPNE).

6.5. INSTRUCCIONES DE MANEJO DE STRINGS

El microprocesador INTEL 8088-8086 provee una serie de instrucciones específicas para facilitar el manejo de strings o cadenas de caracteres. Las instrucciones para manejo de strings, se dividen en los siguientes grupos:

1. Prefijos de repetición
2. De movimiento

3. De comparación
4. De búsqueda
5. De carga y almacenamiento en memoria

Las instrucciones de manejo de strings asumen, por default, que el string fuente se ubica en el segmento de datos con el desplazamiento indicado por el registro SI (DS:SI), y que el string destino se ubica en el segmento extra con el desplazamiento indicado por el registro DI (ES:DI).

Los registros SI y DI se actualizan automáticamente después de cada operación, en 1 o en 2 según sean los operandos de tipo byte o palabra respectivamente.

Ese incremento puede ser positivo o negativo, según el estado de la bandera de dirección (DF): si DF es 0, el incremento es positivo; y si DF es 1, el incremento es negativo.

Prefijos de repetición (5)

Dado que la mayoría de las instrucciones operan sobre un elemento individual de un string, existen los denominados prefijos de repetición, que permiten operar sobre todos los elementos de un string.

Estos prefijos de repetición no son instrucciones, sino que complementan a las instrucciones de strings. Se colocan delante de los códigos de operación de las instrucciones de string, y permiten la repetición de una misma operación con strings.

Todos los prefijos de repetición actúan en función del contenido del registro CX, en un modo similar a las **instrucciones de control de ciclos**.

REP - Repetir operación de string.

REPE - Repetir operación de string, mientras igual.

REPZ - Repetir operación de string, mientras cero
(equivalente a REPE).

REPNE - Repetir operación de string, mientras no igual.

REPNZ - Repetir operación de string, mientras no cero.
(equivalente a REPNE).

De movimiento (3)

Permiten copiar un elemento de un string en otro string. Mediante los prefijos de repetición es posible mover mas de un elemento. La cantidad de elementos a mover se debe especificar en el registro CX.

MOVS - Mover string de bytes o de palabras.

MOVSB - Mover string de bytes (equivalente a MOVS).

MOVSW - Mover string de palabras (equivalente a MOVS).

De comparación (3)

Sirven para comparar un elemento de un string con un elemento de otro string. Mediante los prefijos de repetición es posible comparar mas de un elemento. La cantidad de elementos a comparar se debe especificar en el registro CX.

CMPS - Comparar string de bytes o de palabras.

CMPSB - Comparar string de bytes (equivalente a CMPS).

CMPSW - Comparar string de palabras (equivalente a CMPS).

De búsqueda (3)

Permiten explorar un string para buscar un elemento que contenga un determinado valor.

SCAS - Buscar en string de bytes o de palabras.

SCASB - Buscar en string de bytes (equivalente a SCAS).

SCASW - Buscar en string de palabras (equivalente a SCAS).

De carga y almacenamiento en memoria (6). Una vez hallado un elemento de un string, estas instrucciones permiten mover dicho elemento (byte o palabra) al acumulador (AL o AX) y viceversa. No se aplican los prefijos de repetición.

LODS - Cargar en AL o AX string de bytes o de palabras.

LODSB - Cargar en AL string de bytes (equivalente a LODS).

LODSW - Cargar en AX string de palabras (equivalente a LODS).

STOS - Almacenar AL o AX en string de bytes o palabras.

STOSB - Almacenar AL en string de bytes (equivalente a STOS).

STOSW - Almacenar AX en string de palabras (equivalente a STOS)

6.6. INSTRUCCIONES DE STACK

Este grupo de instrucciones permite el manejo del stack para almacenar y recuperar datos y direcciones de memoria. Como fue mencionado anteriormente, el stack es una parte de la memoria, residente en el segmento de stack, que se estructura bajo un funcionamiento de pila (First In First Out), es decir que se salva información en un sentido y se la recupera en el sentido contrario.

El stack es manejado mediante tres registros: SS (Stack Segment), SP (Stack Pointer) y BP (Base Pointer).

El **SS** contiene la dirección base del segmento de stack, y se usa como registro de segmento para armar las direcciones de los elementos dentro del stack.

El **SP** contiene el desplazamiento de la primera posición libre dentro del stack; de este modo, un elemento que se almacene en el stack, se guardado en la dirección SS:SP.

El **BP** contiene el desplazamiento de la dirección de memoria donde comienza el stack.

Cada elemento del stack es una palabra (16 bits).

Las instrucciones de stack se pueden clasificar en dos grupos:

1. De propósitos generales
2. De transferencia de banderas

De propósitos generales (2)

Permiten cargar/recuperar datos y direcciones hacia/desde el stack. No afectan ninguna de las banderas de estado.

PUSH - Colocar una palabra al tope del stack.

POP - Extraer una palabra del tope del stack.

De transferencia de banderas (2)

Permiten cargar/recuperar el registro de banderas hacia/desde el stack.

PUSHF - Colocar registro de banderas al tope del stack.

POPF - Extraer registro de banderas del tope del stack.

6.7. INSTRUCCIONES DE CONTROL DEL PROCESADOR

Este grupo de instrucciones permiten regular el funcionamiento del microprocesador 8088-8086 desde un programa. En general, las instrucciones de control del procesador deben ser usadas muy cuidadosamente, dado que alteran las condiciones de procesamiento.

Hay tres tipos de instrucciones de control del procesador:

1. De operación de banderas
2. De sincronización externa
3. No operación

De operación de banderas (7)

Permiten alterar el contenido de las distintas banderas de estado, de modo de poder controlar el contexto de procesamiento de un programa.

CLC - Borrar bandera de acarreo.

CLD - Borrar bandera de dirección.

CLI - Borrar bandera de interrupción.

CMC - Complementar bandera de acarreo.

STC - Fijar bandera de acarreo.

STD - Fijar bandera de dirección.

STI - Fijar bandera de interrupción.

De sincronización externa (4)

Estas instrucciones se usan para la sincronización del microprocesador INTEL 8088-8086 con eventos externos.

HLT - Parar el procesador.

WAIT - Esperar la finalización de un coprocesador.

ESC - Escape (transmitir información a un coprocesador).

LOCK - Bloquear el bus.

De no operación (1)

Esta instrucción no ejecuta nada.

NOP - No operación.

6.8. INSTRUCCIONES DE INTERRUPCION

Las interrupciones son mecanismos a través de los cuales el microprocesador se comunica con los distintos dispositivos. Las instrucciones de interrupción permiten a un usuario generar sus propias interrupciones para poder manipular por software algunos aspectos del microprocesador.

Existen 3 instrucciones de interrupción:

INT - Interrupción.

INTO - Interrupción si overflow.

IRET - Retorno de interrupción.

CAPITULO 7

CLASIFICACION DE LAS PSEUDO-OPS

Como fue visto anteriormente, las pseudo-ops son, a diferencia de las instrucciones, directivas al compilador, y, por lo tanto, no generan código objeto.

El macro-assembler dispone de una gran variedad de pseudo-ops distintas, las que dan gran ductilidad en la escritura de un programa.

Las pseudo-ops pueden ser clasificadas en distintos grupos, según sus funciones:

1. De datos
2. Condicionales
3. De listado
4. De macros

Cada uno de estos grupo tiene, uno ó más subgrupos de pseudo-ops, según determinadas especificaciones.

En el capítulo 8 se estudiarán en detalle cada una de las pseudo-ops.

7.1. PSEUDO-OPS DE DATOS

Estas pseudo-ops son las más usadas, ya que permiten definir símbolos y datos, establecer referencias externas y otra serie de facilidades muy utilizadas en cualquier programa assembler.

Se dividen en los siguientes subgrupos:

1. Definición de símbolos
2. Definición de datos
3. Referencias externas
4. Definición de segmentos y procedimientos
5. Definición de bloques
6. Control del ensamblador

Definición de símbolos

Sirven para asignar nombres simbólicos a expresiones.

Después de haber asignado la expresión al símbolo, este puede ser usado en cualquier parte del programa en vez de la expresión.

EQU - Asigna un símbolo a una expresión fija.

= - Asigna un símbolo a una expresión variable.

Definición de datos

Permiten reservar espacio de memoria para variables del programa. Opcionalmente, se puede dar un valor inicial a cada variable.

DB - Definir byte.

DW - Definir palabra.

DD - Definir doble palabra.

DQ - Definir cuádruple palabra.

DT - Definir diez bytes.

Referencias externas

Se utilizan para compartir información entre módulos que, residen en archivos distintos y que deben ser linkeados juntos.

PUBLIC - Definir símbolo público.

EXTRN - Definir símbolo externo.

INCLUDE- Incluir archivo fuente.

Definición de segmentos y procedimientos

SEGMENT- Declaración de comienzo de segmento.

ENDS- Declaración de fin de segmento.

ASSUME- Asignar registros de segmento por default.

PROC- Declaración de comienzo de procedimiento.

ENDP- Declaración de fin de procedimiento.

Definición de bloques

Permiten definir diferentes estructuras de datos y de código.

GROUP - Agrupa segmentos.

NAME - Nombre de un módulo.

LABEL - Asigna un atributo a un nombre.

RECORD- Define un registro.

STRUC- Define una estructura.

Control del ensamblador

ORG- Declaración del origen del código o de los datos.

END- Declaración de fin del módulo fuente.

EVEN- Asignar contador de posiciones en un número par.

.RADIX - Asignar base de numeración default.

7.2. PSEUDO-OPS CONDICIONALES

Estas pseudo-ops hacen que el compilador incluya o ignore ciertas porciones del programa fuente, según que una cierta condición sea verdadera o falsa en tiempo de compilación.

Esto permite crear programas de tipo multipropósito, es decir que hagan ciertas operaciones bajo ciertas condiciones. También brinda facilidades para diseñar corridas de testeó durante la prueba de un programa.

Para lograr la compilación condicional de un bloque de programa, dicho bloque debe comenzar con una pseudo-op de tipo IF y finalizar con una pseudo-op de tipo ENDIF.

Las pseudo-ops condicionales son:

IFxx- Testeo de una determinada condición (xx).

ELSE- Opción por condición falsa.

ENDIF- Declaración de fin de un bloque condicional.

7.3. PSEUDO-OPS DE LISTADO

Estas pseudo-operaciones son usadas para indicarle al compilador que información (y en que formato) se debe obtener en el listado de salida, de modo de obtener una presentación lo más clara y prolija posibles.

Se pueden dividir en los siguientes grupos:

1. Formato del listado
2. Control del listado
3. Comentarios
4. Mensajes
5. Control de listado de bloques asociados a una condición falsa

Formato del listado

PAGE- Formato de la página del listado.

TITLE- Definición del título del listado.

SUBTTL- Definición del subtítulo del listado.

Control del listado

.LIST- Habilita el listado del programa fuente.

.XLIST- Deshabilita el listado del programa fuente.

.CREF- Habilita el listado de referencias cruzadas.

.XCREF- deshabilita el listado de referencias cruzadas.

Comentarios

COMMENT- Comentario en el programa fuente.

Mensajes

%OUT - Emitir mensaje por pantalla durante la compilación.

Control de listado de bloques asociados a una condición falsa

.LFCOND- Listar bloques asociados a una condición falsa.

.SFCOND- Suprimir el listado de bloques asociados a una condición falsa.

.TFCOND- Invertir el modo de listado de los bloques asociados a una condición falsa.

7.4. PSEUDO-OPS DE MACROS

Una macro es un conjunto de sentencias fuente (tanto de instrucciones como de pseudo-ops), que puede aparecer varias veces en un programa. En el capítulo 11 veremos más en detalle este tema.

Las pseudo-ops para macros se pueden dividir en los siguientes subgrupos:

1. De propósitos generales
2. De repetición
3. Condicionales
4. De listado

De propósitos generales

Permiten efectuar una serie de operaciones generales con las macros.

MACRO- Declaración de comienzo de una macro.

ENDM- Declaración de fin de un bloque de repetición o de fin de una macro.

LOCAL - Definición de rótulos dentro de una macro.

EXITM - Fuerza el fin de la expansión de una macro.

PURGE - Borrar las macros de la memoria.

De repetición

Indican al compilador que repita una secuencia de sentencias dentro de una macro.

REPT - Repetir bloque de sentencias un cierto número de veces.

IRP - Repetir bloque de sentencias con un valor numérico cada vez.

IRPC - Repetir bloque de sentencias con un valor alfanumérico cada vez.

Condicionales

Tienen el mismo sentido que las pseudo-ops condicionales ya vistas (ver 7.2., en este mismo capítulo).

De listado

Permiten definir una serie de precisiones acerca del listado de las macros.

.LALL - Listar macros y expansiones.

.SALL - Suprimir el listado de las macros y expansiones.

.XALL - Listar sólo las macros que generan código objeto.

CAPITULO 8

SET DE PSEUDO-OPERACIONES

&

Formato: **texto&texto**

Descripción: Es un operador que se utiliza para concatenar textos o símbolos, permitiendo crear, por ejemplo, rótulos u operandos "hechos a medida".

Comentarios:

- Si un parámetro de la macro es un string, debe ir precedido por el operador & dentro de las comillas para que se sustituya en la expansión de la macro.
- Para crear un símbolo a partir de un texto y de un parámetro de la macro, es necesario interponer un & entre ellos.

Ejemplo 1: ERRGEN MACRO X
 ERROR&X: PUSH BX
 ABX MOV BX,"&X"
 AB&X JMP ERROR
 ENDM

La expansión de la macro para la invocación de la misma a través de ERRGEN A será:

```
ERRORA PUSH BX
ABX  MOV  BX,"A"
ABA  JMP  ERROR
```

Ejemplo 2: MENSAJE MACRO COND,SUBRU

```
    J&COND FIN
    CALL &SUBRU
FIN:
ENDM
```

La expansión de la macro para la invocación de la misma a través de MENSAJE GE,SUB1 será:

```
    JGE FIN
    CALL SUB1
FIN:
```

Ejemplo 3: HACER MACRO COND,TEXTO

```
    J&COND FIN
    TEXTO
FIN:
ENDM
```

La expansión de la macro para la invocación de la misma a través de HACER GE,<MOV AX,BYTE PTR [DI]> ser :

```
    JGE FIN
    MOV AX,BYTE PTR [DI]
FIN:
```

Ejemplo 4: DEF_TABLA MACRO SUF,LONGI

```
    TAB&SUF DB    LONGI DUP(?)
    ENDM
```

La expansión de la macro para la invocación de la misma a través de DEF_TABLA X,5 será:

```
    TABX  DB    5 DUP(?)
```

;;

Formato: **;;comentario**

Descripción: Es un operador que hace que se omitan el comentario (texto ubicado a continuación de ;;) en la expansión de la macro.

Comentarios:

- El comentario precedido por ;; nunca saldrá impreso, aun usando la directiva .LALL.
- Al usar el operador ;; se obtienen programas que ocupan menos memoria y, por lo tanto, la compilación es más rápida.
- Los comentarios precedidos por el operador ; aparecerán en la expansión.
- En general, es conveniente usar el operador ; para aquellos comentarios estrictamente necesarios y dejar ;; para el resto.

Ejemplo 1: SUMAR MACRO X,Y,Z

```
MOV AX,X ;;AX=X
MOV BX,Y ;;BX=Y
ADD AX,BX ;;AX=AX+BX
MOV Z,AX ;;Z=AX
ENDM
```

La expansión de la macro para la invocación de la misma a través de SUMAR UNO,DOS,TRES ser :

```
MOV AX,UNO
MOV BX,2
ADD AX,BX
MOV TRES,AX
```

Formato: -caracter

Descripción: Es un operador de macro que se utiliza para indicarle al compilador que el carácter debe interpretarse en sentido literal, y no como símbolo.

Comentarios:

- Las expresiones -X y <X> son equivalentes.

Ejemplo 1: MAC1 MACRO X

```
MOV AX,X
MAC2 -X
ENDM
MAC2 MACRO Y
PUSH Y
ENDM
```

La expansión que resulta de la invocación a través de MAC1 PEPE será:

```
MOV AX,PEPE
PUSH X
```

%

Formato: %expresión

Descripción: Es un operador de macro que sirve para convertir una expresión (que, generalmente, es un símbolo) a un número en la base de numeración activa. Permite usar números como argumentos al invocar a macros, en vez de argumentos interpretados

Comentarios:

- El operador % sólo puede ser usado como argumento de una macro.

Ejemplo 1: MACRO1 MACRO X

```
LB = 0
REPT X
LB=LB+1
MACRO2 %LB
ENDM
ENDM
MACRO2 MACRO Y
ERROR&Y DB "ERROR &Y",0
ENDM
```

A través de la invocación MACRO1 3, se generará lo siguiente:

```

ERROR1 DB "ERROR 1",0
ERROR2 DB "ERROR 2",0
ERROR3 DB "ERROR 3",0

```

=

Formato: nombre = expresión

Descripción: Asigna a un nombre simbólico el valor de una expresión. La asignación no es fija, es decir que, eventualmente, "nombre" puede cambiar de valor.

"expresión" puede ser:

- una constante numérica;
- una referencia de dirección (usando cualquiera de los modos de direccionamiento);
- cualquier combinación de símbolos y operaciones que pueda evaluarse como un valor numérico;
- otro nombre simbólico.

Comentarios:

- La directiva = sólo se puede usar para asignar valores numéricos.
- La directiva = es especialmente útil en operaciones iterativas y en macros.

Ejemplos: CONST = 56 ;definición de una constante

VALOR = CONST/2 ;expresión numérica

CONST = CONST+1 ;redefinición de un símbolo

DIR_1 = BYTE PTR ES:[SI]

DIR_2 = WORD PTR ES:[SI]

ASSUME Asumir

Formato: ASSUME reg-seg:nombre-seg[,...]

o

ASSUME NOTHING

Descripción: Le indica al compilador que registro de segmento se va a utilizar para direccionar cada segmento dentro del modulo. Para acceder al segmento de código se debe usar siempre el registro CS; para acceder a los segmentos de datos, extra y de stack, se utilizan por default los registros DS, ES y SS respectivamente, según el modo de direccionamiento de los operandos dereandos de cada instrucción.

La directiva ASSUME va, generalmente, a continuación de la declaración SEGMENT.

Si no se coloca ASSUME, en cada instrucción se debe explicitar el registro de segmento de cada operando.

Operandos:

"reg-seg" puede ser DS, CS, SS o ES.

"nombre-seg" puede ser:

- el nombre asignado por la directiva SEGMENT
- el nombre asignado por la directiva GROUP
- SEG variable
- SEG rótulo
- la palabra NOTHING

Con NOTHING se anula la última declaración ASSUME, debiéndose especificar un registro de segmento como prefijo de los operandos de memoria en cada instrucción.

Comentarios:

- El mínimo ASSUME requerido para un programa assembler es:

```
ASSUME CS:seg-código
```

dado que, de omitirse, el compilador dará error cada vez que haya una instrucción de bifurcación a un rótulo dentro del actual procedimiento o a otro procedimiento. Esto ocurre pues falta la referencia de segmento en el argumento de bifurcación.

- Cuando se referencia a un operando usando un registro de segmento diferente al definido en ASSUME, el compilador genera automáticamente un código de un byte de tamaño para la instrucción, donde se indica el segmento que sustituye al default.

Por ejemplo, si el campo VECTOR fue definido en el segmento extra y el registro definido en ASSUME para el segmento de datos es DS, cada instrucción que haga referencia a VECTOR debe estar precedida por el prefijo de segmento ES; en este caso, el compilador generará un código de un byte para esas instrucciones.

- La directiva ASSUME no inicializa los registros de segmentos. Para inicializar el registro DS, se usa la siguiente secuencia de instrucciones:

```
MOV AX,DATOS      ;DATOS = nombre del segmento de datos
MOV DS,AX
```

Análogamente, el segmento extra se inicializa con:

```
MOV AX,EXTRA      ;EXTRA = nombre del segmento extra
MOV ES,AX
```

Los registros SS y CS son inicializados por el cargador de programas del DOS.

- Los segmentos de datos deben colocarse delante del segmento de código; de lo contrario, el compilador puede señalar errores en los datos.

Ejemplo 1: Modulo con segmento de datos y segmento de código:

```
DATOS SEGMENT
...
DATOS ENDS
CODIGO SEGMENT
    ASSUME CS:CODIGO,DS:DATOS,ES:NOTHING
...
CODIGO ENDS
```

Ejemplo 2: Modulo con datos y código en un sólo segmento:

```
UNICO SEGMENT
    ASSUME CS:UNICO,DS:UNICO
...
UNICO ENDS
```

Ejemplo 3: Modulo con segmento de datos, segmento de código y segmento de stack:

```
DATOS SEGMENT
...
DATOS ENDS
PILA SEGMENT
...
PILA ENDS
CODIGO SEGMENT
    ASSUME CS:CODIGO,DS:DATOS,SS:STACK
...
CODIGO ENDS
```

COMMENT Comentario

Formato: COMMENTdelimitador texto delimitador

Descripción: Permite insertar comentarios en el programa sin necesidad de tener que especificar el operador ; en cada línea. El primer carácter no blanco que está a continuación de COMMENT actúa como delimitador inicial del comentario. El delimitador final es otro carácter igual al delimitador inicial.

Comentarios:

- Un COMMENT definido dentro del cuerpo de una macro no será listado, a menos que se haya especificado .LALL.

Ejemplo 1: COMMENT *Este es un comentario definido con COMMENT que usa un asterisco como delimitador de texto.

```
.....
.....
.....
```

El final del texto queda determinado por la nueva aparición del asterisco*

.CREF / .XCREF Controlar listado de referencias cruzadas

Formato: .CREF

.XCREF

Descripción: Estas pseudo-ops controlan la salida o no del listado de referencias cruzadas (cross reference).

.CREF habilita el listado de referencias cruzadas; es la opción por default.

.XCREF suprime el listado hasta que se encuentre la siguiente directiva .CREF (si la hubiera).

Las referencias cruzadas se usan como ayuda en la depuración de un programa. Consta de un listado de los símbolos, ordenados alfabéticamente, con indicación de las líneas donde es referenciado cada uno.
Ejemplo:

```
...
.XCREF ;deshabilita listado de referencias cruzadas
...
.CREF ;restaura listado de referencias cruzadas.
```

DB Definir byte

Formato: [nombre-variable] DB expresión[,...]

Descripción: Permite reservar espacio en memoria para una variable de tipo byte (8 bits), inicializando o no dicho byte y los posteriores.

"nombre-variable" es opcional y es el nombre asignado al primer byte.

Operandos:

"expresión" es el valor inicial, y puede ser:

- Una constante (positiva o negativa), o una expresión de constantes.
- Un signo "?", que indica valor indefinido.
- Un string (cadena de caracteres encerrados entre comillas).
- n DUP(expresión), que indica la repetición n veces de "expresión". "expresión" podría incluir otra declaración DUP.

Comentarios:

- La cantidad de expresiones está limitado por la longitud de la línea.

Ejemplos:

```
VALORES DB 30,-15,20
MAXSS DB 255 ;número máximo sin signo
MINSS DB 0 ;número mínimo sin signo
MAXCS DB 127 ;número máximo con signo
MINCS DB -128 ;número mínimo con signo
HEXA DB 0DH ;constante hexadecimal
BINA DB 11001101B ;constante binaria
EXPRES DB 12 * 3 ;expresión de constantes
TABLA DB 0,0,0,0
TABLA_1 DB 4 DUP(0)
TABLA_2 DB 4 DUP(?)
TABLA_3 DB 1,2,?,?
TABLA_4 DB 4 DUP(3 DUP(2),4)
MENSA DB "ERROR"
MENSA_1 DB "ERROR",13,"$"
MENSA_2 DB 'ERROR'
TAB_MEN DB 4 DUP(?),"ERROR",32,"$"
```

DD Definir doble palabra

Formato: [nombre-variable] DD expresión[,...]

Descripción: Permite reservar espacio en memoria para una variable de tipo doble palabra (32 bits), inicializando o no dicha doble palabra y las posteriores.

"nombre-variable" es opcional y es el nombre asignado a la primera doble palabra.

Operandos:

"expresión" es el valor inicial, y puede ser:

- Una constante (positiva o negativa), o una expresión de constantes.
- Un signo "?", que indica valor indefinido.
- Una dirección de memoria (segmento y desplazamiento).
- n DUP(expresión), que indica la repetición n veces de "expresión". "expresión" podría incluir otra declaración DUP.

Comentarios:

- La cantidad de expresiones está limitado por la longitud de la línea.
- El almacenamiento en memoria es: en la dirección más baja la palabra menos significativa, y en la dirección más alta la palabra más significativa. A su vez, cada palabra se almacena con el byte menos significativo en la dirección más baja, y con el byte más significativo en la dirección más alta. Por ejemplo, 12345678H se almacena en memoria como 78 56 34 12.

Ejemplos: VALORES DD 300,-150,2000

```
MAXSS DD 4294967295 ;Nro. máximo sin signo
MINSS DD 0 ;Nro. mínimo sin signo
MAXCS DD 2147483647 ;Nro. máximo con signo
MINCS DD -2147483648 ;Nro. mínimo con signo
DD 120 * 3 ;expresión de cntes
TABLA DD 4,?,3 DUP(0)
DIREC DD TABLA ;dirección de TABLA
```

DQ Definir cuádruple palabra

Formato: [nombre-variable] DQ expresión[,...]

Descripción: Permite reservar espacio en memoria para una variable de tipo cuádruple palabra (64 bits), inicializando o no dicha cuádruple palabra y las posteriores.

"nombre-variable" es opcional y es el nombre asignado a la primera cuádruple palabra.

Esta directiva se usa especialmente en aplicaciones de punto flotante.

Operandos: "expresión" es el valor inicial, y puede ser:

- Una constante (positiva o negativa), o una expresión de constantes.
- Un signo "?", que indica valor indefinido.
- n DUP(expresión), que indica la repetición n veces de "expresión". "expresión" podría incluir otra declaración DUP.

Comentarios:

- La cantidad de expresiones está limitado por la longitud de la línea.
- El almacenamiento en memoria es: en la dirección más baja la doble palabra menos significativa, y en la dirección más alta la doble palabra más significativa. A su vez, cada doble palabra se almacena con la palabra menos significativa en la dirección más baja, y con la palabra más significativa en la dirección más alta. Por último, cada palabra se almacena con el byte menos significativo en la dirección más baja, y con el byte más significativo en la dirección más alta. El orden resultante es el inverso a nivel byte. Por ejemplo, 0123456789ABCDEFH se almacena en memoria como EF CD AB 89 67 45 23 01.

Ejemplos: VALORES DQ 300,-150,2000

```
MAXSS DQ 18446744073709551615
MINSS DQ 0
MAXCS DQ 9223372036854775807
MINCS DQ -9223372036854775808
DQ 120 * 3
TABLA DQ 4,?,3 DUP(0)
```

DT Definir diez bytes

Formato: [nombre-variable] DT expresión[,...]

Descripción: Permite reservar 10 bytes de memoria para almacenar dígitos decimales empaquetados.

El primer byte se reserva para el signo, y los restantes 9 bytes para almacenar 18 dígitos decimales.

El signo positivo se almacena como 00H, y el signo negativo se almacena como 80H.

"nombre-variable" es opcional y es el nombre asignado al primer byte.

Operandos: "expresión" es el valor inicial, y puede ser:

- Una constante (positiva o negativa), o una expresión de constantes.
- Un signo "?", que indica valor indefinido.
- n DUP(expresión), que indica la repetición n veces de "expresión". "expresión" podría incluir otra declaración DUP.

Comentarios:

- La cantidad de expresiones está limitado por la longitud de la línea.

Ejemplos:

```
NMAS DT 0123456789
NMENOS DT -0123456789
```

Estas declaraciones son equivalentes a:

```
NMAS DB 00H,4 DUP(?),01H,23H,45H,67H,89H
NMENOS DB 80H,4 DUP(?),01H,23H,45H,67H,89H
```

DW Definir palabra

Formato: [nombre-variable] DW expresión[,...]

Descripción: Permite reservar espacio en memoria para una variable de tipo palabra (16 bits), inicializando o no dicha palabra y las posteriores.

"nombre-variable" es opcional y es el nombre asignado a la primera palabra.

Operandos:

"expresión" es el valor inicial, y puede ser:

- Una constante (positiva o negativa), o una expresión de constantes.
- Un signo "?", que indica valor indefinido.
- La componente de desplazamiento de una dirección de memoria.
- n DUP(expresión), que indica la repetición n veces de "expresión". "expresión" podría incluir otra declaración DUP.

Comentarios:

- La cantidad de expresiones está limitado por la longitud de la línea.
- El almacenamiento en memoria es: el byte menos significativo (bits 0 a 7) en la dirección más baja, y el byte más significativo (bits 8 a 15) en la dirección más alta. Por ejemplo, 0123H se almacena como 23 01.

Ejemplos: VALORES DW 300,-150,2000

```
MAXSS DW 65535
MINSS DW 0
MAXCS DW 32767
MINCS DW -32768
DW 120 * 3
TABLA DW 4,?,3 DUP(0)
DIREC DW TABLA ;DIREC=despl. de TABLA
```

END Final

Formato: END [expresión]

Descripción: Es de uso obligatorio en todo programa: indica el final del programa fuente y le señala al compilador el punto final del ensamblado. "expresión" corresponde a la dirección de comienzo de ejecución del programa fuente. Generalmente se especifica un rótulo.

Comentarios:

- Si el programa es de un sólo modulo fuente, entonces "expresión" es de uso optativo.
- Si el programa está compuesto por varios módulos fuente, que se linkearán juntos para obtener un sólo modulo ejecutable, sólo el modulo principal debe llevar "expresión"; los restantes módulos deben usar END sólo.

Ejemplos:

```
...
PRINCI PROC
```

```

...
...
PRINCI ENDP
SECUND PROC
...
SECUN ENDP
END PRINCI ;Fin del modulo principal

```

ENDM Final de macro

Formato: **ENDM**

Descripción:

Indica el final de una macro o de alguna de las estructuras repetitivas REPT, IRP e IRPC.

Ejemplo 1:

Macro para obtener la suma $Z=X+Y$

```

SUMAR MACRO X,Y,Z
    MOV AX,X
    MOV BX,Y
    ADD AX,BX
    MOV Z,AX
ENDM

```

Ejemplo 2:

Macro para reservar 5 palabras de memoria, inicializadas con los cuadrados de los 5 primeros números naturales:

```

CUAD MACRO
    IRP VALOR,<1,2,3,4,5>
        DW VALOR*VALOR
    ENDM
ENDM

```

La expansión de esta macro dará:

```

DW 1
DW 4
DW 9
DW 16
DW 25

```

ENDP Final de procedimiento

Formato: **nombre-procedimiento ENDP**

Descripción:

Indica el final del procedimiento "nombre-procedimiento".

Comentarios:

- "nombre-procedimiento" debe ser el mismo que se utilizó al definir el comienzo del procedimiento mediante la pseu PROC.

Ejemplos:

```

PROC1 PROC
...
...
RET
PROC1 ENDP

```

ENDS Final de segmento - Final de estructura

Formato: **nombre-segmento ENDS**

nombre-estructura ENDS

Descripción:

Indica el final del segmento "nombre-segmento", o bien de la estructura "nombre-estructura".

Comentarios:

- "nombre-segmento" debe ser el mismo que se utilizó al definir el comienzo del segmento mediante la pseudo-op SEGMENT.
- "nombre-estructura" debe ser el mismo que se utilizó al definir el comienzo de la estructura mediante la pseudo-op STRUC.

Ejemplos:

```
SEG1  SEGMENT
```

```
...
```

```
...
```

```
SEG1  ENDS
```

```
ESTRUC1 STRUC
```

```
...
```

```
...
```

```
ESTRUC1 ENDS
```

EQU Equivalencia

Formato: **nombre EQU expresión**

Descripción:

Asigna a un nombre simbólico el valor de una expresión. La asignación es fija, es decir que "nombre" no puede cambiar de valor.

"expresión" puede ser:

- una constante numérica;
- una referencia de dirección (usando cualquiera de los modos de direccionamiento);
- cualquier combinación de símbolos y operaciones que pueda evaluarse como un valor numérico;
- otro nombre simbólico.

Comentarios:

- EQU no puede ser usada dentro de una estructura (entre STRUC y ENDS).
- EQU es especialmente útil para poder acceder a una variable o a un rótulo con un atributo diferente, usándolo con el operador PTR para asignar un nuevo atributo.

Ejemplos:

```
LONGITUD  DW  0
```

```
BYTE_1 EQU  BYTE PTR LONGITUD
```

```
BYTE_2 EQU  BYTE PTR LONGITUD+1
```

```
ROTU  EQU  FAR PTR ROTULO
```

```
CONST EQU  56
```

```
VALOR EQU  CONST/2
```

```
CARAC EQU  BYTE PTR ES:[SI]
```

```
CONTADOR EQU  CX
```

```
MOVER EQU  MOV
```

EVEN Par

Formato: **EVEN**

Descripción:

Le fuerza un valor par al contador de posiciones, es decir que lo coloca en el primer byte de una palabra.

Con esta directiva, se asegura que el código o los datos que vienen a continuación comiencen en una dirección par de memoria.

Si el contador de posiciones es un número par, EVEN no hace nada; en cambio, si el contador de posiciones es impar, EVEN hace que el compilador agregue en esa posición una instrucción NOP (no operación), que tiene un byte de tamaño.

Comentarios:

- El 8088-8086 tarda más tiempo en transferir datos que residen en una dirección impar que en una dirección par.

En este sentido, el uso de EVEN produce una disminución del tiempo de transferencia.

Ejemplos:

Si el contador de posiciones es 0099H

EVEN

hace que se genere un NOP en la dirección 0099H y que el contador pase a tener el valor 009AH.

EXITM Salir de la macro

Formato: EXITM

Descripción:

Fuerza la terminación de la expansión de una macro.

EXITM se usa dentro de una macro o dentro de algún bloque repetitivo (REPT, IRP o IRPC), cuando, como consecuencia de una pseudo-op condicional anterior, se desea terminar la expansión de la macro.

Ejemplo:

Macro para reservar N bytes con valores 1 a N. El valor de N deber ser menor o igual a 5; si es mayor que este número, se reservarán sólo 5 bytes.

```
ALOCAR MACRO N
VALOR = 0
REPT N
VALOR = VALOR+1
IF VALOR EQ 6
EXITM
ENDIF
DB VALOR
ENDM
ENDM
```

Invocando la macro mediante ALOCAR 4, la expansión será:

```
DB 1
DB 2
DB 3
DB 4
```

En cambio, si se invoca la macro con ALOCAR 10, la expansión será :

```
DB 1
DB 2
DB 3
DB 4
DB 5
```

EXTRN Externo

Formato: EXTRN nombre:tipo[,...]

Descripción:

Identifica a los símbolos que fueron definidos (y declarados como públicos mediante la pseudo-op PUBLIC) en otro modulo.

Operandos:

"nombre" corresponde al símbolo definido y declarado como PUBLIC en otro modulo, y puede ser:

- un nombre de variable;
- un rótulo (inclusive si es un nombre de procedimiento).

"tipo" es un atributo, y puede ser:

- BYTE, WORD o DWORD, si "nombre" es una variable de tipo byte, palabra o doble palabra, respectivamente;
- NEAR o FAR, si "nombre" es el rótulo de identificación de un procedimiento que está en el mismo segmento de código (NEAR) o en otro segmento de código (FAR);
- ABS, si "nombre" es una constante de 16 bits definida con = o con EQU en el modulo de origen.

Comentarios:

- La declaración EXTRN X:ABS equivale a X EQU expr. o a X = expr., donde la expresión es dada por el linkeditor, que la toma desde el modulo dónde X está declarada como PUBLIC.
- La declaración EXTRN puede aparecer ya sea dentro del mismo segmento en que se utiliza la referencia a "nombre" o, bien, fuera de todos los segmentos. En este último caso, para acceder a las variables se debe especificar un prefijo de segmento en el acceso a los datos, o usar la declaración ASSUME DS:SEG variable.

Ejemplo 1:

Acceso al procedimiento PROC1, residente en otro modulo y en el mismo segmento:

```
EXTRN PROC1:NEAR
```

Ejemplo 2:

Acceso al procedimiento PROC1, residente en otro modulo y en otro segmento:

```
EXTRN PROC1:FAR
```

Ejemplo 3:

Acceso a la variable NOMBRE, de tipo palabra, perteneciente a otro modulo:

```
EXTRN NOMBRE:WORD
```

Ejemplo 4:

Acceso por bifurcación a la instrucción rotulada con ROTU, en otro modulo y en el mismo segmento:

```
EXTRN ROTU:NEAR
```

Ejemplo 5:

Acceso por bifurcación a la instrucción rotulada con ROTU, en otro modulo y en otro segmento:

```
EXTRN ROTU:FAR
```

Ejemplo 6:

Acceso a la variable DATO, de tipo byte, perteneciente a otro modulo:

```
EXTRN DATO:BYTE
```

GROUP Grupo

Formato: nombre GROUP nombre-segmento[,...]

Descripción:

Permite agrupar dos o más segmentos lógicos en un sólo segmento físico, bajo un nombre único.

Después de hacer la declaración de GROUP, todos los elementos de los segmentos agrupados pueden ser direccionados con un único valor del registro de segmento correspondiente.

El conjunto de los segmentos agrupados puede ocupar hasta un máximo de 64 Kbytes.

Operandos:

"nombre" es el nombre asignado al grupo de segmentos, y puede ser el mismo que alguno de los "nombre-segmento" definidos a continuación.

"nombre-segmento" puede ser:

- un nombre de segmento, definido con SEGMENT;
- un operador SEG variable;
- un operador SEG rótulo.

Comentarios:

- Para direccionar a todos los segmentos del grupo, basta con inicializar el correspondiente registro de segmento asociado al grupo.
 - El nombre de grupo puede ser incluido en una directiva ASSUME.
 - El nombre de grupo se puede utilizar como prefijo de segmento de un operando.
 - El operador OFFSET debe llevar el nombre del grupo como prefijo de segmento si la variable referenciada está dentro de un grupo.
 - Mediante la directiva GROUP se pueden combinar segmentos de distinto tipo.
- Este concepto es utilizado en los programas .COM, que ocupan un único segmento físico. Una alternativa para crear un programa de un solo segmento es de mezclar código y datos dentro del segmento; esto presenta la desventaja de tener que referenciar cada dato con un prefijo de segmento.

Ejemplo 1:

Agrupamiento de tres segmentos de código, dos de ellos en un modulo y el tercero en otro modulo:

Primer modulo:

```
GRUPO GROUP SEG1,SEG2
SEG1 SEGMENT
    ASSUME CS:GRUPO
    ...
    ...
SEG1 ENDS
SEG2 SEGMENT
    ASSUME CS:GRUPO
    ...
    ...
SEG2 ENDS
END
```

Segundo modulo:

```
GRUPO GROUP SEG3
SEG3 SEGMENT
    ASSUME CS:GRUPO
    ...
    ...
SEG3 ENDS
END
```

Ejemplo 2:

Agrupamiento de un segmento de código y un segmento de datos, de un mismo modulo:

```
GRUPO GROUP DATOS,CODIGO
DATOS SEGMENT
    ...
    ...
DATOS ENDS
CODIGO SEGMENT
    ASSUME CS:GRUPO,DS:GRUPO
    ...
    ...
CODIGO ENDS
END
```

Ejemplo 3:

Este ejemplo consiste en un programa que usa dos procedimientos externos. Ambos procedimientos, PROC1 y PROC2, están definidos con atributo NEAR y, además, están en segmentos de código que tienen distinto nombre que el segmento de código del programa que los invoca; PROC1 está en CODSEG_1 y sus datos están en DATSEG_1, mientras que PROC_2 está en CODSEG_2 y sus datos están en DATSEG_2. Dado que los procedimientos con atributo de distancia NEAR sólo pueden ser llamados desde dentro del mismo segmento, entonces se deberán agrupar los segmentos distintos bajo un mismo nombre, de modo que puedan ser invocados por el programa principal como si se trataran de llamadas a procedimientos dentro del mismo segmento.

En el ejemplo, se agrupan bajo el nombre CGRUPO, los segmentos de código de ambos procedimientos, CODSEG_1 y CODSEG_2 respectivamente, y el del programa principal, CSEG. Asimismo, se agrupan bajo el nombre DGRUPO los segmentos de datos de los dos procedimientos, DATSEG_1 y DATSEG_2 respectivamente (el programa principal no tiene segmento de datos).

Luego de la definición de los grupos, estos se tratan como si fueran segmentos individuales.

```
;Definición de segmentos externos
CODSEG_1    SEGMENT
            EXTRN  PROC1:NEAR
...
CODSEG_1    ENDS

CODSEG_2    SEGMENT
            EXTRN  PROC2:NEAR
...
CODSEG_2    ENDS

DATSEG_1    SEGMENT
...
DATSEG_1    ENDS

DATSEG_2    SEGMENT
...
DATSEG_2    ENDS
;Definición de grupos de segmentos de código y de datos

CGRUPO GROUP CSEG,CODSEG_1,CODSEG_2
DGRUPO GROUP DATSEG_1,DATSEG_2
;Programa principal
CSEG  SEGMENT
        ASSUME CS:CGRUPO,DS:DGRUPO,SS:STACK
INICIO PROC
...
...
        MOV  AX,DGRUPO
        MOV  DS,AX
...
...
        CALL PROC1
        CALL PROC2
...
...
        RET
INICIO ENDP
CSEG  ENDS
        END INICIO
```

IF	Si (condicional)	
Formato:	IFxxx [condición]	IFxxx [condición]
....		
.... 0		
ENDIF	ELSE	
		
		

ENDIF

Descripción:

Las pseudo-ops del tipo IF son condicionales, y hacen que el compilador incluya o no un grupo de sentencias fuente, a partir de evaluar, al tiempo de compilación, si una condición es verdadera o falsa.

Si la condición es verdadera, entonces el compilador incluye todo el bloque IF (grupo de sentencias fuente comprendidas entre las directivas IF y ENDIF o entre las directivas IF y ELSE).

Si la condición es falsa, el compilador incluye el bloque ELSE (grupo de sentencias fuente comprendidas entre las directivas ELSE y ENDIF). El uso de ELSE es opcional.

Las siguientes son las distintas variantes de IF:

DIRECTIVA	CONDICION
IF	expresión
IFE	expresión distinta de cero
IF1	expresión igual a cero
IF2	paso 1 del compilador
IFDEF	paso 2 del compilador
IFNDEF	símbolo definido o declarado externo
IFB	símbolo no definido ni declarado externo
IFNB	<arg> argumento en blanco
IFIDN	<arg> argumento no en blanco
IFIDF	<arg1>,<arg2> argumento 1 igual a argumento 2
	<arg1>,<arg2> argumento 1 diferente de argumento 2

Comentarios:

- IFB e IFNB se usan para chequear la existencia de argumentos en una macro.
- IFB, IFNB, IFIDN e IFIDF deben incluir los argumentos entre paréntesis angulares.
- Las directivas condicionales pueden anidarse.

Ejemplo 1:

Incluir un grupo de sentencias si el símbolo TEST es cero:

```
TEST = 0
IFE TEST
...
...
ENDIF
```

Otra forma de escribir el mismo bloque será:

```
TEST = 0
IF TEST EQ 0
...
...
ENDIF
```

Ejemplo 2:

Pseudo-ops condicionales anidadas, sin opción ELSE:

```
IF condición 1
....
....
    IF condición 2
        ....
        ....
    ENDIF
....
....
```

ENDIF

Ejemplo 3:

Pseudo-ops condicionales anidadas, con opción ELSE:

```
IDIOMA = 0
IDIOMA_1 = 1
IDIOMA_2 = 2
    IFE IDIOMA
    ....
    ....
    ELSE
        IFE IDIOMA_1
        ....
        ....
        ELSE
            IF IDIOMA_3
            ....
            ....
            ENDIF
        ENDIF
    ENDIF
ENDIF
```

Ejemplo 4:

Macro que lee el nombre y apellido de una persona; sus argumentos son NOMBRE_1, NOMBRE_2 y APELLIDO, y su sintaxis contempla la posibilidad de aceptar los datos de una persona que no tenga el segundo nombre:

```
IDENT MACRO NOMBRE_1,NOMBRE_2,APELLIDO
    ....
    ....
    IFNB <NOMBRE_2>
    ....
    ....
    ENDIF
    ....
    ....
    ENDM
```

Una posible llamada a esta macro puede ser:

```
IDENT JUAN,JOSE,PEREZ
```

con lo que el compilador incluirá el grupo de sentencias del bloque IFE <NOMBRE_2>.

En cambio, con la llamada:

```
IDENT JUAN,,PEREZ
```

el compilador ignorar dicho bloque condicional.

INCLUDE Incluir

Formato: **INCLUDE nombre-archivo**

Descripción:

Incluye el archivo de sentencias fuente identificado mediante "nombre-archivo".

El conjunto de sentencias fuente del archivo se incluye inmediatamente a continuación de la sentencia INCLUDE.

Comentarios:

- No se permite el anidamiento de INCLUDE.
- En el listado del fuente generado por el compilador, las sentencias incluidas comienzan en la columna 32 de cada línea; mientras que en la columna 30 aparece una "C", y la columna 31 queda en blanco.

- Las macros que residen en archivos separados (bibliotecas de macros) deben ser incluidas en el primer paso del compilador mediante el uso de la directiva condicional IF1.
- Mediante INCLUDE se elimina la necesidad de repetir grupos de sentencias comunes en el mismo o en diferentes módulos.

Ejemplo 1:

```
;Archivo PROLOGO.ASM
ASSUME CS:CODIGO,DS:DATOS
MOV AX,DATOS
MOV DS,AX
;Programa que usa INCLUDE
CODIGO SEGMENT
    INCLUDE PROLOGO.ASM
    ....
    ....
CODIGO ENDS
```

Ejemplo 2:

Incluir una biblioteca de macros en el primer paso del compilador.

```
IF1
    INCLUDE    MACRO.LIB
ENDIF
```

IRP Repetir

Formato: IRP nombre,<lista-argumentos>

Descripción:

Le indica al compilador que repita el conjunto de sentencias fuente, comprendidas entre las directivas IRP y ENDM, una vez para cada uno de los ítems de "<lista-argumentos>" en cada referencia a "nombre".

En cada paso, se reemplaza "nombre" por el próximo elemento de "<lista-argumentos>".

Operandos:

"nombre" es un nombre de variable.

"<lista-argumentos>" es una lista de argumentos numéricos, separados por coma.

Comentarios:

- La lista de argumentos debe, obligatoriamente, estar encerrada entre paréntesis angulares ("<" y ">").
- Si la lista de argumentos es nula (<>), el bloque de sentencias se repite una vez, eliminando "nombre".
- Se pueden incluir bloques IRP/ENDM en cualquier programa, y no sólo en macros.

Ejemplo 1:

Macro que almacena 5 palabras, inicializando cada una de ellas con la potencia cúbica de cada uno de los valores indicados en la lista de argumentos:

```
CUBO MACRO VALOR
    IRP VALOR,<1,2,3,4,5>
        DW VALOR*VALOR*VALOR
    ENDM
ENDM
```

A través de la invocación:

```
CUBO POT_3
```

La expansión de la macro será:

```
POT_3 DW 1
      DW 8
      DW 27
      DW 64
      DW 125
```

IRPC Repetir

Formato: IRPC nombre,<string>

IRPC nombre,string

Descripción:

Le indica al compilador que repita el conjunto de sentencias fuente, comprendidas entre las directivas IRPC y ENDM, una vez para cada uno de los caracteres de "string" en cada referencia a "nombre".

En cada paso, se reemplaza "nombre" por el próximo carácter de "string".

Operandos:

"nombre" es un nombre de variable.

"string" es una lista de caracteres, uno a continuación del otro.

Comentarios:

- Esta pseudo-op es similar a IRP, excepto que la lista de argumentos se sustituye por un string.
- El uso de paréntesis angulares ("<" y ">") es optativo.
- Si el string es nulo (<>), el bloque de sentencias se repite una vez, eliminando "nombre".
- Se pueden incluir bloques IRPC/ENDM en cualquier programa, y no sólo en macros.

Ejemplo 1:

Macro que almacena 5 bytes, inicializando cada uno de ellos con el código ASCII de cada uno de los caracteres del string.

```
PALABRA MACRO CARACR
    IRPC CARAC,ABCDE
    DB CARAC
ENDM
ENDM
```

A través de la invocación:

```
PALABRA FRASE
```

La expansión de la macro será :

```
FRASE DB 65
      DB 66
      DB 67
      DB 68
      DB 69
```

o, lo que es lo mismo:

```
FRASE DB "A"
      DB "B"
      DB "C"
      DB "D"
      DB "E"
```

LABEL Rótulo

Formato: **nombre LABEL atributo**

Descripción:

Sirve para asignar diferentes nombres a variables o a rótulos, con la posibilidad adicional de modificar sus atributos.

Operandos:

"nombre" puede ser:

- el nombre de una estructura;
- una variable;
- un rótulo (inclusive, nombres de procedimientos).

"atributo" puede ser:

- BYTE, WORD o DWORD, para variables;
- nombre de una estructura;
- nombre de un RECORD;
- NEAR o FAR para indicar la distancia de un procedimiento.

Comentarios:

- LABEL no puede ser usada dentro de una definición de estructura (bloque STRUC/ENDS).
- Los nombres de estructura y de RECORD son tipos de datos definidos por el usuario mediante las pseudo-ops STRUC y RECORD, respectivamente.

Ejemplo 1:

Asignar atributo FAR a una instrucción:

```
ACAF LABEL FAR
ACA: MOV DX,0
```

...

ACA es un rótulo de tipo NEAR, y ACAF es una redefinición de ACA, pero con atributo FAR. Esta redefinición permite la bifurcación a la instrucción rotulada ACA: desde un segmento distinto en el que está definida.

Ejemplo 2:

Asignar atributo de BYTE a una tabla definida con elementos de tipo palabra:

```
TABLAB LABEL BYTE
TABLAW DW 2F24H,036AH,0817H,03H
```

La misma dirección de memoria puede ser referenciada como BYTE a través del uso del rótulo TABLAB, o como WORD a través del uso del rótulo TABLAW.

.LALL / .SALL / .XALL Listado de expansión de macros

Formato: .LALL

.SALL

.XALL

Descripción:

Controlan la aparición o no de las expansiones de las macros en el listado del fuente generado por el compilador.

.LALL produce un listado completo de las sentencias de la macro, incluyendo las líneas de comentarios.

.SALL produce la exclusión de todo texto de la macro en el listado.

.XALL produce un listado de las sentencias fuente de la macro que generan código objeto, pasando por alto las líneas de comentarios y las pseudo-ops que no reservan espacio de memoria.

Comentarios:

- .XALL es la opción default.
- .LALL es útil para obtener una copia completa del programa fuente, con las macros y sus expansiones incluidas.
- .SALL es útil para listar un programa fuente, una vez que hayan sido depuradas las macros incluidas en ,1.

Ejemplo:

```
.LALL ;habilita el listado completo de macros
```

....

....

....

```
.SALL ;suprime el listado de macros
```

....

....

....

```
.XALL ;lista sentencias que producen código objeto
```

.LFCOND / .SFCOND / .TFCOND Listado de bloques
asociados a una
condición falsa

Formato: .LFCOND

.SFCOND

.TFCOND

Descripción:

Permiten especificar la impresión de los bloques condicionales falsos en el listado del fuente generado por el compilador.

.LFCOND lista los bloques asociados a una condición falsa.

.SFCOND suprime el listado de los bloques asociados a una condición falsa.

.TFCOND invierte el modo de listado de los bloques falsos; es decir, si está activo, lo desactiva, y si está inactivo, lo activa.

Comentarios:

- .LFCOND es la opción default.
- El uso de .SFCOND permite obtener un listado del fuente que incluya sólo las partes del programa que se compilan.

Ejemplo:

```
.LFCOND      ;listar bloques condicionales falso
....
....
....
.SFCOND      ;suprimir listado de bloques condicionales falsos
....
....
....
.TFCOND      ;invertir modo de listado de bloques condicionales falsos
```

.LIST / .XLIST Listado del fuente

Formato: **.LIST**

.XLIST

Descripción:

Controlan la emisión o no del listado del fuente generado por el compilador.

.LIST habilita la emisión del listado.

.XLIST suprime la emisión del listado.

Comentarios:

- Por default, el compilador siempre emite un listado del programa fuente (opción .LIST); si se desea que algún bloque no salga impreso, basta con especificar .XLIST y .LIST al comienzo y al final del bloque, respectivamente.

Ejemplo:

```
....
....
.XLIST      ;suprime listado del fuente
....
....
.LIST       ;habilita listado del fuente
```

LOCAL Local

Formato: **LOCAL rótulo[,...]**

Descripción:

Le indica al compilador los rótulos que debe cambiar cada vez que expande la macro.

Si una macro contiene sentencias rotuladas, entonces se debe tener en cuenta que esa macro utilizada más de una vez en el mismo programa, se expande con el mismo rótulo.

Para salvar este problema, los rótulos se deben declarar como LOCAL.

Si se usa, LOCAL debe ser la primera sentencia del cuerpo de una macro, es decir que debe estar inmediatamente a continuación de la directiva MACRO, y no debe llevar campo de comentarios.

Comentarios:

- LOCAL sólo puede utilizarse dentro de la definición de una macro.
- No se pueden incluir comentarios (ni la directiva COMMENT ni el uso de ";") entre las pseudo-ops MACRO y LOCAL.
- La declaración de un rótulo como LOCAL permite que pueda ser reutilizado en otras macros.
- Cada vez que se expande una macro, el compilador se encarga de asignar un nuevo nombre interno a los rótulos declarados LOCAL. Nunca se producen duplicaciones de rótulos.
- Los símbolos creados por el compilador para los rótulos declarados LOCAL son de la forma XXX0000 a XXXFFFF, 1 donde XXX es un nombre.

Ejemplo:

```
ESPERA MACRO CONT
    LOCAL PROXIMO
    PUSH CX
    MOV CX,CONT
PROXIMO: LOOP PROXIMO
    POP CX
ENDM
```

Esta macro puede ser llamada más de una vez en un mismo programa:

```
ESPERA 100
....
ESPERA 200
....
ESPERA 500
....
```

Cada vez que sea expandida la macro ESPERA, el compilador irá cambiando el nombre del rótulo PROXIMO, declarado LOCAL.

MACRO Macro-instrucción

Formato: nombre MACRO lista-parámetros

Descripción:

Define la macro-instrucción identificada con "nombre" y le asocia una lista de parámetros, que se corresponden con los argumentos al momento de ser invocada.

Para invocar a una macro se debe especificar el nombre de la macro seguido de la lista de argumentos a pasar.

Cada uno de los argumentos se corresponde con cada uno de los parámetros de "lista-parámetros".

El compilador, al hallar una invocación a una macro, la expande; es decir, copia, en ese punto del programa, el bloque de sentencias que forman el cuerpo de la macro reemplazando los parámetros por los argumentos.

Las líneas generadas por la expansión se señalan con un signo "+" en la columna 31; la columna 32 queda en blanco, y la columna 33 se corresponde con la columna 1 del módulo fuente.

Comentarios:

- La lista de parámetros es opcional, pues puede ocurrir que se defina una macro que no devuelva ningún resultado, sino que efectúe una determinada acción como ser, por ejemplo, borrar la pantalla.
- El número de argumentos en la invocación de una macro no necesariamente debe coincidir con el número de parámetros. Si hay más argumentos que parámetros, se ignoran los argumentos sobrantes; si hay más parámetros que argumentos, los parámetros sobrantes toman valor nulo.
- El nombre de una macro puede ser una pseudo-op o una instrucción; de ser así, se utiliza la definición de la macro en vez del significado normal. Para restaurar el significado original del nombre, se debe utilizar la pseudo-op PURGE.

Ejemplo 1:

Macro para sumar dos valores y guardar el resultado.

```
SUMA MACRO X,Y,Z
    MOV AX,X
    MOV BX,Y
    ADD AX,BX
    MOV Z,AX
ENDM
```

La expansión de esta macro para SUMA A,B,C ser :

```
MOV AX,A
MOV BX,B
ADD AX,BX
MOV C,AX
```

Ejemplo 2:

Macro sin parámetros.

```
APILAR MACRO
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX
ENDM
```

La expansión de esta macro será:

```
PUSH AX
PUSH BX
PUSH CX
PUSH DX
```

NAME Nombre de modulo objeto

Formato: NAME nombre

Descripción:

Asigna un nombre al modulo objeto.

Comentarios:

- El nombre del modulo objeto no puede ser una palabra reservada.
- NAME no puede tener un rótulo.
- No puede haber más que una directiva NAME por modulo fuente.
- El nombre del modulo objeto se pasa al linkeditor.

El compilador toma el nombre del modulo objeto de acuerdo con el siguiente criterio:

- a) Si existe NAME, de esta pseudo-op.
- b) Si no existe NAME, de la pseudo-op TITTLE. El nombre son los seis primeros caracteres del texto de esa directiva, siempre que sean legales como nombre de archivo.
- c) Si no existen ni NAME ni TITTLE (o existe, pero los caracteres son ilegales), entonces el nombre del modulo objeto se toma del nombre del modulo fuente pero con extensión .OBJ.

Ejemplo:

```
NAME MODULO1 ;asigna el nombre MODULO1 al modulo objeto
```

ORG Origen

Formato: ORG expresión

Descripción:

Inicializa el contador de posiciones con el valor resultante de evaluar la expresión.

"expresión" es tomada como un número de 16 bits, y su resultado indica la dirección (desplazamiento dentro del segmento) a partir de la cual el compilador almacenar el código de máquina del programa (origen del código).

Comentarios:

- En general, ORG se escribe al comienzo del programa.
- Para programa que se van a transformar en .COM, es necesario especificar ORG 100H.
- "expresión" puede indicarse mediante el carácter "\$", que devuelve el actual valor del contador de posiciones.
- ORG no se puede incluir dentro de un bloque STRUC/ENDS (estructura).

Ejemplos:

```
ORG 100H ;256 bytes respecto al origen del segmento
```

```
ORG $+4 ;contador de posiciones + 4 (reserva 4 bytes)
```

%OUT Salida

Formato: %OUT texto

Descripción:

Sirve para escribir un texto (mensaje) por pantalla durante la compilación del programa.

Comentarios:

- El mensaje se imprime cuando el compilador llega a la línea donde est %OUT.
- %OUT se utiliza especialmente para indicar que camino se ha elegido en una determinada condición.
- Si el listado del fuente es por pantalla, el compilador ignora todas las sentencias %OUT.

Ejemplo:

```
IF MODO EQ 0
    %OUT Modo normal
    ....
    ....
ELSE
    %OUT Modo Prueba
    ....
    ....
ENDIF
```

PAGE Página

Formato: PAGE [líneas][,caracteres]

Descripción:

Define las dimensiones (longitud y ancho) de las páginas del listado del programa fuente que genera el compilador.

La longitud de la página se mide en líneas, y el ancho se mide en caracteres o columnas.

Cada página del listado del programa fuente lleva impreso al tope el número de página y el número de capítulo

Comentarios:

- Los valores por default son de 66 líneas y 80 caracteres, para largo y ancho de página respectivamente.
- El rango de valores para el largo de página es de 10 a 255 líneas.
- El rango de valores para el ancho de página es de 60 a 132 caracteres.
- El compilador produce un salto de página en el listado del programa fuente cuando se llena una página de acuerdo a las dimensiones especificadas, o cuando encuentra una directiva PAGE.
- Si se coloca el carácter "+" en el lugar de cantidad de líneas, el compilador incrementa el número de capítulo del listado, y reinicializa la numeración de páginas desde el número 1.
- Si se especifica PAGE sin operandos, se genera un salto de página y se incrementa en 1 el número de página.

Ejemplos:

```
PAGE          ;66 líneas de 80 caracteres
PAGE  ,132    ;66 líneas de 132 caracteres
PAGE  72      ;72 líneas de 80 caracteres
PAGE  72,85   ;72 líneas de 85 caracteres
PAGE  +       ;nuevo capítulo
```

PROC Procedimiento

Formato: **nombre PROC [atributo]**

Descripción:

Indica el comienzo del procedimiento identificado con "nombre".

Un procedimiento es un bloque de instrucciones que sirve para realizar una tarea determinada, y que puede ser invocado desde varios lugares del programa. En general, un procedimiento puede ser considerado como una subrutina.

Operandos:

- "nombre" es un rótulo con el que se identifica el procedimiento; cuando se lo invoca, se hace referencia a "nombre".
- "atributo" es el atributo de distancia, y puede ser NEAR o FAR. Se usa para especificar desde donde puede ser invocado el procedimiento. El valor por default es NEAR.

Un procedimiento NEAR sólo puede ser invocado desde el mismo segmento de código donde fue definido o desde un segmento que tiene el mismo valor definido en ASSUME CS (mismo nombre).

Al invocar un procedimiento NEAR, se guarda en el stack sólo el desplazamiento (valor de IP) de la próxima instrucción, que se recuperará cuando se retorne del procedimiento.

Un procedimiento FAR puede ser invocado desde cualquier segmento de código.

Al invocar un procedimiento FAR, se guardan en el stack el segmento (valor de CS) y el desplazamiento (valor de IP) de la próxima instrucción, en ese orden, que se recuperarán cuando se retorne del procedimiento.

Comentarios:

- Un procedimiento se ejecuta a través de una instrucción de transferencia, a través de una instrucción CALL, o directamente.
- Si se desea que un procedimiento de un modulo pueda ser ejecutado desde otros módulos, se lo debe declarar con la pseudo-op PUBLIC.
- El procedimiento principal de un programa debe ser siempre de tipo FAR.

Ejemplo 1:

Dos procedimientos en el mismo segmento de código:

```
CODIGO SEGMENT
    ASSUME CS:CODIGO
PROC1 PROC FAR
    ....
    CALL  PROC2
    ....
    RET
PROC1 ENDP
PROC2 PROC NEAR
    ....
    ....
    RET
PROC2 ENDP
CODIGO ENDS
```

Ejemplo 2:

Dos procedimientos en segmentos de código distintos:

```
CODIGO SEGMENT
    ASSUME CS:CODIGO
PROC1 PROC FAR
```

```

....
CALL  PROC2
....
RET
PROC1 ENDP
CODIGO ENDS
CODIGO1 SEGMENT
PROC2 PROC  FAR
....
....
RET
PROC2 ENDP
CODIGO1 ENDS

```

PUBLIC Público

Formato: **PUBLIC símbolo[...]**

Descripción:

Declara como públicos los símbolos especificados. Esto significa que dichos símbolos pueden ser usados por otros módulos, los que serán linkeditados juntos.

Operandos:

"símbolo" puede ser:

- un nombre de variable.
- una constante de 16 bits.
- un nombre de procedimiento.

Comentarios:

- Los módulos que usan símbolos declarados como PUBLIC, deben declarar dichos símbolos como externos mediante la pseudo-op EXTRN.
- La pseudo-op PUBLIC puede aparecer en cualquier parte del programa, pero resulta conveniente y práctico colocar todas las declaraciones PUBLIC al comienzo del programa.

Ejemplo 1:

Procedimiento que va a ser usado por otros módulos:

```

PUBLIC PROC1
PROC1 PROC  FAR
PUSH  BP
MOV   BP,SP
....
....
MOV   SP,BP
POP   BP
RET
PROC1 ENDP

```

El modulo que invocar a PROC1 debe incluir la declaración EXTRN:

```
EXTRN  PROC1:FAR
```

Ejemplo 2:

Variable definida en un modulo y también usada en otro modulo:

```

PUBLIC TOTAL
TOTAL DW    0
....
....

```

El modulo que invoque a la variable debe tener la declaración EXTRN:

```
EXTRN  TOTAL:WORD
```

Ejemplo 3:

Bifurcación a la instrucción rotulada ETIQ desde otro modulo, en el mismo segmento físico de código:

```
SEG1  SEGMENT
      PUBLIC ETIQ
      ....
      ....
      ETIQ: ....      ;ETIQ es NEAR (tiene ":")
      ....
      ....
      SEG1  ENDS
```

El modulo desde el cual se va a efectuar la bifurcación a ETIQ, debe incluir la declaración EXTRN:

```
      EXTRN ETIQ:NEAR
SEG2  SEGMENT
      ....
      ....
      JMP  ETIQ
      ....
      ....
      SEG2  ENDS
```

Ejemplo 4:

Bifurcación a la instrucción rotulada ETIQ desde otro modulo, en otro segmento físico de código:

```
SEG1  SEGMENT
      PUBLIC ETIQ
      ....
      ....
      ETIQ  ....      ;ETIQ es FAR (no tiene ":")
      ....
      ....
      SEG1  ENDS
```

El modulo desde el cual se va a efectuar la bifurcación a ETIQ, debe incluir la declaración EXTRN:

```
      EXTRN ETIQ:FAR
SEG2  SEGMENT
      ....
      ....
      JMP  ETIQ
      ....
      ....
      SEG2  ENDS
```

PURGE Purgar

Formato: **PURGE nombre-macro[,...]**

Descripción:

Borra las macros especificadas, para permitir que se reutilice el espacio de memoria.

Comentarios:

- PURGE se usa cuando, por ejemplo, se incluye una biblioteca de macros y no se usan todas las macros definidas en ella.

Ejemplo:

```
INCLUDE MACRO.LIB
      ....
      ....
      PURGE  MACRO1,MACRO2
      ....
```

....

.RADIX Raíz

Formato: .RADIX expresión

Descripción:

Permite cambiar la base de numeración default. El compilador toma por default la base decimal, es decir que considera a los números sin sufijo como números en base 10.

Los sufijos son:

- B Binario
- D Decimal
- O Octal
- Q Octal
- H Hexadecimal

Operandos:

"expresión" es un valor numérico, siempre expresado en base 10, independientemente de la base de numeración que esté activa.

Comentarios:

- .RADIX no afecta a DD, DQ o DT, pues siempre suponen decimal, a menos que se indique un sufijo de tipo de datos.
- Si se especifica .RADIX 16, hay que tener en cuenta que los números hexadecimales terminados en D son considerados por el compilador como valores decimales.

Ejemplos:

```
.RADIX 16 ;Base 16 (hexadecimal)
DB 11 ;11H = 17
DB 11D ;se interpreta como 11D (decimal)
DB 01AD ;error, pues A no es un dígito decimal
.RADIX 2 ;Base 2 (binaria)
DB 11 ;11B = 3
.RADIX 10 ;Base 10 (decimal)
DB 11 ;11D = 11
```

RECORD Registro

Formato: nombre-registro RECORD nombre-campo:long[=exp][,...]

Descripción:

Define un conjunto de campos a nivel de bits, y los agrupa bajo un nombre de registro.

Con RECORD no se reserva memoria, es sólo una definición de campos. Cada campo es definido mediante su nombre, su longitud (en bits) y un valor, que es opcional, que se toma como valor default.

Una vez definido el RECORD, el nombre del RECORD se puede usar para reservar e inicializar memoria.

Operandos:

- "nombre-registro" es un rótulo que identifica al RECORD.
- "nombre-campo" es un rótulo que identifica un campo dentro del registro.
- "long" es el número de bits de un campo.
- "exp" es opcional, y corresponde al valor por default de un campo.

Comentarios:

- El tamaño de un campo puede ser de 1 a 16 bits. Si es mayor que 8 bits, el compilador utiliza 2 bytes enteros, de lo contrario, el compilador usa sólo un byte.
- Si el número total de bits es menor que 8 ó 16, los campos se ajustan a la derecha, es decir, al bit menos significativo del byte o de la palabra.
- Para reservar memoria e inicializar campos utilizando un registro previamente definido, se usa la siguiente sentencia:

```
[nombre] nombre-registro<[exp][,...]>
```

o

[nombre] nombre-registro expresión DUP(<[exp]>)
 "nombre" es opcional, y corresponde al nombre del primer byte del área de memoria reservada.
 "nombre-registro", que es el nombre de la estructura RECORD previamente definida.
 "exp" es una expresión opcional, y corresponde al valor inicial de un campo. Los valores iniciales de los campos van separados por comas, y encerrados entre paréntesis angulares. Si no se especifican valores iniciales, los campos se inicializan con los valores declarados en la definición del RECORD.
 En el segundo formato, "expresión" es un valor que indica la cantidad de veces que se repite un valor inicial; en este caso, va seguido de la declaración DUP.
 - La directiva RECORD es similar a la directiva STRUC, con la única diferencia que mientras que STRUC funciona a nivel de bytes, RECORD lo hace a nivel de bits.

Ejemplos:

Definición del registro R, de 16 bits, con los siguientes campos:

- Campo A de 7 bits. Valor default = 5.
- Campo B de 3 bits. Valor default = 1.
- Campo C de 2 bits. Valor default = ninguno.
- Campo D de 4 bits. Valor default = ninguno.

R RECORD A:7=5,B:3=1,C:2,D:4

Reserva de memoria e inicialización de la variable X, utilizando la definición del RECORD R:

X R<6,,2>

Después de la ejecución de esta sentencia, se reservan 16 bits para X, con los siguientes valores:

- Campo A = 0000110B = 6 (valor tomado de la lista)
- Campo B = 001B = 1 (valor por default)
- Campo C = 10B = 2 (valor tomado de la lista)
- Campo D = xxxxB = valor indeterminado

La variable X ser , entonces 0000 1100 0110 xxxxB = 0C 6XH, que se almacenar , como toda palabra, con el byte menos significativo en la dirección más baja y con el byte más significativo en la dirección más alta. Es decir que se verá como 6X 0C.

Utilización de los datos definidos:

MOVE AX,OFFSET X ;se recoge el desplazamiento de X en un registro que no es registro de segmento

AND DX,MASK B ;equivale a la instrucción AND DX,0000000111000000B

MOV CL,WIDTH B ;obtiene el número de bits del campo B. Equivale a la instrucción MOV CL,3

REPT Repetir

Formato: REPT expresión

Descripción:

Le indica al compilador que repita el bloque de sentencias comprendidas entre REPT y ENDM la cantidad de veces dada por "expresión".

Comentarios:

- El bloque REPT/ENDM no necesariamente debe residir dentro de una definición de macro.

Ejemplo:

Macro para reservar una cantidad de bytes en memoria:

ALOCAR MACRO TABLA,LONGI

TABLA EQU THIS BYTE

VALOR = 0

REPT LONGI

VALOR = VALOR+1

DB VALOR

ENDM

ENDM

La invocación a la macro mediante ALOCAR TABLA1,5 produce la siguiente expansión:

```
TABLA1 DB 1
        DB 2
        DB 3
        DB 4
        DB 5
```

SEGMENT Segmento

Formato: **nombre SEGMENT [alineación] [combinación] ['clase']**

Descripción:

Marca el inicio de un segmento de programa fuente. El final de un segmento se indica con la directiva ENDS.

Un segmento es un bloque de sentencias que puede contener definición de variables (datos), instrucciones (código ejecutable) o una combinación de datos e instrucciones.

Existen cuatro tipos de segmentos: de datos, de código, extra o auxiliar y de stack.

Un modulo fuente puede ser un segmento, parte de un segmento, varios segmentos, partes de varios segmentos o combinaciones de los anteriores.

Operandos:

"alineación" es opcional, e indica en que posición de memoria se almacena el segmento; la alineación puede ser:

- BYTE, cuando el segmento puede comenzar en cualquier dirección.
- WORD, cuando el segmento debe comenzar en el primer byte de una palabra, es decir, en una dirección múltiplo de 2.
- PARA, cuando el segmento debe comenzar en el primer byte de un párrafo, es decir, en una dirección múltiplo de 16. Este es el valor por default.
- PAGE, cuando el segmento debe comenzar en el primer byte de una página, es decir, en una dirección múltiplo de 256.

"combinación" es opcional, e indica la manera en que el segmento se combinar con otros segmentos de la misma clase; la combinación puede ser:

- PUBLIC, cuando el segmento se concatena con otros segmentos de la misma clase. Esta concatenación se produce en el momento de linkedición.
- COMMON, cuando el segmento se superpone con otros segmentos de la misma clase; significa que todos los segmentos COMMON de la misma clase comenzar n en la misma dirección de memoria, y se superpondrán entre sí (swapping).

La longitud asignada por el linkeditor es la longitud máxima de todos los segmentos COMMON.

- AT expresión, hace que el segmento se ubique en la dirección dada por la expresión.

Esta dirección no se usa para forzar la carga del segmento en una dirección fija; lo que se permite es de poder definir variables dentro del segmento. Normalmente se usa para referenciar determinadas zonas de memoria como los vectores de interrupción o la memoria ROM.

- STACK, cuando el segmento es el de stack. Al menos debe existir un segmento stack para crear con el linker un modulo ejecutable.
- MEMORY, cuando el segmento se ubicará en una dirección de memoria más alta que la de los otros segmentos que aparecen durante la linkedición. Se puede aplicar, por ejemplo, para acceder a la memoria más allá de los límites del programa.

Sólo puede haber un segmento de este tipo; si existieran varios, sólo se tomará como tal el primero, mientras que los restantes se procesarán como COMMON.

"clase" se utiliza para indicar el orden en el cual son almacenados los segmentos en memoria: los segmentos con la misma clase serán almacenados consecutivamente, mientras que los segmentos con diferente clase van a ser almacenados en el orden en el cual el linkeditor los encuentra. La clase se especifica entre comillas simples.

Comentarios:

- Los operandos "alineación", "combinación" y "clase" se le pasan al linkeditor.
- Pueden existir segmentos anidados. En este caso, el compilador los procesa como si no lo fueran, juntando la primera parte de cada segmento partido con la segunda.

- No se permiten segmentos que se superpongan, excepto aquellos con "alineación" COMMON.

Ejemplo 1:

Segmento alineado a párrafo:

```
SEG1  SEGMENT
....
....
SEG1  ENDS
```

Ejemplo 2:

Segmento de stack:

```
SEG1  SEGMENT STACK
....
....
SEG1  ENDS
```

Ejemplo 3:

Segmentos anidados: segmento DATOS dentro de segmento CODIGO:

```
CODIGO SEGMENT
....
....
DATOS  SEGMENT
....
....
DATOS  ENDS
....
....
CODIGO ENDS
```

Ejemplo 4:

Agrupamiento de segmentos lógicos distintos en un único segmento físico. Dichos segmentos pueden pertenecer o no al mismo módulo fuente:

```
SEG1  SEGMENT PUBLIC 'DATOS'
....
....
SEG1  ENDS
SEG2  SEGMENT PUBLIC 'DATOS'
....
....
SEG2  ENDS
```

Ejemplo 5:

Segmento correspondiente a la memoria de pantalla gráfica. El buffer de la pantalla comienza en la dirección B800H:0000H (dirección física B8000H):

```
PANTALLA  SEGMENT AT B800H
....
....
PANTALLA  ENDS
```

Ejemplo 6:

Segmento correspondiente a los vectores de interrupción del BIOS. La dirección es 0H:

```
INTERRUP  SEGMENT AT 0
....
....
INTERRUP  ENDS
```

STRUC Estructura

Formato: nombre-estructura STRUC

....
dato

....

nombre-estructura ENDS

Descripción:

Define un conjunto de campo a nivel de bytes. Para la definición de cada campo de la estructura se emplean las directivas estándar de definición de datos: DB, DW, DD, DQ y DT.

La definición de una estructura comienza con STRUC y finaliza con ENDS. "nombre-estructura" debe ser el mismo en ambas sentencias. Entre ambas directivas se incluyen las directivas de definición de datos, que determinan a la estructura.

Con STRUC no se reserva memoria, es sólo una definición de campos. Cada campo es definido mediante su nombre, su tipo y un valor inicial, el que se toma como valor por default.

Una vez definida la estructura, el nombre de la STRUC se puede usar para reservar e inicializar memoria.

Comentarios:

- Para reservar memoria e inicializar campos utilizando una estructura previamente definida, se usa la siguiente sentencia:

[nombre] nombre-estructura<[exp][,...]>

o

[nombre] nombre-estructura expresión DUP(<[exp]>)

"nombre" es opcional, y corresponde al nombre del primer byte del área de memoria reservada.

"nombre-estructura" es el nombre de la estructura previamente definida.

"exp" es una expresión opcional, y corresponde al valor inicial de un campo. Los valores iniciales de los campos van separados por comas, y encerrados entre paréntesis angulares. Si no se especifican valores iniciales, los campos se inicializan con los valores declarados en la definición de la STRUC.

En el segundo formato, "expresión" es un valor que indica la cantidad de veces que se repite un valor inicial; en este caso, va seguido de la declaración DUP.

- Los campos de una estructura pueden ser modificables o no modificables: los campos simples son modificables, mientras que los campos múltiples son no modificables.

- La referencia a los campos se hace mediante:

variable.campo

"variable" es el nombre con el cual se reservó espacio de memoria para una estructura previamente definida.

"campo" es uno de los campos de la estructura.

- La directiva STRUC es similar a la directiva RECORD, con la única diferencia que mientras que STRUC funciona a nivel de bytes, RECORD lo hace a nivel de bits.

- La directiva STRUC es especialmente usada en el pasaje de parámetros a subrutinas a través del stack.

Ejemplo 1:

Definición de la estructura S:

S STRUC

CAMPO1 DB 1,2 ;campo múltiple, no modif.

CAMPO2 DB 10 DUP(?) ;campo múltiple, no modif.

CAMPO3 DW 5 ;campo simple, modificable

CAMPO4 DB 'ABC' ;campo simple, modificable

S ENDS

Esta estructura tiene las siguientes características:

- Consta de cuatro campos, dos simples y dos múltiples.
- Su longitud es de 17 bytes (2+10+2+3).
- Los campos CAMPO1, CAMPO3 y CAMPO4 tienen un valor inicial (valor default).
- El campo CAMPO2 no tiene valor inicial.

Reserva de memoria e inicialización de la variable X, utilizando la estructura S:

X S<,,7,'XYZ'>

Se reservan 17 bytes para la variable X; de este modo, los datos dentro de X quedarán con los siguientes valores:

```
X.CAMPO1 = 1,2
X.CAMPO2 = 10 DUP(?)
X.CAMPO3 = 7
X.CAMPO4 = 'XYZ'
```

Algunos ejemplos de utilización son los siguientes:

```
MOV AX,[BX].CAMPO3
```

Esta instrucción es equivalente a `MOV AX,[BX+12]`; el valor 12 corresponde al desplazamiento de CAMPO3 dentro de la estructura. Observar que no es necesario especificar el nombre de la estructura ni de la variable, sino sólo el nombre del campo.

```
MOV AX,X.CAMPO3
```

Esta instrucción es equivalente a `MOV AX,X+12`.

Ejemplo 2:

Uso de STRUC para el pasaje de parámetros a subrutinas:

```
PARAMS STRUC
    DW  ?    ;BP salvado
    DD  ?    ;dirección de retorno
P2 DD  ?    ;dirección del parámetro 2
P1 DD  ?    ;dirección del parámetro 1
PARAMS ENDS
```

Esta estructura tiene las siguientes características:

- Consta de cuatro simples.
- Su longitud es de 14 bytes (2+4+4+4).
- Ninguno de los campos tienen un valor inicial.

Ejemplo de la utilización de esta estructura es:

```
LES SI,[BP].P1
```

Esta instrucción carga en ES un valor de segmento y en SI un valor de desplazamiento; equivale a la instrucción `LES SI,[BP]+10`, dado que P1 está ubicado a un desplazamiento de 10 bytes respecto al inicio de la estructura PARAMS.

Ejemplo 3:

Definición de una estructura y reserva de memoria e inicialización:

```
VECTOR STRUC
    DB  ?
    DW  ?
    DW  ?
VECTOR ENDS
```

```
LISTA VECTOR<'A',0,0>,<'B',21,0>,<'B',0,11>
```

SUBTTL Subtítulo

Formato: **SUBTTL texto**

Descripción:

Permite definir un texto que saldrá impreso como subtítulo del listado del fuente cada vez que se produzca un salto de hoja.

Comentarios:

- El subtítulo puede tener hasta 60 caracteres de longitud.
- El subtítulo se imprime, centrado, en la tercera línea de cada página del listado, a continuación del título.
- Se pueden especificar por modulo todas las directivas SUBTTL que se deseen.
- Si no se utiliza ninguna directiva SUBTTL, la tercera línea del listado aparecerá en blanco.
- Si después de usar una directiva SUBTTL se desea que vuelva a salir el subtítulo en blanco, se debe incluir una directiva SUBTTL pero sin texto.

Ejemplo:

SUBTTL Modulo secundario

TITLE Título

Formato: TITLE texto

Descripción:

Permite definir un texto que saldrá impreso como título del listado del fuente cada vez que se produzca un salto de hoja.

Comentarios:

- El título puede tener hasta 60 caracteres de longitud.
- El título se imprime, justificado a izquierda, en la segunda línea de cada página del listado.
- No se puede especificar más de una directiva TITLE por modulo.
- Si no se incluyó una directiva NAME en el modulo fuente, el nombre del modulo objeto es tomado por el compilador de los primeros seis caracteres del texto de la directiva TITLE.
- Si no se utilizó ni TITLE ni NAME, el nombre del modulo objeto es el mismo que el del modulo fuente, pero con extensión .OBJ.

Ejemplo:

TITLE MODULO PRINCIPAL

Si no se especificó la directiva NAME, el nombre del modulo objeto ser MODULO.OBJ.

CAPITULO 9

SET DE INSTRUCCIONES

En este capítulo veremos en detalle cada una de las instrucciones assembler del microprocesador INTEL 8088-8086. Utilizaremos las siguientes abreviaturas y símbolos:

Tipos de operandos

acum = acumulador de 8 ó 16 bits (AX,AL)

acum8 = acumulador de 8 bits (AL)

acum16 = acumulador de 16 bits (AX)

desp = desplazamiento

mem = variable de memoria de 8 ó 16 bits

mem8 = variable de memoria de 8 bits

mem16 = variable de memoria de 16 bits

mem32 = variable de memoria de 32 bits

reg = registro de 8 ó 16 bits

reg8 = registro de 8 bits

reg16 = registro de 16 bits

seg = registro de segmento (CS,DS,ES,SS)

val = valor inmediato de 8 ó 16 bits

val8 = valor inmediato de 8 bits

val16 = valor inmediato de 16 bits

Modificación de banderas

1 = Bandera encendida incondicionalmente

0 = Bandera apagada incondicionalmente

* = Bandera alterada

? = Bandera con valor indefinido

- = Bandera no alterada

AAA Ajuste ASCII en suma (Ascii Adjust for Addition)

Formato: AAA

Descripción: Ajusta el resultado de una suma decimal desempaquetada. AAA se utiliza cuando se hace una operación de suma entre dos operandos decimales desempaquetados, de modo que el resultado sea también un decimal desempaquetado. Se asume que el resultado de la suma se encuentra en el registro AL.

Lógica:

Si parte baja de AL > 9 ó AF=1

AL=AL+6
 AH=AH+1
 AF=1
 CF=AF

Parte alta de AL=0

El ajuste consiste en dejar el resultado de la suma en los 4 bits de menor orden de AL y en llenar con ceros los 4 bits de mayor orden de AL. Si el resultado de la suma es superior a 9, AAA incrementa en 1 el registro AH y fija la bandera de acarreo auxiliar.

Banderas:

OF	DF	IF	TF	SF	ZF	AF	PF	CF
?	-	-	-	?	?	*	?	*

Ejemplo 1: AH=00H ; AL=08H ; BL=06H

ADD AL,BL
 AL=0EH AF=0
 BL=06H

AAA
 AX=0104H AF=1 CF=1
 BL=06H

Ejemplo 2: AH=05H ; AL=08H ; BL=09H

ADD AL,BL
 AL=11H AF=1
 BL=09H

AAA
 AX=0207H AF=1 CF=1
 BL=09H

Ejemplo 3: AH=00H ; AL=02H ; BL=04H

ADD AL,BL
 AL=06H AF=0
 BL=04H

AAA
 AX=0006H AF=0 CF=0
 BL=04H

AAD Ajuste ASCII en división (Ascii Adjust for Division)

Formato: AAD

Descripción: Convierte un dividendo desempaquetado a un valor binario y lo almacena en el registro AL. AAd asume que el dividendo desempaquetado está almacenado en el registro AX, donde AH contiene la parte alta y AL la parte baja. AAD se debe usar antes de efectuar la operación de división.

Lógica:

AL=AH*10+AL
 AH=0

Multiplica por 10 el contenido de AH y lo suma al contenido de AL, almacenando el resultado en AL y rellenando con ceros AH.

Banderas:

OF	DF	IF	TF	SF	ZF	AF	PF	CF
?	-	-	-	*	*	?	*	?

Ejemplo 1: AX=0509H (59 desempaquetado) ; BL=07H

AAD
 AX=003BH (59 binario)
 BL=07H

DIV BL
 AH=03H
 AL=08H
 BL=07H

Ejemplo 2: AX=0103H (13 desempaquetado) ; BL=02H

AAD

AX=000DH (13 binario)

BL=02H

DIV BL

AH=01H

AL=06H

BL=02H

AAM Ajuste ASCII en multiplicación (Ascii Adjust for Multiply)

Formato: AAM

Descripción: Ajusta el resultado de una multiplicación desempaquetada. AAM asume que el resultado de la multiplicación está almacenado en el registro AX, donde AH contiene la parte alta y AL la parte baja.

Para que AAM opere correctamente, ambos operandos que se multiplican deben ser decimales desempaquetados.

Lógica:

AH=cociente AL/10

AL=resto de AL/10

Divide por 10 el contenido de AL y almacena el cociente en AH y el resto en AL.

Banderas:

OF DF IF TF SF ZF AF PF CF

? - - - * * ? * ?

Ejemplo 1: AL=09H (9 desempaquetado)

BL=07H (7 desempaquetado)

MUL BL

AX=003FH (63 binario)

BL=07H

AAM

AH=06H (6 desempaquetado)

AL=03H (3 desempaquetado)

BL=07H

Ejemplo 2: AL=04H (4 desempaquetado)

BL=05H (5 desempaquetado)

MUL BL

AX=0014H (20 binario)

BL=05H

AAM

AH=02H (2 desempaquetado)

AL=00H (0 desempaquetado)

BL=05H

AAS Ajuste ASCII en resta (Ascii Adjust for Subtraction)

Formato: AAS

Descripción: Ajusta el resultado en AL de la resta de dos números decimales desempaquetados, convirtiéndolo en un valor decimal desempaquetado.

Lógica: Si parte baja de AL > 9 o AF=1

AL=AL-6 , AH=AH-1 , AF=1

CF=AF

Parte alta de AL=0H

El ajuste consiste en dejar el resultado de la resta en los 4 bits menos significativos de AL, rellenando con ceros los 4 bits más significativos.

Banderas: OF DF IF TF SF ZF AF PF CF
 ? - - - ? ? * ? *

Ejemplo 1: AL=08H , AH=00H , BL=06H

SUB AL,BL
 AL=02H AF=0
 BL=06H

AAS
 AX=0002H AF=0 CF=0
 BL=06H

Ejemplo 2: AL=08H , AH=01H , BL=09H

SUB AL,BL
 AL=FFH AF=1
 BL=09H

AAS
 AX=0009H AF=1 CF=1
 BL=09H

ADC Sumar con acarreo (ADd with Carry)

Formato: ADC destino, fuente

Descripción: Efectúa la suma con acarreo de los dos operandos, y almacena el resultado en el operando destino. El operando destino puede ser un registro o una variable de memoria, y el operando fuente puede ser un registro, una variable de memoria o un valor inmediato. No se pueden sumar dos variables de memoria

La suma con acarreo significa que, a la suma de ambos operandos le agrega el valor de la bandera de acarreo (CF) de la ultima instrucción ejecutada. ADC es muy útil para sumar cantidades de mayor precisión que 16 bits.

Lógica: destino = destino + fuente

destino = destino + CF

Si CF=1, entonces el resultado de la suma se incrementa en 1; si CF=0 el resultado de la suma no se modifica.

Banderas: OF DF IF TF SF ZF AF PF CF
 * - - - * * * * *

Ejemplo 1: AL=FEH

BL=12H
 CF=0

ADC AL,BL
 AL=10H
 BL=12H
 CF=1

En este ejemplo, el resultado de ADC es igual al que se produciría de usar ADD pues, antes de la instrucción CF es 0. ADC agrega al resultado el valor actual de CF, es decir el que quedó establecido tras la ejecución de la ultima instrucción.

Ejemplo 2: Suma de dos operandos de 32 bits cada uno:

operando fuente : E2A47692H

operando destino: 1265C524H

El operando fuente está almacenado en los registros AX (parte baja) y BX (parte alta), y el operando destino está almacenado en los registros CX (parte baja) y DX (parte alta). Para efectuar esta suma, primero se deben sumar las partes bajas de ambos operandos con una operación ADD, y luego efectuar la suma con acarreo de las partes altas.

AX=7692H
 BX=E2A4H
 CX=C524H
 DX=1265H
 CF=0

ADD AX,CX ;sumar las partes bajas
 AX=3BB6H

CX=C524H
 CF=1
 ADC BX,DX ;sumar las partes altas
 BX=F50AH
 DX=1265H
 CF=0

El resultado de la suma es F50A3BB6H. La parte baja (3BB6H) está almacenada en el registro AX, y la parte alta (F50AH) está almacenada en el registro BX.

ADD Suma (ADDITION)

Formato: ADD destino, fuente

Descripción: Efectúa la suma de los dos operandos, y almacena el resultado en el operando destino.

Los operandos pueden ser bytes o palabras, dependiendo de la especificación de los mismos.

El operando destino puede ser un registro o una variable de memoria, y el operando fuente puede ser un registro, una variable de memoria o un valor inmediato. No se pueden sumar dos variables de memoria

Lógica: destino = destino + fuente

Banderas: OF DF IF TF SF ZF AF PF CF
 * - - - * * * * *

Ejemplo 1: VAR=10H

ADD VAR,10H
 VAR=20H

Ejemplo 2: AX=1200H

BX=31ABH
 ADD BX,AX
 AX=1200H
 BX=43ABH

Ejemplo 3: AL=12H
 VAR=20H

ADD AL,VAR
 AL=32H
 VAR=12H

Ejemplo 4: AL=12H
 VAR=20H

ADD VAR,AL
 AL=12H
 VAR=32H

Ejemplo 5: AL=12H

ADD AL,A8H
 AL=BAH

AND Y lógico (Logical AND)

Formato: AND destino, fuente

Descripción: Efectúa la operación de Y lógico entre dos operandos y almacena el resultado en el operando destino. Los operandos pueden tener tipo byte o palabra, y ser dos registros, un registro y una variable de memoria, un registro y una constante o una variable de memoria y una constante, pero no pueden ser, simultáneamente, dos variables de memoria.

Lógica: destino=destino AND fuente

CF=0

OF=0

AND opera a nivel de bits: fija en 1 el bit del operando destino, por cada posición en que sean simultáneamente 1 los respectivos bits de los operandos fuente y destino; para cualquier otra combinación, el bit de resultado es 0. La siguiente, es la tabla del AND para las distintas combinaciones de bits:

OP1	OP2	AND
0	0	0
0	1	0
1	0	0
1	1	1

Como regla general, se puede establecer que los bits testeados con un 0 se convierten en cero, y los bits testeados con un 1 no modifican su valor. AND fija en 0 las banderas OF (overflow) y CF (acarreo), dado que ninguna operación lógica produce overflow y acarreo.

Banderas: OF DF IF TF SF ZF AF PF CF
0 - - - * * ? * 0

Ejemplo 1:

AX=FEDCH = 1111 1110 1101 1100B
BX=1234H = 0001 0010 0011 0100B
AND AX,BX
AX=1214H = 0001 0010 0001 0100B

Ejemplo 2: BX=E455H = 1110 0100 0101 0101B
MEM=0259H = 0000 0010 0101 1001B
AND BX,MEM
BX=0051H = 0000 0000 0101 0001B

Ejemplo 3: AL=CCH = 1100 1100B
MEM=59H = 0101 1001B
AND MEM,AL
MEM=48H = 0100 1000B

Ejemplo 4: BH=E4H = 1110 0100B
AND BH,0AH
BH=00H = 0000 0000B

Ejemplo 5: MEM=02H = 0000 0010B
AND MEM,FFH
MEM=02H = 0000 0010B

CALL LLamar a un procedimiento (CALL a procedure

Formato: CALL destino

Descripción: Bifurca incondicionalmente a un procedimiento, salvando previamente en el stack la dirección de la próxima instrucción (dirección de retorno), para continuar con su ejecución después de ejecutar el procedimiento. El procedimiento llamado puede ser:

NEAR: significa que está en el mismo segmento que el procedimiento llamador; en este caso, se almacena en el stack solo el desplazamiento de la próxima instrucción.

FAR: significa que está en un segmento distinto al del procedimiento llamador; en este caso, se almacenan en el stack el segmento y el desplazamiento, en ese orden, de la próxima instrucción.

A su vez, la llamada puede ser directa o indirecta.

Una llamada directa es cuando se referencia a un rótulo relativo a CS y no a variables. En este caso, se supone siempre que el procedimiento llamado es de tipo NEAR a menos que se especifique FAR en la declaración del destino. Una llamada indirecta es cuando se referencia a variables de memoria o a registros.

En el caso de variables de memoria se puede especificar WORD PTR para llamadas a procedimientos NEAR o DWORD PTR para llamadas a procedimientos FAR. Si se indica una variable WORD, el contenido

de esa palabra de memoria sera el desplazamiento de la dirección de bifurcación; si se indica una variable DWORD, la primera palabra contendrá el segmento y la segunda el desplazamiento de la dirección de bifurcación. En el caso de registros, los mismos deben ser de 16 bits y pueden estar entre corchetes. Indicarán, en forma indirecta, el desplazamiento de la dirección de bifurcación.

Lógica: Si procedimiento es FAR

SP=SP-2

CS --> stack

CS=segmento de destino

SP=SP-2

IP --> stack

IP=desplazamiento de destino

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: CALL PROC1 ;llamada NEAR directa

Ejemplo 2: CALL WORD PTR MEM ;llamada NEAR indirecta con variable de memoria

Ejemplo 3: CALL DWORD PTR MEM ;llamada FAR indirecta con variable de memoria

Ejemplo 4: CALL WORD PTR [BX][SI] ;llamada NEAR indirecta con registros

Ejemplo 5: CALL DWORD PTR [BX] ;llamada FAR indirecta con registro

Ejemplo 6: CALL BX ;llamada NEAR indirecta con registro

Ejemplo 7: CALL DWORD PTR SS:MEM[SI] ;llamada FAR indirecta con registros

CBW Convertir byte en palabra (Convert Byte to Word)

Formato: CBW

Descripción: Transforma a palabra el contenido del registro AL y almacena el resultado en el registro AX. Esta instrucción asume que el contenido de AL es un número con signo. para efectuar la conversión, copia el bit 7 de AL en todos los bits de AH; es decir, expande el signo de AL.

CBW es especialmente útil para efectuar operaciones aritméticas entre un byte y una palabra.

Lógica:

bit 0 de AH=bit 7 de AL

bit 1 de AH=bit 7 de AL

bit 2 de AH=bit 7 de AL

bit 3 de AH=bit 7 de AL

bit 4 de AH=bit 7 de AL

bit 5 de AH=bit 7 de AL

bit 6 de AH=bit 7 de AL

bit 7 de AH=bit 7 de AL

Si AL es menor que 80H, AH ser fijado en 00H.

Si AL es mayor que 7FH, AH ser fijado en FFH.

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: AL=C2H

CBW

AX=FFC2H

Ejemplo 2: AL=4AH

BX=354AH

CBW

AX=004AH

ADD BX,AX
BX=3594H

CLC Borrar bandera de acarreo (CLear Carry flag)

Formato: CLC

Descripción: Pone en cero la bandera de acarreo (CF), sin afectar ninguna otra bandera.

Lógica: CF=0

Banderas: OF DF IF TF SF ZF AF PF CF
 - - - - - - - - 0

Ejemplo: CLC
 CF=0

CLD Borrar bandera de dirección (CLear Direction flag)

Formato: CLD

Descripción: Pone en cero la bandera de dirección (DF), sin afectar ninguna otra bandera.

Luego de ejecutarse esta instrucción, los string serán tratados desde la dirección mas alta a la dirección mas baja o, lo que es lo mismo, de izquierda a derecha. Esto significa que los registros índice SI y DI, utilizados para el manejo de string, se autoincrementarán automáticamente después de cada operación de string. Lógica:

DF=0

Banderas: OF DF IF TF SF ZF AF PF CF
 - 0 - - - - - - -

Ejemplo: CLD
 DF=0

CLI Borrar bandera de interrupciones (CLear Interrupt flag)

Formato: CLI

Descripción: Pone en cero la bandera de interrupciones externas (IF), sin afectar ninguna otra bandera. Después de ejecutarse esta instrucción, ninguna interrupción externa o enmascarable será atendida por el microprocesador. CLI es especialmente útil para ejecutar una tarea de alta prioridad, que no debe ser interrumpida. También se la utiliza en las rutinas de atención de interrupciones para evitar que estas sean interrumpidas mientras se están ejecutando.

Lógica: IF=0

Banderas: OF DF IF TF SF ZF AF PF CF
 - - 0 - - - - - -

Ejemplo: CLI
 IF=0

CMC Complementar bandera de acarreo (CoMplement Carry flag)

Formato: CMC

Descripción: Cambia el valor de la bandera de acarreo (CF) por su complemento, sin alterar ninguna otra bandera. Lógica:

CF=CF

Si CF es cero, CF es fijada en uno.

Si CF es uno, CF es fijada en cero.

Banderas: OF DF IF TF SF ZF AF PF CF
 - - - - - - - *

Ejemplo 1: CF=0
 CMC

 CF=1

Ejemplo 2: CF=1
 CMC

 CF=0

CMP Comparar dos operandos (CoMPare two operands)

Formato: CMP destino, fuente

Descripción: Efectúa la resta del operando fuente al operando destino, pero no almacena el resultado. Esta instrucción fija las banderas de estado, de modo que puedan ser consultadas por alguna de las instrucciones de bifurcación condicional para verificar el resultado. Los operandos pueden ser de tipo byte o palabra, pero ambos deben ser del mismo tipo. Se pueden comparar registro con registro, registro con memoria, registro con valor inmediato y memoria con registro. En cambio, no se pueden comparar dos posiciones de memoria, memoria con valor inmediato ni dos valores inmediatos.

Lógica: destino - fuente

Banderas: OF DF IF TF SF ZF AF PF CF

 * _ _ _ * * * * *

Ejemplo: AX=40H

 BX=30H

 CMC AX,BX

 JL MENOR ;bifurca a MENOR si AX < BX

CMPS Comparar string (CoMPare String)

Formato: CMPS string_destino,string_fuente

Descripción: Compara un elemento, de tipo byte o palabra, del string fuente con un elemento del string destino, y fija las banderas de estado de acuerdo al resultado de la operación. El string fuente se halla en el segmento de datos, y su dirección es obtenida con el registro DS para el segmento y con el registro SI para el desplazamiento. El string destino se debe definir en el segmento auxiliar, y su dirección esta dada por el registro ES para el segmento y por el registro DI para el desplazamiento. En realidad, la instrucción CMPS resta el operando destino del operando fuente y fija las banderas de estado sin almacenar el resultado de la operación. Observar que la resta es fuente-destino, es decir, en el sentido inverso al de la instrucción CMP, donde se hace destino - fuente. Esto se debe tener en cuenta al usar la instrucción de bifurcación condicional posterior. Después de la comparación los registro DI y SI se actualizan automáticamente, de modo que quedan apuntando a los próximos elementos de ambos string. Esta actualización depende del estado de la bandera de dirección:

- Si DF=0, DI y SI se incrementan;
- Si DF=1, DI y SI se decrementan.

Por otra parte, la cantidad de unidades en que se incrementan o decrementan DI y SI depende del tipo de los operandos comparados:

- Si los operandos son de tipo BYTE, DI y SI se actualizan en 1;
- Si los operandos son de tipo WORD, DI y SI se actualizan en 2.

CMPS se puede utilizar con alguno de los prefijos de repetición (REP, REPZ, REPNE, REPNZ) para comparar dos string completos de bytes o de palabras. En este caso, el número de elementos a comparar se debe especificar en el registro CX. Es importante resaltar que los operandos especificados en CMPS son utilizados por el compilador únicamente para verificar el tipo de operandos a comparar y para ver si se indica algún registro de segmento. CMPS realiza la operación DS:SI-ES:DI sin usar realmente los operandos de la instrucción, por lo que antes se deben inicializar los registros DI y SI con los valores correspondientes. Otra observación a considerar es que se puede resignar el segmento del operando fuente pero no el del operando destino. Por ejemplo:

 CMPS DESTINO,ES:FUENTE

En este caso, se efectúa una comparación entre dos elementos dentro del segmento direccionado por ES (tener en cuenta que DESTINO est direccionado por ES). Si se desean comparar dos elementos dentro del segmento direccionado por DS, habría que hacer previamente la asignación ES=DS.

Lógica: string _ fuente - string _ destino

Si tipo de operandos es BYTE, valor=1

Si tipo de operandos es WORD, valor=2

Si DF=0

SI=SI+valor

DI=DI+valor

Si DF=1

SI=SI-valor

DI=DI-valor

Banderas: OF DF IF TF SF ZF AF PF CF

* - - - * * * * *

Ejemplo 1: MOV SI,ES:FUENTE ;SI=despl. del string FUENTE

MOV DI,ES:DEST ;DI=despl. del string DEST

CMPS DEST,FUENTE ;Compara un elemento de FUENTE con DEST

JNE DISTINTO ;Bifurca si no son iguales

Ejemplo 2: LEA SI,FUENTE ;SI=despl. del string FUENTE

LEA DI,DEST ;DI=despl. del string DEST

CMPS [DI],[SI] ;Compara un elemento de FUENTE con DEST

JNE DISTINTO ;Bifurca si no son iguales

Este ejemplo es similar al anterior, pero usando DI y SI explícitamente.

Ejemplo 3:

;Comparar 100 elementos del string FUENTE, en el segmento ;de datos, con el string DEST, en el segmento extra.

;

CLD ;Manejo de strings de izquierda a derecha

LEA SI,FUENTE ;SI=despl. del string FUENTE

LEA DI,DEST ;DI=despl. del string DEST

MOV CX,100 ;CX=cantidad de elementos

REPE CMPS [DI],[SI] ;Compara 100 elementos

JNE DISTINTO ;Bifurca si no son iguales

Ejemplo 4:

;Comparar 100 elementos del string FUENTE, en el segmento

;de datos, con el string DEST, también en el segmento de datos.

;

CLD ;Manejo de strings de izquierda a derecha

PUSH DS ;

POP ES ;ES=DS

LEA SI,FUENTE ;SI=despl. del string FUENTE

LEA DI,DEST ;DI=despl. del string DEST

MOV CX,100 ;CX=cantidad de elementos

REPE CMPS [DI],[SI] ;Compara 100 elementos

JNE DISTINTO ;Bifurca si no son iguales

CMPSB Comparar string de bytes (CoMPare Byte String)

Formato: CMPSB

Descripción: Es una particularización de la instrucción CMPS para operandos de tipo byte. Para esta instrucción son válidas las mismas consideraciones que para CMPS.

Lógica: DS[SI]-ES[DI]

Si DF=0

SI=SI+1

DI=DI+1

Si DF=1

SI=SI-1

DI=DI-1

Banderas: OF DF IF TF SF ZF AF PF CF

* - - - * * * * *

Ejemplo 1:

;Comparar 100 bytes del string FUENTE, en el segmento de datos, con el string DEST, en el segmento extra.

```
CLD                ;Manejo de strings de izquierda a derecha
LEA  SI,FUENTE     ;SI=despl. del string FUENTE
LEA  DI,DEST       ;DI=despl. del string DEST
MOV  CX,100        ;CX=cantidad de elementos
REPE CMPSB        ;Compara 100 bytes
JNE  DISTINTO      ;Bifurca si no son iguales
```

Ejemplo 2:

;Comparar 100 bytes del string FUENTE, en el segmento de datos, con el string DEST, tambien en el
segmento de datos

```
CLD                ;Manejo de strings de izquierda a derecha
PUSH DS
POP  ES            ;ES=DS
LEA  SI,FUENTE     ;SI=despl. del string FUENTE
LEA  DI,DEST       ;DI=despl. del string DEST
MOV  CX,100        ;CX=cantidad de elementos
REPE CMPSB        ;Compara 100 bytes
JNE  DISTINTO      ;Bifurca si no son iguales
```

CMPSW Comparar string de palabras (CoMPare Word String)

Formato: CMPSW

Descripción: Es una particularización de la instrucción CMPS para operandos de tipo palabra.

Para esta instrucción son validas las mismas consideraciones que para CMPS.

Lógica: DS[SI]-ES[DI]

Si DF=0

SI=SI+2
DI=DI+2

Si DF=1

SI=SI-2
DI=DI-2

Banderas: OF DF IF TF SF ZF AF PF CF

* - - - * * * * *

Ejemplo 1:

;Comparar 100 palabras del string FUENTE, en el segmento de datos, con el string DEST, en el segmento
extra.

```
CLD                ;Manejo de strings de izquierda a derecha
LEA  SI,FUENTE     ;SI=despl. del string FUENTE
LEA  DI,DEST       ;DI=despl. del string DEST
MOV  CX,100        ;CX=cantidad de elementos
REPE CMPSW        ;Compara 100 palabras
JNE  DISTINTO      ;Bifurca si no son iguales
```

Ejemplo 2:

;Comparar 100 palabras del string FUENTE, en el segmento de datos, con el string DEST, tambien en el
segmento de datos.

```
CLD                ;Manejo de strings de izquierda a derecha
PUSH DS
POP  ES            ;ES=DS
LEA  SI,FUENTE     ;SI=despl. del string FUENTE
LEA  DI,DEST       ;DI=despl. del string DEST
MOV  CX,100        ;CX=cantidad de elementos
REPE CMPSW        ;Compara 100 palabras
JNE  DISTINTO      ;Bifurca si no son iguales
```

CWD convertir palabra en doble palabra(Convert Word to Doubleword)

Formato: CWD

Descripción: Transforma a doble palabra el contenido del registro AX y almacena el resultado en los registros DX (parte alta) y AX (parte baja). Esta instrucción asume que el contenido de AX es un número con signo. para efectuar la conversión, copia el bit 15 de AX en todos los bits de DX; es decir, expande el signo de AX. CWD es especialmente útil para efectuar operaciones aritmeticas entre una palabra y una doble palabra.

Lógica:

bit 0 de DX=bit 15 de AX
bit 1 de DX=bit 15 de AX
bit 2 de DX=bit 15 de AX
bit 3 de DX=bit 15 de AX
bit 4 de DX=bit 15 de AX
bit 5 de DX=bit 15 de AX
bit 6 de DX=bit 15 de AX
bit 7 de DX=bit 15 de AX
bit 8 de DX=bit 15 de AX
bit 9 de DX=bit 15 de AX
bit 10 de DX=bit 15 de AX
bit 11 de DX=bit 15 de AX
bit 12 de DX=bit 15 de AX
bit 13 de DX=bit 15 de AX
bit 14 de DX=bit 15 de AX
bit 15 de DX=bit 15 de AX

Si AX es menor que 8000H, DX ser fijado en 0000H.

Si AX es mayor que 7FFFH, DX ser fijado en FFFFH.

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: AX=C24AH

CWD

DX=FFFF AX=C24AH

Ejemplo 2: AX=4A11H

BX=0035H

CWD

DX=0000H AX=4A11H

IDIV BX

AX=0165H (cociente)

DX=0028H (resto)

DAA Ajuste decimal en suma (Decimal Adjust for Addition)

Formato: DAA

Descripción: Ajusta el resultado de una suma decimal empaquetada. DAA se utiliza cuando se efectúa una operación de suma entre dos operandos decimales empaquetados, de modo que el resultado sea tambien un decimal empaquetado. Se asume que el resultado de la suma se encuentra en el registro AL.

Lógica: Si parte baja de AL > 9 o AF=1

AL=AL+06H

AF=1

Si AL > 99 o CF=1

AL=AL+60H

CF=1

El ajuste consiste en colocar el dígito menos significativo de la suma en los 4 bits de menor orden de AL, y en colocar el dígito mas significativo de la suma en los 4 bits de mayor orden de AL.

Banderas: OF DF IF TF SF ZF AF PF CF

? - - - * * * * *

Ejemplo 1: AL=68H
BL=17H
ADD AL,BL
AL=7FH AF=0
DAA
AL=85H AF=1
Ejemplo 2: AL=51H
BL=78H
ADD AL,BL
AL=C9H CF=0
DAA
AL=29H CF=1

DAS Ajuste decimal en resta (Decimal Adjust for Subtraction)

Formato: DAS

Descripción: Ajusta el resultado de una resta decimal empaquetada; es la equivalente para la resta a la instrucción de ajuste decimal en suma (DAA). DAS se utiliza cuando se efectúa una operación de resta entre dos operandos decimales empaquetados, de modo que el resultado sea también un decimal empaquetado. Se asume que el resultado de la resta se encuentra en el registro AL.

Lógica: Si parte baja de AL > 9 o AF=1

AL=AL-06H

AF=1

Si AL > 99 o CF=1

AL=AL-60H

CF=1

El ajuste consiste en colocar el dígito menos significativo de la resta en los 4 bits de menor orden de AL, y en colocar el dígito más significativo de la resta en los 4 bits de mayor orden de AL.

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - * * * * *

Ejemplo 1: AL=55H
BL=26H
SUB AL,BL
AL=2FH AF=1
DAS
AL=29H AF=1

Ejemplo 2: AL=59H
BL=68H
SUB AL,BL
AL=F1H CF=1
DAS
AL=91H CF=1

El resultado de la resta es un número negativo. AL representa el decimal desempaqueado del complemento del resultado; es decir que el resultado de la resta es de -9.

DEC Decrementar destino en uno (DECrement destination by one)

Formato: DEC destino

Descripción: Decrementa en 1 el contenido del operando destino, que puede ser un registro o una variable de memoria, de tipo BYTE o WORD.

Lógica: destino=destino-1

Banderas: OF DF IF TF SF ZF AF PF CF

* - - - * * * * -

Ejemplo 1: AX=0000H
DEC AX

AX=FFFFH

Ejemplo 2: MEM=23H
DEC MEM
MEM=22H

DIV División sin signo (DIVision, unsigned)

Formato: DIV fuente

Descripción: Divide, sin considerar el signo, el contenido del dividendo por el operando fuente (divisor), y devuelve el cociente y el resto. El operando fuente puede ser un registro o una variable de memoria, de tipo BYTE o WORD. El dividendo es siempre del doble de longitud que el divisor: si el divisor es de tipo BYTE, el dividendo esta en AX; si el divisor es de tipo WORD, el dividendo esta en DX (parte alta) y AX (parte baja). Si el divisor es de tipo BYTE, el cociente se almacena en el registro AL y el resto en el registro AH; si el divisor es de tipo WORD, el cociente se almacena en el registro AX y el resto en el registro DX. Si el cociente es mayor que el valor máximo que se puede almacenar según si el divisor es de tipo BYTE (FFH) o de tipo WORD (FFFFH), entonces tanto el cociente como el resto quedan indefinidos y se genera una interrupción 00H (error en división).

Se produce overflow en una división sin signo cuando:

- el divisor es cero;
- el dividendo es 256 veces más grande que el divisor (divisor tipo BYTE);
- el dividendo es 65.536 veces más grande que el divisor (divisor tipo WORD).

Lógica: Si fuente tipo BYTE

AX/fuente

AL=cociente

AH=resto

Si cociente > FFH

INT 00H

Si fuente tipo WORD

DX,AX/fuente (DX=parte alta, AX=parte baja)

AX=cociente

DX=resto

Si cociente > FFFFH

INT 00H

Banderas: OF DF IF TF SF ZF AF PF CF

? - - - ? ? ? ? ?

Ejemplo 1: AX=0013H = 19
BL=02H = 2

DIV BL

AX=0109H

AL=09H = 9 (cociente)

AH=01H = 1 (resto)

Ejemplo 2: DX,AX=00000013H = 19
BX=0002H = 2

DIV BX

DX,AX=00010009H

AX=0009H = 9 (cociente)

DX=0001H = 1 (resto)

ESC Escape (ESCape)

Formato: ESC código de operación externo, fuente

Descripción: Permite la comunicación con otros coprocesadores a través del envío al bus de datos del código de operación y del contenido del operando fuente. El microprocesador INTEL 8088-8086 está diseñado para operar con una variedad de coprocesadores que ejecutan diferentes funciones como, por ejemplo, el 8087 (coprocesador matemático) y el 8089 (coprocesador de E/S). ESC se utiliza para indicarle a

un coprocesador que comience a realizar alguna tarea; mientras tanto, el 8088-8086 continua ejecutando la instrucción siguiente. Es decir que, a menos que se le indique otra cosa, habría dos procesadores trabajando simultaneamente. Por este motivo es que ESC se utiliza en combinación con la instrucción WAIT, de modo que el 8088-8086 pare y espere que el coprocesador termine con la instrucción en curso. Normalmente, WAIT se especifica delante de ESC para asegurarse que el código de operación sea recogido por el coprocesador. El primer operando es un valor inmediato de 6 bits (00H a 3FH), y sirve para especificar a la vez el coprocesador y el código de operación. El segundo operando es un registro o una variable de memoria. Si es un registro, ESC no hace nada; si es una variable de memoria, ESC envía al bus de datos el contenido de esa posición de memoria. Entonces, esta instrucción puede servir para mandarle a un coprocesador una instrucción y su operando.

Lógica: Si fuente es un registro NOP

Si fuente es una variable de memoria contenido de fuente --> bus de datos

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo: WAIT ;poner al procesador en estado IDLE

ESC 21,TABLA ;bus de datos = contenido de TABLA

HLT Alto (HaLT)

Formato: HLT

Descripción: Detiene el funcionamiento del microprocesador. Este puede volver a activarse si:

- se restaura el sistema (RESET);
- se recibe una interrupción externa, siempre que esten habilitadas las interrupciones enmascarables (bandera IF=1).

La instrucción HLT puede ser usada, por ejemplo, para detener el funcionamiento del microprocesador hasta que se introduzca un caracter por el teclado (interrupción externa).

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo: HLT ;parar el microprocesador

IDIV División con signo (Integer DIVision, signed)

Formato: IDIV fuente

Descripción: Divide, considerando el signo, el contenido del dividendo por el operando fuente (divisor), y devuelve el cociente y el resto. El operando fuente puede ser un registro o una variable de memoria, de tipo BYTE o WORD. El dividendo es siempre del doble de longitud que el divisor: si el divisor es de tipo BYTE, el dividendo esta en AX; si el divisor es de tipo WORD, el dividendo esta en DX (parte alta) y AX (parte baja). Si el divisor es de tipo BYTE, el cociente se almacena en el registro AL y el resto en el registro AH; si el divisor es de tipo WORD, el cociente se almacena en el registro AX y el resto en el registro DX. Si el cociente es positivo y mayor que el valor maximo o negativo y menor que el valor m;nimo que se puede almacenar segun si el divisor es de tipo BYTE (7FH y 81H, respectivamente) o de tipo WORD (7FFFH y 8001H, respectivamente), entonces tanto el cociente como el resto quedan indefinidos y se genera una interrupción 00H (error en división).

Se produce overflow en una división con signo cuando:

- el divisor es cero
- el cociente es mayor que +127 o menor que -128 (divisor tipo BYTE);
- el cociente es mayor que +32.767 o menor que -32.768 (divisor tipo WORD).

Lógica: Si fuente tipo BYTE

AX/fuente

AL=cociente

AH=resto

Si cociente > 7FH o cociente < 81H INT 00H

Si fuente tipo WORD

DX,AX/fuente (DX=parte alta, AX=parte baja)

AX=cociente

DX=resto
Si cociente > 7FFFH o cociente < 8001H INT 00H

Banderas: OF DF IF TF SF ZF AF PF CF
? - - - ? ? ? ? ?

Ejemplo 1: AX=FFEDH = -19
BL=02H = 2

IDIV BL
AX=01F7H
AH=01H = 1 (resto)
AL=F7H = -9 (cociente)

Ejemplo 2: DX,AX=FFFFFFEDH = -19
BX=0002H = 2

IDIV BX
DX,AX=0001FFF7H
DX=0001H = 1 (resto)
AX=FFF7H = -9 (cociente)

IMUL Multiplicación con signo (Integer MULtiply)

Formato: IMUL fuente

Descripción: Multiplica, considerando el signo, el contenido del acumulador por el operando fuente y devuelve el resultado del producto. El operando fuente puede ser un registro o una variable de memoria, de tipo BYTE o WORD. Dependiendo del tipo del operando fuente, se considerara la parte del acumulador a multiplicar: si es de tipo BYTE, se toma AL; si es de tipo WORD, se toma AX. El resultado de la multiplicación es siempre del doble de longitud que el operando fuente: si es de tipo BYTE, el resultado se almacenara en el registro AX; si es de tipo WORD, el resultado se almacenara en DX (parte alta) y AX (parte baja).

Lógica: Si fuente tipo BYTE

AX=AL*fuentes

Si fuente tipo WORD

DX,AX=AX*fuentes (DX=parte alta, AX=parte baja)

Si parte alta del resultado = expansión del signo de la parte baja

CF=0

Si parte alta del resultado <> expansión del signo de la parte baja

CF=1

OF=CF

Banderas: OF DF IF TF SF ZF AF PF CF
* - - - ? ? ? ? *

Ejemplo 1: AL=FEH = -2
BL=12H = 18

IMUL BL
AX=FFDCH = -36 CF=0 OF=0

Ejemplo 2: AL=7FH = 127
BL=12H = 18

IMUL BL
AX=08EEH = 2.286 CF=1 OF=1

Ejemplo 3: AX=FFFEH = -2
BX=0012H = 18

IMUL BX
DX,AX=FFFFFFDCH = -36 CF=0 OF=0

Ejemplo 4: AX=7FFFH = 32.767
BX=0012H = 18

IMUL BX
DX,AX=0008FFEEH = 589.806 CF=1 OF=1

IN Entrada de byte o palabra (INput byte or word)

Formato: IN acumulador, puerta E/S

Descripción: Transfiere un dato al acumulador desde la puerta de E/S al procesador indicado. En acumulador se debe indicar AL o AX, dependiendo de si el dato a transferir es un byte o una palabra, respectivamente. La puerta de E/S puede ser:

- un valor fijo entre 0 y 255;
- un valor variable contenido en el registro DX; se pueden direccionar 64K puertas (0 a 65.535).

Lógica: Si acumulador es AL

AL = dato de la puerta de E/S

Si acumulador es AX

AX = dato de la puerta de E/S

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: DX=0034H

IN AX,DX ;toma una palabra desde la puerta 34H y la almacena en el registro AX.

Ejemplo 2: IN AL,200H ;toma un byte desde la puerta 200 y lo almacena en el registro AL.

INC Incrementar destino en uno (INCrement destination by one)

Formato: INC destino

Descripción: Suma una unidad al operando destino, que puede ser un registro o una variable de memoria de tipo BYTE o WORD. La instrucción INC es muy útil para el manejo de contadores de ciclos y para acceder a posiciones consecutivas de memoria.

Lógica: destino=destino+1

Banderas: OF DF IF TF SF ZF AF PF CF

* - - - * * * * -

Ejemplo 1: CX=AAFFH

INC CX

CX=AB00H

Ejemplo 2: MEM=19H

INC MEM

MEM=1AH

INT Interrupción (INTerrupt)

Formato: INT código _ interrupción

Descripción: Genera una interrupción al microprocesador y obtiene, del vector de interrupciones correspondiente al código de interrupción especificado, la dirección de la rutina de atención de esa interrupción. Esa dirección es cargada en los registros CS e IP. El código de interrupción es un dato inmediato, cuyo valor puede ser entre 0 y 255. Después de ejecutarse la instrucción INT, el microprocesador comienza la ejecución de la rutina de atención de la interrupción. En el capítulo 12 se estudian en detalle el proceso de interrupción y los distintos tipos de interrupciones soportados por el microprocesador INTEL 8088-8086.

Lógica: SP=SP-2

registro de banderas ---> stack

IF=0

TF=0

SP=SP-2

CS ---> stack

CS= código _ interrupción x 4 + 2

SP=SP-2

IP ---> stack

IP= código_interrupción x 4

Banderas: OF DF IF TF SF ZF AF PF CF
 - - 0 0 - - - -

Ejemplo 1: MOV AX,15H

INT 21H ;Invocación a la función 15H de la interrupción 21H (llamada al MS-DOS)

Ejemplo 2: INT 03H ;Interrupción 03H

INTO Interrupción si hay overflow (Interrupt if Overflow)

Formato: INTO

Descripción: Genera una interrupción 04H si existe overflow, es decir si esta encendida la bandera de overflow (OF=1); de lo contrario, se continua con la ejecución de la próxima instrucción.

Lógica: Si OF=1

SP=SP-2

registro de banderas ---> stack

IF=0

TF=0

SP=SP-2

CS ---> stack

CS = valor contenido en la palabra de dirección 12H (4 x 4 + 2)

SP=SP-2

IP ---> stack

IP = valor contenido en la palabra de dirección 10H (4 x 4)

Esto es equivalente a:

Si OF=1

INT 04H

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo: ADD AX,BX

INTO ;Genera una interrupción 04H si se produjo overflow en la suma.

IRET Retorno de interrupción (Interrupt RETurn)

Formato: IRET

Descripción: Devuelve el control a la dirección de retorno salvada en el stack por una instrucción INT previa, y recupera el registro de banderas. IRET se utiliza para finalizar una rutina de atención de interrupción.

Lógica:

IP <--- stack

SP=SP+2

CS <--- stack

SP=SP+2

Registro de banderas <--- stack

SP=SP+2

Banderas: OF DF IF TF SF ZF AF PF CF

* * * * *

Ejemplo: IRET ;retorno de interrupción

JA Bifurcar si superior (Jump if Above)

Formato: JA desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición CF=0 (no acarreo) y ZF=0 (no cero). Si CF=1 (hay acarreo) o ZF=1 (es cero), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JA, es decir que es un valor con signo de 8 bits.

El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JA se utiliza para evaluar una relación entre dos valores sin considerar el signo.

JA es equivalente a la instrucción JNBE.

Lógica: Si CF=0 y ZF=0

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0023H

CMP AX,BX ;Comparar AX con BX

JA MAYOR ;Bifurcar a MAYOR si AX > BX

Según los valores en AX y en BX, se produce la bifurcación al rótulo MAYOR.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JA MAYOR ;Bifurcar a MAYOR si AX > BX

En este caso, también se produce la bifurcación al rótulo MAYOR, dado que AX es mayor que BX en valor absoluto y JA no evalúa la bandera de signo.

JAE Bifurcar si superior o igual (Jump if Above or Equal)

Formato: JAE desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición CF=0 (no acarreo). Si CF=1 (hay acarreo), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JAE, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JAE se utiliza para evaluar una relación entre dos valores sin considerar el signo.

JAE es equivalente a las instrucciones JNB y JNC.

Lógica: Si CF=0

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0023H BX=0023H

CMP AX,BX ;Comparar AX con BX

JAE BIEN ;Bifurcar a BIEN si AX > BX o si AX = BX

Según los valores en AX y en BX, se produce la bifurcación al rótulo BIEN.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JAE BIEN ;Bifurcar a BIEN si AX > BX o si AX = BX

En este caso, también se produce la bifurcación al rótulo BIEN, dado que AX es mayor que BX en valor absoluto y JAE no evalúa la bandera de signo.

JB Bifurcar si inferior (Jump if Below)

Formato: JB desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición CF=1 (hay acarreo). Si CF=0 (no acarreo), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JB, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JB se utiliza para evaluar una relación entre dos valores sin considerar el signo.

JB es equivalente a las instrucciones JNAE y JC.

Lógica: Si CF=1

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -
Ejemplo 1: ;AX=0045H BX=0023H

CMP BX,AX ;Comparar BX con AX

JB MENOR ;Bifurcar a MENOR si BX < AX

Según los valores en AX y en BX, se produce la bifurcación al rótulo MENOR.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JB MENOR ;Bifurcar a MENOR si AX < BX

En este caso, no se produce la bifurcación al rótulo MENOR, dado que AX es mayor que BX en valor absoluto y JB no evalúa la bandera de signo.

JBE Bifurcar si inferior o igual (Jump if Below or Equal)

Formato: JBE desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición CF=1 (hay acarreo) o ZF=1 (es cero). Si CF=0 (no acarreo) y ZF=0 (no es cero), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JBE, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JBE se utiliza para evaluar una relación entre dos valores sin considerar el signo. JBE es equivalente a la instrucción JNA.

Lógica: Si CF=1 o ZF=1

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -
Ejemplo 1: ;AX=0045H BX=0045H

CMP BX,AX ;Comparar BX con AX

JBE BIEN ;Bifurcar a BIEN si BX < AX o si BX = AX

Según los valores en AX y en BX, se produce la bifurcación al rótulo BIEN.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JBE BIEN ;Bifurcar a BIEN si AX < BX o si AX = BX

En este caso, no se produce la bifurcación al rótulo BIEN, dado que AX es mayor que BX en valor absoluto y JBE no evalúa la bandera de signo.

JC Bifurcar si acarreo (Jump if Carry)

Formato: JC desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición CF=1 (hay acarreo). Si CF=0 (no acarreo), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JC, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JC se utiliza para evaluar una relación entre dos valores sin considerar el signo. JC es equivalente a las instrucciones JNAE y JB.

Lógica: Si CF=1

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -
Ejemplo 1: ;AX=0045H BX=0023H

CMP BX,AX ;Comparar BX con AX

JC MENOR ;Bifurcar a MENOR si BX < AX

Según los valores en AX y en BX, se produce la bifurcación al rótulo MENOR.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JC MENOR ;Bifurcar a MENOR si AX < BX

En este caso, no se produce la bifurcación al rótulo MENOR, dado que AX es mayor que BX en valor absoluto y JC no evalúa la bandera de signo.

JCXZ Bifurcar si CX es cero (Jump if CX is Zero)

Formato: JCXZ desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición CX=0. Si CX≠0, no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JCXZ, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control.

Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JCXZ es especialmente útil al comienzo de un ciclo, para no ingresar al mismo si el contador (registro CX) es cero.

Lógica: Si CX=0

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo: DEC CX ;CX=CX-1

JCXZ CERO ;Bifurcar a CERO si CX=0

JE Bifurcar si igual (Jump if Equal)

Formato: JE desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición ZF=1 (es cero). Si ZF=0 (no es cero), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JE, es decir que es un valor con signo de 8 bits.

El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JE es equivalente a la instrucción JZ.

Lógica: Si ZF=1

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0045H

CMP AX,BX ;Comparar AX con BX

JE IGUAL ;Bifurcar a IGUAL si AX = BX

Según los valores en AX y en BX, se produce la bifurcación al rótulo IGUAL.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JE IGUAL ;Bifurcar a IGUAL si AX = BX

En este caso, no se produce la bifurcación al rótulo IGUAL, dado que AX es distinto que BX.

JG Bifurcar si mayor (Jump if Greater)

Formato: JG desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición ZF=0 (no es cero) y SF=OF (signo positivo y no overflow o signo negativo y overflow). Si ZF=1 (es cero) o SF=OF (signo positivo y overflow o signo negativo y no overflow), no hay transferencia de control.

El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JG, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JG se utiliza para evaluar una relación entre dos valores considerando el signo. JG es equivalente a la instrucción JNLE.

Lógica: Si ZF=0 y SF=OF

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=00A3H BX=0045H

CMP AX,BX ;Comparar AX con BX

JG MAYOR ;Bifurcar a MAYOR si AX > BX

Según los valores en AX y en BX, se produce la bifurcación al rótulo MAYOR.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JG MAYOR ;Bifurcar a MAYOR si AX > BX

En este caso, no se produce la bifurcación al rótulo MAYOR, dado que AX es menor que BX y JG evalúa la bandera de signo.

JGE Bifurcar si mayor o igual (Jump if Greater or Equal)

Formato: JGE desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición SF=OF (signo positivo y no overflow o signo negativo y overflow). Si SF=OF (signo positivo y overflow o signo negativo y no overflow), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JGE, es decir que es un valor con signo de 8 bits.

El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JGE se utiliza para evaluar una relación entre dos valores considerando el signo.

JGE es equivalente a la instrucción JNL.

Lógica: Si SF=OF

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0045H

CMP BX,AX ;Comparar BX con AX

JGE BIEN ;Bifurcar a BIEN si BX > AX o si AX = BX

Según los valores en AX y en BX, se produce la bifurcación al rótulo BIEN.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JGE BIEN ;Bifurcar a BIEN si AX > BX o si AX = BX

En este caso, no se produce la bifurcación al rótulo BIEN, dado que AX es menor que BX y JGE evalúa la bandera de signo.

JL Bifurcar si menor (Jump if Less)

Formato: JL desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición SF≠OF (signo positivo y overflow o signo negativo y no overflow). Si SF=OF (signo positivo y no overflow o signo negativo y overflow), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JL, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JL se utiliza para evaluar una relación entre dos valores considerando el signo. JL es equivalente a la instrucción JNGE.

Lógica: Si SF≠OF

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=00A3H BX=0045H

CMP BX,AX ;Comparar BX con AX

JL MENOR ;Bifurcar a MENOR si BX < AX

Según los valores en AX y en BX, se produce la bifurcación al rótulo MENOR.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JL MENOR ;Bifurcar a MENOR si AX < BX

En este caso, también se produce la bifurcación al rótulo MENOR, dado que AX es menor que BX y JLE evalúa la bandera de signo.

JLE Bifurcar si menor o igual (Jump if Less or Equal)

Formato: JLE desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición ZF=1 (es cero) o SF/=OF (signo positivo y overflow o signo negativo y no overflow). Si ZF=0 (no es cero) y SF=OF (signo positivo y no overflow o signo negativo y overflow), no hay transferencia de control.

El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JLE, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JLE se utiliza para evaluar una relación entre dos valores considerando el signo. JLE es equivalente a la instrucción JNG.

Lógica: Si ZF=1 o SF/=OF

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0045H

CMP AX,BX ;Comparar AX con BX

JLE BIEN ;Bifurcar a BIEN si AX < BX o si AX = BX

Según los valores en AX y en BX, se produce la bifurcación al rótulo BIEN.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JLE BIEN ;Bifurcar a BIEN si AX < BX o si AX = BX

En este caso, también se produce la bifurcación al rótulo BIEN, dado que AX es menor que BX y JLE evalúa la bandera de signo.

JMP Bifurcar (JuMP)

Formato: JMP operando

Descripción: Transfiere el control incondicionalmente a la dirección indicada en el operando.

La dirección a la cual se transfiere el control se obtiene mediante la composición de los registros CS, para el segmento, e IP, para el desplazamiento. El operando puede ser un rótulo, un registro o una variable de memoria, y puede ser expresado usando los diferentes modos de direccionamiento provistos por el microprocesador INTEL 8088-8086. Para el caso en que el operando es un rótulo, la dirección de bifurcación es la dirección de memoria donde está definido ese rótulo (bifurcación directa). Para el caso en que el operando es un registro o una variable de memoria, la dirección de bifurcación es el contenido del registro o de la variable (bifurcación indirecta). El operando determina si la bifurcación incondicional es dentro del mismo segmento (intra-segmento) o hacia otro segmento (inter-segmento). En el caso de las bifurcaciones intra-segmento, el operando modifica sólo el valor del registro IP, y puede ser:

- Un rótulo de tipo SHORT (significa que la dirección de bifurcación debe estar entre -128 y +127 bytes de distancia de la instrucción JMP);
- Un rótulo con atributo de distancia NEAR (significa que la dirección de bifurcación debe estar entre -32 y +32 Kbytes de distancia de la instrucción JMP)
- Una cantidad de 16 bits contenida en un registro o en una variable de memoria de tipo WORD.

En el caso de las bifurcaciones inter-segmento, el operando modifica el valor de IP y de CS (es decir que no tienen ninguna limitación de direccionamiento), y puede ser:

- un rótulo con atributo de distancia FAR;
- una cantidad de 32 bits indicada por dos registros;
- una cantidad de 32 bits contenida en una dirección apuntada por un registro o en una variable de

memoria de tipo DOUBLEWORD.
 Lógica: Si bifurcación intersegmento
 CS=segmento
 Si bifurcación directa
 IP=desplazamiento
 Si bifurcación indirecta
 IP= IP + desplazamiento
 Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -
 Ejemplo 1: JMP ROTULO ;Bifurcación directa
 Ejemplo 2: JMP FAR PTR ROTULO ;Bifurcación directa intersegmento
 Ejemplo 3: JMP [BX] ;Bifurcación indirecta intrasegmento
 Ejemplo 4: JMP FAR PTR [BX] ;Bifurcación indirecta intersegmento
 Ejemplo 5: SALTO DW PROC ;Desplazamiento de PROC

 JMP SALTO ;Bifurcación indirecta intrasegmento
 Ejemplo 6: SALTO DD PROC ;Dirección completa de PROC

 JMP SALTO ;Bifurcación indirecta intersegmento
 Ejemplo 7: JMP SHORT ACA ;Bifurcación directa (corta) intrasegmento

JNA Bifurcar si no superior (Jump if Not Above)

Formato: JNA desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición CF=1 (hay acarreo) o ZF=1 (es cero). Si CF=0 (no acarreo) y ZF=0 (no es cero), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JNA, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JNA se utiliza para evaluar una relación entre dos valores sin considerar el signo. JNA es equivalente a la instrucción JBE.

Lógica: Si CF=1 o ZF=1
 IP = IP + desplazamiento
 Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -
 Ejemplo 1: ;AX=0045H BX=0045H
 CMP BX,AX ;Comparar BX con AX
 JNA BIEN ;Bifurcar a BIEN si BX < AX o si BX = AX
 Según los valores en AX y en BX, se produce la bifurcación al rótulo BIEN.
 Ejemplo 2: ;AX=FFFFH BX=0000H
 CMP AX,BX ;Comparar AX con BX
 JNA BIEN ;Bifurcar a BIEN si AX < BX o si AX = BX

En este caso, no se produce la bifurcación al rótulo BIEN, dado que AX es mayor que BX en valor absoluto y JNA no evalúa la bandera de signo.

JNAE Bifurcar si no superior ni igual (Jump if Not Above nor Equal)

Formato: JNAE desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición CF=1 (hay acarreo). Si CF=0 (no acarreo), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JNAE, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se

debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JNAE se utiliza para evaluar una relación entre dos valores sin considerar el signo.

JNAE es equivalente a las instrucciones JB y JC.

Lógica: Si CF=1

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0023H

CMP BX,AX ;Comparar BX con AX

JNAE MENOR ;Bifurcar a MENOR si BX < AX

Según los valores en AX y en BX, se produce la bifurcación al rótulo MENOR.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JNAE MENOR ;Bifurcar a MENOR si AX < BX

En este caso, no se produce la bifurcación al rótulo MENOR, dado que AX es mayor que BX en valor absoluto y JNAE no evalúa la bandera de signo.

JNB Bifurcar si no inferior (Jump if Not Below)

Formato: JNB desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición CF=0 (no acarreo). Si CF=1 (hay acarreo), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JNB, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JNB se utiliza para evaluar una relación entre dos valores sin considerar el signo.

JNB es equivalente a las instrucciones JAE y JNC.

Lógica: Si CF=0

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0023H BX=0023H

CMP AX,BX ;Comparar AX con BX

JNB BIEN ;Bifurcar a BIEN si AX > BX o si AX = BX

Según los valores en AX y en BX, se produce la bifurcación al rótulo BIEN.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JNB BIEN ;Bifurcar a BIEN si AX > BX o si AX = BX

En este caso, también se produce la bifurcación al rótulo BIEN, dado que JNB no evalúa la bandera de signo.

JNBE Bifurcar si no inferior ni igual (Jump if Not Below nor Equal)

Formato: JNBE desplazamiento

Descripción: Transfiere el control a la dirección IP+desplazamiento, si se cumple la condición CF=0 (no acarreo) y ZF=0 (no cero). Si CF=1 (hay acarreo) o ZF=1 (es cero), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JNBE, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JNBE se utiliza para evaluar una relación entre dos valores sin considerar el signo. JNBE es equivalente a la instrucción JA.

Lógica: Si CF=0 y ZF=0

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0023H

CMP AX,BX ;Comparar AX con BX

JNBE MAYOR ;Bifurcar a MAYOR si AX > BX

Según los valores en AX y en BX, se produce la bifurcación al rótulo MAYOR.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JNBE MAYOR; Bifurcar a MAYOR si AX > BX

En este caso, también se produce la bifurcación al rótulo MAYOR, dado que AX es mayor que BX en valor absoluto y JNBE no evalúa la bandera de signo.

JNC Bifurcar si no acarreo (Jump if Not Carry)

Formato: JNC desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición CF=0 (no acarreo). Si CF=1 (hay acarreo), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JNC, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JNC se utiliza para evaluar una relación entre dos valores sin considerar el signo.

JNC es equivalente a las instrucciones JAE y JNB.

Lógica: Si CF=0

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0023H BX=0023H

CMP AX,BX ;Comparar AX con BX

JNC BIEN ;Bifurcar a BIEN si AX > BX o si AX = BX

Según los valores en AX y en BX, se produce la bifurcación al rótulo BIEN.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JNC BIEN ;Bifurcar a BIEN si AX > BX o si AX = BX

En este caso, también se produce la bifurcación al rótulo BIEN, dado que JNB no evalúa la bandera de signo.

JNE Bifurcar si no igual (Jump if Not Equal)

Formato: JNE desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición ZF=0 (no es cero). Si ZF=1 (es cero), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JNE, es decir que es un valor con signo de 8 bits.

El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JNE es equivalente a la instrucción JNZ.

Lógica: Si ZF=0

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0045H

CMP AX,BX ;Comparar AX con BX

JNE BIEN ;Bifurcar a BIEN si AX > BX o si AX < BX

Por ser iguales los valores en AX y en BX, no se produce la bifurcación al rótulo BIEN.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JNE BIEN ;Bifurcar a BIEN si AX > BX o si AX < BX

En este caso, sí se produce la bifurcación al rótulo BIEN, dado que AX es distinto a BX.

JNG Bifurcar si no mayor (Jump if Not Greater)

Formato: JNG desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición ZF=1 (es cero) o SF/=OF (signo positivo y overflow o signo negativo y no overflow). Si ZF=0 (no es cero) y SF=OF (signo positivo y no overflow o signo negativo y overflow), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JNG, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JNG se utiliza para evaluar una relación entre dos valores considerando el signo. JNG es equivalente a la instrucción JLE.

Lógica: Si ZF=1 o SF/=OF

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0045H

CMP AX,BX ;Comparar AX con BX

JNG BIEN ;Bifurcar a BIEN si AX < BX o si AX = BX

Según los valores en AX y en BX, se produce la bifurcación al rótulo BIEN.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JNG BIEN ;Bifurcar a BIEN si AX < BX o si AX = BX

En este caso, también se produce la bifurcación al rótulo BIEN, dado que AX es menor que BX y JNG evalúa la bandera de signo.

JNGE Bifurcar si no mayor ni igual (Jump if Not Greater nor Equal)

Formato: JNGE desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición SF/=OF (signo positivo y overflow o signo negativo y no overflow). Si SF=OF (signo positivo y no overflow o signo negativo y overflow), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JNGE, es decir que es un valor con signo de 8 bits.

El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JNGE se utiliza para evaluar una relación entre dos valores considerando el signo.

JNGE es equivalente a la instrucción JL.

Lógica: Si SF/=OF

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=00A3H BX=0045H

CMP BX,AX ;Comparar BX con AX

JNGE MENOR ;Bifurcar a MENOR si BX < AX

Según los valores en AX y en BX, se produce la bifurcación al rótulo MENOR.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JNGE MENOR ;Bifurcar a MENOR si AX < BX

En este caso, también se produce la bifurcación al rótulo MENOR, dado que AX es menor que BX y JNGE evalúa la bandera de signo.

JNL Bifurcar si no menor (Jump if Not Less)

Formato: JNL desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple SF=OF (signo positivo y no overflow o signo negativo y overflow). Si SF/=OF (signo positivo y overflow o signo negativo y no overflow), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JNL, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits.

JNL se utiliza para evaluar una relación entre dos valores considerando el signo.

JNL es equivalente a la instrucción JGE.

Lógica: Si SF=OF

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0045H

CMP BX,AX ;Comparar BX con AX

JNL BIEN ;Bifurcar a BIEN si BX > AX o si BX = AX

Según los valores en AX y en BX, se produce la bifurcación al rótulo BIEN.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JNL BIEN ;Bifurcar a BIEN si AX > BX o si AX = BX

En este caso, no se produce la bifurcación al rótulo BIEN, dado que AX es menor que BX y JNL evalúa la bandera de signo.

JNLE Bifurcar si no menor ni igual (Jump if Not Less nor Equal)

Formato: JNLE desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición ZF=0 (no es cero) y SF=OF (signo positivo y no overflow o signo negativo y overflow). Si ZF=1 (es cero) o SF/=OF (signo positivo y overflow o signo negativo y no overflow), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JNLE, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JNLE se utiliza para evaluar una relación entre dos valores considerando el signo. JNLE es equivalente a la instrucción JG.

Lógica: Si ZF=0 y SF=OF

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=00A3H BX=0045H

CMP BX,AX ;Comparar BX con AX

JNLE MAYOR ;Bifurcar a MAYOR si BX > AX

Según los valores en AX y en BX, se produce la bifurcación al rótulo MAYOR.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JNLE MAYOR ;Bifurcar a MAYOR si AX > BX

En este caso, no se produce la bifurcación al rótulo MAYOR, dado que AX es menor que BX y JNLE evalúa la bandera de signo.

JNO Bifurcar si no overflow (Jump if Not Overflow)

Formato: JNO desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición OF=0 (no hay overflow). Si OF=1 (hay overflow), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JNO, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits.

Lógica: Si OF=0

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0045H

ADD AX,BX ;AX=AX+BX

JNO NO_OVER ;Bifurcar a NO_OVER si no hay overflow

Según los valores en AX y en BX, se produce la bifurcación al rótulo NO_OVER.

Ejemplo 2: ;AX=00AAH BX=00BBH

ADD AL,BL ;AL=AL+BL

JNO NO_OVER ;Bifurcar a NO_OVER si no hay overflow

En este caso, no se produce la bifurcación al rótulo NO_OVER, dado que la operación de suma produce overflow.

JNP Bifurcar si no paridad (Jump if Not Parity)

Formato: JNP desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición PF=0 (paridad impar). Si OF=1 (paridad par), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JNP, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JNP es equivalente a la instrucción JPO.

Lógica: Si PF=0

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0045H

ADD AX,BX ;AX=AX+BX --> AX=008AH

JNP IMPAR ;Bifurcar a IMPAR si es paridad impar

Según los valores en AX y en BX, se produce la bifurcación al rótulo IMPAR pues 8AH tiene una cantidad impar de bits encendidos.

Ejemplo 2: ;AX=0045H BX=0048H

ADD AX,BX ;AX=AX+BX ---> AX=008DH

JNP IMPAR ;Bifurcar a IMPAR si es paridad impar

En este caso, no se produce la bifurcación al rótulo IMPAR, pues 8DH tiene una cantidad par de bits encendidos.

JNS Bifurcar si no signo/si positivo (Jump if Not Sign/if positive)

Formato: JNS desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición SF=0 (signo positivo). Si SF=1 (signo negativo), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JNS, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a

donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JNS evalúa sólo el signo del resultado de una operación, sin considerar la existencia o no de overflow.

Lógica: Si SF=0

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0046H BX=0045H

CMP AX,BX ;Comparar AX con BX

JNS POSI ;Bifurcar a POSI si signo positivo

Según los valores de AX y BX, se produce la bifurcación al rótulo POSI.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JNS POSI ;Bifurcar a POSI si signo positivo

En este caso, no se produce la bifurcación al rótulo POSI, dado que la comparación resulta negativa.

JNZ Bifurcar si no cero (Jump if Not Zero)

Formato: JNZ desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición ZF=0 (no es cero). Si ZF=1 (es cero), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JNZ, es decir que es un valor con signo de 8 bits.

El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JNZ es equivalente a la instrucción JNE.

Lógica: Si ZF=0

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0045H

CMP AX,BX ;Comparar AX con BX

JNZ BIEN ;Bifurcar a BIEN si AX > BX o si AX < BX

Por ser iguales los valores en AX y en BX, no se produce la bifurcación al rótulo BIEN.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JNZ BIEN ;Bifurcar a BIEN si AX > BX o si AX < BX

En este caso, si se produce la bifurcación al rótulo BIEN, dado que AX es distinto a BX.

JO Bifurcar si overflow (Jump if Overflow)

Formato: JO desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición OF=1 (hay overflow). Si OF=0 (no hay overflow), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JO, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits.

Lógica: Si OF=1

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0045H

ADD AX,BX ;AX=AX+BX

JO OVERF ;Bifurcar a OVERF si hay overflow

Según los valores en AX y en BX, no se produce la bifurcación al rótulo OVERF.

Ejemplo 2: ;AX=00AAH BX=00BBH

ADD AL,BL ;AL=AL+BL

JO OVERF ;Bifurcar a OVERF si hay overflow

En este caso, si se produce la bifurcación al rótulo OVERF, dado que la operación de suma produce overflow.

JP Bifurcar si hay paridad (Jump if Parity Odd)

Formato: JP desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición PF=1 (paridad par). Si PF=0 (paridad impar), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JP, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JP es equivalente a la instrucción JPE.

Lógica: Si PF=1

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0045H

ADD AX,BX ;AX=AX+BX --> AX=008AH

JP PAR ;Bifurcar a PAR si es paridad par

Según los valores en AX y en BX, no se produce la bifurcación al rótulo PAR pues 8AH tiene una cantidad impar de bits encendidos.

Ejemplo 2: ;AX=0045H BX=0048H

ADD AX,BX ;AX=AX+BX ---> AX=008DH

JP PAR ;Bifurcar a PAR si es paridad par

En este caso, si se produce la bifurcación al rótulo PAR, pues 8DH tiene una cantidad par de bits encendidos.

JPE Bifurcar si paridad par (Jump if Parity Even)

Formato: JPE desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición PF=1 (paridad par). Si PF=0 (paridad impar), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JPE, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JPE es equivalente a la instrucción JP.

Lógica: Si PF=1

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0045H

ADD AX,BX ;AX=AX+BX --> AX=008AH

JPE PAR ;Bifurcar a PAR si es paridad par

Según los valores en AX y en BX, no se produce la bifurcación al rótulo PAR pues 8AH tiene una cantidad impar de bits encendidos.

Ejemplo 2: ;AX=0045H BX=0048H

ADD AX,BX ;AX=AX+BX ---> AX=008DH

JPE PAR ;Bifurcar a PAR si es paridad par

En este caso, si se produce la bifurcación al rótulo PAR, pues 8DH tiene una cantidad par de bits encendidos.

JPO Bifurcar si paridad impar (Jump if Parity Odd)

Formato: JPO desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición PF=0 (paridad impar). Si PF=1 (paridad par), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JPO, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JPO es equivalente a la instrucción JNP.

Lógica: Si PF=0

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0045H

ADD AX,BX ;AX=AX+BX --> AX=008AH

JPO IMPAR ;Bifurcar a IMPAR si es paridad impar

Según los valores en AX y en BX, se produce la bifurcación al rótulo IMPAR pues 8AH tiene una cantidad impar de bits encendidos.

Ejemplo 2: ;AX=0045H BX=0048H

ADD AX,BX ;AX=AX+BX ----> AX=008DH

JPO IMPAR ;Bifurcar a IMPAR si es paridad impar

En este caso, no se produce la bifurcación al rótulo IMPAR, pues 8DH tiene una cantidad par de bits encendidos.

JS Bifurcar si signo (Jump on Sign)

Formato: JS desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición SF=1 (signo negativo). Si SF=0 (signo positivo), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JS, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JS evalúa sólo el signo del resultado de una operación, sin considerar la existencia o no de overflow.

Lógica: Si SF=1

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0046H BX=0047H

CMP AX,BX ;Comparar AX con BX

JS NEGA ;Bifurcar a NEGA si signo negativo

Según los valores de AX y BX, se produce la bifurcación al rótulo NEGA.

Ejemplo 2: ;AX=0000H BX=FFFFH

CMP AX,BX ;Comparar AX con BX

JS NEGA ;Bifurcar a NEGA si signo negativo

En este caso, no se produce la bifurcación al rótulo NEGA, dado que la comparación resulta positiva.

JZ Bifurcar si cero (Jump if Zero)

Formato: JZ desplazamiento

Descripción: Transfiere el control a la dirección IP + desplazamiento, si se cumple la condición ZF=1 (es cero). Si ZF=0 (no es cero), no hay transferencia de control. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción JZ, es decir que es un valor con signo de 8 bits.

El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. JZ es equivalente a la instrucción JE.

Lógica: Si ZF=1

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;AX=0045H BX=0045H

CMP AX,BX ;Comparar AX con BX

JZ IGUAL ;Bifurcar a IGUAL si AX = BX

Según los valores en AX y en BX, se produce la bifurcación al rótulo IGUAL.

Ejemplo 2: ;AX=FFFFH BX=0000H

CMP AX,BX ;Comparar AX con BX

JZ IGUAL ;Bifurcar a IGUAL si AX = BX

En este caso, no se produce la bifurcación al rótulo IGUAL, dado que AX es distinto que BX.

LAHF Cargar AH con banderas (Load AH from Flags)

Formato: LAHF

Descripción: Copia las banderas SF (signo), ZF (cero), AF (acarreo auxiliar), PF (paridad) y CF (acarreo) en los bits 7, 6, 4, 2 y 0, respectivamente, del registro AH. El contenido de los bits 5, 3 y 1 queda indefinido.

Esta instrucción se usa para leer el contenido de las banderas de estado y efectuar alguna operación con ellas.

Lógica: bit 7 de AH = SF

bit 6 de AH = ZF

bit 4 de AH = AF

bit 2 de AH = PF

bit 0 de AH = CF

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo: ;SF=0 ZF=0 AF=0 PF=0 CF=0

;AH=4AH

LAHF

;AH=02H

LDS Cargar registro de segmento de datos (Load Data Segment Register)

Formato: LDS destino , fuente

Descripción: Carga el contenido de la doble palabra que esta en la dirección del operando fuente en:

- el operando destino, la palabra menos significativa (dirección mas baja);
- el registro DS, la palabra mas significativa (dirección mas alta).

El operando destino debe ser un registro, pero no un registro de segmento; el operando fuente debe ser una variable de memoria de 32 bits, ya sea definida como DD o con atributo DWORD PTR.

LDS es similar a la instrucción LES, pero utilizando el registro DS.

Lógica: DS = contenido de la palabra en la dirección fuente+2

destino = contenido de la palabra en la dirección fuente

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ROTU DD MEM ;Define ROT como doble palabra, asignándole como contenido la
;dirección de MEM

...

LDS BX,ROT ;BX = desplazamiento de MEM ;DS = segmento de MEM

Ejemplo 2: ROT DW 1234H ;Define ROT como palabra, asignándole el valor 1234H

DW 5678H ;A continuación almacena una palabra con el valor 5678H

...

LDS BX,DWORD PTR ROT ;BX=5678H DS=1234H

LEA Cargar dirección efectiva (Load Effective Address)

Formato: LEA destino , fuente

Descripción: Carga el desplazamiento del operando fuente en el operando destino. El operando destino debe ser un registro, pero no un registro de segmento; el operando fuente debe ser un rótulo o una variable de memoria de tipo BYTE o WORD. La instrucción LEA es muy útil para el pasaje de parámetros por dirección a procedimientos a través del stack. Es similar al operador OFFSET, aunque este no permite ser usado con los registros índices.

Lógica: destino = desplazamiento de fuente

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo: TAB DB 10 DUP (?) ;Definición de una tabla

...

LEA AX,TAB[SI] ;AX = desplazamiento de TAB + SI

LES Cargar registro de segmento extra (Load Extra Segment Register)

Formato: LES destino,fuente

Descripción: Carga el contenido de la doble palabra que esta en la dirección del operando fuente en:

- el operando destino, la palabra menos significativa (dirección mas baja);
- el registro ES, la palabra mas significativa (dirección mas alta).

El operando destino debe ser un registro, pero no un registro de segmento; el operando fuente debe ser una variable de memoria de 32 bits, ya sea definida como DD o con atributo DWORD PTR.

LES es similar a la instrucción LDS, pero utilizando el registro ES.

Lógica: ES = contenido de la palabra en la dirección fuente+2

destino = contenido de la palabra en la dirección fuente

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ROTU DD MEM ;Define ROT como doble palabra, asignándole como contenido la dirección de MEM

...

LES BX,ROT ;BX = desplazamiento de MEM ,ES = segmento de MEM

Ejemplo 2: ROT DW 1234H ;Define ROT como palabra, asignándole el valor 1234H

DW 5678H ;A continuación almacena una palabra con el valor 5678H

...

LES BX,DWORD PTR ROT ;BX=5678H ES=1234H

LOCK Bloquear el bus (LOCK the bus)

Formato: LOCK

Descripción: LOCK no es exactamente una instrucción, sino que es un prefijo que acompaña a una instrucción y que activa la señal de LOCK mientras dicha instrucción se ejecuta. Es decir que impide la utilización del bus por otros procesadores, evitando de esa forma el acceso a los recursos compartidos por los procesadores. En sistemas de multiprocesadores, con recursos compartidos, siempre es necesario un mecanismo de este tipo para el control del acceso a los recursos. LOCK es, en esencia, una herramienta para el manejo de semáforos.

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo: El siguiente ejemplo muestra la utilización de LOCK para impedir el acceso a una serie de datos en memoria, utilizando una técnica de semáforos. Si el valor del semáforo es 0 (verde), se puede acceder; si el valor del semáforo es 1 (rojo), no se puede acceder.

SEMAF DB 0 ;semáforo inicializado en 0

...

CHEQUEAR: MOV AL,1 ;AL=1 para bloquear bus

LOCK XCHG SEMAF,AL ;SEMAF=1 (rojo)

;AL=estado del semáforo

CMP AL,1 ;semáforo en rojo?

JE CHEQUEAR ;si, volver a intentar ;no, realizar proceso

...

MOV SEMAF,0 ;SEMAF=0 (verde)

...

LODS Cargar string (LOaD String)

Formato: LODS string_fuente

Descripción: Transfiere un elemento, de tipo byte o palabra, del string fuente al acumulador. Si el string fuente es de tipo BYTE, la transferencia es al registro AL; si el string fuente es de tipo WORD, la transferencia es al registro AX. El string fuente se halla en el segmento de datos, y su dirección es obtenida con el registro DS para el segmento y con el registro SI para el desplazamiento.

Después de la transferencia al acumulador, el registro SI se actualiza automáticamente, de modo que quede apuntando al próximo elemento del string a transferir. Esta actualización depende del estado de la bandera de dirección:

- Si DF=0, SI se incrementa;
- Si DF=1, SI se decrementa.

Por otra parte, la cantidad de unidades en que se incrementa o decrementa SI depende del tipo del operando transferido:

- Si el operando es de tipo BYTE, SI se actualiza en 1;
- Si el operando es de tipo WORD, SI se actualiza en 2.

Es importante resaltar que el operando especificado en LODS es utilizado por el compilador únicamente para verificar el tipo de operando a transferir y para ver si se indicó algún registro de segmento. LODS mueve el byte o la palabra que esta en DS:SI al registro AL o al registro AX sin usar realmente el operando de la instrucción, por lo que antes se debe inicializar el registro SI con el valor correspondiente.

Lógica: Si string _ fuente es tipo BYTE

AL = string _ fuente
valor=1

Si string _ fuente es tipo WORD

AX = string _ fuente
valor=2

Si DF=0

SI = SI + valor

Si DF=1

SI = SI - valor

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: FUENTE DB 50 DUP(?) ;String FUENTE, de tipo BYTE

....

CLD ;DF=0 (incrementar SI)

LEA SI,FUENTE ;SI = desplazamiento de FUENTE

LODS FUENTE ;AL=FUENTE SI=SI+1

Ejemplo 2: FUENTE DB 50 DUP(?) ;String FUENTE, de tipo BYTE

....

CLD ;DF=0 (incrementar SI)

LEA SI,FUENTE ;SI = desplazamiento de FUENTE

LODS ES:FUENTE ;AL=ES:[SI] SI=SI+1

LODSB Cargar string de bytes (LOaD Byte String)

Formato: LODSB

Descripción: Es una particularización de la instrucción LODS para un operando de tipo BYTE. Para esta instrucción son validas las mismas consideraciones que para LODS.

Lógica: AL = DS:[SI]

Si DF=0

SI=SI+1

Si DF=1

SI=SI-1

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: FUENTE DB 50 DUP(?) ;String FUENTE, de tipo BYTE

...

```
CLD          ;DF=0 (incrementar SI)
LEA SI,FUENTE ;SI = desplazamiento de FUENTE
LODSB        ;AL=FUENTE SI=SI+1
```

Ejemplo 2: FUENTE DB 50 DUP(?) ;String FUENTE, de tipo BYTE

...

```
CLD          ;DF=0 (incrementar SI)
LEA SI,FUENTE ;SI = desplazamiento de FUENTE
PUSH DS      ;Salvar DS
PUSH ES
POP DS        ;DS=ES
LODSB        ;AL=ES:[SI] SI=SI+1
POP DS        ;Recuperar DS
```

LODSW Cargar string de palabras (LOaD Word String)

Formato: LODSW

Descripción: Es una particularización de la instrucción LODS para un operando de tipo WORD. Para esta instrucción son validas las mismas consideraciones que para LODS.

Lógica: AX = DS:[SI]

Si DF=0

SI=SI+2

Si DF=1

SI=SI-2

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: FUENTE DW 50 DUP(?) ;String FUENTE, de tipo WORD

...

```
CLD          ;DF=0 (incrementar SI)
LEA SI,FUENTE ;SI = desplazamiento de FUENTE
LODSW        ;AL=FUENTE SI=SI+2
```

Ejemplo 2: FUENTE DW 50 DUP(?) ;String FUENTE, de tipo WORD

...

```
CLD          ;DF=0 (incrementar SI)
LEA SI,FUENTE ;SI = desplazamiento de FUENTE
PUSH DS      ;Salvar DS
PUSH ES
POP DS        ;DS=ES
LODSW        ;AL=ES:[SI] SI=SI+2
POP DS        ;Recuperar DS
```

LOOP Bucle (LOOP)

Formato: LOOP desplazamiento

Descripción: Decrementa en 1 el contenido del contador (registro CX) y bifurca a la dirección IP+desplazamiento en caso que CX sea distinto de cero; si CX es cero, entonces se ejecuta la siguiente instrucción. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción LOOP, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. LOOP se utiliza para implementar ciclos.

Lógica: CX=CX-1


```

    SI CX /= 0
    IP = IP + desplazamiento
Banderas: OF DF IF TF SF ZF AF PF CF
           - - - - - - - - -

```

Ejemplo 1: ;Calculo de la suma de los números del 1 al 10 y almacenar el resultado en el registro AX

```

    MOV    AX,0      ;Inicializar AX
    MOV    CX,10     ;Inicializar el contador
SUMAR: ADD  AX,CX     ;AX=AX+CX
    LOOP  SUMAR

```

Ejemplo 2: ;Encontrar el primer byte distinto de cero en una tabla de bytes

```

    TABLA DB  50 DUP(?)
    ...
    MOV    CX,LENGTH TABLA ;CX = longitud de TABLA
    MOV    SI,-1          ;SI=valor inicial
OTRO: INC  SI              ;Incrementar índice
    CMP    TABLA[SI],0    ;Comparar con cero
    JNE    SALIR           ;No es cero. Salir del ciclo
    LOOP  OTRO             ;Es cero. Seguir con el próximo byte

```

LOOPE Bucle si igual (LOOP if Equal)

Formato: LOOPE desplazamiento

Descripción: Decrementa en 1 el contenido del contador (registro CX) y bifurca a la dirección IP+desplazamiento en caso que CX sea distinto de cero y ZF=1; si CX=0 o ZF=0, entonces se ejecuta la siguiente instrucción. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción LOOPE, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. LOOPE se utiliza para implementar ciclos, y es equivalente a la instrucción LOOPZ.

Lógica: CX=CX-1

```

    SI CX /= 0 y ZF=1
    IP = IP + desplazamiento
Banderas: OF DF IF TF SF ZF AF PF CF
           - - - - - - - - -

```

Ejemplo: ;Encontrar el primer byte distinto de cero en una tabla de bytes

```

    TABLA DB  50 DUP(?)
    ...
    MOV    CX,LENGTH TABLA ;CX = longitud de TABLA
    MOV    SI,-1          ;SI = valor inicial
OTRO: INC  SI              ;Incrementar índice
    CMP    TABLA[SI],0    ;Comparar con cero
    LOOPE  OTRO            ;Es cero. Seguir con el próximo byte

```

LOOPNE Bucle si no igual (LOOP if Not Equal)

Formato: LOOPNE desplazamiento

Descripción: Decrementa en 1 el contenido del contador (registro CX) y bifurca a la dirección IP+desplazamiento en caso que CX sea distinto de cero y ZF=0; si CX=0 o ZF=1, entonces se ejecuta la siguiente instrucción. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción LOOPNE, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. LOOPNE se utiliza para implementar ciclos, y es equivalente a la instrucción LOOPNZ.

Lógica: CX=CX-1

SI CX /= 0 y ZF=0

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo: ;Encontrar el primer byte igual a cero en una tabla de bytes

TABLA DB 50 DUP(?)

...

MOV CX,LENGTH TABLA ;CX = longitud de TABLA

MOV SI,-1 ;SI = valor inicial

OTRO: INC SI ;Incrementar índice

CMP TABLA[SI],0 ;Comparar con cero

LOOPNE OTRO ;No es cero. Seguir con el próximo byte

LOOPNZ Bucle si no cero (LOOP if Not Zero)

Formato: LOOPNZ desplazamiento

Descripción: Decrementa en 1 el contenido del contador (registro CX) y bifurca a la dirección IP+desplazamiento en caso que CX sea distinto de cero y ZF=0; si CX=0 o ZF=1, entonces se ejecuta la siguiente instrucción. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción LOOPNZ, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. LOOPNZ se utiliza para implementar ciclos, y es equivalente a la instrucción LOOPNE.

Lógica: CX=CX-1

SI CX /= 0 y ZF=0

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo: ;Encontrar el primer byte igual a cero en una tabla de bytes

TABLA DB 50 DUP(?)

...

MOV CX,LENGTH TABLA ;CX = longitud de TABLA

MOV SI,-1 ;SI = valor inicial

OTRO: INC SI ;Incrementar índice

CMP TABLA[SI],0 ;Comparar con cero

LOOPNZ OTRO ;No es cero. Seguir con el próximo byte

LOOPZ Bucle si cero (LOOP if Zero)

Formato: LOOPZ desplazamiento

Descripción: Decrementa en 1 el contenido del contador (registro CX) y bifurca a la dirección IP+desplazamiento en caso que CX sea distinto de cero y ZF=1; si CX=0 o ZF=0, entonces se ejecuta la siguiente instrucción. El desplazamiento debe estar comprendido entre -128 y +127 bytes de distancia de la instrucción LOOPZ, es decir que es un valor con signo de 8 bits. El desplazamiento puede estar indicado por un rótulo, que corresponde a la dirección de memoria a donde se debe transferir el control. Para efectuar la bifurcación, el desplazamiento extiende su signo a una cantidad de 16 bits. LOOPZ se utiliza para implementar ciclos, y es equivalente a la instrucción LOOPE.

Lógica: CX=CX-1

SI CX /= 0 y ZF=1

IP = IP + desplazamiento

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo: ;Encontrar el primer byte distinto de cero en una tabla de bytes

```
TABLA DB 50 DUP(?)
...
MOV CX,LENGTH TABLA ;CX = longitud de TABLA
MOV SI,-1 ;SI = valor inicial
OTRO: INC SI ;Incrementar índice
CMP TABLA[SI],0 ;Comparar con cero
LOOPZ OTRO ;Es cero. Seguir con el próximo byte
```

MOV Mover (MOVE)

Formato: MOV destino,fuente

Descripción: Copia un byte o una palabra desde el operando fuente al operando destino, sin alterar el contenido del operando fuente. El operando destino puede ser un registro o una variable de memoria, y el operando fuente puede ser un registro, una variable de memoria o un valor inmediato, pero ambos deben ser del mismo tipo (byte o palabra). La cantidad de información que se transfiere queda determinada por el tipo de los operandos. Las siguientes son las restricciones que presenta la instrucción MOV:

- No se puede mover información entre dos variables de memoria. Para poder hacerlo, primero se debe copiar el contenido de la dirección de origen a un registro general, y luego mover el contenido de ese registro a la dirección destino.
- No se puede mover un valor inmediato a un registro de segmento. Para poder hacerlo también se debe utilizar un registro intermedio.
- No se puede utilizar el registro CS como operando destino.

Lógica: destino = fuente

Banderas: OF DF IF TF SF ZF AF PF CF

```
- - - - -
Ejemplo 1:    AX=ABCDH
               BX=1234H
MOV  AX,BX
       AX=1234H
       BX=1234H
```

```
Ejemplo 2: MOV  DX,CS ;mover una palabra, registro a registro
Ejemplo 3: MOV  SUM,AX ;mover una palabra, registro a memoria
Ejemplo 4: MOV  AL,ORI ;mover un byte, registro a memoria
Ejemplo 5: MOV  CL,BL ;mover un byte, registro a memoria
Ejemplo 6: MOV  DL,30H ;mover un byte, constante a registro
Ejemplo 7: MOV  DES,30H ;mover un byte, constante a memoria
```

MOVS Mover string (MOVE String)

Formato: MOVS string_destino,string_fuente

Descripción: Copia un elemento, de tipo byte o palabra, del string fuente al string destino. El string fuente se halla en el segmento de datos, y su dirección es obtenida con el registro DS para el segmento y con el registro SI para el desplazamiento. El string destino se debe definir en el segmento auxiliar, y su dirección esta dada por el registro ES para el segmento y por el registro DI para el desplazamiento. Después de la copia los registro DI y SI se actualizan automáticamente, de modo que quedan apuntando a los próximos elementos de ambos string. Esta actualización depende del estado de la bandera de dirección:

- Si DF=0, DI y SI se incrementan;
- Si DF=1, DI y SI se decrementan.

Por otra parte, la cantidad de unidades en que se incrementan o decrementan DI y SI depende del tipo de los operandos comparados:

- Si los operandos son de tipo BYTE, DI y SI se actualizan en 1;
- Si los operandos son de tipo WORD, DI y SI se actualizan en 2.

MOVS se puede utilizar con alguno de los prefijos de repetición (REP, REPZ, REPNE, REPNZ) para copiar un string completo de bytes o de palabras en otro. En este caso, el número de elementos a copiar se debe especificar en el registro CX. Es importante resaltar que los operandos especificados en MOVS son utilizados por el compilador únicamente para verificar el tipo de operandos a copiar y para ver si se indicó algún registro de segmento. MOVS realiza la operación ES:DI=DS:SI sin usar realmente los operandos de la instrucción, por lo que antes se deben inicializar los registros DI y SI con los valores correspondientes.

Otra observación a considerar es que se puede reasignar el segmento del operando fuente pero no el del operando destino. Por ejemplo:

```
MOVS DESTINO,ES:FUENTE
```

En este caso, se efectúa una copia de un elemento que está dentro del segmento direccionado por ES a otro elemento dentro del mismo segmento (tener en cuenta que DESTINO está direccionado por ES). Para mover elementos dentro del segmento direccionado por DS, habría que hacer previamente la asignación ES=DS.

Lógica: string_destino = string_fuente

Si tipo de operandos es BYTE, valor=1

Si tipo de operandos es WORD, valor=2

Si DF=0

```
SI = SI + valor
```

```
DI = DI + valor
```

Si DF=1

```
SI = SI - valor
```

```
DI = DI - valor
```

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;Mover 1 byte desde el string FUENTE, en el segmento de datos,

;al string DEST, en el segmento extra

```
CLD ;DF=0 (incrementar SI y DI)
```

```
LEA SI,FUENTE ;SI = desplazamiento de FUENTE
```

```
LEA DI,DEST ;DI = desplazamiento de DEST
```

```
MOVS DEST,FUENTE ;Copiar un byte
```

Ejemplo 2: CLD ;DF=0 (incrementar SI y DI)

```
LEA SI,FUENTE ;SI = desplazamiento de FUENTE
```

```
LEA DI,DEST ;DI = desplazamiento de DEST
```

```
MOVS [DI],[SI] ;Copiar un byte
```

;Este ejemplo es similar al anterior, pero usando DI y SI explícitamente.

Ejemplo 3: ;Copiar 100 elementos del string FUENTE, en el segmento de datos,

;al string DEST, en el segmento extra.

```
CLD ;DF=0 (incrementar SI y DI)
```

```
LEA SI,FUENTE ;SI = desplazamiento de FUENTE
```

```
LEA DI,DEST ;DI = desplazamiento de DEST
```

```
MOV CX,100 ;CX = cantidad de elementos
```

```
REP MOVS [DI],[SI] ;Copia 100 elementos
```

Ejemplo 4: ;Copiar 100 elementos del string FUENTE, en el segmento de datos,

;al string DEST, también en el segmento de datos.

```
CLD ;DF=0 (incrementar SI y DI)
```

```
PUSH DS
```

```
POP ES ;ES=DS
```

```
LEA SI,FUENTE ;SI = desplazamiento de FUENTE
```

```
LEA DI,DEST ;DI = desplazamiento de DEST
```

```
MOV CX,100 ;CX = cantidad de elementos
```

```
REP MOVS [DI],[SI] ;Copiar 100 elementos
```

MOVSb Mover string de bytes (MOVE Byte String)

Formato: MOVSb

Descripción: Es una particularización de la instrucción MOVs para operandos de tipo byte. Para esta instrucción son validas las mismas consideraciones que para MOVs.

Lógica: ES[DI] = DS[SI]

Si DF=0

SI=SI+1

DI=DI+1

Si DF=1

SI=SI-1

DI=DI-1

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;Copiar 100 bytes del string FUENTE, en el segmento de datos,
;al string DEST, en el segmento extra.

```
CLD          ;DF=0 (incrementar SI y DI)
LEA SI,FUENTE ;SI = desplazamiento de FUENTE
LEA DI,DEST   ;DI = desplazamiento de DEST
MOV CX,100    ;CX = cantidad de elementos
REP MOVSb     ;Copiar 100 bytes
```

Ejemplo 2: ;Copiar 100 bytes del string FUENTE, en el segmento de datos,
;al string DEST, también en el segmento de datos.

```
CLD          ;DF=0 (incrementar SI y DI)
PUSH DS
POP ES        ;ES=DS
LEA SI,FUENTE ;SI = desplazamiento de FUENTE
LEA DI,DEST   ;DI = desplazamiento de DEST
MOV CX,100    ;CX = cantidad de elementos
REP MOVSb     ;Copiar 100 bytes
```

MOVSw Mover string de palabras (MOVE Word String)

Formato: MOVSw

Descripción: Es una particularización de la instrucción MOVs para operandos de tipo palabra.

Para esta instrucción son validas las mismas consideraciones que para MOVs.

Lógica: ES[DI] = DS[SI]

Si DF=0

SI=SI+2

DI=DI+2

Si DF=1

SI=SI-2

DI=DI-2

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: ;Copiar 100 palabras del string FUENTE, en el segmento de datos,
;al string DEST, en el segmento extra.

```
CLD          ;DF=0 (incrementar SI y DI)
LEA SI,FUENTE ;SI = desplazamiento de FUENTE
LEA DI,DEST   ;DI = desplazamiento de DEST
MOV CX,100    ;CX = cantidad de elementos
REP MOVSw     ;Copiar 100 palabras
```

Ejemplo 2: ;Copiar 100 palabras del string FUENTE, en el segmento de datos,

;al string DEST, también en el segmento de datos.
 CLD ;DF=0 (incrementar SI y DI)
 PUSH DS
 POP ES ;ES=DS
 LEA SI,FUENTE ;SI = desplazamiento de FUENTE
 LEA DI,DEST ;DI = desplazamiento de DEST
 MOV CX,100 ;CX = cantidad de elementos
 REP MOVSW ;Copiar 100 palabras

MUL Multiplicación, sin signo (MULtiply, unsigned)

Formato: MUL fuente

Descripción: Multiplica, sin considerar el signo, el contenido del acumulador por el operando fuente y devuelve el resultado del producto. El operando fuente puede ser un registro o una variable de memoria, de tipo BYTE o WORD. Dependiendo del tipo del operando fuente, se considerará la parte del acumulador a multiplicar: si es de tipo BYTE, se toma AL; si es de tipo WORD, se toma AX. El resultado de la multiplicación es siempre del doble de longitud que el operando fuente: si es de tipo BYTE, el resultado se almacenará en el registro AX; si es de tipo WORD, el resultado se almacenará en DX (parte alta) y AX (parte baja).

Lógica: Si fuente tipo BYTE

$AX = AL * \text{fuente}$

Si fuente tipo WORD

$DX, AX = AX * \text{fuente}$ (DX = parte alta, AX = parte baja)

Si parte alta del resultado = 0

CF=0

Si parte alta del resultado $\neq$ 0

CF=1

OF=CF

Banderas: OF DF IF TF SF ZF AF PF CF

* - - - ? ? ? ? *

Ejemplo 1: AL=64H = 100

BL=02H = 2

MUL BL

AX=00C8H = 200 CF=0 OF=0

Ejemplo 2: AX=1234H

BX=00F1H

MUL BX

DX,AX=001122F4H CF=1 OF=1

NEG Negación (NEGate)

Formato: NEG destino

Descripción: Obtiene el complemento a 2 del operando destino (es decir, calcula su valor negativo), y devuelve el resultado en el mismo operando. Esta instrucción es equivalente a restar el operando destino de cero. El operando puede ser un registro o una variable de memoria, de tipo BYTE o WORD.

Lógica: destino = 0 - destino

Banderas: OF DF IF TF SF ZF AF PF CF

* - - - * * * * *

Ejemplo 1: AL=F2H = -14

NEG AL

AL=0EH = 14

Ejemplo 2: BH=43H = 67

NEG BH

BH=BDH = -67

NOP No operación (No Operation)

Formato: NOP

Descripción: El microprocesador no hace nada. Pasa a ejecutar la próxima instrucción.

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo: NOP ;No operar

NOT No lógico (Logical NOT)

Formato: NOT destino

Descripción: Efectúa la negación lógica del operando (es decir, calcula su complemento a 1), y almacena el resultado sobre el mismo operando. NOT invierte cada uno de los bits del operando destino. El operando puede ser un registro o una variable de memoria, de tipo BYTE o WORD.

Lógica: Si destino es tipo BYTE

destino = FFH - destino

Si destino es tipo WORD

destino = FFFFH - destino

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: AL=F2H

NOT AL

AL=0DH

Ejemplo 2: BH=43H

NEG BH

BH=BCH

OR O lógico inclusivo (Logical inclusive OR)

Formato: OR destino,fuente

Descripción: Efectúa la operación de O inclusivo lógico entre dos operandos y almacena el resultado en el operando destino. Los operandos pueden ser de tipo BYTE o WORD, y ser dos registros, un registro y una variable de memoria, un registro y una constante o una variable de memoria y una constante, pero no pueden ser, simultáneamente, dos variables de memoria.

Lógica: destino = destino OR fuente

CF=0

OF=0

OR opera a nivel de bits: fija en 1 el bit del operando destino, por cada posición de los operandos en la cual uno o ambos bits contengan un 1; en las posiciones en que ambos operandos contienen 0, el bit de resultado es 0. La siguiente, es la tabla del OR para las distintas combinaciones de bits:

A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

Como regla general, se puede establecer que los bits testeados con un 1 se convierten en uno, y los bits testeados con un 0 no modifican su valor. OR fija en 0 las banderas OF (overflow) y CF (acarreo), dado que ninguna operación lógica produce overflow y acarreo.

Banderas: OF DF IF TF SF ZF AF PF CF

0 - - - * * ? * 0

Ejemplo 1: AX=FEDCH = 1111 1110 1101 1100B

BX=1234H = 0001 0010 0011 0100B

OR AX,BX

Ejemplo 2: AX=FEFCH = 1111 1110 1111 1100B
 BX=E455H = 1110 0100 0101 0101B
 MEM=0259H = 0000 0010 0101 1001B
 OR BX, MEM
 BX=E65DH = 1110 0110 0101 1101B
 Ejemplo 3: MEM=59H = 0101 1001B
 AL=CCH = 1100 1100B
 OR MEM, AL
 MEM=DDH = 1101 1101B
 Ejemplo 4: BH=E4H = 1110 0100B
 OR BH, 0AH
 BH=EEH = 1110 1110B
 Ejemplo 5: MEM=02H = 0000 0010B
 OR MEM, FFH
 MEM=FFH = 1111 1111B

OUT Salida de byte o palabra (OUTput byte or word)

Formato: OUT puerta E/S ,acumulador

Descripción: Transfiere un dato desde el acumulador a la puerta de E/S del procesador indicada.
 En acumulador se debe indicar AL o AX, dependiendo de si el dato a transferir es un byte o una palabra, respectivamente. La puerta de E/S puede ser:
 - un valor fijo entre 0 y 255;
 - un valor variable contenido en el registro DX; en este caso, se pueden direccionar 64K puertas (0a65.535).

Lógica: Si acumulador es AL, puerta = AL

Si acumulador es AX, puerta = AX

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: DX=0034H

OUT DX, AX ;Transfiere la palabra contenida en AX a la puerta 34H.

Ejemplo 2: IN 200H, AH ;Transfiere el byte contenido en AH a la puerta 200H.

POP Sacar palabra del stack (POP word off stack to destination)

Formato: POP destino

Descripción: Descarga una palabra desde el tope del stack, lo almacena en el operando destino e incrementa en 2 el puntero del stack (SP). El operando destino puede ser un registro general, un registro de segmento (excepto CS) o una variable de memoria. La instrucción POP es la complementaria de PUSH y, en general, siempre se debe usar la misma cantidad de POP y PUSH de modo de recuperar toda la información salvada.

Lógica: destino = palabra apuntada por SS:SP

SP=SP+2

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: POP AX ;AX = palabra del stack

esto es equivalente a: MOV AX, [SP] ;AX=SS:[SP]

ADD SP, 2 ;SP=SP+2

Ejemplo 2: POP MEM ;MEM = palabra del stack

esto es equivalente a: MOV AX, [SP] ;AX=SS:[SP]

MOV MEM, AX ;MEM=AX

ADD SP, 2 ;SP=SP+2

POPF Descargar banderas del stack (POP Flags off stack)

Formato: POPF

Descripción: Descarga bits específicos de la palabra ubicada al tope del stack en el registro de banderas, e incrementa en 2 el puntero del stack (SP). La instrucción POPF es la complementaria a PUSHF, y sirven para resguardar y recuperar el registro de banderas.

Lógica:

bit 11 de palabra direccionada por SS:SP ---> OF

bit 10 de palabra direccionada por SS:SP ---> DF

bit 9 de palabra direccionada por SS:SP ---> IF

bit 8 de palabra direccionada por SS:SP ---> TF

bit 7 de palabra direccionada por SS:SP ---> SF

bit 6 de palabra direccionada por SS:SP ---> ZF

bit 4 de palabra direccionada por SS:SP ---> AF

bit 2 de palabra direccionada por SS:SP ---> PF

bit 0 de palabra direccionada por SS:SP ---> CF

SP=SP+2

Banderas: OF DF IF TF SF ZF AF PF CF

* * * * *

Ejemplo: POPF ;Recuperar registro de banderas

PUSH Colocar palabra en el stack (PUSH word onto stack)

Formato: PUSH fuente

Descripción: Decrementa en 2 el puntero del stack (SP) y luego almacena el operando fuente al tope del stack(que esta apuntado por SP).El operando fuente debe ser una palabra, y puede ser un registro general, un registro de segmento(excepto CS) o una variable de memoria.PUSH es la instrucción complementaria a POP

Lógica: SP = SP - 2 palabra apuntada por SS:SP=fuente

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: PUSH AX ;Colocar AX al tope del stack

Esto es equivalente a: SUB SP,2 ;SP=SP-2

MOV [SP],AX ;SS:[SP]=AX

Ejemplo 2: PUSH MEM ;Colocar MEM al tope del stack

Esto equivale a: SUB SP,2 ;SP=SP-2

MOV AX,MEM ;AX=MEM

MOV [SP],AX ;SS:[SP]=AX

PUSHF Cargar banderas al tope del stack (PUSH Flags onto stack)

Formato: PUSHF

Descripción: Decrementa en 2 el puntero del stack (SP) y luego almacena el registro de banderas en bits específicos de la palabra situada al tope del stack. La instrucción PUSHF es la complementaria a POPF, y sirven para resguardar y recuperar el registro de banderas.

Lógica: SP = SP - 2

OF ---> bit 11 de palabra direccionada por SS:SP

DF ---> bit 10 de palabra direccionada por SS:SP

IF ---> bit 9 de palabra direccionada por SS:SP

TF ---> bit 8 de palabra direccionada por SS:SP

SF ---> bit 7 de palabra direccionada por SS:SP

ZF ---> bit 6 de palabra direccionada por SS:SP

AF ---> bit 4 de palabra direccionada por SS:SP

PF ---> bit 2 de palabra direccionada por SS:SP

CF ---> bit 0 de palabra direccionada por SS:SP

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo: PUSHF ;Guardar registro de banderas en stack

RCL Rotar a la izquierda a través de acarreo (Rotate through Carry Left)

Formato: RCL destino, contador

Descripción: Efectúa la rotación, de derecha a izquierda y a través de la bandera de acarreo, de los bits del operando destino tantos bits como la cantidad indicada en el contador. El operando destino puede ser un registro o una variable de memoria, de tipo BYTE o WORD. El contador puede ser el valor inmediato 1 o un número sin signo contenido en CL.

Lógica: tmp = contador

Mientras tmp <> 0

CF_tmp=CF

CF=bit mas alto de destino

destino=destino*2+CF_tmp

tmp=tmp-1

Si contador = 1

Si bit mas alto de destino <> CF

OF=1

Si bit mas alto de destino = CF

OF=0

Si contador <> 1

OF queda indefinida

Los bits del operando que se van desplazando a la izquierda, reingresan al mismo por la derecha, pasando a través de la bandera de acarreo (CF); es decir que la bandera de acarreo funciona como si fuera el bit mas bajo del operando destino. CF queda, entonces, con el valor del ultimo bit rotado que no forma parte del operando destino.

Banderas: OF DF IF TF SF ZF AF PF CF

* - - - - - - *

Ejemplo 1: AX=E2CAH = 1110001011001010B
CF=0

RCL AX,1

AX=C594H = 1100010110010100B

CF=1

Ejemplo 2: AX=E2CAH = 1110001011001010B
CF=0

CL=02H = 0010B

RCL AX,CL

AX=8B29H = 1000101100101001B

CF=1

Ejemplo 3: AL=5EH = 01011110B
CF=0

CL=03H = 0011B

RCL AL,CL

AL=F1H = 11110001B

CF=0

El siguiente es el detalle, paso a paso, del funcionamiento de RCL en este ejemplo:

- PASO 1: CF_tmp=0 AL=1011 1100 CF=0

- PASO 2: CF_tmp=0 AL=0111 1000 CF=1

- PASO 3: CF_tmp=1 AL=1111 0001 CF=0

RCR Rotar a la derecha a través de acarreo (Rotate through Carry Right)

Formato: RCR destino,contador

Descripción: Efectúa la rotación, de izquierda a derecha y a través de la bandera de acarreo, de los bits del operando destino tantos bits como la cantidad indicada en el contador. El operando destino puede ser un registro o una variable de memoria, de tipo BYTE o WORD. El contador puede ser el valor inmediato 1 o un número sin signo contenido en CL.

Lógica: tmp = contador

Mientras tmp <> 0

CF_tmp=CF

CF=bit mas bajo de destino

destino=destino/2

bit mas alto de destino = CF _ tmp

tmp=tmp-1

Si contador = 1

Si bit mas alto de destino <> bit anterior

OF=1

Si bit mas alto de destino = bit anterior

OF=0

Si contador <> 1

OF queda indefinida

Los bits del operando que se van desplazando a la derecha, reingresan al mismo por la izquierda, pasando a través de la bandera de acarreo (CF); es decir que la bandera de acarreo funciona como si fuera el bit mas alto del operando destino. CF queda, entonces, con el valor del ultimo bit rotado que no forma parte del operando destino.

Banderas: OF DF IF TF SF ZF AF PF CF

* - - - - - - - *

Ejemplo 1: AX=E2CBH = 1110001011001011B
CF=0

RCR AX,1

AX=7165H = 0111000101100101B

CF=1

Ejemplo 2: AX=E2CBH = 1110001011001011B
CF=0

CL=02H = 0010B

RCR AX,CL

AX=B8B2H = 1011100010110010B

CF=1

Ejemplo 3: AL=5EH = 01011110B
CF=0

CL=03H = 0011B

RCR AL,CL

AL=8BH = 10001011B

CF=1

El siguiente es el detalle, paso a paso, del funcionamiento de RCR en este ejemplo:

- PASO 1: CF_tmp=0 CF=0 AL=0010 1111

- PASO 2: CF_tmp=0 CF=1 AL=0001 0111

- PASO 3: CF_tmp=1 CF=1 AL=1000 1011

REP Repetir operación de string (REPeat string operation)

Formato: REP

Descripción: REP no es exactamente una instrucción, sino que es un prefijo de instrucción, que se utiliza con las instrucciones de manejo de strings y en particular con las de movimiento de strings y con las de almacenamiento de strings. REP hace que la instrucción de string que esta a continuación se repita la cantidad de veces indicada en el registro CX. Un aspecto a tener en cuenta es que las instrucciones con prefijos de repetición son interrumpibles, es decir que el microprocesador reconocerá una interrupción antes de procesar el siguiente elemento del string.

Lógica: Mientras CX $\neq$ 0

ejecutar instrucción siguiente

CX=CX-1

Banderas: Las banderas son modificadas según la instrucción de string que esta con el prefijo REP.

Ejemplo 1: ;Copiar un string de 100 palabras, residente en el segmento de

;datos, sobre otro residente en el segmento extra.

MOV CX,100 ;CX=Cantidad de elementos

LEA SI,FUENTE ;SI=desplazamiento de FUENTE

; (segmento de datos)

LEA DI,DEST ;DI=desplazamiento de DEST

; (segmento extra)

REP MOVSW ;Copiar 100 palabras:

;ES:[DI]=DS:[SI]

Ejemplo 2: ;Copiar un string de 100 bytes, residente en el segmento de datos,

;sobre otro residente también en el segmento de datos.

MOV CX,100 ;CX=Cantidad de elementos

LEA SI,FUENTE ;SI=desplazamiento de FUENTE

; (segmento de datos)

LEA DI,DEST ;DI=desplazamiento de DEST

; (segmento extra)

PUSH DS

POP ES ;ES=DS

REP MOVSB ;ES:[DI]=DS:[SI] (100 veces)

REPE Repetir operación de string, si igual (REPeat string operation, if Equal)

Formato: REPE

Descripción: REPE es un prefijo de instrucción que se utiliza con las instrucciones de manejo de strings, en particular con las de comparación de strings y con las de exploración de strings. REPE hace que la instrucción de string que esta a continuación se repita la cantidad de veces indicada en el registro CX, mientras ZF=1.

Los prefijos REPE y REPZ son equivalentes. Un aspecto a tener en cuenta es que las instrucciones con prefijos de repetición son interrumpibles, es decir que el microprocesador reconocerá una interrupción antes de procesar el siguiente elemento del string.

Lógica: Mientras CX $\neq$ 0 y ZF=1

ejecutar instrucción siguiente

CX=CX-1

Banderas: Las banderas son modificadas según la instrucción de string que esta con el prefijo REPE.

Ejemplo: ;Encontrar el primer valor diferente

;en dos strings de 100 bytes cada uno.

MOV CX,100 ;CX=Cantidad de elementos

LEA SI,FUENTE ;SI=desplazamiento de FUENTE

; (segmento de datos)

LEA DI,DEST ;DI=desplazamiento de DEST

; (segmento extra)

REPE CMPS [DI],[SI] ;Comparar elementos mientras sean iguales

CMP CX,0 ;" CX=0 ?

JE NINGUNO ;Si. Son todos iguales

;Los valores distintos están en DS:[SI-1] y en ES:[DI-1]

REPNE Repetir operación de string, si no igual(REPeat string operación, if Not Equal)

Formato: REPNE

Descripción: REPNE es un prefijo de instrucción que se utiliza con las instrucciones de manejo de strings, en particular con las de comparación de strings y con las de exploración de strings. REPNE hace que la instrucción de string que esta a continuación se repita la cantidad de veces indicada en el registro CX, mientras ZF=0. Los prefijos REPNE y REPNZ son equivalentes. Un aspecto a tener en cuenta es que las instrucciones con prefijos de repetición son interrumpibles, es decir que el microprocesador reconocerá una interrupción antes de procesar el siguiente elemento del string.

Lógica: Mientras CX <> 0 y ZF=0

ejecutar instrucción siguiente

CX=CX-1

Banderas: Las banderas son modificadas según la instrucción de string que est con el prefijo REPNE.

Ejemplo: ;Encontrar el primer valor igual

;en dos strings de 100 bytes cada uno.

MOV CX,100 ;CX=Cantidad de elementos

LEA SI,FUENTE ;SI=desplazamiento de FUENTE

; (segmento de datos)

LEA DI,DEST ;DI=desplazamiento de DEST

; (segmento extra)

REPNE CMPS [DI],[SI] ;Comparar elementos mientras sean distintos

CMP CX,0 ;" CX=0 ?

JE NINGUNO ;Si. Son todos distintos

;Los valores iguales están en DS:[SI-1] y en ES:[DI-1]

REPNZ Repetir operación de string, si no cero (REPeat string operation, if Not Zero)

Formato: REPNZ

Descripción: REPNZ es un prefijo de instrucción que se utiliza con las instrucciones de manejo de strings, en particular con las de comparación de strings y con las de exploración de strings. REPNZ hace que la instrucción de string que esta a continuación se repita la cantidad de veces indicada en el registro CX, mientras ZF=0. Los prefijos REPNE y REPNZ son equivalentes. Un aspecto a tener en cuenta es que las instrucciones con prefijos de repetición son interrumpibles, es decir que el microprocesador reconocerá una interrupción antes de procesar el siguiente elemento del string.

Lógica: Mientras CX <> 0 y ZF=0

ejecutar instrucción siguiente

CX=CX-1

Banderas: Las banderas son modificadas según la instrucción de string que est con el prefijo REPNZ.

Ejemplo: ;Encontrar el primer valor igual en dos strings de 100 bytes cada uno.

MOV CX,100 ;CX=Cantidad de elementos

LEA SI,FUENTE ;SI=desplazamiento de FUENTE

; (segmento de datos)

LEA DI,DEST ;DI=desplazamiento de DEST

; (segmento extra)

REPNZ CMPS [DI],[SI] ;Comparar elementos mientras

;sean distintos

CMP CX,0 ;" CX=0 ?

JE NINGUNO ;Si. Son todos distintos

;Los valores iguales están

;en DS:[SI-1] y en ES:[DI-1]

REPZ Repetir operación de string, si cero (REPeat string operación, if Zero)

Formato: REPZ

Descripción: REPZ es un prefijo de instrucción que se utiliza con las instrucciones de manejo de strings, en particular con las de comparación de string y con las de exploración de string. REPZ hace que la instrucción de string que esta a continuación se repita la cantidad de veces indicada en el registro CX, mientras ZF=1. Los prefijos REPE y REPZ son equivalentes. Un aspecto a tener en cuenta es que las instrucciones con prefijos de repetición son interrumpibles, es decir que el microprocesador reconocerá una interrupción antes de procesar el siguiente elemento del string.

Lógica: Mientras CX <> 0 y ZF=1

ejecutar instrucción siguiente

CX=CX-1

Banderas: Las banderas son modificadas según la instrucción de string que esta con el prefijo REPZ.

Ejemplo: ;Encontrar el primer valor diferente en dos strings de 100 bytes cada uno.

```
MOV    CX,100      ;CX=Cantidad de elementos
LEA     SI,FUENTE   ;SI=desplazamiento de FUENTE
                ; (segmento de datos)
LEA     DI,DEST     ;DI=desplazamiento de DEST
                ; (segmento extra)
REPZ    CMPS  [DI],[SI] ;Comparar elementos mientras sean iguales
CMP     CX,0        ;" CX=0 ?
JE      NINGUNO     ;Si. Son todos iguales. Los valores distintos están en DS:[SI-1] y en ES:[DI-1]
```

RET Retornar de un procedimiento (RETurn)

Formato: RET [valor]

Descripción: Retorna de un procedimiento, previamente invocado mediante CALL, utilizando como dirección de retorno la que fue salvada en el stack por CALL. Dicha dirección de retorno corresponde a la dirección de la instrucción siguiente a la instrucción CALL. Según haya sido el tipo de llamada, el compilador genera un retorno distinto:

- Si el procedimiento fue definido como NEAR, es decir que esta dentro del mismo segmento de código, la dirección de retorno se obtiene extrayendo la palabra que esta al tope del stack. Dicha palabra corresponde al desplazamiento.
- Si el procedimiento fue definido como FAR, es decir que esta en otro segmento de código, la dirección de retorno se obtiene extrayendo dos palabras del stack: la primera corresponde al desplazamiento y la segunda al segmento.

El valor es un parámetro adicional y, si es especificado, se suma al registro SP después de haber extraído la dirección de retorno. El uso de un valor en la instrucción RET sirve para dejar libre el área del stack que haya sido afectado al pasaje de parámetros por dirección.

Lógica: IP <--- stack

SP = SP + 2

Si procedimiento es FAR

CS <--- stack

SP = SP + 2

Si hay valor opcional

SP = SP + valor

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: RET ;Retorno de un procedimiento

Ejemplo 2: RET 4 ;Retorno de un procedimiento liberando 4 bytes del stack

ROL Rotación a la izquierda (ROtate Left)

Formato: ROL destino,contador

Descripción: Efectúa la rotación, de derecha a izquierda, de los bits del operando destino tantos bits como la cantidad indicada en el contador. El operando destino puede ser un registro o una variable de memoria, de tipo BYTE o WORD. El contador puede ser el valor inmediato 1 o un número sin signo contenido en CL.

Lógica: tmp = contador

Mientras tmp <> 0

CF=bit mas alto de destino

destino=destino*2+CF

tmp=tmp-1

Si contador = 1

Si bit mas alto de destino <> CF

OF=1

Si bit mas alto de destino = CF

OF=0

Si contador <> 1

OF queda indefinida

Los bits del operando que se van desplazando a la izquierda, reingresan al mismo por la derecha.

La bandera de acarreo (CF) queda con el valor del ultimo bit rotado o, lo que es lo mismo, con el valor del bit mas bajo del resultado.

Banderas: OF DF IF TF SF ZF AF PF CF

* - - - - - *

Ejemplo 1: AX=E2CAH = 1110001011001010B

ROL AX,1

AX=C595H = 1100010110010101B CF=1

Ejemplo 2: AX=E2CAH = 1110001011001010B

CL=04H = 0011B

ROL AX,CL

AX=2CAEH = 0010110010101110B CF=0

Ejemplo 3: AL=51H = 01010001B

CL=03H = 0011B

ROL AL,CL

AL=8AH = 10001010B

El siguiente es el detalle, paso a paso, del funcionamiento de ROL en este ejemplo:

- PASO 1: CF=0 AL=1010 0010

- PASO 2: CF=1 AL=0100 0101

- PASO 3: CF=0 AL=1000 1010

ROR Rotación a la derecha (ROtate Right)

Formato: ROR destino, contador

Descripción: Efectúa la rotación, de izquierda a derecha, de los bits del operando destino tantos bits como la cantidad indicada en el contador. El operando destino puede ser un registro o una variable de memoria, de tipo BYTE o WORD. El contador puede ser el valor inmediato 1 o un número sin signo contenido en CL.

Lógica: tmp = contador

Mientras tmp <> 0

CF=bit mas bajo de destino

destino=destino/2

bit mas alto de destino=CF

tmp=tmp-1

Si contador = 1

Si bit mas alto de destino <> bit anterior

OF=1

Si bit mas alto de destino = bit anterior

OF=0

Si contador <> 1

OF queda indefinida

Los bits del operando que se van desplazando a la derecha, reingresan al mismo por la izquierda. La bandera de acarreo (CF) queda con el valor del ultimo bit rotado o, lo que es lo mismo, con el valor del bit mas alto del resultado.

Banderas: OF DF IF TF SF ZF AF PF CF
 * - - - - - - *

Ejemplo 1: AX=E2CAH = 1110001011001010B
 ROR AX,1
 AX=7165H = 0111000101100101B CF=0

Ejemplo 2: AX=E2CAH = 1110001011001010B
 CL=04H = 0011B
 ROR AX,CL
 AX=AE2CH = 1010111000101100B CF=1

Ejemplo 3: AL=51H = 01010001B
 CL=03H = 0011B
 ROR AL,CL
 AL=2AH = 00101010B

El siguiente es el detalle, paso a paso, del funcionamiento de ROR en este ejemplo:

- PASO 1: CF=1 AL=1010 1000
- PASO 2: CF=0 AL=0101 0100
- PASO 3: CF=0 AL=0010 1010

SAHF Almacenar AH en banderas (Store AH in Flags)

Formato: SAHF

Descripción: Copia los bits 7, 6, 4, 2 y 0 del registro AH en las banderas SF (signo), ZF (cero), AF (acarreo auxiliar), PF (paridad) y CF (acarreo), respectivamente. Esta instrucción se usa forzar una determinada configuración en las banderas de estado.

Lógica: SF = bit 7 de AH
 ZF = bit 6 de AH
 AF = bit 4 de AH
 PF = bit 2 de AH
 CF = bit 0 de AH

Banderas: OF DF IF TF SF ZF AF PF CF
 - - - - * * * * *

Ejemplo: ;SF=0 ZF=0 AF=0 PF=0 CF=0
 ;AH=4AH
 SAHF
 ;SF=0 ZF=1 AF=0 PF=0 CF=0

SAL Desplazamiento aritmético a la izquierda (Shift Aritmética Left)

Formato: SAL destino , contador

Descripción: Hace el corrimiento a la izquierda de los bits del operando destino, tantas posiciones como las indicadas en el contador, relleno con cero los bits vacantes a la derecha. El operando destino puede ser un registro o una variable de memoria, de tipo BYTE o WORD. El contador puede ser el valor inmediato 1 o un número sin signo contenido en CL. SAL es una instrucción muy útil para efectuar multiplicaciones "manuales", pues el corrimiento de una posición a la izquierda equivale a multiplicar el operando por 2.

SAL es equivalente a la instrucción SHL.

Lógica: tmp = contador

Mientras tmp <> 0

CF=bit mas alto de destino

destino=destino*2


```

    tmp=tmp-1
Si contador = 1
    Si bit mas alto de destino <> CF
        OF=1
    Si bit mas alto de destino = CF
        OF=0
Si contador <> 1
    OF queda indefinida
SAL asume que el operando destino es una cantidad aritmética con signo, por lo que enciende la bandera de
overflow si cambia el signo del operando después del corrimiento. La bandera de acarreo recibe el ultimo bit
desplazado fuera del operando.
Banderas: OF DF IF TF SF ZF AF PF CF
           * - - - * * ? * *
Ejemplo 1:  AL=94H = 10010111B
            SAL  AL,1
            AL=2EH = 00101110B
            CF=1  OF=1  SF=1
Ejemplo 2:  AX=D5CAH = 1101010111001010B
            CL=05H  = 00000101B
            SAL  AX,CL
            AX=B940H = 1011100101000000B
            CF=0  OF=?  SF=1
Ejemplo 3:  BL=CCH = 11001100B
            CL=03H = 00000011B
            SAL  BL,CL
            BL=60H = 01100000
El siguiente es el detalle, paso a paso, del funcionamiento de SAL en este ejemplo:
- PASO 1: CF=1  BL=1001 1000
- PASO 2: CF=1  BL=0011 0000
- PASO 3: CF=0  BL=0110 0000

```

SAR Desplazamiento aritmético a la derecha (Shift Aritmética Right)

Formato: SAR destino , contador

Descripción: Hace el corrimiento a la derecha de los bits del operando destino, tantas posiciones como las indicadas en el contador, rellenando con el bit de signo los bits vacantes a la izquierda. El operando destino puede ser un registro o una variable de memoria, de tipo BYTE o WORD. El contador puede ser el valor inmediato 1 o un número sin signo contenido en CL.

Lógica: tmp = contador

Mientras tmp <> 0

CF=bit mas bajo de destino

destino=destino/2

tmp=tmp-1

Si contador = 1

Si bit mas alto de destino <> bit anterior

OF=1

Si bit mas alto de destino = bit anterior

OF=0

Si contador <> 1

OF queda indefinida

SAR asume que el operando destino es una cantidad aritmética con signo, por lo que enciende la bandera de overflow si cambia el signo del operando después del corrimiento. La bandera de acarreo recibe el ultimo bit desplazado fuera del operando.

Banderas: OF DF IF TF SF ZF AF PF CF

* - - - * * ? * *

Ejemplo 1: AL=94H = 10010111B
 SAR AL,1
 AL=2EH = 11001011B
 CF=1 OF=1 SF=1

Ejemplo 2: AX=45DAH = 0100010111011010B
 CL=05H = 00000101B
 SAR AX,CL
 AX=022EH = 0000001000101110B
 CF=1 OF=? SF=0

Ejemplo 3: BL=CCH = 11001100B
 CL=03H = 00000011B
 SAR BL,CL
 BL=F9H = 11111001B

El siguiente es el detalle, paso a paso, del funcionamiento de SAR en este ejemplo:

- PASO 1: CF=0 BL=1110 0110
- PASO 2: CF=0 BL=1111 0011
- PASO 3: CF=1 BL=1111 1001

SBB Restar con acarreo (SuBtract with Borrow)

Formato: SBB destino,fuente

Descripción: Resta el operando fuente del operando destino y, del resultado, resta la bandera de acarreo, almacenando el resultado en el operando destino. Esta instrucción es similar a SUB, con la diferencia que, además de restar el operando fuente del operando destino, resta la bandera de acarreo del resultado de la operación. SBB es la instrucción equivalente para la resta a la instrucción ADC (suma con acarreo).

SBB es muy útil para restar cantidades de mayor precisión que 16 bits.

Lógica: destino = destino - fuente

destino = destino - CF

Banderas: OF DF IF TF SF ZF AF PF CF

* - - - * * * * *

Ejemplo 1: AL=FEH = 1111 1110B
 BL=12H = 0001 0010B
 CF=1
 SBB AL,BL
 AL=EBH = 1110 1011B

Ejemplo 2: ;Restar dos números contenido en 32 bits

;Fuente: DX (parte alta) y CX (parte baja)

;Destino: BX (parte alta) y AX (parte baja)

SUB AX,CX ;Restar partes bajas

SBB BX,DX ;Restar con acarreo partes altas

SCAS Explorar string (SCAn String)

Formato: SCAS string_destino

Descripción: Permite explorar un string de bytes o palabras, comparando el elemento del string destino con el contenido del acumulador (AL en caso que el string sea de bytes o AX en caso que el string sea de palabras).

El string destino esta direccionado por DI en el segmento extra, es decir ES:[DI]. La instrucción SCAS es similar a la instrucción CMPS, pero tomando el acumulador como operando fuente. Es decir que la operación que ejecuta SCAS es: acumulador (AL o AX)-ES:[DI]

Del mismo modo que CMPS, SCAS fija las banderas de estado después de la comparación. Asimismo, el registro DI se actualiza automáticamente, de modo que quede apuntando al próximo elemento a explorar del string. Esta actualización depende del estado de la bandera de dirección:

- Si DF=0, DI se incrementa;
- Si DF=1, DI se decrementa.

Por otra parte, la cantidad de unidades en que se incrementa o decrementa DI depende del tipo del operando:

- Si el operando es de tipo BYTE, DI se actualiza en 1;
- Si el operando es de tipo WORD, DI se actualiza en 2.

Es importante resaltar que el operando especificado en SCAS es utilizado por el compilador únicamente para verificar el tipo de operando a explorar y para ver si se indicó algún registro de segmento. SCAS compara el byte o la palabra que esta en ES:DI con el contenido del registro AL o con el contenido del registro AX sin usar realmente el operando de la instrucción, por lo que antes se debe inicializar el registro DI con el valor correspondiente. SCAS se puede utilizar con los prefijos de repetición. Esto permite, por ejemplo, la búsqueda de un elemento de un string que cumpla una determinada condición.

Lógica: Si string_fuente es tipo BYTE

AL-ES:[DI]

valor=1

Si string_fuente es tipo WORD

AX-ES:[DI]

valor=2

Si DF=0

DI=DI+valor

Si DF=1

DI=DI-valor

Banderas: OF DF IF TF SF ZF AF PF CF

* _ _ _ * * * * *

Ejemplo 1: ;Buscar en el string DEST, de 100 bytes, en el segmento extra,

; el primer elemento con el carácter blanco.

CLD ;DF=0 (incrementar DI)

LEA DI,DEST ;DI=desplazamiento de DEST

MOV CX,100 ;CX=cantidad de bytes

MOV AL,' ' ;AL=valor a buscar

REPNE SCAS DEST ;Explorar string

JZ ESTA ;DI contiene el desplazamiento del elemento hallado

Ejemplo 2: ;Buscar en el string DEST, de 100 bytes, en el segmento extra,

; el primer elemento distinto del carácter blanco.

CLD ;DF=0 (incrementar DI)

LEA DI,DEST ;DI=desplazamiento de DEST

MOV CX,100 ;CX=cantidad de bytes

MOV AL,' ' ;AL=valor a buscar

REPE SCAS DEST ;Explorar string

JNZ ESTA ;DI contiene el desplazamiento del elemento hallado

SCASB Explorar string de bytes (SCAn Byte String)

Formato: SCASB

Descripción: Es una particularización de la instrucción SCAS para un operando de tipo BYTE. Para esta instrucción son validas las mismas consideraciones que para SCAS.

Lógica:

AL-ES:[DI]

Si DF=0

DI=DI+1

Si DF=1

DI=DI-1

Banderas: OF DF IF TF SF ZF AF PF CF

* _ _ _ * * * * *

Ejemplo 1: ;Buscar en el string DEST, de 100 bytes, en el segmento extra,

; el primer elemento con el carácter blanco.

CLD ;DF=0 (incrementar DI)

LEA DI,DEST ;DI=desplazamiento de DEST

```

MOV    CX,100      ;CX=cantidad de bytes
MOV    AL,' '      ;AL=valor a buscar
REPNE  SCASB       ;Explorar string
JZ     ESTA        ;DI contiene el desplazamiento del elemento hallado
Ejemplo 2: ;Buscar en el string DEST, de 100 bytes, en el segmento extra,
           ;el primer elemento distinto del carácter blanco.
CLD          ;DF=0 (incrementar DI)
LEA    DI,DEST     ;DI=desplazamiento de DEST
MOV    CX,100      ;CX=cantidad de bytes
MOV    AL,' '      ;AL=valor a buscar
REPE   SCASB       ;Explorar string
JNZ    ESTA        ;DI contiene el desplazamiento del elemento hallado

```

SCASW Explorar string de palabras (SCAN Word String)

Formato: SCASW

Descripción: Es una particularización de la instrucción SCAS para un operando de tipo WORD. Para esta instrucción son validas las mismas consideraciones que para SCAS.

Lógica:

AX-ES:[DI]

Si DF=0

DI=DI+2

Si DF=1

DI=DI-2

Banderas: OF DF IF TF SF ZF AF PF CF

* - - - * * * * *

Ejemplo 1: ;Buscar en el string DEST, de 100 palabras, en el segmento extra,
;el primer elemento con el valor cero.

```

CLD          ;DF=0 (incrementar DI)
LEA    DI,DEST ;DI=desplazamiento de DEST
MOV    CX,100  ;CX=cantidad de bytes
MOV    AX,0    ;AX=valor a buscar
REPNE  SCASW   ;Explorar string
JZ     ESTA    ;DI contiene el desplazamiento del elemento hallado

```

Ejemplo 2: ;Buscar en el string DEST, de 100 palabras, en el segmento extra,
;el primer elemento distinto de cero.

```

CLD          ;DF=0 (incrementar DI)
LEA    DI,DEST ;DI=desplazamiento de DEST
MOV    CX,100  ;CX=cantidad de bytes
MOV    AX,0    ;AX=valor a buscar
REPE   SCASW   ;Explorar string
JNZ    ESTA    ;DI contiene el desplazamiento del elemento hallado

```

SHL Desplazamiento lógico a la izquierda (SHift logical Left)

Formato: SHL destino,contador

Descripción: Hace el corrimiento a la izquierda de los bits del operando destino, tantas posiciones como las indicadas en el contador, rellenando con cero los bits vacantes a la derecha. El operando destino puede ser un registro o una variable de memoria, de tipo BYTE o WORD. El contador puede ser el valor inmediato 1 o un

número sin signo contenido en CL. SHL es una instrucción muy útil para efectuar multiplicaciones "manuales", pues el corrimiento de una posición a la izquierda equivale a multiplicar el operando por 2. SHL es equivalente a la instrucción SAL.

Lógica: tmp = contador

Mientras tmp <> 0

CF=bit mas alto de destino

destino=destino*2

tmp=tmp-1

Si contador = 1

Si bit mas alto de destino <> CF

OF=1

Si bit mas alto de destino = CF

OF=0

Si contador <> 1

OF queda indefinida

SHL asume que el operando destino es una cantidad aritmética con signo, por lo que enciende la bandera de overflow si cambia el signo del operando después del corrimiento. La bandera de acarreo recibe el ultimo bit desplazado fuera del operando.

Banderas: OF DF IF TF SF ZF AF PF CF

* - - - * * ? * *

Ejemplo 1: AL=94H = 10010111B

SHL AL,1

AL=2EH = 00101110B

CF=1 OF=1 SF=1

Ejemplo 2: AX=D5CAH = 1101010111001010B

CL=05H = 00000101B

SHL AX,CL

AX=B940H = 1011100101000000B

CF=0 OF=? SF=1

Ejemplo 3: BL=CCH = 11001100B

CL=03H = 00000011B

SHL BL,CL

BL=60H = 01100000

El siguiente es el detalle, paso a paso, del funcionamiento de SHL en este ejemplo:

- PASO 1: CF=1 BL=1001 1000

- PASO 2: CF=1 BL=0011 0000

- PASO 3: CF=0 BL=0110 0000

SHR Desplazamiento lógico a la derecha (SHift logical Right)

Formato: SHR destino , contador

Descripción: Hace el corrimiento a la derecha de los bits del operando destino, tantas posiciones como las indicadas en el contador, rellenando con ceros los bits vacantes a la izquierda. El operando destino puede ser un registro o una variable de memoria, de tipo BYTE o WORD. El contador puede ser el valor inmediato 1 o un número sin signo contenido en CL. SHR asume que el operando destino es una cantidad sin signo, por lo que, a diferencia de la instrucción SAR, no preserva el signo del operando. SHR es una instrucción muy útil para efectuar divisiones "manuales", pues el corrimiento de una posición a la derecha equivale a dividir el operando por 2.

Lógica: tmp=contador

Mientras tmp <> 0

CF=bit mas bajo de destino

destino=destino/2

tmp=tmp-1

Si contador = 1

Si bit mas alto de destino <> bit anterior

OF=1
 Si bit mas alto de destino = bit anterior
 OF=0
 Si contador <> 1
 OF queda indefinida
 La bandera de acarreo recibe el ultimo bit desplazado fuera del operando.
 Banderas: OF DF IF TF SF ZF AF PF CF
 * - - - * * ? * *

Ejemplo 1: AL=94H = 10010111B
 SHR AL,1
 AL=4EH = 01001011B
 CF=1 OF=1 SF=0

Ejemplo 2: AX=45DAH = 0100010111011010B
 CL=05H = 00000101B
 SHR AX,CL
 AX=022EH = 0000001000101110B
 CF=1 OF=? SF=0

Ejemplo 3: BL=CCH = 11001100B
 CL=03H = 00000011B
 SHR BL,CL
 BL=19H = 00011001B

El siguiente es el detalle, paso a paso, del funcionamiento de SHR en este ejemplo:

- PASO 1: CF=0 BL=0110 0110
 - PASO 2: CF=0 BL=0011 0011
 - PASO 3: CF=1 BL=0001 1001

STC Encender bandera de acarreo (SeT Carry flag)

Formato: STC

Descripción: Pone en uno la bandera de acarreo (CF), sin afectar ninguna otra bandera.

Lógica: CF=1

Banderas: OF DF IF TF SF ZF AF PF CF
 - - - - - - - 1

Ejemplo: STC
 CF=1

STD Encender bandera de dirección (SeT Direction flag)

Formato: STD

Descripción: Pone en uno la bandera de dirección (DF), sin afectar ninguna otra bandera. Luego de ejecutarse esta instrucción, los string serán tratados desde la dirección mas baja a la dirección mas alta o, lo que es lo mismo, de derecha a izquierda. Esto significa que los registros índice SI y DI, utilizados para el manejo de string, se autodecrementaran automáticamente después de cada operación de string.

Lógica: DF=1

Banderas: OF DF IF TF SF ZF AF PF CF
 - 1 - - - - - -

Ejemplo: STD
 DF=1

STI Encender bandera de interrupciones (SeT Interrupt flag)

Formato: STI

Descripción: Pone en uno la bandera de interrupciones externas (IF), sin afectar ninguna otra bandera.

Después de ejecutarse esta instrucción, quedan habilitadas las interrupciones externas o enmascarables, es decir que serán atendidas por el microprocesador. STI se la utiliza en las rutinas de atención de interrupciones, una vez que la rutina completó su acción, para habilitar la atención de las interrupciones externas.

Lógica: IF=1

Banderas: OF DF IF TF SF ZF AF PF CF

- - 1 - - - - -

Ejemplo: CLI

IF=1

STOS Almacenar string (STOre String)

Formato: STOS string _ destino

Descripción: Transfiere un elemento, de tipo byte o palabra, desde el acumulador al string destino. Si el string destino es de tipo BYTE, la transferencia es desde el registro AL; si el string destino es de tipo WORD, la transferencia es desde el registro AX. La dirección del string destino se obtiene con el registro ES para el segmento y con el registro DI para el desplazamiento. Después de la transferencia, el registro DI se actualiza automáticamente, de modo que quede apuntando al próximo elemento del string a transferir. Esta actualización depende del estado de la bandera de dirección:

- Si DF=0, DI se incrementa;
- Si DF=1, DI se decrementa.

Por otra parte, la cantidad de unidades en que se incrementa o decrementa DI depende del tipo del operando transferido:

- Si el operando es de tipo BYTE, DI se actualiza en 1;
- Si el operando es de tipo WORD, DI se actualiza en 2.

Es importante resaltar que el operando especificado en STOS es utilizado por el compilador únicamente para verificar el tipo de operando a transferir y para ver si se indicó algún registro de segmento. STOS mueve el contenido del registro AL o el contenido del registro AX al byte o a la palabra que esta en ES:DI sin usar realmente el operando de la instrucción, por lo que antes se debe inicializar el registro DI con el valor correspondiente. STOS puede ser utilizado con los prefijos de repetición; en ese caso, se puede llenar un string con un determinado valor.

Lógica: Si string _ destino es tipo BYTE

string _ destino = AL valor=1

Si string _ destino es tipo WORD

string _ destino = AX valor=2

Si DF=0

DI=DI+valor

Si DF=1

DI=DI-valor

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: FUENTE DB 50 DUP(?) ;String FUENTE, de tipo BYTE

...

CLD ;DF=0 (incrementar DI)

LEA DI,DEST ;DI=desplazamiento de DEST

STOS DEST ;DEST=AL DI=DI+1

Ejemplo 2: FUENTE DW 50 DUP(?) ;String FUENTE, de tipo WORD

....

CLD ;DF=0 (incrementar DI)

LEA DI,DEST ;DI=desplazamiento de DEST

STOS ES:DEST ;ES:[DI]=AL DI=DI+2

Ejemplo 3: ;Rellenar con asteriscos los 100 bytes del string DEST

CLD ;DF=0 (incrementar DI)

LEA DI,DEST ;DI=desplazamiento de DEST

MOV AL,'*' ;AL=valor a cargar

MOV CX,100 ;CX=Cantidad de bytes

REP STOS DEST ;Almacenar 100 bytes

STOSB Almacenar string de bytes (STOre Byte String)

Formato: STOSB

Descripción: Es una particularización de la instrucción STOS para un operando de tipo BYTE.

Para esta instrucción son validas las mismas consideraciones que para STOS.

Lógica: ES:[DI]=AL

Si DF=0

DI=DI+1

Si DF=1

DI=DI-1

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: FUENTE DB 50 DUP(?) ;String FUENTE, de tipo BYTE

....

CLD ;DF=0 (incrementar DI)

LEA DI,DEST ;DI=desplazamiento de DEST

STOSB ;ES:[DI]=AL DI=DI+1

Ejemplo 2: ;Rellenar con asteriscos los 100 bytes del string DEST

CLD ;DF=0 (incrementar DI)

LEA DI,DEST ;DI = desplazamiento de DEST

MOV AL,'*' ;AL = valor a cargar

MOV CX,100 ;CX = Cantidad de bytes

REP STOSB ;Almacenar 100 bytes

STOSW Almacenar string de palabras (STORe Word String)

Formato: STOSW

Descripción: Es una particularización de la instrucción STOS para un operando de tipo WORD.

Para esta instrucción son validas las mismas consideraciones que para STOS.

Lógica: ES:[DI]=AX

Si DF=0

DI=DI+2

Si DF=1

DI=DI-2

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - -

Ejemplo 1: FUENTE DW 50 DUP(?) ;String FUENTE, de tipo WORD

....

CLD ;DF=0 (incrementar DI)

LEA DI,DEST ;DI=desplazamiento de DEST

STOSW ;ES:[DI]=AX DI=DI+2

Ejemplo 2: ;Rellenar con asteriscos las 50 palabras del string DEST

CLD ;DF=0 (incrementar DI)

LEA DI,DEST ;DI=desplazamiento de DEST

MOV AX,'**' ;AX = valor a cargar

MOV CX,50 ;CX=Cantidad de palabras

REP STOSW ;Almacenar 50 palabras

SUB Resta (SUBtraction)

Formato: SUB destino,fuente

Descripción: Resta el operando destino del operando fuente, y almacena el resultado en el operando destino.

Se puede restar un registro de otro, un registro de una variable de memoria, una variable de memoria de un registro, un valor inmediato de un registro o un valor inmediato de una variable de memoria. En cambio, no es posible restar una variable de memoria de otra. Los operandos pueden ser de tipo BYTE o WORD, dependiendo de como están especificados, pero ambos deben ser del mismo tipo. El microprocesador INTEL 8088-8086 hace la operación de resta mediante la complementación a 2 del sustraendo (operando fuente) y la suma del resultado al minuendo (operando destino).

Lógica: destino = destino - fuente

Esto es equivalente a:

fuente = NEG fuente

destino = destino + fuente

Banderas: OF DF IF TF SF ZF AF PF CF
* - - - * * * * *

Ejemplo: AL=FEH

BL=12H

SUB AL,BL

AL=ECH

TEST Test / Y lógico no almacenable(TEST/ nondestructive logical AND)

Formato: TEST destino,fuente

Descripción: Efectúa la operación de Y lógico entre dos operandos y fija las banderas sin almacenar el resultado. TEST es igual a la instrucción AND, pero sin el almacenamiento del resultado. Los operandos pueden tener tipo byte o palabra, y ser dos registros, un registro y una variable de memoria, un registro y una constante o una variable de memoria y una constante, pero no pueden ser, simultáneamente, dos variables de memoria.

Lógica: destino AND fuente

CF=0

OF=0

La siguiente, es la tabla del AND para las distintas combinaciones de bits:

A	B	A*B
0	0	0
0	1	0
1	0	0
1	1	1

Como regla general, se puede establecer que los bits testeados con un 0 se convierten en cero, y los bits testeados con un 1 no modifican su valor. TEST fija en 0 las banderas OF (overflow) y CF (acarreo), dado que ninguna operación lógica produce overflow y acarreo.

Banderas: OF DF IF TF SF ZF AF PF CF

0 - - - * * ? * 0

Ejemplo: AX=ECC8H = 1110 1100 1100 1000B

BX=1234H = 0001 0010 0011 0100B

TEST AX,BX

JZ CERO ;Bifurca a CERO si los bits de fuente con 1, se corresponden con bits 0 de destino

WAIT Esperar (WAIT)

Formato: WAIT

Descripción: WAIT, que se utiliza junto con la instrucción ESC, permite la comunicación con otros coprocesadores. WAIT coloca al procesador en estado de espera (idle) y chequea una línea de entrada llamada TEST cada cinco intervalos de reloj. Mientras dura la espera, el procesador puede atender interrupciones, pero, al terminar la ejecución de las correspondientes rutinas de atención (retorno de interrupción), vuelve al estado de espera. Cuando la línea TEST es activada por el coprocesador, ya sea porque esta libre o porque esta preparado para la ejecución de alguna instrucción, el microprocesador abandona el estado de espera y ejecuta la instrucción que esta a continuación de WAIT.

Banderas: OF DF IF TF SF ZF AF PF CF

- - - - - - - -

Ejemplo: WAIT ;Esperar que el coprocesador se libere

ESC 21H, MEM ;Pasar información al coprocesador

XCHG Intercambiar (eXCHanGe)

Formato: XCHG destino , fuente

Descripción: Intercambia los contenidos de los operandos: el contenido del operando fuente se transfiere al operando destino y viceversa. Los operandos pueden ser dos registros (excepto registros de segmento) o un registro y una variable de memoria. Por lo tanto, no se pueden intercambiar dos variables de memoria. Los operandos pueden ser de tipo BYTE o WORD, dependiendo de como están especificados, pero ambos deben ser del mismo tipo.

Lógica: tmp =destino

destino = fuente

fuentes = tmp

Bandas: OF DF IF TF SF ZF AF PF CF

Ejemplo: AX=1234H
BX=5678H
XCHG AX,BX
AX=5678H
BX=1234H

XLAT Traducir (Translate)

Formato: XLAT nombre_tabla

Descripción: XLAT realiza un posicionamiento sobre una tabla de bytes para obtener el valor correspondiente a un índice, y almacena dicho valor sobre el registro AL. Antes de ejecutar XLAT se debe cargar la dirección de inicio de la tabla en el registro BX y el desplazamiento dentro de ella en el registro AL.

De este modo, XLAT accede al byte que esta en la dirección de memoria BX+AL, y su contenido lo almacena en AL. La tabla debe ser de tipo byte y puede tener una longitud máxima de 256 bytes; el primer byte está en el desplazamiento cero.

Lógica: AL = byte direccionado por BX+AL

Bandas: OF DF IF TF SF ZF AF PF CF

Ejemplo: TABLA DB 1,2,4,8,16,32,64 ;Definición de TABLA

...
MOV BX,OFFSET TABLA ;BX=desplazamiento de TABLA
MOV AL,6 ;Recuperar 7mo. elemento
XLAT TABLA ;AL=64

XOR O lógico exclusivo (logical eXclusive OR)

Formato: XOR destino,fuente

Descripción: Efectúa el O exclusivo lógico entre los operandos fuente y destino, y almacena el resultado en el operando destino. Los operandos pueden tener tipo byte o palabra, y ser dos registros, un registro y una variable de memoria, un registro y un valor inmediato o una variable de memoria y un valor inmediato, pero no pueden ser, simultáneamente, dos variables de memoria.

Lógica: destino = destino "O lógico exclusivo" fuente

CF=0

OF=0

XOR fija en 1 el bit del resultado, por cada posición de los operandos en la cual ambos contengan bits distintos (un operando con 1 y el otro con 0); en las posiciones en que ambos operandos tienen bits iguales (1 0), el bit de resultado es 0. La siguiente, es la tabla del XOR para las distintas combinaciones de bits:

A	B	A@B
0	0	0
0	1	1
1	0	1
1	1	0

Como regla general, se puede establecer que los bits testeados con un 1 se invierten, y los bits testeados con un 0 no modifican su valor. XOR fija en 0 las banderas OF (overflow) y CF (acarreo), dado que ninguna operación lógica produce overflow y acarreo.

Banderas: OF DF IF TF SF ZF AF PF CF
0 - - - * * ? * 0

Ejemplo 1: AX=57CCH = 0101011111001100B
BX=E455H = 1110010001010101B

XOR AX,BX

AX=B399H = 1011001110011001B

Ejemplo 2: BX=E455H = 1110010001010101B
MEM=0259H = 0000001001011001B

XOR BX,MEM

BX=E60CH = 1110011000001100B

Ejemplo 3: MEM=59H = 01011001B
AL=CCH = 11001100B

XOR MEM,AL

MEM=95H = 10010101B

Ejemplo 4: BL=55H = 01010101B

XOR BL,0AH

BL=5FH = 01011111B

Ejemplo 5: MEM=02H = 00000010B

XOR MEM,FFH

MEM=FDH = 11111101B

CAPITULO 10 - SUBRUTINAS

Como fue definido anteriormente, un programa es una secuencia de instrucciones que, mediante el procesamiento de datos, permite obtener un determinado resultado. Este concepto presenta una serie de ventajas y desventajas, frente a cada problema particular que se desea resolver; por ejemplo, a veces existen limitaciones para la longitud de un programa, o bien aparecen dificultades para la lectura y comprensión de un programa fuente. Estos inconvenientes fueron estudiados ampliamente por las diversas teorías que tratan acerca de las técnicas de programación, obteniéndose, como principal conclusión, los conceptos de modularidad y estructura. En este sentido, se puede, entonces, definir un programa como una sucesión de módulos, donde cada modulo cumple una determinada función. El concepto de modulo introduce la idea de subprograma; un subprograma es una parte de un programa, que esta compuesta por una sucesión de instrucciones y que realiza una función muy determinada. La implementación de subprogramas depende del lenguaje de programación que se utilice, aunque generalmente se los conoce como subrutinas. En el macro-assembler existen dos tipos de subprogramas: subrutinas o procedimientos, y macros. En este capítulo se estudiará la implementación de subrutinas, mientras que el capítulo siguiente tratará acerca de las macros.

10.1. DEFINICION DE SUBRUTINAS

Una subrutina es un conjunto de sentencias que se escribe una sola vez en un programa y que puede ser utilizada todas las veces que sea necesario. En macro-assembler las subrutinas se denominan procedimientos, y siempre comienzan con una directiva PROC, y finalizan con una directiva ENDP. Una subrutina es invocada, cada vez que haga falta usarla, mediante la instrucción de llamada CALL. Cuando finaliza la ejecución de una subrutina, el control vuelve a la instrucción del programa siguiente a la de la llamada. Todas las subrutinas deben tener, como última instrucción ejecutable, una instrucción RET.

10.2. MECANISMO DE LLAMADA Y RETORNO

Al encontrar una instrucción CALL, el INTEL 8088-8086 almacena la dirección de retorno (el contenido del puntero de instrucción y, de ser necesario, el contenido del registro de segmento de código) al tope del stack y, luego, bifurca a la dirección señalada en la instrucción de llamada. Cuando, mientras esta ejecutando una subrutina, el 8088-8086 encuentra una instrucción RET, extrae del tope del stack la dirección de retorno y la carga en el puntero de instrucción, con lo que continúa la ejecución del programa desde el punto siguiente al que se invocó la subrutina. El siguiente ejemplo muestra como es la disposición de una subrutina, y que cambios se producen en la memoria cuando esta es llamada y cuando finaliza su ejecución:

04F0		CALL MY_PROC	;llamada a la subrutina
04F3	SIGUE: MOV	AX,BX	;instrucción siguiente a la llamada
		...	
0500	MY_PROC	PROC	;inicio de la subrutina
0500		MOV CL,6	;primer instrucción de la subrutina
		...	;instrucciones
		...	;adicionales
051E		RET	;retorno al programa llamador
051F	MY_PROC	ENDP	;final de la subrutina

Antes de la ejecución de CALL MY_PROC, el estado del stack y del IP es el siguiente:

SP=01FEH SS:01FEH=0000H
 SS:01FCH=0000H
 SS:01FAH=0000H
 SS:01F8H=0000H

IP=04F3H

Después de la ejecución de CALL MY_PROC:

SP=01FCH SS:01FEH=0000H
 SS:01FCH=04F3H
 SS:01FAH=0000H
 SS:01F8H=0000H

IP=0500H

Por último, después de la ejecución de RET:

SP=01FEH SS:01FEH=0000H
 SS:01FCH=04F3H
 SS:01FAH=0000H
 SS:01F8H=0000H

IP=04F3H

10.3. USO DE SUBROUTINAS

El INTEL 8088-8086 permite bifurcar a subrutinas ubicadas dentro del mismo segmento de código (subrutinas intrasegmento), o ubicadas en otro segmento de código (subrutinas intersegmento). Las subrutinas intrasegmento son aquellas que se definen con atributo de distancia NEAR, y la longitud de la dirección de retorno es de 16 bits, correspondientes al desplazamiento (contenido del puntero de instrucción); en cambio, las subrutinas intersegmento, son las que se definen con atributo de distancia FAR, y la longitud de la dirección de retorno es de 32 bits, de los cuales 16 bits corresponden al segmento (contenido del registro de segmento de código), y los restantes 16 bits corresponden al desplazamiento (contenido del puntero de instrucción). Existen dos modos de invocar subrutinas: llamada directa y llamada indirecta. La diferencia entre ambos ,todos consiste en el operando que se indica en la instrucción CALL, pero no existe diferencia entre ambos modos en lo que respecta al mecanismo de llamada y retorno. Una llamada directa es aquella en que el operando es un rótulo, el que se corresponde con el punto de entrada de una subrutina (declaración PROC). Por ejemplo:

CALL RUTINA

RUTINA es el rótulo de inicio de una subrutina que puede ser NEAR o FAR, dependiendo si esta en el mismo segmento de código o no. Una llamada indirecta es aquella en que el operando no es un rótulo, sino que es un registro o una posición de memoria, utilizados con cualquiera de los modos de direccionamiento.

Por ejemplo:

CALL BX	;la dirección de la subrutina es la indicada por ;el contenido del registro BX, que representa el ;desplazamiento en el mismo segmento de código
---------	--

CALL [BX]	;la dirección de la subrutina esta dada por el
-----------	--

;contenido de la dirección indicada en el registro BX

```
CALL WORD PTR ES:[BX][SI];la dirección de la subrutina esta dada  
;por una posición dentro del segmento extra.
```

Una subrutina puede invocar a otra u otras subrutinas. Por ejemplo, una subrutina que lee un carácter desde el teclado, lo decodifica, y luego llama a otra de varias subrutinas, según el resultado. El llamado a una subrutina desde otra subrutina se denomina anidamiento; en muchos casos, las subrutinas anidadas se refieren como niveles. Como la dirección de retorno de las subrutinas se almacena al tope del stack, siempre se retorna al nivel inmediato anterior.

Un ejemplo del uso de subrutinas anidadas es el siguiente:

```
CALL RUT1  
  
RUT1 PROC  
MOV AX,BX  
CALL RUT2  
...  
...  
RET  
RUT1 ENDP  
RUT2 PROC  
MOV AL,BL  
...  
...  
RET  
RUT2 ENDP
```

En este ejemplo, RUT2 es llamada desde RUT1; es decir, RUT1 y RUT2 están anidadas.

10.4. PASAJE DE PARAMETROS

Cuando se invoca una subrutina, generalmente se le deben pasar parámetros, que corresponden a los datos de entrada y salida de la misma. El pasaje de parámetros se hace por valor o por dirección, a través de registros o a través del stack. Pasar parámetros por valor significa que la subrutina recibirá los datos y podrá trabajar directamente sobre ellos. Pasar parámetros por dirección significa que la subrutina recibirá las direcciones donde residen los datos. En el pasaje de parámetros por valor, la subrutina no modifica los campos donde residen los datos en el programa llamador; en cambio, en el pasaje de parámetros por dirección, la subrutina trabaja sobre la misma zona de memoria donde están definidos los datos en el programa llamador. Si los datos, o sus direcciones, residen en registros, la subrutina deberá hacer referencia a esos mismos registros. Si los datos, o sus direcciones, residen en el stack, la subrutina debe extraerlos del mismo, respetando el orden en el que fueron almacenados. En las siguientes secciones estudiaremos el pasaje de parámetros por dirección a través del stack. Para ello, deberemos considerar ambos módulos, el llamador y el llamado, y la estructura del stack.

10.4.1. Interfaz con subrutinas de tipo NEAR.

Una subrutina NEAR reside en el mismo segmento de código que el programa llamador y accede al mismo segmento de datos que este. Por lo tanto, las direcciones a pasar serán sólo los desplazamientos.

Módulo llamador

Para pasar parámetros a una subrutina de tipo NEAR, se deben guardar en el stack solo el desplazamiento de las variables. Los pasos que se deben seguir para pasar parámetros son los siguientes:

- Obtener el desplazamiento de cada una de las variables a pasar mediante una instrucción LEA (cargar dirección efectiva) o mediante el uso del operador OFFSET, y colocarlo al tope del stack.
- Invocar a la subrutina mediante una instrucción CALL. El esquema general del pasaje de parámetros por dirección a una subrutina NEAR (usando la instrucción LEA), es el siguiente:

```

LEA    AX,VAR1      ;AX=desp. de la variable VAR1
PUSH   AX           ;salvar desp. de la variable VAR1
LEA    AX,VAR2      ;AX=desp. de la variable VAR2
PUSH   AX           ;salvar desp. de la variable VAR2
....
LEA    AX,VARn      ;AX=desp. de la variable VARn
PUSH   AX           ;salvar desp. de la variable VARn
CALL  MODULO        ;llamada a la subrutina MODULO

```

Si, en vez de la instrucción LEA, se utiliza el operador OFFSET, resulta lo siguiente:

```

PUSH   OFFSET VAR1  ;salvar desp. de la variable VAR1
PUSH   OFFSET VAR2  ;salvar desp. de la variable VAR2
....
PUSH   OFFSET VARn  ;salvar desp. de la variable VARn
CALL  MODULO        ;llamada a la subrutina MODULO

```

Modulo llamado

En general, para n parámetros, la estructura de la pila, después de haber efectuado el pasaje de variables, es la siguiente:

```

SS:SP      ----> dirección de retorno
SS:SP+2    ----> desplazamiento parámetro n
...
SS:SP+2*(N-I+1) ----> desplazamiento parámetro i
...
SS:SP+2*N  ----> desplazamiento parámetro 1

```

Para poder trabajar con los datos dentro del stack, es necesario poder tratarlo a este como si fuera un área de memoria de datos; por lo tanto, habrá que utilizar el registro BP para direccionar esos datos (SS es el registro de segmento asociado a desplazamientos referidos con BP). Pero, para poder usar BP, es necesario, primero, resguardar su valor en el stack, y asignarle un nuevo valor. Ese nuevo valor que se carga en BP es el actual valor de SP, de modo que BP y SP apunten a la misma dirección dentro del stack.

El acceso a los parámetros se debe hacer teniendo en cuenta que:

- La primera palabra del stack (BP) esta ocupada por el viejo valor de BP.
- La segunda palabra del stack (BP+2) esta ocupada por la dirección de retorno.
- La tercera palabra del stack (BP+6) esta ocupada por el desplazamiento del parámetro n .
- La palabra $BP+4+2*(n-i)$ esta ocupada por el desplazamiento del parámetro i .

Después de haber accedido a los parámetros y operado con ellos, se debe extraer del tope del stack el viejo valor de BP de modo de restaurar la estructura original del stack. A este punto, el tope del stack contiene la dirección de retorno al programa llamador. Como se explica mas arriba, antes de invocar una subrutina, se deben cargar en el stack los valores que se pasaran como parámetros. El espacio ocupado en el stack por estos valores, que equivale en bytes al doble de la cantidad de parámetros, sirve sólo durante la ejecución de la subrutina, y, por lo tanto, al regreso de la misma, debería dejarse liberado. Para hacer esto (liberar el espacio de los parámetros), basta con darle como argumento a la instrucción de retorno el valor, en bytes, usado por los parámetros. De este modo la instrucción RET con un valor hará que se sume ese valor al registro SP. El siguiente, es un ejemplo completo del pasaje de parámetros por dirección, a través del stack, a una subrutina NEAR. Escribir la subrutina, de tipo NEAR, SUMA, que efectúe la suma de dos números y devuelva su resultado. Los parámetros que recibe SUMA son, en ese orden, RESULTADO como parámetro de salida, y VALOR 2 y VALOR 1 como parámetros de entrada. Escribir el programa llamador que invoque a la subrutina SUMA.

```

;          SUBROUTINA SUMA
;

```

```

CODIGO      SEGMENT      PARA PUBLIC 'CODIGO'
              PUBLIC      SUMA
              ASSUME      CS:CODIGO
SUMA         PROC:NEAR
              PUSH  BP           ;Guardar viejo valor de BP
              MOV   BP,SP       ;asignar nuevo valor a BP
              PUSH  AX           ;resguardar registro AX
              PUSH  BX           ;resguardar registro BX
              MOV   BX,[BP]+10   ;BX=desp. de VALOR 1
              MOV   AX,[BX]      ;AX=VALOR 1
              MOV   BX,[BP]+8    ;BX=desp. de VALOR 2
              ADD   AX,[BX]      ;AX=VALOR 1 + VALOR 2
              MOV   BX,[BP]+6    ;BX=desp. de RESULTADO
              MOV   [BX],AX      ;RESULTADO=VALOR 1 + VALOR 2
              POP   BX           ;restaurar BX
              POP   AX           ;restaurar AX
              POP   BP           ;restaurar viejo valor de BP
              RET    6
SUMA         ENDP
CODIGO      ENDS

;          PROGRAMA L_SUMA --- Invoca a la subrutina SUMA
;
;          EXTRN SUMA:NEAR
;
DATOS      SEGMENT      PARA PUBLIC 'DATOS'
VALOR_1    DW           548
VALOR_2    DW           168
RESULTADO  DW           ?
DATOS      ENDS
;
PILA       SEGMENT      STACK
              DB          128 DUP('pila')
PILA       ENDS
;
CODIGO     SEGMENT
L_SUMA     PROC
              ASSUME      CS:CODIGO,DS:DATOS,SS:PILA
;
;poner dirección de retorno al DOS o al DEBUG
;
              PUSH  DS
              SUB   AX,AX
              PUSH  AX
;
;direccionar segmento de datos
;
              MOV   AX,DATOS
              MOV   DS,AX
;
;pasar direcciones de los parámetros
;
              LEA   AX,VALOR_1   ;AX=desp. de VALOR_1
              PUSH  AX
              LEA   AX,VALOR_2   ;AX=desp. de VALOR_2

```



```

                PUSH    AX
                LEA     AX,RESULTADO      ;AX=desp. de RESULTADO
                PUSH    AX
;llamado a la subrutina SUMA
;
                CALL    SUMA
;
;final
;
                RET
L_SUMA        ENDP
CODIGO        ENDS
                END     L_SUMA

```

10.4.2. Interfaz con subrutinas de tipo FAR.

El trabajo con subrutinas de tipo FAR es, en general, similar que con subrutinas de tipo NEAR; la única diferencia consiste en que, además del desplazamiento, se debe considerar también la componente de segmento para cada dirección que se pasa.

Modulo llamador

Para pasar parámetros a una subrutina de tipo FAR, se debe guardar en el stack el desplazamiento y el segmento de cada una de las variables. Los pasos que se deben seguir para pasar parámetros son los siguientes:

- a) Para cada una de las variables a pasar:
 - i. Obtener el segmento, mediante la invocación directa del registro de segmento o mediante el operador SEG, y colocarlo al tope del stack.
 - ii. Obtener el desplazamiento, mediante una instrucción LEA (cargar dirección efectiva) o mediante el operador OFFSET, y colocarlo al tope del stack.
- b) Invocar a la subrutina mediante una instrucción CALL.

El esquema general del pasaje de parámetros por dirección a una subrutina FAR (usando, para segmento y desplazamiento, la invocación directa del registro de segmento y la instrucción LEA, respectivamente), es el siguiente:

```

PUSH    DS          ;salvar seg. de la variable VAR1
LEA     AX,VAR1      ;AX=desp. de la variable VAR1
PUSH    AX          ;salvar desp. de la variable VAR1
PUSH    DS          ;salvar seg. de la variable VAR2
LEA     AX,VAR2      ;AX=desp. de la variable VAR2
PUSH    AX          ;salvar desp. de la variable VAR2
....
PUSH    DS          ;salvar seg. de la variable VARn
LEA     AX,VARn      ;AX=desp. de la variable VARn
PUSH    AX          ;salvar desp. de la variable VARn
CALL    MODULO       ;llamada a la subrutina MODULO

```

Si se utilizan, para segmento y desplazamiento, los operadores SEG y OFFSET, respectivamente:

```

PUSH    SEG VAR1     ;salvar seg. de la variable VAR1
PUSH    OFFSET VAR1  ;salvar desp. de la variable VAR1
PUSH    SEG VAR2     ;salvar seg. de la variable VAR2
PUSH    OFFSET VAR2  ;salvar desp. de la variable VAR2
....
PUSH    SEG VARn     ;salvar seg. de la variable VARn
PUSH    OFFSET VARn  ;salvar desp. de la variable VARn
CALL    MODULO       ;llamada a la subrutina MODULO

```

Modulo llamado

En general, para n parámetros, la estructura de la pila, después de haber efectuado el pasaje de variables, es la siguiente:

SS:SP ----> desp. de la dirección de retorno
 SS:SP+2 ----> seg. de la dirección de retorno
 SS:SP+4 ----> desplazamiento parámetro n
 SS:SP+6 ----> segmento parámetro n

SS:SP+4*(N-I+1) ----> desplazamiento parámetro i
 SS:SP+4*(N-I+1)+2 ----> segmento parámetro i

SS:SP+4*N ----> desplazamiento parámetro 1
 SS:SP+4*N+2 ----> segmento parámetro 1

Al igual que con subrutinas NEAR, se debe salvar al tope del stack el valor de BP, y luego asignarle a BP un nuevo valor, que ser el de SP.

El acceso a los parámetros se debe hacer teniendo en cuenta que:

- La primera palabra del stack (BP) esta ocupada por el viejo valor de BP.
- La segunda palabra del stack (BP+2) esta ocupada por el desplazamiento de la dirección de retorno.
- La tercera palabra del stack (BP+4) esta ocupada por el segmento de la dirección de retorno.
- La cuarta palabra del stack (BP+6) esta ocupada por el desplazamiento del parámetro n.
- La quinta palabra del stack (BP+8) esta ocupada por el segmento del parámetro n.
- La palabra BP+6+4*(n-i) esta ocupada por el desplazamiento del parámetro i.
- La palabra BP+8+4*(n-i) esta ocupada por el segmento del parámetro i.

Después de haber accedido a los parámetros y operado con ellos, se debe extraer del tope del stack el viejo valor de BP de modo de restaurar la estructura original del stack. A este punto, el tope del stack contiene la dirección de retorno (desplazamiento y segmento), al programa llamador. El espacio ocupado en el stack por las direcciones de los parámetros es, en bytes, cuatro veces la cantidad de parámetros. Para liberar ese espacio basta con darle como argumento a la instrucción de retorno el valor, en bytes, usado por los parámetros. De este modo la instrucción RET con un valor hará que se sume ese valor al registro SP.

El siguiente, es un ejemplo completo del pasaje de parámetros por dirección, a través del stack, a una subrutina NEAR. Escribir la subrutina, de tipo FAR, SUMA, que efectúe la suma de dos números y devuelva su resultado. Los parámetros que recibe SUMA son, en ese orden, RESULTADO como parámetro de salida, y VALOR 2 y VALOR 1 como parámetros de entrada. Todos los parámetros son pasados por dirección a través del stack. Escribir el programa llamador que invoque a la subrutina SUMA.

```
; SUBROUTINA SUMA
CODIGO SEGMENT PARA PUBLIC 'CODIGO'
        PUBLIC SUMA
        ASSUME CS:CODIGO

SUMA PROC:FAR
        PUSH BP ;Guardar viejo valor de BP
        MOV BP,SP ;asignar nuevo valor a BP
        PUSH AX ;resguardar registro AX
        PUSH BX ;resguardar registro BX
        PUSH ES ;resguardar registro ES
        LES BX,[BP]+10 ;ES=seg. de VALOR 1
        ;BX=desp. de VALOR 1
        MOV AX,ES:[BX] ;AX=VALOR 1
        LES BX,[BP]+8 ;ES=seg. de VALOR 2
        ;BX=desp. de VALOR 2
        ADD AX,ES:[BX] ;AX=VALOR 1 + VALOR 2
        LES BX,[BP]+6 ;ES=seg. de RESULTADO
        ;BX=desp. de RESULTADO
        MOV ES:[BX],AX ;RESULTADO=VALOR 1 + VALOR 2
        POP ES ;restaurar ES
        POP BX ;restaurar BX
        POP AX ;restaurar AX
        POP BP ;restaurar viejo valor de BP
        RET 12
```

```

SUMA      ENDP
CODIGO    ENDS

;          PROGRAMA L_SUMA --- Invoca a la subrutina SUMA
          EXTRN SUMA:FAR

;
DATOS     SEGMENT      PARA PUBLIC 'DATOS'
VALOR_1   DW           548
VALOR_2   DW           168
RESULTADO DW           ?
DATOS     ENDS
PILA      SEGMENT      STACK
          DB           128 DUP('pila')
PILA      ENDS
CODIGO    SEGMENT
L_SUMA    PROC FAR
          ASSUME        CS:CODIGO,DS:DATOS,SS:PILA

;
;poner dirección de retorno al DOS o al DEBUG
;
          PUSH DS
          SUB  AX,AX
          PUSH AX

;
;direccionar segmento de datos
;
          MOV  AX,DATOS
          MOV  DS,AX

;
;pasar direcciones de los parámetros
;
          PUSH DS          ;cargar seg. de VALOR_1
          LEA  AX,VALOR_1   ;AX=desp. de VALOR_1
          PUSH AX          ;cargar desp. de VALOR_1
          PUSH DS          ;cargar seg. de VALOR_2
          LEA  AX,VALOR_2   ;AX=desp. de VALOR_2
          PUSH AX          ;cargar desp. de VALOR_2
          PUSH DS          ;cargar seg. de RESULTADO
          LEA  AX,RESULTADO ;AX=desp. de RESULTADO
          PUSH AX          ;cargar desp. de RESULTADO

;
;llamado a la subrutina SUMA
;
          CALL SUMA

;
;final
;
          RET
L_SUMA    ENDP
CODIGO    ENDS
          END  L_SUMA

```

10.4.3. Uso de la directiva STRUC.

La directiva STRUC es muy útil para el manejo de parámetros. Se utiliza, en especial, para:

- a) Especificar la estructura del stack.
- b) Simplificar el acceso a los parámetros.
- c) Simplificar el cálculo del argumento de retorno.

Para analizar cada uno de estos aspectos, tomemos de ejemplo una subrutina FAR.

Estructura del stack

La porción del stack involucrada con el llamado a una subrutina se puede definir a través de una estructura:

```
PARAMS      STRUC
              DW    ?      ;BP salvado por la subrutina
RETORNO      DD    ?      ;dirección de retorno
VARn  DD    ?      ;dirección del parámetro VARn
...
VARi  DD    ?      ;dirección del parámetro VARi
...
VAR2  DD    ?      ;dirección del parámetro VAR2
VAR1  DD    ?      ;dirección del parámetro VAR1
PARAMMS      ENDS
```

Para el caso de una subrutina NEAR, la diferencia consiste en el tamaño de los datos de las variables de la estructura: en lugar de ser de tipo doble palabra deben ser de tipo palabra, ya que las direcciones pasadas corresponden sólo al valor del desplazamiento de las variables.

Acceso a los parámetros

Igual que antes, el acceso a los parámetros se efectúa mediante referencias a BP.

Pero ya no es necesario "hacer las cuentas" para conocer en qué posición del stack está la dirección que se desea recoger, pues ahora este dato viene dado por la estructura que se definió. Por ejemplo, para obtener la dirección del parámetro VARi y copiar su contenido en el registro AX:

```
LES    SI,[BP].VARi    ;ES=segmento de VARi
                          ;SI=desplazamiento de VARi
MOV    AX,ES:[SI]      ;AX=valor de VARi
```

Argumento de retorno

Para calcular el argumento de retorno se debe definir un símbolo, que será siempre el mismo, independiente del número de parámetros. La definición de este símbolo será:

```
NRET    EQU    OFFSET VAR1-OFFSET RETORNO
```

y en la subrutina, la instrucción de retorno será:

```
RET     NRET
```

CAPITULO 11 MACRO-INSTRUCCIONES

11.1. INTRODUCCION

Las macros, al igual que las subrutinas, son conjuntos de instrucciones que se escriben una sólo vez y pueden ser utilizadas todas las veces que sea necesario. Sin embargo, a diferencia de las subrutinas, las macros se expanden cuando se las invoca. Esto significa que, cuando se llama a una macro dentro del programa, el compilador copia (expande) las instrucciones de la macro a partir del punto de llamada. El uso de macros tiene algunas ventajas y desventajas, que siempre deberán ser tenidas en cuenta antes de su utilización. Como principal desventaja, el uso de macros produce programas fuente mas largos, dado que la secuencia de instrucciones de una macro se copia en el programa fuente cada vez que se la invoca. En cuanto a las ventajas, la mas importante es la facilidad de programación, pues una macro se escribe sólo una vez, y el compilador es el que se encarga de copiarla cada vez que se la invoca. En este sentido, el uso de macros es ventajosa en lo referido a la velocidad de programación y prueba, dado que no es necesario probar una macro en cada parte del programa en que se la usa sino que basta con verificar su correcto funcionamiento sólo una vez. Otra importante ventaja es la posibilidad de usar parámetros, lo cual significa que una misma macro puede ser usada para la misma función, pero con variables distintas cada vez. El uso de macros es, también, importante desde el punto de vista que permite "crear" instrucciones no existentes en el assembler. Por ejemplo, se puede preprogramar una macro que limpie todos los registros generales, y cada vez que haya que borrar todos los registros, se llama a esa macro en lugar de escribir todas las instrucciones. Es por esto último que, en muchos lados, las macros se refieren como macro-instrucciones.

11.2. DEFINICION DE MACROS

Básicamente, una macro tiene 3 partes: el header o encabezamiento, que incluye el nombre de la macro y los parámetros que utiliza; el cuerpo, donde están escritas todas las sentencias de assembler que definen lo que hace la macro; el final, que marca el punto de terminación de la macro.

Un ejemplo de una macro es el siguiente:

SUMA	MACRO	VALOR1,VALOR2,RESULT	;encabezamiento
	MOV	AX,VALOR1	;cuerpo
	ADD	AX,VALOR2	;de la
	MOV	RESULT,AX	;macro
	ENDM		;final

11.3. USO DE MACROS

Existen dos modos de usar macros en los programas assembler: a través de la escritura en el programa fuente, o a través de la invocación a una biblioteca de macros.

11.3.1. Escritura de macros en el programa fuente.

En este caso, se deben definir las macros al comienzo del programa fuente, y luego invocarlas cuando sea necesario. El uso de macros de este modo tiene la desventaja de limitar la cantidad de memoria disponible para el programa. Otro inconveniente que se produce es que, si una determinada macro debe ser usada en otros programas, entonces se deberá reescribir en cada programa que la necesite.

El siguiente es un ejemplo completo de macros escritas en un programa fuente:

TITLE NEWDIAG - Programa que gráfica una línea en diagonal

```
;        Compilado como: MASM NEWDIAG;
;        Linkeditado como: LINK NEWDIAG;
;*****
```

```

;      Definiciones de macros
;*****
PUSH_REGS    MACRO      Lista_reg
;;      Salvar registros al tope del stack.
            IRP          reg,<Lista_reg>
            PUSH        reg
            ENDM
            ENDM

POP_REGS     MACRO      Lista_reg
;;      Recuperar registros desde el stack.
            IRP          reg,<Lista_reg>
            POP         reg
            ENDM
            ENDM

MOV_CURSOR  MACRO
;;      Mover cursor a las posiciones de fila y columna indicadas
;;      en los registros DH y DL, respectivamente.
            MOV         AH,2           ;;Seleccionar opción de movimiento del cursor
            INT         10H
            ENDM

PRINT_AL    MACRO
;;      Imprimir el carácter indicado en AL.
            MOV         CX,1           ;;Exhibir 1 solo carácter
            MOV         AH,14         ;;Seleccionar opción para display
            INT         10H
            ENDM

;*****
;      Comienzo del programa.
;*****
PILA        SEGMENT      PARA STACK 'PILA'
            DB          64 DUP ('STACK ')

PILA        ENDS
CODIGO      SEGMENT      PARA 'CODIGO'
DIAG_LINE   PROC          FAR
            ASSUME       CS:CODIGO,SS:PILA
;      Cargar en el stack los valores para que el
;      programa pueda retornar al DEBUG
            PUSH         DS           ;Direc. de seg. en el stack
            MOV         AX,0         ;Limpiar un registro
            PUSH         AX          ;Desplazamiento cero en el stack
            PUSH_REG     <AX,BX,CX,DX> ;Salvar registros
            MOV         AH,15        ;Fijar BH para pagina de display
            INT         10H
            MOV         AH,0         ;Seleccionar 80x25, B/N, alfab,t.
            MOV         AL,2
            INT         10H
            MOV         DX,0         ;Inicio en fila 0, columna 0

SET_CRSR:   MOV_CURSOR
            MOV         AL,2         ;Fijar nueva posición del cursor
            PRINT_AL
            INC         DH           ;Apuntar a la próxima fila
            INC         DL           ;Apuntar a la próxima columna
            CMP         DH,25        ;Final de la pantalla?
            JNE         SET_CRSR

```

```

                                POP_REGS    <DX,CX,BX,AX>      ;Recuperar registros
                                RET          ;Finalizar
DIAG_LINE    ENDP
CODIGO       ENDS
                                END          DIAG_LINE

```

11.3.2. Bibliotecas de macros.

Las bibliotecas de macros son archivos de disco que contienen definiciones de macros, que pueden ser necesarias en varios programas. Una vez creada una biblioteca de macros, el archivo puede ser leído en cualquier programa fuente; de este modo, todas las macros pertenecientes a esa biblioteca están disponibles para ser usadas en el programa. El uso de las macros de una biblioteca se efectúa mediante la referencia al nombre.

11.3.2.1. Creación de bibliotecas de macros.

Una biblioteca de macros se crea con cualquier programa editor de textos. Las macros de una biblioteca deben ser rutinas de propósitos generales, que puedan ser usadas en cualquier programa. Es decir, las macros de una biblioteca deberían ser diseñadas para que puedan ser entendidas fácilmente, y para que no generen dificultades en los programas que las usen. Las siguientes, son algunas recomendaciones a tener en cuenta para diseñar bibliotecas de macros:

- Documentar las macros tanto como sea posible, mediante la inclusión de comentarios. Esto es importante si se tiene en cuenta que una macro debe ser entendida por todo aquel que la use.
- Usar el operador ;; para entrar comentarios, y usar el tabulador para separar campos. Esto ayuda a disminuir el tamaño de los programas y, por lo tanto, a hacer más rápido el ensamblado.
- Las macros deben ser lo más generales posibles.
- Definir como LOCAL los rótulos de cada macro.
- Salvar los registros que usa la macro, excepto que sean registros de salida. En general, es conveniente colocarlos al tope del stack al comienzo de la macro y extraerlos del stack al final de la macro.
- Si es necesario, invocar otras macros desde la macro que se está definiendo. Esto hace que no se repitan bloques de instrucciones.

11.3.2.2. Invocación de bibliotecas de macros.

Para leer una biblioteca de macros en un programa fuente, se debe usar la pseudo-op INCLUDE con el nombre del archivo.

Por ejemplo:

```
INCLUDE    MACRO.LIB
```

Cuando el compilador encuentra esta sentencia, lee el archivo MACRO.LIB en el programa fuente.

Sin embargo, el uso del INCLUDE en estas condiciones hace que el compilador lea la biblioteca de macros en ambas pasadas. Para evitar esto, es conveniente encerrar el INCLUDE en la directiva condicional IF1.

En este caso, el ejemplo anterior quedaría como:

```
IF1
INCLUDE MACRO.LIB
ENDIF
```

Así, la biblioteca de macros será leída por el compilador sólo durante la primera pasada. Una desventaja que surge del uso de bibliotecas de macros es que cuando el compilador incluye una biblioteca todas las macros que existen en ella son leídas y almacenadas en el área de trabajo de memoria que usa el compilador. De este modo, las macros que no serán usadas están ocupando, innecesariamente, espacio de memoria que necesita el compilador. Esto puede llegar a producir alguna condición de error de fuera de memoria.

Para evitar este posible problema, se puede utilizar la directiva PURGE para sacar las macros innecesarias.

Por ejemplo:

```
INCLUDE    MACRO.LIB
PURGE CLS,HOME,LOCATE
```

En el ejemplo, se incluye una biblioteca de macros residente en el archivo MACRO.LIB, y se dejan de lado las macros CLS, HOME y LOCATE, que pertenecen a esa biblioteca.

CAPITULO 12 INTERRUPCIONES

Este capítulo esta dedicado a las interrupciones del microprocesador INTEL 8088-8086. En una primera parte, veremos una serie de conceptos generales y, posteriormente, analizaremos mas en detalle los distintos tipos de interrupciones.

12.1. INTRODUCCION

Cualquier microprocesador de cualquier computadora, actúa como un regulador del sistema, es decir que esta en conocimiento de todo lo que ocurre. Así, por ejemplo, cuando se están transfiriendo datos entre la memoria y un disco, el microprocesador es el responsable de la ejecución de las instrucciones que manejan dicha transferencia. El microprocesador debe, también, estar habilitado para poder detectar la ocurrencia de eventos externos que requieran su atención. Supongamos, por ejemplo, una microcomputadora y su teclado; sabemos que cuando se oprime una tecla, la microcomputadora recibe el carácter a través de una puerta de E/S. El microprocesador debe, entonces, saber en que momento tiene que leer esa puerta de E/S.

Una posibilidad es que el microprocesador haga un poleo (recorrida cíclica) permanente de los distintos recursos del sistema para ver si hay algo en las puertas de E/S. Este mecanismo resulta muy lento, pues el microprocesador dispondría de poco tiempo para ejecutar instrucciones. Otra posibilidad es a través de las interrupciones. Por ejemplo, cuando se oprime una tecla del teclado, este le envía una señal al microprocesador; entonces, el microprocesador recibe y decodifica esa señal, y acude a leer la puerta de E/S del teclado para recoger de allí el carácter tipeado. La señal mencionada anteriormente se denomina interrupción, y el mecanismo del microprocesador que es responsable de atenderla se denomina mecanismo de interrupción.

12.2. TIPOS DE INTERRUPCIONES

Así como se mencionó la existencia de interrupciones debidas a eventos externos (interrupciones externas), que son activadas por los periféricos, existen otros tipos de interrupciones, denominadas internas.

Las interrupciones internas pueden ser generadas por el usuario o por el propio microprocesador.

Las interrupciones generadas por el usuario se hacen a través del uso de instrucciones específicas; estas instrucciones envían una señal de interrupción al microprocesador, quien la decodifica e interpreta y, luego, actúa en consecuencia. Las interrupciones generadas por el microprocesador son aquellas producidas por incidentes de programas. Por ejemplo, el microprocesador se interrumpe a si mismo cuando, en un programa, se intenta hacer una división por cero. En resumen, una interrupción es un mecanismo que altera el funcionamiento normal del microprocesador; es, por lo tanto, un elemento de hardware.

12.3. MECANISMO DE INTERRUPCIONES. ENTORNO DE UN PROGRAMA

Una interrupción se puede producir en cualquier momento; no existe modo alguno para que el microprocesador pueda predecirla. De este modo, es necesario que el mecanismo de interrupciones suministre dos facilidades básicas; en primer lugar, cuando llega la señal de interrupción debe ser posible optar por deshabilitar el microprocesador de lo que esta haciendo en ese momento; en segundo termino, para poder atender una interrupción, el microprocesador debe poder salvar la información de lo que esta haciendo en ese momento, es decir, debe poder resguardar el entorno del programa que esta ejecutando de manera de que se pueda retomar la ejecución del programa desde el punto en el que fue interrumpido. El primer requerimiento (opción a deshabilitar el microprocesador de lo que esta haciendo), esta satisfecho a través del uso de una bandera de control denominada IF (Interrupt Flag); si IF=0, el microprocesador ignorara cualquier señal de interrupción que le sea enviada desde el exterior; sólo cuando IF=1, el microprocesador responder a requerimientos de interrupciones externas. El uso de una bandera para satisfacer o no cierto tipo de interrupciones introduce los conceptos de interrupciones enmascarables e interrupciones no enmascarables. Las interrupciones enmascarables son las provocadas por eventos externos, y su atención o no esta manejada por el uso de la bandera de interrupciones IF; las interrupciones no enmascarables son las internas, y no se manejan con ninguna bandera: estas interrupciones son siempre atendidas.

El segundo requerimiento (resguardo del entorno del programa), es satisfecho por el hardware a través de un mecanismo que consiste en cargar en el stack el registro de banderas, deshabilitar interrupciones externas

(IF=0) y cargar en el stack los registros CS e IP. Una vez que ha sido resguardado el actual estado del procesador (entorno del programa), el mecanismo de interrupciones transfiere el control a un programa especial residente en una dirección específica de memoria. Este programa especial, llamado rutina de atención de interrupción, es responsable del manejo del evento externo que provocó la interrupción.

Cuando esta tarea se completa, el control retorna al programa que se estaba ejecutando cuando se produjo la interrupción; este proceso, denominado vuelta o retorno de interrupción, recupera el estado del procesador sacando del stack los registros IP, CS y de banderas, en ese orden (que es el orden inverso en el que fueron guardados al tope del stack).

Es importante notar que el mecanismo de interrupción descrito no considera el resguardo de los demás registros; entonces, si alguna de las rutinas de atención de interrupciones altera algún registro, esta debería resguardarlo y luego recuperarlo de modo que el programa interrumpido no se "de cuenta".

12.4. RUTINAS DE ATENCION DE INTERRUPCIONES

Las rutinas de atención de interrupciones son programas especiales que satisfacen los requerimientos solicitados por las interrupciones. Cada interrupción tiene asociada una rutina de atención, en una determinada dirección de memoria. Cada señal de interrupción va acompañada por un código de interrupción, de un byte de tamaño, que contiene un código único. Este código, que es recibido por el microprocesador inmediatamente a continuación de la señal de interrupción, permite identificar la interrupción y, en consecuencia, determinar la dirección de memoria de la correspondiente rutina de atención de esa interrupción. Al ser el código de interrupción de 1 byte de tamaño, puede haber hasta 256 códigos distintos. El código de interrupción va de 00 a FF en notación hexadecimal.

El Kbyte mas bajo de la memoria (dirección física 00000H a 003FFH) esta reservado para una tabla de direcciones de las rutinas de atención de interrupciones; cada entrada de esta tabla de direcciones se denomina vector de interrupción.

Cada vector de interrupción tiene 4 bytes de tamaño y contiene una dirección de memoria, donde el valor de un segmento de código esta en los dos bytes mas altos, y el valor de un desplazamiento esta en los dos bytes mas bajos. El procesador, para seleccionar el vector de interrupción que corresponde, multiplica por 4 el código de interrupción que recibió. Luego, coloca el valor del segmento de código en CS y el valor del desplazamiento en IP, y transfiere el control (comienza a ejecutar) a la rutina correspondiente.

La siguiente tabla permite observar los distintos tipos de interrupciones contemplados por el microprocesador INTEL 8088-8086:

DIRECCION	CODIGOS	DESCRIPCION
0000-007F	00 - 1F	Interrupciones del BIOS
0080-00FF	20 - 3F	Interrupciones del DOS
0100-017F	40 - 5F	Interrupciones del BIOS
0180-019F	60 - 67	Interrupciones del usuario
01A0-01BF	68 - 6F	Reservado
01C0-01DF	70 - 77	Interrupciones de E/S
01E0-01FF	78 - 7F	Reservado
0200-03C3	80 - F0	Interrupciones del BASIC
03C4-03FF	F1 - FF	Reservado

Posteriormente se vera, mas en detalle, cada una de las interrupciones.

12.5. EJEMPLO DEL MECANISMO DE INTERRUPCIONES

El siguiente es un ejemplo completo, con la descripción de cada uno de los pasos, de lo que sucede cuando se produce una interrupción:

- El microprocesador esta por ejecutar la instrucción MOV CX,30.
- Llega una interrupción, cuyo código es 4AH.
- Se almacenan, al tope del stack, los actuales valores de las banderas de estado, CS e IP.
- El microprocesador obtiene en el vector de interrupción correspondiente, que se encuentra en la dirección 0000:0128 (4AH * 4), una nueva dirección. Esta nueva dirección, F000:1805, corresponde a la dirección de inicio de la rutina de atención de la interrupción con código 4AH.
- Carga F000H en CS y 1805H en IP, y comienza la ejecución desde esa nueva dirección.

- f) Al encontrar una instrucción de retorno de interrupción, carga del stack los valores de IP, CS y banderas de estado, y continúa la ejecución con la instrucción MOV CX,30 (punto donde se produjo la interrupción).

12.6. INTERRUPCIONES DEL BIOS

Cuando se enciende la computadora, se ejecutan una serie de instrucciones que residen en la memoria ROM (Read Only Memory). Estas instrucciones permiten que se carguen en la memoria los programas necesarios para la inicialización y administración de los recursos del equipo. Estos programas están incluidos dentro del BIOS (Basic Input Output System), que maneja todo lo referente a las operaciones de E/S, y dentro del DOS (Disk Operating System), que regula el uso de los diferentes recursos del sistema. El BIOS dispone de una serie de interrupciones que facilitan acceder a la potencia del microprocesador 8088-8086.

Las interrupciones para el BIOS corresponden a los tipos 00H a 1FH y 40H a 5FH, sin embargo, no todas están actualmente usadas.

Básicamente, las interrupciones del BIOS se pueden dividir en los siguientes grupos:

- 1) Interrupciones del microprocesador 8088-8086
- 2) Interrupciones del controlador 8259
- 3) Puntos de entrada al BIOS
- 4) Rutinas del usuario
- 5) Punteros a tablas de datos

Las interrupciones del BIOS pueden ser accedidas a través de la ejecución de la instrucción INT con el código que corresponda.

12.6.1. Interrupciones del microprocesador 8088-8086.

Estas interrupciones son todas internas, generadas por el propio microprocesador, cuando se cumple determinada condición de excepción durante el proceso de las instrucciones.

Interrupción 00H --- Error en división. Se activa si una instrucción de división (DIV o IDIV), produce un resultado de mayor longitud que el registro del resultado (AL o AX, según el tamaño de los operandos), o si se efectúa una división por cero. En ambos casos, hace finalizar la ejecución del programa, emite un mensaje de error y devuelve el control al Sistema Operativo (DOS).

Interrupción 01H --- Ejecución paso a paso. Permite que los programas puedan ejecutarse "paso a paso", es decir, de a una instrucción por vez, para facilitar la depuración. El comando T (Trace) del DEBUG usa la interrupción 01H.

Interrupción 02H --- Interrupción no enmascarable. Es usada por el BIOS para reportar errores de almacenamiento de datos en expansiones de memoria. A diferencia de otras interrupciones externas que pueden ser enmascaradas mediante el encendido y apagado de la bandera IF, la interrupción 02H no es enmascarable. El coprocesador matemático 8087 también usa la interrupción 02H para reportar errores.

Interrupción 03H --- Punto de ruptura. Permite la ejecución de un programa hasta que el microprocesador alcance una dirección especificada de parada.

Interrupción 04H --- Overflow. Se activa si la bandera de overflow esta encendida (OF=1) o cuando el 8088-8086 ejecuta una instrucción INTO (Interrupt if Overflow).

Interrupción 05H --- Imprimir pantalla. Hace que lo que esta escrito en la pantalla se transfiera a la impresora, salvando, y luego recuperando, la actual posición del cursor. La interrupción 05H produce el mismo efecto que al oprimir la tecla <PRINT_SCREEN>.

12.6.2. Interrupciones del controlador 8259.

El controlador de interrupciones 8259 es un chip de la tarjeta madre del sistema (System Board), que puede aceptar señales de petición de interrupciones (es decir, interrupciones enmascarables) de hasta 8 dispositivos diferentes. Este controlador actúa como una oficina electrónica de trafico, que le pasa al procesador la señal de pedido de interrupción y el código de identificación del dispositivo. Los dispositivos conectados al controlador 8259 son: el temporizador del sistema, el teclado y el controlador de disco.

Interrupción 08H --- Timer del sistema. El timer del sistema es un chip de la tarjeta madre, identificado como 8353. El timer emite una señal de interrupción 08H cada 0,0549254 segundos, es decir, unas 18,2 veces por segundo aproximadamente. Cada interrupción del timer se denomina "tic", y produce la actualización de un contador de tiempo. Este contador es una doble palabra, que se encuentra en el área de datos del BIOS.

Interrupción 09H --- Teclado. Se activa cuando se presiona alguna tecla del teclado, y corresponde a una interrupción del sistema. Un usuario puede acceder al teclado a través de la interrupción 16H.

Interrupción 0EH --- Unidad de diskette. El controlador de discos usa esta interrupción cuando una unidad de diskettes debe leer o grabar información; corresponde a una interrupción del sistema. Un usuario puede acceder a una unidad de diskettes a través de la interrupción 13H.

12.6.3. Puntos de entrada al BIOS.

Algunas interrupciones de este grupo efectúan funciones de E/S, permitiendo transferir información a o desde los periféricos del sistema; otras, efectúan operaciones de control.

Interrupción 10H --- E/S de pantalla. Dispone de 16 diferentes funciones (según el valor de A)H, que se usan para fijar el modo de funcionamiento de la pantalla, y para transferir gráficos y/o textos. En el capítulo correspondiente a la pantalla se verán en detalle todas las posibilidades de esta interrupción.

Interrupción 11H --- Chequeo del equipamiento. Esta interrupción determina cual es la configuración de la computadora, devolviendo una determinada mascara en el registro AX.

El siguiente es el detalle de los distintos bits de AX:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
		0				0								1	

- bits 15 a 14: Cantidad de impresoras conectadas.
- bit 12 : Existencia del adaptador de juegos:
 - 0 = no existente
 - 1 = existente
- bits 11 a 9 : Cantidad de tarjetas de comunicación RS-232 instaladas.
- bits 7 a 6 : Cantidad de unidades de diskettes:
 - 00=1, 01=2, 10=3, 11=4
- bits 5 a 4 : Identificación del monitor:
 - 00 = sin usar
 - 01 = 40 x 25, B/N, Adaptador Color/Gráficos
 - 10 = 80 x 25, B/N, Adaptador Color/Gráficos
 - 11 = 80 x 25, B/N, Adaptador Monocromático
- bits 3 a 2 : Cantidad de memoria RAM en la tarjeta madre del sistema:
 - 00 = 16 Kbytes
 - 01 = 32 Kbytes
 - 10 = 48 Kbytes
 - 11 = 64 Kbytes
- bit 1 : Identificación del coprocesador matemático 8087:
 - 0 = no instalado
 - 1 = instalado

Según estas especificaciones, para una computadora con monitor monocromático, dos unidades de diskettes y una impresora, la interrupción 11H devolver 4271H en el registro AX. El uso de esta interrupción es importante en el desarrollo de software para microcomputadoras de las cuales no se conoce su configuración; de este modo, según el resultado de AX se puede optar por una u otra rutina.

Interrupción 12H --- Tamaño de memoria. Devuelve en el registro AX la cantidad de bloques de 1 Kbyte de tamaño instalados en la tarjeta madre del sistema. No se incluye la memoria de las tarjetas de expansión.

Interrupción 13H --- E/S sobre disco. Permite efectuar algunas operaciones sobre la información en discos, ya sean diskette o winchester, direccionando a pistas o sectores. Antes de activar esta interrupción, se deben cargar algunos datos de entrada en los registros AH, DL, DH, CH, CL, AL y BX, dependiendo la operación a efectuar. Posteriormente se estudiara en detalle las posibilidades de E/S sobre disco.

Interrupción 14H --- E/S en port de comunicaciones. Permite transmitir y recibir información a través del port de comunicaciones serial (RS-232).

Interrupción 15H --- E/S sobre extensiones del sistema. Brinda una serie de posibilidades para acceder a distintas extensiones del sistema como ser, por ejemplo, efectuar E/S sobre cassettes.

Interrupción 16H --- E/S de teclado. Permite acceder al teclado, brindando una gran cantidad de posibilidades. Posteriormente, se verán en detalle todas las variantes que ofrece esta interrupción.

Interrupción 17H --- E/S de impresora. Facilita la comunicación con hasta 3 impresoras distintas. Permite imprimir un carácter, inicializar una impresora o leer el estado de una impresora.

Interrupción 18H --- BASIC. Llama al BASIC residente en la memoria ROM.

Interrupción 19H --- Re-inicialización del sistema. Fuerza la re-inicialización de la computadora.

Interrupción 1AH --- Hora del día. Permite acceder al contador del timer del sistema; con ella es posible fijar o leer la hora y la fecha, fijar un lapso de alarma, etc. Posteriormente se verán en detalle todas las posibilidades que brinda esta interrupción.

12.6.4. Rutinas del usuario.

Estas interrupciones son activadas automáticamente por el BIOS durante su operación normal; sin embargo, no producen ninguna acción, es decir que tienen asignados sendos vectores de interrupciones que apuntan a rutinas de atención que, simplemente tienen una instrucción IRET (vuelta de interrupción). Para poder usar estas interrupciones, es necesario escribir las rutinas de atención y cargar las direcciones de las mismas en los correspondientes vectores de interrupciones. Esta tarea se puede realizar a través de rutinas de atención de interrupciones del DOS.

Interrupción 1BH --- Pausa del teclado. Es activada por la rutina de atención de la interrupción de 09H (teclado). La rutina de atención de la interrupción 1BH tiene una instrucción IRET, de modo que cuando es activada, lo único que hace es volver de la interrupción; sin embargo, el DOS cambia el apuntador de esta interrupción a la rutina de atención de la interrupción 23H del DOS (Dirección de terminación por CTRL-BREAK).

Interrupción 1CH --- Tic del timer del sistema. Es invocada por la rutina que atiende la interrupción 08H (timer del sistema), con lo que la interrupción 1CH se ejecuta 18.2 veces por segundo. El correspondiente vector de la interrupción 1CH apunta a una instrucción IRET, de modo que actualmente no realiza ninguna tarea. Sin embargo, se podría programar la rutina para que, por ejemplo, se muestre la hora en pantalla cada vez que se produce un pulso del cronómetro. El uso de la interrupción 1CH tiene una gran cantidad de aplicaciones.

12.6.5. Punteros a tablas de datos.

Las interrupciones 1DH, 1EH, 1FH, 41H y 46H no son, precisamente, vectores de interrupciones, sino que son, en realidad, direcciones de tablas de datos usadas por otras interrupciones. De este modo, la dirección apuntada por la interrupción 1DH es la de una tabla usada por la interrupción 10H; la dirección apuntada por la interrupción 1EH es la de una tabla de diskette, usada por la interrupción 40H; y la dirección apuntada por las interrupciones 41H y 46H es la de una tabla de disco fijo (winchester), usada por la interrupción 13H. En cambio, la posición de la interrupción 1FH esta reservada para un puntero a una tabla de caracteres gráficos, la cual es cargada por un archivo del sistema operativo llamado GRAFTABL.COM.

12.7. INTERRUPCIONES DEL DOS

Así como existe un conjunto de interrupciones reservadas para el uso del BIOS, también existe otro grupo que están reservadas para uso del DOS. Las interrupciones reservadas para el DOS son 20H a 3FH, aunque no son todas utilizadas. La siguiente tabla detalla cada una de las interrupciones del DOS:

CODIGO	DESCRIPCION
20H	Terminar programa
21H	Llamada a función
22H	Dirección de terminación por fin de programa
23H	Dirección de terminación por CTRL-BREAK
24H	Manejador de error crítico
25H	Lectura absoluta de disco
26H	Grabación absoluta en disco
27H	Finalizar, pero quedar residente
28H-2EH	Sin uso (reservadas)
2FH	Impresora
30H-3FH	Sin uso (reservadas)

12.7.1. Interrupción 21H --- Llamada a función. De todas las interrupciones del DOS, la mas ampliamente usada es la 21H, de llamada a función (Call System), que brinda una gran cantidad de posibilidades en el aprovechamiento de los recursos del sistema. Cada función disponible de la interrupción 21H esta identificada

por un código, que se debe cargar en el registro AH; además, muchas de esas funciones, requieren valores de entrada en determinados registros y, del mismo modo, devuelven en ciertos registros, valores de salida. La siguiente tabla indica las distintas funciones provistas para la interrupción 21H:

AH	DESCRIPCION
01H	Leer un carácter del teclado
02H	Imprimir un carácter por pantalla
03H	Leer un carácter del port serie RS-232
04H	Enviar un carácter al port serie RS-232
05H	Imprimir un carácter por impresora
06H	Imprimir un carácter por pantalla
06H	Leer un carácter del teclado
07H	Leer un carácter del teclado
08H	Leer un carácter del teclado
09H	Imprimir una cadena de caracteres por pantalla
0AH	Leer caracteres del teclado
0BH	Obtener estado del teclado
0CH	Limpiar buffer del teclado

AH	DESCRIPCION
0DH	Restaurar disco
0EH	Seleccionar unidad por default
0FH	Abrir archivo existente
10H	Cerrar archivo
11H	Buscar la primera entrada
12H	Buscar la siguiente entrada
13H	Borrar archivo
14H	Lectura secuencial
15H	Grabación secuencial
16H	Crear archivo
17H	Renombrar archivo
19H	Obtener el número de la unidad default
1AH	Establecer el DTA (Area de Transferencia de Disco)
1BH	Obtener dirección de la FAT de la unidad default
1CH	Obtener dirección de la FAT de una unidad determinada
21H	Lectura random de un registro
22H	Grabación random de un registro
23H	Obtener cantidad de registros del archivo
24H	Especificar registro random
25H	Colocar vector de interrupción
27H	Lectura random de un bloque de registros
28H	Grabación random de un bloque de registros
2AH	Obtener fecha
2BH	Colocar fecha
2CH	Obtener hora
2DH	Colocar hora
2FH	Obtener dirección de la DTA
31H	Terminar ejecución y dejar residente en memoria
35H	Obtener vector de interrupción
39H	Crear un subdirectorío
3AH	Borrar un subdirectorío
3BH	Cambiar de subdirectorío activo
3CH	Crear un archivo
3DH	Abrir archivo
3EH	Cerrar archivo

3FH	Leer archivo
40H	Grabar archivo
41H	Borrar archivo
42H	Mover el puntero de lectura/escritura
43H	Obtener/fijar el atributo de un archivo
47H	Obtener el directorio activo
48H	Ubicar bloque de memoria
49H	Liberar bloque de memoria
4AH	Modificar bloques de memoria ubicados
4BH	Cargar o ejecutar un programa
4EH	Buscar primer nombre de archivo en directorio activo
4FH	Buscar siguiente nombre de archivo
56H	Renombrar un archivo
57H	Obtener/establecer fecha y hora de un archivo

En los siguientes capítulos se verán mas en detalle las distintas funciones de la interrupción 21H del DOS.

12.8. CAMBIO DE UN VECTOR DE INTERRUPCION

Habíamos visto que un vector de interrupción es la dirección de una rutina especial del sistema operativo, cuya función es la de satisfacer o atender la interrupción. Estas rutinas vienen provistas con el sistema operativo, pero son susceptibles de ser modificadas: es decir, se pueden instalar rutinas apropiadas para que las interrupciones sean tratadas en forma particular. Para poder instalar una rutina de atención de interrupción, es necesario modificar el contenido del correspondiente vector de interrupción, de modo que apunte a la nueva rutina. Posteriormente veremos como instalar una rutina y que se mantenga residente en memoria. Existen tres métodos para cambiar un vector de interrupción:

- 1) con dos MOV
- 2) con un MOV encadenado
- 3) con la interrupción 21H

12.8.1. Con dos MOV.

En este método se utilizan dos instrucciones MOV para cargar la dirección de la nueva rutina en el vector de interrupción: uno para cargar el segmento y otro para cargar el desplazamiento. Previamente a las instrucciones de MOV se deben inhibir las interrupciones (excepto la interrupción no enmascarable, que se activa ante situaciones urgentes). Después de la ejecución de ambos MOV, se deben desinhibir las interrupciones. El siguiente es el esquema del código assembler para este método:

```

DESP EQU 4*(NRO_INT-1)           ;desplazamiento del vector de la interrupción NRO_INT
XOR AX,AX                       ;AX=0
MOV ES,AX                       ;ES=0
CLI                             ;inhibir interrupciones
MOV ES:DESP,OFFSET RUTINA ;colocar desplazamiento de la rutina en el vector de interrupción
MOV ES:DESP+2,SEG RUTINA ;colocar segmento de la rutina en el vector de interrupción
STI                             ;desinhibir interrupciones

```

El uso de este método tiene el inconveniente de que no puede evitar que se produzca una interrupción no enmascarable entre un MOV y el otro. Para resolver este problema, en general se utilizan cualquiera de los otros dos métodos.

12.8.2. Con un MOV encadenado.

Este método es, en esencia, idéntico al anterior, pero sustituye las dos instrucciones de MOV por una sola, que es un MOVS (MOV encadenado) con un prefijo de repetición. Es decir, que considera como string la dirección que va a copiar en el vector de interrupción.

Esta consideración se hace para evitar el riesgo de que se produzca una interrupción no enmascarable entre los dos MOV. El siguiente es el esquema del código assembler para este método:

```

....
DIREC DD RUTINA
....
DESP EQU 4*(NRO_INT-1)           ;desplazamiento del vector de la interrupción NRO_INT

```

	XOR	DI,DI		;DI=0
	MOV	ES,DI		;ES=0
	MOV	DI,DESP		;DI=desplazamiento del vector
	LEA	SI,DIREC	;SI=desplazamiento de la dirección de memoria donde esta almacenada	
				;la dirección de la rutina
	MOV	CX,2		;CX=2 (nro. de palabras a copiar)
	CLD		;manejo de strings de izquierda a derecha (parte alta a parte baja)	
	CLI		;inhibir interrupciones	
REP	MOVSW		;mover dos palabras	
	STI		;desinhibir interrupciones	

12.8.3. Con la interrupción 21H.

Este es el método mas practico pues hace que la operación sea independiente de la versión del sistema operativo. El código de función es 25H. El siguiente es el esquema del código assembler para este método:

PUSH	DS		;salvar DS en el stack
MOV	DS,SEG RUTINA		;DS=segmento de la rutina
MOV	DX,OFFSET RUTINA		;DX=desplazamiento de la rutina
MOV	AL,NRO_INT		;AL=número de la interrupción
MOV	AH,25H		;código de la función
INT	21H		;llamada a función del DOS
POP	DS		;recuperar DS

Así como existe el código 25H para cargar un vector de interrupción, existe también el código 35H que obtiene un vector de interrupción. Las especificaciones para ambas funciones son las siguientes:

25H	cargar vector de interrupción	AL=número de interrupción	
			DS=segmento de la rutina
			DX=desplazamiento de la rutina
35H	Obtener vector de interrupción	AL=número de interrupción	
			ES=segmento de la rutina
			BX=desplazamiento de la rutina

Ambas funciones corresponden a la interrupción 21H.

CAPITULO 13 COMIENZO Y TERMINACION DE PROGRAMAS

En este capítulo veremos los distintos mecanismos que se desencadenan, a nivel del sistema operativo, al comenzar la ejecución de un programa y al finalizar. También trataremos el mecanismo de programas residentes.

13.1. EL PREFIJO DE SEGMENTO DE PROGRAMA (PSP)

Un programa se invoca desde el procesador de comandos a través del ingreso de su nombre: por ejemplo, si se desea ejecutar el programa CUENTAS.EXE, se debe ingresar el comando CUENTAS.

Una vez que el procesador de comandos acepta el comando ingresado y verifica que se trata de un comando externo (archivo .EXE o .COM), invoca una función del DOS quien, antes de pasarle el control al programa, construye un bloque de 256 bytes (100H) de tamaño a partir de la primera posición de memoria disponible. Esta zona de memoria es el Prefijo de Segmento del Programa (Program Segment Prefix, PSP) que contiene una serie de campos usados para control.

La siguiente es la estructura del PSP:

DESP	LONGITUD	CAMPO
0H	2H	Instrucción INT 20H
2H	2H	Tamaño de memoria disponible
4H	1H	Reservado
5H	5H	Llamada al dispatcher
AH	4H	Dirección de terminación
EH	4H	Dirección de CTRL-BREAK
12H	4H	Dirección de error crítico
16H	16H	Usado por el DOS
2CH	2H	Segmento del entorno (environment)
2EH	2EH	Area de trabajo del DOS
5CH	10H	Bloque de control de archivo (FCB) sin abrir
6CH	14H	Bloque de control de archivo (FCB) sin abrir
80H	1H	Longitud de los parámetros
81H	7FH	Parámetros

El significado de algunos de los campos es el siguiente:

Instrucción INT 20H. La interrupción 20H hace que un programa finalice y devuelva el control al DOS para que este libere todos los recursos usados por el programa. Al comienzo del programa se debe guardar al tope del stack una dirección de retorno mediante la secuencia de instrucciones:

```
PUSH DS
SUB AX,AX
PUSH AX
```

Como se puede ver, la dirección que se almacena al tope del stack es DS:0. Teniendo en cuenta que inicialmente el registro DS apunta al inicio del PSP, la dirección guardada en el stack es, justamente, la de la primera posición del PSP. Al terminar un programa con la instrucción RET, se recupera del stack la dirección guardada, es decir la dirección de comienzo del PSP en CS:IP. Por lo tanto, la siguiente instrucción que se ejecuta es INT 20H, que esta codificada en los 2 primeros bytes del PSP. El mecanismo de finalización descrito es el que se utiliza en programas .EXE, haciendo que el registro CS apunte al PSP.

En los programas .COM, la finalización es directa a través de INT 20H, dado que CS apunta al PSP.

Tamaño de memoria disponible. Este campo contiene la cantidad de memoria disponible para el programa, medida en párrafos (bloques de 16 bytes).

Dirección de terminación. Corresponde a la dirección del código a ejecutarse cuando finalice el programa. Este dato es tomado por la rutina de atención de la interrupción 20H. En general, este campo apunta al procesador de comandos.

Dirección de CTRL-BREAK. Contiene la dirección del código a ejecutarse en caso de que se opriman la teclas CTRL-BREAK.

Dirección de error crítico. Contiene la dirección de la rutina de tratamiento de errores críticos.

Bloque de control de archivo (FCB) sin abrir. Un bloque de control de archivo (File Control Block, FCB) es una estructura que contiene una serie de datos acerca de un archivo en particular. En el capítulo correspondiente a archivos, estudiaremos el FCB en detalle. El PSP incluye dos FCB, sin abrir, que corresponden a sendos archivos especificados como parámetros al ingresar el comando. Por ejemplo, hay comandos como COPY que utilizan archivos como parámetros. Cada uno de estos archivos tiene ciertas especificaciones que se indican en su correspondiente FCB.

Longitud de los parámetros. Contiene el número de caracteres de los parámetros. Se considera parámetro a todo carácter que esta a continuación del nombre del comando, incluido el carácter blanco que hace las veces de separador entre el comando y los parámetros propiamente dichos. Así, por ejemplo, para el comando CUENTAS XXX este campo del PSP contendrá el valor 4, que corresponde al blanco y a las tres equis. Inicialmente este campo es el Area de Transferencia a Disco (Disk Transfer Area). El DTA se estudiara en detalle en el capítulo correspondiente a archivos.

Parámetros. Este campo incluye la totalidad de los parámetros. Por ejemplo, si el comando ingresado es EJEM ARCHI1.DAT ARCHI2.DAT, este campo contendrá la serie de caracteres " ARCHI1.DAT ARCHI2.DAT".

13.2. COMIENZO DE PROGRAMAS

Cuando el DOS le pasa el control a un programa, algunos registros se inicializan con determinados valores:

- DS y ES apuntan al PSP.
- AL contiene un código que indica si es correcta o no la especificación de unidad correspondiente al primer nombre de archivo pasado como parámetro. Un valor cero, indica que dicha especificación es correcta.
- El contenido de AH es similar al de AL, pero para el segundo nombre de archivo.
- El DTA apunta al desplazamiento 80H del PSP; este es el DTA por default.

En particular, para los programas .COM se producen las siguientes inicializaciones:

- CS y SS apuntan al PSP.
- IP se inicializa en 100H, que corresponde al byte siguiente al PSP; es decir, que la primera instrucción del programa esta inmediatamente a continuación del PSP.
- SP se inicializa con la dirección mas alta dentro del PSP.

Por otra parte, para los programas .EXE se producen las siguientes inicializaciones:

- CS e IP se cargan con los valores asignados por el linkeditor; es decir, con los valores indicados en la instrucción END o con cero si no se especificó ningún valor.
- SS apunta al segmento de stack.
- SP se carga con la dirección mas alta dentro del stack mas 100H; es decir, que se reserva un stack de, al menos 100H bytes (50H palabras).

13.3. TERMINACION DE PROGRAMAS

Existen, bajo DOS, diferentes maneras de terminación de programas. Básicamente, podemos clasificar estas maneras de terminación en dos grupos: terminar y devolver recursos o terminar y dejar residente.

13.3.1. Terminar y devolver recursos.

Es la manera "normal" o mas común de terminación de programas: cuando finaliza la ejecución de un programa, el sistema operativo se encarga de liberar todos los recursos usados por el mismo (memoria, tablas, etc.) y pasar el control al procesador de comandos o al shell que corresponda. Habíamos visto que el primer campo del PSP correspondía al código de la instrucción INT 20H, que se ejecutaba cuando finalizaba un programa con la instrucción RET. Similarmente, un programa puede finalizar con la función 00H de la interrupción 21H. Esto es idéntico a la interrupción 20H. Existe otro modo de finalización de programas que, además de hacer lo mismo que la interrupción 20H, devuelve un código de retorno en el registro AL. Para obtener esto, se debe invocar la función 4CH de la interrupción 21H. El uso de un código de retorno permite evaluar el resultado de una operación a través del uso del lenguaje de comandos.

Por ejemplo:

```
MASM PROG/L
IF ERRORLEVEL 1 GOTO :MAL
LINK PROG
```

En este caso, si la compilación del programa termina con errores, el compilador devuelve un código de retorno distinto de 1, lo que permite tomar la decisión de, por ejemplo, señalar el error y terminar.

Una particularidad de esta función es que cierra todos los archivos que hayan quedado abiertos.

Otra aplicación de la función 4CH de la interrupción 21H es cuando un programa se invoca desde otro programa a través del MS-DOS. En este caso, se debe consultar el contenido de AL, que es donde queda el código de retorno.

13.3.2. Terminar y dejar residente.

Un programa se define como residente cuando es cargado en memoria para su ejecución y, una vez que finaliza, se mantiene en memoria sin liberar el área que le fuera asignada. Todo programa de usuario se carga a partir de la primera posición disponible después del área reservada para el sistema operativo. Cuando finaliza su ejecución, el área de memoria usada se libera y queda disponible para cargar el próximo programa. En el caso de un programa residente, no se produce la liberación de la memoria usada por el programa. El programa residente es como si fuera una extensión del DOS. Por lo tanto, luego de dejar residente un programa, disminuye la cantidad de memoria disponible para nuevos programas. Se pueden cargar tantos programas residentes como se quiera, en tanto quede suficiente memoria para poder ejecutar los demás programas que se invoquen. No sólo pueden instalarse programas como residentes; también es posible que ciertos datos se hagan residentes en memoria. Los programas residentes se utilizan, en general, para reemplazar rutinas de atención de interrupciones; los programas se cargan en memoria cuando son invocados, modifican un vector de interrupción para que apunte a la rutina propia y finalizan indicándole al MS-DOS que se mantienen residentes. También es posible dejar residentes en memoria ciertos datos para, por ejemplo, comunicación entre programas. La información que se deja residente permanece en memoria mientras el DOS esta funcionando. Al reinicializar la computadora (REBOOT), la información instalada como residente deja de estarlo. Para dejar información residente en memoria, se utilizan la interrupción 27H y la función 31H de la interrupción 21H.

Interrupción 27H. Antes de ejecutar la instrucción INT 27H, se debe cargar en el registro DX la dirección (desplazamiento respecto del segmento) de la posición siguiente a la última que se quiere dejar residente. La instrucción INT 27H deja residente la porción de programa entre DX y la dirección anterior a la de la instrucción. Con INT 27H se puede dejar residente hasta un máximo 64 Kbytes. En general, un programa que utiliza esta técnica tiene normalmente dos partes bien diferenciadas: la parte de instalación de la información a dejar residente y la información a dejar residente.

El siguiente ejemplo corresponde a la instalación de una rutina de atención de interrupción:

```
; Esquema de instalación de una rutina de atención de la interrupción NRO_INT
; Segmento correspondiente a los vectores de interrupciones
INTERRUP    SEGMENT        AT 0H
              ORG            NRO_INT*4
DIREC LABEL DWORD                      ;dirección rutina
INTERRUP    ENDS
; Segmento principal
CODIGO      SEGMENT
              ASSUME        CS:CODIGO,DS:CODIGO
              ORG            100                ;por ser archivo .COM
EMPEZAR:
              JMP            INSTALAR            ;bifurca a la rutina de instalación
              ....
; Rutina propia de atención de la interrupción NRO_INT
RUTINA      PROC
              CLI                ;inhibir interrupciones
              PUSH    ...        ;salvar registros afectados
              PUSH    ...        ;en el stack
              ....              ;instrucciones propias de la rutina
              POP     ...        ;restaurar
              POP     ...        ;registros
              STI                ;desinhibir interrupciones
              IRET              ;vuelta de interrupción
RUTINA      ENDP
; Procedimiento de instalación de la rutina
; Almacena la dirección de la rutina en el vector de interrupción y deja residente la rutina
INSTALAR    PROC
              ASSUME        ES:INTERRUP
              MOV            AX,0
              MOV            ES,AX
```

	CLI		;inhibir interrupciones
	MOV	DIREC,OFFSET RUTINA	;cargar desplazamiento
	MOV	DIREC+2,CS	;segmento
	MOV	DX,OFFSET INSTALAR	;DX=última dirección de
			;la rutina a instalar + 1
	STI		;desinhibir interrupciones
	INT	27H	;terminar, y dejar residente
INSTALAR	ENDP		
;			
CODIGO	ENDS		
	END	EMPEZAR	

A la vista del ejemplo desarrollado, podemos hacer algunas consideraciones:

- Todo programa residente conviene escribirlo como .COM, para utilizar un único segmento.
- El punto de entrada al programa es la rutina principal (EMPEZAR), que se ocupara de modificar el contenido del vector de interrupción e indicar la terminación y residencia del programa (procedimiento INSTALAR).
- La rutina de interrupción propiamente dicha (procedimiento RUTINA) no se ejecuta cuando se ejecuta la rutina principal, sino cuando se produzca la interrupción correspondiente.
- La rutina principal sólo es ejecutada al invocar el programa desde el procesador de comandos, y no cuando se produce la interrupción.

Un aspecto que se debe considerar es que, cuando se desea hacer una rutina de atención de interrupción que contemple sólo algunas de las variantes de la interrupción, se debe guardar la dirección de la rutina original. Por ejemplo, si queremos hacer alguna acción específica cuando se tepea la tecla F1, debemos cargar una rutina propia de atención de la interrupción 09H y controlar que la tecla oprimida haya sido F1; si no lo es, debemos restaurar la rutina original de la interrupción 09.

Función 31H del DOS (interrupción 21H)

Esta función es similar a la interrupción 27H en el sentido que también permite finalizar y dejar residente un programa. Pero, además:

- Permite reportar un código de retorno en AL.
- Supera la limitación de 64 Kbytes de máximo tamaño a dejar residente que permite la interrupción 27H. La cantidad de memoria a dejar residente se debe especificar en el registro DX, medida en párrafos (múltiplos de 16 bytes).
- Se debe tener en cuenta que el PSP del programa también queda en memoria, pues esta en la dirección mas baja desde donde se cargó el programa para su ejecución.

El resto de las consideraciones son las mismas que para la interrupción 27H.

CAPITULO 14 FECHA Y HORA DEL SISTEMA

14.1. RELOJ DE INTERVALOS

El reloj de intervalos o timer del sistema es un chip de la tarjeta madre, identificado como 8353, manejado por el controlador 8259. El reloj de intervalos envía, periódicamente, una señal al microprocesador lo que permite mantener actualizado un contador de tiempo. Cada una de esas señales se denomina "tic" del reloj.

Se producen 18,2 "tics" por segundo; es decir, hay un "tic" del reloj cada 0,0549254 segundos. El contador que se actualiza con cada "tic" del reloj es una doble palabra (32 bits) residente en el área de datos del BIOS

(parte baja de la memoria). Cada 24 horas se producen 1.573.040 "tics" del reloj; cuando el contador alcanza ese valor (1800B0H), es vuelto a cero automáticamente por el BIOS y se guarda una marca de medianoche (Midnight signal). Este contador es el que se utiliza para calcular la hora del día que mantiene el sistema operativo.

14.2. INTERRUPCIONES DEL BIOS

El BIOS tiene 3 interrupciones asociadas al reloj de intervalos: la interrupción 08H, la interrupción 1CH y la interrupción 1AH.

La interrupción 08H es la que se produce automáticamente en cada "tic" del reloj; en otras palabras, un "tic" del reloj es una interrupción 08H. Cada vez que se produce un "tic" del reloj, la rutina que atiende la interrupción 08H, además de actualizar el valor del contador, genera una interrupción 1CH. La rutina de atención de la interrupción 1CH no efectúa ninguna tarea específica (sólo contiene una instrucción IRET), pero podría ser modificada para que realice alguna función como ser la de mostrar el reloj en pantalla, hacer sonar una alarma, etc.

A través de la interrupción 1AH (Hora del día) se puede obtener o modificar el valor del contador del reloj de intervalos. Ambas funciones se indican con un código de función en el registro AH:

AH	Operación	Entrada	Resultados
0	Leer el actual valor del contador	Ninguna CX=parte alta del contador DX=parte baja del contador	
	AL=0 si el contador no pasó de 24 horas después de la última lectura		
1	Cargar un valor en el contador	Ninguno CX=parte alta del contador DX=parte baja del contador	

Existen algunas microcomputadoras que incluyen un circuito de reloj que conserva la hora y el día aún estando apagado el equipo. Este reloj, mantenido a través de una batería, se conoce como reloj de tiempo real. La interrupción 1AH brinda una serie de funciones adicionales para el reloj de tiempo real. Esas funciones también se indican con un código en AH:

AH	Operación	Entrada	Resultados
2	Leer hora del reloj de tiempo real	Ninguna CH=Horas en BCD CL=Minutos en BCD DH=Segundos en BCD CF=1 si el reloj no esta operativo	

AH	Operación	Entrada	Resultados
3	Cargar hora en el reloj de tiempo real	CH=Horas en BCD CL=Minutos en BCD DH=Segundos en BCD DL=0	Ninguno
4	Leer fecha del reloj de tiempo real	Ninguna CX = Año en BCD (1980 a 2099) DH=Mes en BCD DL=Día en BCD CF=1 si el reloj no esta operativo	
5	Cargar día en el reloj de tiempo real	CH=Año en BCD (1980 a 2099) DH=Mes en BCD DL=Día en BCD	Ninguno
6	Establecer hora de alarma	CH=Horas en BCD CL=Minutos en BCD DH=Segundos en BCD	CF=1 si el reloj no esta operativo o si la alarma ya esta habilitada
7	Borrar alarma	Ninguna	Ninguno

Los siguientes son ejemplos de posibles aplicaciones de la interrupción 1AH:

Generación de números al azar

Como el contador del reloj de intervalos cambia continuamente su valor (18,2 veces por segundo), su contenido, en un determinado instante, se puede considerar como un número al azar. Usando, por ejemplo, los 16 bits de la parte baja del contador se puede obtener un número entre 0 y 65.535. Para generar un número al azar entre 0 y n, se debe dividir por (n+1) el valor de la parte baja del contador y tomar el resto de dicha división como el número generado al azar. En la generación de números al azar mediante este método se debe considerar la posibilidad de overflow al efectuar la división.

Fijar hora de alarma

Para hacer esto es necesario preparar una rutina de atención de la interrupción 4AH (alarma) que produzca un determinado sonido, y almacenar la dirección de esa rutina en el vector de la interrupción 4A.

Una vez efectuados estos pasos, se debe escribir un procedimiento que cargue la hora de alarma y la habilite. Para la obtención de sonidos se debe tener en cuenta el pasaje de información al parlante de la computadora a través de las instrucciones IN y OUT, y considerar las frecuencias de los distintos tonos que se quieren lograr.

14.3. INTERRUPCIONES DEL DOS

El DOS brinda, a través de la interrupción 21H, una serie de operaciones con el día y la hora bastantes similares a las ofrecidas por la interrupción 1AH del BIOS. Las siguientes son las funciones que ofrece la interrupción 21H:

AH	Operación	Entrada	Resultados
2A	Leer fecha	Ninguna CX=Año DH=Mes DL=Día	
2B	Cargar fecha	CX=Año (1980 a 2099) DH=Mes DL=Día	AL=0 si es fecha valida AL=FF si es fecha invalida
2C	Leer hora	Ninguna CH=Horas CL=Minutos DH=Segundos DL=Centesimas	
2D	Cargar hora	CH=Horas (0 a 23) CL=Minutos DH=Segundos DL=Centesimas	AL=0 si es hora valida AL=FF si es hora invalida

Las funciones de la interrupción 21H para fecha y hora difieren de las funciones ofrecidas por las interrupciones del BIOS en:

- La hora y la fecha son valores binarios en vez de ser BCD.
- La fecha y la hora son valores temporarios que se mantienen en la memoria; su cambio no afecta el funcionamiento del reloj de tiempo real, si es que este existe.
- La hora tiene mayor precisión, pues maneja hasta centésimas de segundos.

Los siguientes son ejemplos de posibles aplicaciones de la interrupción 21H:

Medición del tiempo de ejecución de programas

Esto es posible debido a la precisión en centésimas de segundos. Para hacerlo, primero se debe leer la hora, luego ejecutar el programa y, finalmente, volver a leer la hora. El tiempo de ejecución del programa lo da la diferencia entre la hora final y la hora inicial.

Generar demoras

La generación de demoras tiene aplicación en acciones tales como la producción de sonidos a través del parlante (medir el tiempo de sostenido de una nota), o la visualización en pantalla de figuras gráficas (determinar durante cuanto tiempo aparece una figura o establecer cuanto tiempo debe pasar para mostrar la próxima figura). Un programa que genera demoras debe, en general, leer primero la hora, luego sumarle a la misma un lapso de tiempo, obtener un tiempo destino (ajustando los valores de horas, minutos, segundos y centésimas) y, por último, leer repetidamente la hora hasta que se alcance la hora destino.

CAPITULO 15 LA PANTALLA

15.1. GENERALIDADES DE HARDWARE

La pantalla es un tubo de rayos catódicos (Cathodic Ray Tube, CRT), que es controlada por un chip denominado controlador CRT. Sirve para representar la información mediante puntos luminosos, llamados pixels o pells, que se activan o no durante el barrido del haz de electrones del tubo de rayos catódicos sobre la superficie de la pantalla. Este barrido es de izquierda a derecha y de arriba hacia abajo. El funcionamiento de la pantalla depende del adaptador de pantalla (Display Adapter), que es una tarjeta ubicada en uno de las ranuras o slots de la microcomputadora.

15.1.1. El adaptador de pantalla.

Las principales funciones del adaptador de pantalla son las de manejar la memoria de pantalla y de generar los caracteres a imprimir a través de un generador de caracteres en memoria ROM; en otras palabras, el adaptador

de pantalla es la interface entre el microprocesador y el monitor. Los adaptadores de pantalla pueden ser monocromático o color, con o sin graficación e, incluso, con distintas capacidades de graficación. Existe una gran variedad de marcas y modelos distintos: EGA, CGA, Hercules, etc.

Nosotros estudiaremos la pantalla en modo texto (adaptador monocromático) y la pantalla en modo gráfico (adaptador color/gráficos Color/ Graphics Adapter, CGA), ambas de uso generalizado.

Con el adaptador monocromático, la pantalla sólo puede funcionar en modo texto, mientras que con el adaptador color/gráficos, puede funcionar en ambos modos.

15.1.2. Generación de caracteres.

Todos los caracteres que aparecen en pantalla están definidos sobre una matriz bidimensional de puntos o pixels; así, un carácter es una serie de pixels encendidos sobre esta matriz. En modo texto, sólo se pueden ver caracteres. En total, se pueden ver hasta 256 caracteres: los primeros 128 corresponden al código ASCII, mientras que los restantes 128 corresponden a alguna de las tantas extensiones que existen del código ASCII. Para un adaptador monocromático cada carácter se representa en una matriz de 9 x 14 pixels, mientras que para un adaptador color/gráficos cada carácter se representa en una matriz de 8 x 8 pixels.

En modo gráfico, se puede ver cualquier figura, dado que se puede acceder a cada pixel de la pantalla. La superficie de la pantalla es una matriz bidimensional (filas y columnas), cuyas características están dadas por la resolución. La resolución de una pantalla se mide en unidades de representación; de este modo, la resolución para una pantalla en modo texto es de 40 x 25 caracteres (baja resolución) o de 80 x 25 caracteres (alta resolución), mientras que la resolución para una pantalla en modo gráfico es de 160 x 100 pixels (baja resolución), o de 320 x 200 pixels (media resolución) o de 640 x 200 pixels (alta resolución). En algunas microcomputadoras existe, también, la resolución superalta, que tiene 640 x 400 pixels. Para trabajar con baja resolución es necesario el uso de la programación directa del procesador de la pantalla, ya que el BIOS no soporta esta resolución. En realidad, existen hasta 15 modos de trabajo distintos, variando en la cantidad de pixels y los colores que se pueden lograr. Los nombrados son los mas usuales. Cualquier punto de la pantalla se lo identifica a través de una coordenada (X,Y). El origen de las coordenadas, tanto en modo texto como en modo gráfico, es la esquina superior izquierda de la pantalla; la primera fila es la 0, y la primera columna también es la 0. El sentido positivo para X (columnas) es de izquierda a derecha, mientras que para Y (filas) es de arriba a abajo. El cursor indica la posición activa de la pantalla; en modo gráfico no existe un cursor físico, pero si un cursor lógico (las aplicaciones "generan" su propio cursor físico).

15.1.3. Memoria de pantalla.

Ambos tipos de adaptadores representan la información en la pantalla mediante el manejo de memoria mapeada (Memory-mapped display), que se conoce como la memoria o buffer de pantalla. La memoria de pantalla esta físicamente situada en la tarjeta del adaptador, pero lógicamente se encuentra dentro del espacio de direcciones del microprocesador 8088-8086. La dirección inicial y tamaño de la memoria de pantalla dependen del tipo de adaptador, pero siempre se encuentran entre las direcciones A0000H a BFFFFH. La memoria de pantalla para el adaptador monocromático comienza en la dirección B0000H y el tamaño es de 4 Kbytes, que se corresponde con el espacio necesario para representar todos los datos de una pantalla, carácter a carácter. La memoria de pantalla para el adaptador color/gráficos (CGA) comienza en la dirección inicial B8000H y el tamaño es 16 Kbytes. La memoria de pantalla para la CGA se puede dividir en bloques; cada bloque corresponde a una pagina, que es la información que aparece en la pantalla. En un momento dado aparece sólo la información de un bloque, que es el asociado a la pagina activa. En todo momento hay alguna pagina activa, es decir que vemos su contenido por pantalla. Las otras paginas se mantienen residentes en memoria, lo que permite, usando funciones especiales del BIOS, pasar de una pagina a otra. El adaptador, para generar los caracteres que debe mostrar en la pantalla, barre la memoria de pantalla unas 60 veces por segundo, aproximadamente; es decir que escribir caracteres en la pantalla no es otra cosa que mover bytes desde alguna posición de memoria hacia la memoria de pantalla. Posteriormente estudiaremos mas en detalle como es la generación de caracteres en modo gráfico.

15.2. LA PANTALLA EN MODO TEXTO

En modo texto o alfanumérico, la unidad de información representable en la pantalla es el carácter.

Cada carácter tiene asociados dos bytes en la memoria de pantalla: el primero (dirección par) es el byte de código, que contiene el código numérico del carácter y el segundo (dirección impar) es el byte de atributo, que contiene la forma de presentación del carácter. La configuración del byte de atributo es la siguiente:

7 6 5 4 3 2 1 0

B R V A I R V A

Background Foreground. El bit 7 indica el atributo de blinking o parpadeo; los bits 6 a 4 indican los componentes de colores rojo, verde y azul, respectivamente, del fondo (background); y los bits 3 a 0 indican los componentes de intensidad o brillo y de colores rojo, verde y azul, respectivamente, del carácter (foreground). Para el adaptador monocromático y para el CGA en modo blanco y negro, sólo tienen sentido los siguientes atributos:

CODIGO	ATRIBUTO
0000 0111 = 07H	Normal (blanco sobre negro)
0111 0000 = 70H	Vídeo inverso
0000 0001 = 01H	Subrayado
0000 0000 = 00H	Invisible (negro)
0111 0111 = 77H	Invisible (blanco)

Además de estos, también tienen uso los bits de blinking (bit 7) y de intensidad (bit 3). Para el adaptador CGA, el número máximo posible de colores es de 16 para el carácter o foreground (representables en los 4 bits mas bajos del byte de atributo), y de 8 para el fondo o background (representables en los bits 6 a 4). La siguiente tabla muestra los diferentes colores que se pueden obtener para un carácter según los valores del nibble mas bajo del byte de atributo:

VALOR	I	R	V	A	COLOR
0	0	0	0	0	Negro
1	0	0	0	1	Azul
2	0	0	1	0	Verde
3	0	0	1	1	Azul-verde (cyan)
4	0	1	0	0	Rojo
5	0	1	0	1	Magenta
6	0	1	1	0	Marrón
7	0	1	1	1	Blanco
8	1	0	0	0	Gris
9	1	0	0	1	Azul claro
A	1	0	1	0	Verde claro
B	1	0	1	1	Azul-verde (cyan) claro
C	1	1	0	0	Rojo claro
D	1	1	0	1	Magenta claro
E	1	1	1	0	Amarillo
F	1	1	1	1	Blanco intenso

Para el color de fondo son validas las primeras 8 combinaciones de esta tabla (valores 0 a 7), pues sólo se utilizan 3 bits para su representación. Esta tabla se conoce con el nombre de IRGB, que corresponde a las iniciales de cada bit en ingles: Intensity, Red, Green, Blue.

15.2.1. La memoria de pantalla en modo texto.

Para el adaptador CGA, el tamaño de una pagina depende de la resolución: en baja resolución hay 8 paginas de 2.000 bytes (40 x 25 x 2) cada una; en alta resolución hay 4 paginas de 4.000 bytes (80 x 25 x 2) cada una. La siguiente tabla indica la dirección de inicio de cada pagina en la memoria de pantalla:

RESOLUCION	PAGINA	DESPLAZAMIENTO	DIRECCION ABSOLUTA
80 x 25	0	0000H	B8000H
	1	1000H	B9000H
	2	2000H	BA000H
	3	3000H	BB000H
40 x 25	0	0000H	B8000H
	1	0800H	B8800H
	2	1000H	B9000H
	3	1800H	B9800H
	4	2000H	BA000H
	5	2800H	BA800H

6	3000H	BB000H
7	3800H	BB800H

En el cuadro, el desplazamiento es el correspondiente a la memoria de pantalla y la dirección absoluta es la que corresponde al espacio de direcciones del 8088-8086. Se puede ver que cada pagina esta en un desplazamiento par, de modo que hay un desperdicio al final de cada pagina:

- resolución 80 x 25 ---> 4096 - 4000 = 96 bytes/pagina
- resolución 40 x 25 ---> 2048 - 2000 = 48 bytes/pagina

La pagina activa por default es la cero; se puede cambiar rápidamente de pagina mediante una interrupción BIOS (método usado especialmente para producir animación).

15.2.2. Localización de cada carácter en la memoria de pantalla.

Con lo visto, se puede acceder a una determinada coordenada de la pantalla a través del acceso a la correspondiente posición en la memoria de pantalla. Para ello, se debe considerar que el primer byte de cada pagina se corresponde al carácter de coordenadas (0,0), y el segundo byte es el byte de atributo de esa coordenada. Del mismo modo, el tercer byte es el byte dl carácter de coordenadas (1,0), y el cuarto byte es el byte de atributo de esa coordenada. Así sucesivamente, para el carácter de coordenadas (X,Y):

byte de carácter = $P \times \text{tamaño_pagina} + 2 \times Y \times \text{tamaño_fila} + 2 \times X$

byte de atributo = $P \times \text{tamaño_pagina} + 2 \times Y \times \text{tamaño_fila} + 2 \times X + 1$

donde P representa el número de pagina, tamaño_pagina es 2048 ó 4096 (dependiendo de la resolución), X es la columna, tamaño_fila es 40 x 80 (dependiendo, también, de la resolución), e Y es la fila.

El adaptador monocromático tiene una sólo pagina, la activa, de 4 Kb de tamaño. Para este caso, el acceso al carácter de coordenadas (X,Y) es:

byte de carácter = $160 \times Y + 2 \times X$

byte de atributo = $160 \times Y + 2 \times X + 1$

donde 160 es el tamaño de una fila (80 bytes de caracteres y 80 bytes de atributos), X es la columna e Y es la fila.

15.3. LA PANTALLA EN MODO GRAFICO

En modo gráfico, la unidad de información representable en la pantalla es el pixel. Cada pixel tiene asociado un color determinado. La variedad de colores depende del tipo de resolución y de la representación de cada pixel en la memoria de pantalla. En particular, analizaremos media resolución (320 x 200 pixels) y alta resolución (640 x 200 pixels). En alta resolución cada pixel puede ser blanco o negro, dado que tiene asignado un solo bit en la memoria de pantalla. En media resolución cada pixel puede tener uno de cuatro preseleccionados colores, dado que tiene asignado 2 bits en la memoria de pantalla. Uno de esos cuatro colores (el color de fondo o background) puede ser seleccionado de los 16 posibles indicados en la tabla IRGB, siendo el color 0 (negro) el default; los otros tres colores pueden ser tomados de la paleta 0 (rojo, verde y amarillo) o de la paleta 1 (cyan, magenta y blanco). La paleta activa esta indicada en el bit 5 del registro de selección del color, que es un registro especial para el adaptador CGA. Ambas paletas no pueden estar simultáneamente activas, siendo la paleta 1 (cyan, magenta y blanco) la que esta activa por default. En alta resolución, cada pixel tiene asignado un sólo bit de la memoria de pantalla; es decir que cada byte de la memoria de pantalla contiene información de 8 pixels:

7	6	5	4	3	2	1	0
C	C	C	C	C	C	C	C

El color de cada punto esta dado por el valor del respectivo bit, con lo que son posibles sólo dos colores: puede ser 0, que corresponde a color negro, ó 1, que corresponde a color blanco. En media resolución, cada pixel tiene asignados 2 bits de la memoria de pantalla, es decir que cada byte contiene información de 4

pixels:

7	6	5	4	3	2	1	0
C1	C0	C1	C0	C1	C0	C1	C0

La combinación de colores posibles es la siguiente:

C1	C0	COLOR	PALETA 0	PALETA 1
0	0	0 (fondo)		
0	1	1	Verde	Cyan
1	0	2	Rojo	Magenta
1	1	3	Amarillo	Blanco

15.3.1. La memoria de pantalla en modo gráfico.

Cada bloque de la memoria de pantalla corresponde a una serie de filas: el primer bloque corresponde a las filas pares, y el segundo a las filas impares. El tamaño de cada bloque es de 8 Kbytes, es decir la mitad de los 16 Kbytes totales. Por otra parte, cada fila de la pantalla (no importa si es número par o impar) ocupa, en la memoria de pantalla:

- $640 \times 200 \text{ pixels} \times 1 \text{ bit/pixel} = 640 \text{ bits} = 80 \text{ bytes}$ o
- $320 \times 200 \text{ pixels} \times 2 \text{ bits/pixel} = 640 \text{ bits} = 80 \text{ bytes}$

Como existen, en ambos casos, 100 filas por bloque, el espacio ocupado por cada uno de ellos es de $80 \times 100 = 8.000 \text{ bytes}$. Dado que cada bloque comienza en un desplazamiento par, existe un desperdicio de:

$$8.192 - 8.000 = 192 \text{ bytes/bloque}$$

15.3.2. Localización de cada pixel en la memoria de pantalla.

Para determinar cual es la posición en la memoria de pantalla que corresponde a cada pixel se deben tener en cuenta los siguientes aspectos:

- Si es alta resolución, cada byte tiene 8 pixels; si es media resolución, cada byte tiene 4 pixels.
- Si la fila es par, hay que acceder al bloque 0 (desplazamiento 0000H respecto al comienzo de la memoria de pantalla); si la fila es impar, hay que acceder al bloque 1 (desplazamiento 8192H respecto al comienzo de la memoria de pantalla).
- Para acceder al primer bit de cada pixel, se deben efectuar operaciones de rotación de bits. Si, además, se desea modificar el color, se deben hacer las operaciones lógicas AND y OR con las mascarar apropiadas.

15.3.3. Generación de caracteres en modo gráfico.

Habíamos dicho que una de las funciones del adaptador de pantalla era de generar los caracteres que salen impresos sobre la pantalla. Asimismo vimos que se pueden definir hasta 256 caracteres distintos.

Tanto el adaptador monocromático como el adaptador CGA toman los códigos y sus correspondientes caracteres de una tabla residente en memoria ROM. En particular, el adaptador monocromático tiene en ROM una tabla con la totalidad de los 256 caracteres. En cambio, el adaptador CGA tiene en ROM una tabla para los primeros 128 caracteres (códigos 0 a 127), cuya dirección es F000H:FA6EH de la memoria ROM, y una tabla en RAM para los restantes 128 caracteres (códigos 128 a 255). La tabla en RAM se carga desde el sistema operativo, a través del comando GRAFTBL. La dirección de esta tabla en RAM es la apuntada por el vector de interrupción 1FH (la dirección absoluta de este vector de interrupción es 0007CH). Resulta posible, entonces, crear una tabla propia para los caracteres de códigos 128 a 255, dejándola residente en memoria, y cargar su dirección en el vector de interrupción 1FH. Cada carácter se define mediante 8 bytes (uno por línea), por lo que el tamaño de cada una de las tablas es de 1 Kbyte (1024 bytes). Los bits de cada byte indican los pixels activados dentro de cada línea. Por ejemplo, el esquema para la definición de la letra A (a mayúscula) es el siguiente:

		BITS	
BYTE	7 6 5 4 3 2 1 0	BINARIO	HEXA
0	* *	0011 0000	30H
1	* * * *	0111 1000	78H
2	* * * *	1100 1100	CCH
3	* * * *	1100 1100	CCH
4	* * * * *	1111 1100	FCH
5	* * * *	1100 1100	CCH
6	* * * *	1100 1100	CCH
7		0000 0000	00H

Si esto lo lleváramos a sentencias assembler, encontraríamos que para la letra A la definición es:

```
DB 030H,078H,0CCH,0CCH,0FCH,0CCH,0CCH,000H
```

El código asignado a dicha definición es el 41H (65 decimal). Para escribir texto en modo gráfico, primero se debe posicionar el cursor como si se trabajara en modo alfanumérico, y luego escribir el texto mediante la interrupción que corresponda. Debe tenerse en cuenta que:

- No es posible mover caracteres directamente a la memoria de pantalla, pues los bits serán interpretados como pixels.
- No existe control del color de fondo (este es tomado del color de fondo por default).

15.4. INTERRUPTACIONES PARA EL MANEJO DE LA PANTALLA

Existen dos grupos de funciones para el manejo de la pantalla: el primero, a través de la interrupción 10H del BIOS, y el segundo, a través de la interrupción 21H del DOS.

15.4.1. Interrupción 10H del BIOS.

Esta interrupción brinda 17 diferentes operaciones de pantalla, las que se especifican a través del registro AH. El vector de la interrupción 10H apunta a dirección absoluta F3605H (F000:3605), que es donde se encuentra la rutina de atención correspondiente. Las 17 operaciones se pueden dividir en los siguientes grupos:

1. Interface CRT
2. Manejo de caracteres
3. Gráficos
4. Teletipo (TTY)
5. Estado de la pantalla

15.4.1.1. Interfaces CRT.

Estas funciones permiten fijar el modo de la pantalla, establecer las características para el cursor y controlar la página activa. La mayoría de estas opciones se manejan con coordenadas.

Las siguientes son las distintas funciones de interface CRT:

AH	FUNCION
----	---------

00H	Establecer modo de la pantalla Entrada: AL=0 40 x 25, blanco y negro, modo texto AL=1 40 x 25, color, modo texto AL=2 80 x 25, blanco y negro, modo texto AL=3 80 x 25, color, modo texto AL=4 320 x 200, color, modo gráfico AL=5 320 x 200, blanco y negro, modo gráfico AL=6 640 x 200, blanco y negro, modo gráfico Salida: se borra la pantalla
01H	Establecer modo del cursor Entrada: CH (bits 0-4)=Línea inicial CH (bits 5-7)=0 CL (bits 0-4)=Línea final CL (bits 5-7)=0 Salida: Ninguna
02H	Posicionar cursor Entrada: DH=Fila (0 a 24) DL=Columna (0 a 79) BH=Número de página (0 para modo gráfico) Salida: Ninguna
03H	Leer posición actual del cursor Entrada: BH=Número de página (0 para modo gráfico) Salida: DH=Fila (0 a 24) DL=Columna (0 a 79) CH (bits 0-4)=Línea inicial CH (bits 5-7)=0 CL (bits 0-4)=Línea final CL (bits 5-7)=0
05H	Seleccionar página activa (modo texto) Entrada: AL=Nueva página (0 a 7 para modos 0 y 1; 0 a 3 para modos 2 y 3) Salida: Ninguna
06H	Desplazamiento (scroll) hacia arriba de página activa Entrada: AL=Cantidad de líneas. Las líneas de la parte inferior de la ventana se borran; si AL=0, se borra toda la ventana. CH=Fila de la esquina superior izquierda

CL=Columna de la esquina superior izquierda
 DH=Fila de la esquina inferior derecha
 DL=Columna de la esquina inferior derecha
 BH=Atributo a usar sobre la línea en blanco.

Salida: Ninguna

07H Desplazamiento (scroll) hacia abajo de pagina activa

Entrada: AL=Cantidad de líneas.

Las líneas de la parte superior de la ventana se borran; si AL=0, se borra toda la ventana.

CH=Fila de la esquina superior izquierda
 CL=Columna de la esquina superior izquierda
 DH=Fila de la esquina inferior derecha
 DL=Columna de la esquina inferior derecha
 BH=Atributo a usar sobre la línea en blanco.

Salida: Ninguna

Ejemplo 1: Mover el cursor al origen de coordenadas (HOME)

```
MOV DX,0           ;DL=0, DH=0 (origen de coordenadas)
MOV AH,2           ;Mover cursor
INT 10H
```

Ejemplo 2: Mover el cursor a una posición relativa a la actual

```
MOV AH,3           ;Leer posición actual del cursor
INT 10H
ADD DH,2           ;Incrementar en 2 la fila
ADD DL,2           ;Incrementar en 2 la columna
MOV AH,2           ;Mover cursor
INT 10H
```

15.4.1.2. Manejo de caracteres.

Estas funciones permiten transferir caracteres a o desde la pantalla, retornando el cursor a su posición original. Son especialmente útiles para hacer gráficos en modo texto ("pseudo-graficos"); como no producen el avance automático del cursor, son eficientes para desplegar mensajes o prompts. Otra característica de estas funciones es que no reconocen los códigos 07H, 08H, 0AH y 0DH como los caracteres de control

BELL, BS, LF y CR, respectivamente, sino que muestran los caracteres gráficos definidos para esos códigos.

Las siguientes son las distintas funciones de manejo de caracteres:

AH FUNCION

08H Leer carácter y atributo de la actual posición del cursor (modo texto)

Entrada: BH=Número de página

Salida: AL=código del carácter leído
 AH=atributo del carácter leído

El cursor queda en su posición original

09H Escribir carácter y atributo en la actual posición del cursor

Entrada: BH=Número de página

BL=Atributo del carácter (modo texto) =Color del carácter (modo gráfico)

CX=Cantidad de veces a repetir el carácter

AL=código del carácter a escribir

Salida: El cursor queda en su posición original

0AH Escribir sólo el carácter en la actual posición del cursor

Entrada: BH=Número de página

CX=Cantidad de veces a repetir el carácter

AL=código del carácter a escribir

Salida: El cursor queda en su posición original

15.4.1.3. Gráficos.

Las funciones de gráficos permiten escribir y leer pixels, y fijar la paleta de colores a usar para media resolución (320 x 200). Las siguientes son las funciones para gráficos provistas por la interrupción 10H:

AH FUNCION

- 0BH Establecer paleta de colores (modo gráfico 320 x 200)
Entrada: BH=Código de paleta (0 a 127)
 BL=color a ser usado con la paleta
Salida: Ninguna
- 0CH Escribir pixel (modo gráfico)
Entrada: DX=Número de fila (0 a 199)
 CX=Número de columna (0 a 639)
 AL=valor del color. Si el bit 7 de AL es 1, el valor del color se obtiene de una operación XOR entre el valor del color y el actual contenido del punto.
Salida: Ninguna
- 0DH Leer pixel (modo gráfico)
Entrada: DX=Número de fila (0 a 199)
 CX=Número de columna (0 a 639)
Salida: AL=pixel leído

15.4.1.4. Teletipo.

Las funciones de teletipo permiten mostrar en pantalla un carácter o un string. A diferencia de las funciones de manejo de caracteres, las funciones de teletipo tratan efectivamente como caracteres de control (y no como caracteres gráficos) a los códigos 07H, 08H, 0AH y 0DH. Por esta razón es que estas funciones se denominan de teletipo (TTY), pues tratan a la pantalla como si fuera una maquina de teletipo.

Las siguientes son las funciones de teletipo:

AH FUNCION

- 0EH Escribir carácter en la actual posición del cursor y luego avanzar el cursor
Entrada: AL=Código de carácter a escribir
 BL=Color del carácter (modo gráfico)
 BH=Número de página activa (modo texto)
Salida: El cursor se ubica a continuación del carácter

- 13H Escribir string desde la actual posición del cursor
Entrada: ES:BP=dirección del string
 CX=Longitud del string
 DH=Fila inicial
 DL=Columna inicial
 BH=Número de página

MODO 1

- Entrada: BL=Atributo (para todo el string)
 AL=0
Salida: El cursor retorna a su posición original

MODO 2

- Entrada: BL=Atributo (para todo el string)
 AL=1
Salida: El cursor se ubica a continuación del string

MODO 3

- Entrada: AL=2
Salida: El cursor retorna a su posición original

MODO 4

Entrada: AL=3

Salida: El cursor se ubica a continuación del string

La función 13H tiene 4 modos distintos.

En los modos 1 y 2, se especifica un atributo como entrada en el registro BL, que es valido para todo el string; para estos modos, el string es del tipo: carácter, carácter, ..., carácter.

En los modos 3 y 4, cada carácter debe llevar su propio atributo; el string es del tipo: carácter, atributo, carácter, atributo, ...

En los modos 1 y 3, el cursor retorna a la posición original, mientras que en los modos 2 y 4 el cursor se posiciona a continuación del último carácter del string.

Ejemplo 1: Producir un beep

```
MOV AL,7           ;código de BELL
MOV AH,0EH
INT 10H
```

Ejemplo 2: Imprimir un título en el tope de la pantalla

```
TITULO      DB 'Este es un título'
LON_TIT     EQU $-TITULO

...
MOV AL,3           ;seleccionar 80x25, color, texto
MOV AH,0           ;cambiar de modo
INT 10H
LEA BP,TITULO      ;desplazamiento del string
MOV CX,LON_TIT     ;cantidad de caracteres
MOV DX,0           ;comienzo en la esquina sup. izq.
MOV BL,41H         ;color de fondo=rojo
                  ;color de caracteres=azul

MOV AL,0           ;fijar modo 0 (retorno del cursor)
MOV AH,13H         ;seleccionar opción para string
INT 10H            ;mostrar string
```

Ejemplo 3: Igual al ejemplo 2, pero con los atributos de los caracteres del título alternados

```
TITULO1     DB 'Este es un '
TITULO2     DB 'T',42H,'i',41H,'t',42H,'u',41H,'l',42H,'o',41H
LON_TIT2    EQU $-TITULO2

...
MOV AL,3           ;seleccionar 80x25, color, texto
MOV AH,0           ;cambiar de modo
INT 10H
LEA BP,TITULO1     ;desplazamiento del primer string
MOV CX,TITULO2-TITULO1
MOV DX,0
MOV BL,41H         ;color de fondo =rojo, color de caracteres =azul
MOV AL,1           ;fijar modo 1
MOV AH,13H         ;mostrar primer string
INT 10H
MOV AH,3           ;leer posición del cursor
INT 10H
LEA BP,TITULO2     ;desplazamiento del segundo string
MOV CX,LON_TIT2
MOV AL,3           ;fijar modo 3
MOV AH,13H         ;mostrar segundo string
INT 10H
```

15.4.1.5. Estado de la pantalla.

Existe una sola función, dentro de la interrupción 10H, que retorna el estado de la pantalla. Las especificaciones de esta función son las siguientes:

AH	FUNCION
-----------	----------------

0FH	Leer estado actual de la pantalla Entrada: Ninguna Salida: AL=0 40 x 25, blanco y negro, modo texto AL=1 40 x 25, color, modo texto AL=2 80 x 25, blanco y negro, modo texto AL=3 80 x 25, color, modo texto AL=4 320 x 200, color, modo gráfico AL=5 320 x 200, blanco y negro, modo gráfico AL=6 640 x 200, blanco y negro, modo gráfico AH=Número de columnas sobre la pantalla BH=Número de pagina activa
-----	---

Ejemplo: Borrar la pantalla

PUSH AX	
PUSH BX	
MOV AH,15H	;Leer estado de la pantalla
INT 10H	
MOV AH,0	;Fijar modo
INT 10H	
POP BX	
POP AX	

15.4.2. Interrupción 21H del DOS

Bajo MSDOS existen también algunas funciones para manejar la pantalla.

Estas funciones, que se especifican a través de un código en el registro AH, son las siguientes:

AH	FUNCION
-----------	----------------

02H	Escribir un carácter (con chequeo de CTRL-BREAK) Entrada: DL= código del carácter a imprimir Salida: El cursor se ubica a continuación del carácter
09H	Escribir un string Entrada: DS:DX = dirección del string El string debe finalizar con el carácter '\$' Salida: El cursor se ubica a continuación del último carácter del string

Estas dos opciones usan el conjunto de caracteres del mismo modo que las funciones de teletipo de la interrupción 10H del BIOS, es decir que admiten la existencia de los caracteres de control (BELL, LF, CR y BS) como tales. No existe manera, a través de las funciones del DOS, de fijar los atributos para un carácter o de posicionarlo en determinado lugar, o de imprimirlo y hacer que el cursor retorne a la posición original. Todas esas opciones sólo son posibles a través de la interrupción 10H del BIOS. Para la función 09H, que permite imprimir un string, este debe finalizar con un carácter '\$' (código ASCII 24H), que actúa como delimitador. De este modo se puede obtener la longitud del string. El carácter '\$' no sale impreso. Si se quiere imprimir un string que incluya al carácter '\$', se debe hacer en tres pasos: primero, se debe imprimir la parte del string hasta el carácter '\$'; luego se debe imprimir el carácter '\$' en forma individual; y, por último, se debe imprimir la parte del string que está a continuación del carácter '\$'.

CAPITULO 16 EL TECLADO

16.1. GENERALIDADES DE HARDWARE

El teclado es un periférico de tipo carácter (character devise), de entrada de datos. Consiste de un conjunto de teclas que se activan cuando son presionadas. Su funcionamiento es completamente análogo al de una maquina de escribir, con la diferencia que en vez de producir la impresión de caracteres sobre papel emiten códigos que el microprocesador captura e interpreta. El teclado posee un controlador (Keyboard Controller), que cumple las funciones de detectar las teclas que fueron pulsadas y las teclas que fueron liberadas (dejadas de ser presionadas), e informar de ello al microprocesador. Ambas tareas permiten ahorrarle tiempo de ejecución al procesador central, ya que este no se debe ocupar del teclado en una primera instancia.

El BIOS contiene una rutina de atención del teclado, la correspondiente a la interrupción 09H, que se encarga de atender todo requerimiento del mismo. Así, tanto cuando se pulsa una tecla como cuando se libera una tecla, se produce una interrupción 09H generada por el controlador del teclado. Al producirse esta interrupción, toma control la correspondiente rutina BIOS. Existe, también, otra manera de comunicación entre el teclado y el BIOS, que es a través de puertas de E/S. Cada vez que se pulsa o se libera una tecla, el controlador del teclado guarda en su memoria interna un código especial de un byte de longitud. Este byte contiene un código que indica la función efectuada con la tecla ("pulsada" o "liberada") en el bit 7 (valores 1 ó 0 respectivamente), y un identificador de tecla o scan code en los bits restantes. Una vez que el controlador del teclado almacenó un byte en su memoria interna, envía al microprocesador una señal de interrupción 09H. El scan code es un valor entre 1 y 83 (01H a 53H) en el teclado estándar de la microcomputadora, donde cada tecla tiene asociado un valor diferente. La memoria del controlador del teclado puede almacenar hasta 16 bytes, que equivalen a 8 operaciones completas de pulsar y liberar teclas (golpes de tecla o keystrokes), lo que permite tipear cosas aún mientras el microprocesador esta ocupado.

16.1.1. La interrupción 09H.

La rutina de la interrupción 09H toma control cuando el controlador del teclado envía una señal de interrupción al microprocesador, pues se pulsó o se liberó alguna tecla. Esta rutina lee bytes de la memoria del controlador del teclado y los convierte en códigos de caracteres; también verifica si hay varias teclas presionadas sin ninguna liberada, lo que refleja que son teclas pulsadas simultáneamente con alguna de las teclas especiales (SHIFT, CTRL o ALT). Una vez que finalizó su tarea, la rutina almacena los scan code y los códigos de carácter en una porción de memoria, residente en la parte baja de la memoria RAM, denominada memoria o buffer del teclado. El buffer del teclado es manejado como buffer circular o cola: el primero que entra es el primero que sale. La capacidad de este buffer es de 15 golpes de tecla (keystrokes), lo que da suficiente margen para no perder datos cuando se tipea muy rápido. Si se pulsa alguna tecla cuando el buffer esta lleno, el BIOS ignora la tecla y produce la emisión de la alarma (beep). Cuando toma control la rutina de atención de la interrupción 09H por haber pulsado alguna tecla, se almacenan dos bytes en el buffer del teclado: el primero contiene el scan code de la tecla, y el segundo el código ASCII de la misma.

La rutina de atención de la interrupción 09H también maneja dos bytes de control, que se usan para almacenar el estado de las teclas especiales (ambos Shift, Ctrl Alt, Ins, Num Lock, Caps Lock y Scroll Lock). Entonces, las teclas especiales no se almacenan en el buffer, sino que modifican los bytes de control.

El primero de estos dos bytes de control indica el estado de dichas teclas, donde 1 corresponde a tecla activa y 0 a tecla no activa:

BIT	TECLA
7	Ins
6	Caps-Lock
5	Num-Lock
4	Scroll-Lock
3	Alt
2	Ctrl
1	Shift (izquierda)
0	Shift (derecha)

El segundo byte de control indica si una tecla ha sido o no pulsada (valores 1 y 0 respectivamente):

BIT	TECLA
7	Ins
6	Caps-Lock
5	Num-Lock
4	Scroll-Lock
3	Ctrl Num-Lock
2	Sys-Req
1	Sin uso
0	Sin uso

Para obtener el valor ASCII de una tecla pulsada, la rutina verifica el estado de las teclas especiales.

Por ejemplo, para saber si es una letra mayúscula o minúscula.

En muchos casos el segundo código es cero, lo que indica que se pulsó alguna tecla particular como ser las teclas de cursor o las teclas de función. En estas situaciones se debe examinar el primer código (scan code) para identificar la tecla pulsada. Es bastante común la combinación de teclas, es decir una tecla especial junto con otra tecla, como por ejemplo Ctrl-C. En estos casos, el código generado varía según diferentes consideraciones: por ejemplo, toda combinación de la tecla especial Ctrl con una tecla de letra produce un código del tipo xx01 a xx1A (desde a hasta z, respectivamente), donde xx es el scan code de la tecla de letra, mientras que cualquier otra combinación de Ctrl devuelve un código del tipo xx00, donde xx es el scan code de la segunda tecla. También hay algunas combinaciones de teclas especiales, que tienen un significado particular dado que producen alguna acción predeterminada:

Alt-Ctrl-Del: Carga del sistema.

Ctrl-Break: Se invoca a la interrupción BIOS 1BH, que produce un "break" del teclado.

Ctrl-Num Lock: Se suspende la rutina del teclado hasta que se vuelva a pulsar cualquier tecla (excepto Num Lock). Sirve para suspender temporariamente cualquier operación, para reanudarla después.

Shift-Prt Sc: Invoca a la interrupción BIOS 05H que produce la salida por impresora del contenido de la

pantalla (hardcopy). El hardcopy se hace carácter a carácter o pixel a pixel según sea el modo de funcionamiento de la pantalla.

En el apéndice E se detallan los scan code de cada tecla y los códigos que se generan al pulsarlas. Asimismo, se indican los códigos producidos por combinaciones de teclas.

16.2. INTERRUPCIONES PARA EL MANEJO DEL TECLADO

Al igual que para el resto de los dispositivos, existen interrupciones del BIOS e interrupciones del DOS que están asociadas al manejo del teclado.

16.2.1. Interrupciones BIOS.

Existen tres interrupciones BIOS para el manejo del teclado, cuyos códigos son 09H, 1BH y 16H. En la sección anterior vimos en detalle la interrupción 09H, por lo que no es necesario hacer mas agregados. La interrupción 1BH es la que se produce al pulsar simultáneamente las teclas Ctrl-Break cuando el control lo tiene el BIOS. Esta rutina puede ser cambiada por una rutina propia de usuario. La interrupción 16H, en cambio, suministra un conjunto de funciones varias, lo que la hace sumamente adecuada para programas en assembler.

Interrupción BIOS 16H.

Las siguientes, son las funciones que brinda la interrupción 16H (el código de función se pasa en el registro AH):

AH FUNCION

00H Leer un carácter del buffer

Entrada: Ninguna

Salida: AH =Scan code de la tecla pulsada

AL =Código ASCII del carácter

Acción: Lee del buffer del teclado los códigos asociados a una tecla o a una combinación de teclas, almacenándolos en AX, y avanza el puntero del buffer a siguiente carácter. Si el buffer está vacío, espera a que se pulse alguna tecla.

01H Obtener estado del buffer

Entrada: Ninguna

Salida: ZF=0 y AX =códigos del siguiente carácter

ZF=1 (buffer vacío)

Acción: Esta función devuelve el estado del buffer del teclado a través de la bandera de estado ZF: si ZF=0, el buffer está vacío; si ZF=1, el buffer no está vacío (en este caso devuelve, además, los códigos del próximo carácter en el registro AX, sin mover el puntero del buffer).

AH FUNCION

02H Obtener estado de las teclas especiales

Entrada: Ninguna

Salida: AL=Byte de estado del teclado

Acción: Devuelve en el registro AL el primero de los dos bytes de control para las teclas especiales.

16.2.2. Interrupciones DOS.

El DOS dispone de 2 interrupciones asociadas al teclado: la interrupción 21H y la interrupción 23H.

La interrupción 21H tiene, a su vez, un conjunto de funciones que se especifican a través de un código en el registro AX. La interrupción 23H es la que se produce al pulsar simultáneamente las teclas Ctrl-Break cuando el control lo tiene el DOS. Esta rutina puede ser cambiada por una rutina propia de usuario.

Interrupción DOS 21H

Las siguientes, son las funciones para el teclado que brinda la interrupción 21H (el código de función se pasa en el registro AH):

AH FUNCION

01H Esperar para leer un carácter del teclado con eco y chequeo de Ctrl-C.

Entrada: Ninguna

Salida: AL =código ASCII del carácter tipeado

Acción: Espera para la lectura de un carácter del teclado, detectando la presencia de Ctrl-C, lo muestra por pantalla (eco) y lo almacena en el registro AL.

06H Leer un carácter del teclado sin eco ni chequeo de Ctrl-C.

Entrada: DL=0FFH

Salida: AL =código ASCII del carácter leído, si es que existe alguno en el buffer; de lo contrario AL=0

Acción: Lee un carácter del buffer del teclado y lo devuelve en el registro AL, sin mostrarlo por pantalla. Si no hay ningún carácter en el buffer, devuelve 0 en AL. No efectúa el chequeo de Ctrl-C.

07H Esperar para leer un carácter del teclado sin eco ni chequeo de Ctrl-C.

Entrada: Ninguna

Salida: AL =código ASCII del carácter tipeado

Acción: Espera para la lectura de un carácter del teclado, sin detectar la presencia de Ctrl-C, y lo almacena en el registro AL sin mostrarlo por pantalla (eco).

08H Esperar para leer un carácter del teclado sin eco y con chequeo de Ctrl-C.

Entrada: Ninguna

Salida: AL =código ASCII del carácter tipeado

Acción: Espera para la lectura de un carácter del teclado, detectando la presencia de Ctrl-C, y lo almacena en el registro AL sin mostrarlo por pantalla (eco).

0AH Leer un string y almacenarlo en un área de memoria

Entrada: DS:DX =Dirección del área de memoria

(DS:DX)=Cantidad máxima de caracteres a leer

Salida: Ninguna

Acción: Lee un conjunto de caracteres y los almacena en el área de memoria a partir de la dirección indicada por DS:DX + 2. El primer byte del área debe ser distinto de cero e indicar el número máximo de caracteres a teclear, incluido el retorno de carro (CR). El segundo byte se llena por el DOS con la cantidad de caracteres efectivamente ingresados, sin incluir el CR. Esta función efectúa el chequeo de Ctrl-C, y realiza eco por pantalla.

0BH Leer estado del buffer del teclado

Entrada: Ninguna

Salida: AL=FFH si hay algún carácter disponible

AL=00H si el buffer está vacío

Acción: Detecta la existencia o no de caracteres en el buffer del teclado.

0CH Borrar buffer y llamar a una función de entrada de teclado.

Entrada: AL =Número de función del teclado invocada (01H, 06H, 07H, 08H, 0AH)

Salida: La de la función invocada

Acción: Limpia el buffer del teclado y efectúa la función de entrada indicada en el registro AL.

Algunas consideraciones particulares se deben tener en cuenta en el caso que las funciones de lectura del teclado lean teclas especiales. El DOS siempre toma, ante estas teclas (están formando parte de un string o como caracteres individuales), los dos bytes del buffer del teclado, el primero conteniendo 00H y el segundo conteniendo el scan code de la tecla pulsada. En el caso que se esté leyendo un string (función 0AH), las teclas de función se almacenan en dos bytes (el primero con 00H y el segundo con el scan code). En cambio, las teclas de edición (Ins, Del, Home, etc) no devuelven ningún código, sino que producen una acción de edición. En el caso de la lectura de un sólo carácter, será necesario hacer dos lecturas con las funciones del DOS: la primera lectura devolverá 00H y la segunda devolverá el scan code de la tecla. Esta situación debe ser controlada por el programa. Las combinaciones con la tecla Shift (mayúsculas), devuelven el valor que corresponda según la tabla ASCII. Las combinaciones con la tecla Ctrl, devuelven el mismo valor que para el BIOS. La tecla Alt devuelve el scan code de la tecla asociada.

16.2.2.1. Ejemplos de aplicación.

Ejemplo 1: Aceptar S o N como opciones (considerando mayúsculas y minúsculas)

```
LEER:  MOV  AH,1      ;leer una tecla y mostrarla
        INT  21H
        CMP  AL,'S'   ;¿es S?
        JE   SI        ;si es S, bifurcar a SI
        CMP  AL,'s'   ;¿es s?
        JE   SI        ;si es s, bifurcar a SI
```

```

CMP  AL,'N' ;¿es N?
JE   NO      ;si es N, bifurcar a NO
CMP  AL,'n' ;¿es n?
JE   NO      ;si es n, bifurcar a NO
JNE  LEER     ;de lo contrario, leer de nuevo

```

Ejemplo 2: Aceptar ENTER como respuesta

```

LEER  MOV  AH,7      ;leer una tecla sin mostrarla
      INT  21H
      CMP  AL,0DH    ;¿es CR?
      JNE  LEER      ;si no es CR, leer de nuevo

```

Ejemplo 3: Leer las teclas de función F1, F2 y F3

```

      MOV  AH,7      ;leer una tecla sin mostrarla
      INT  21H
      CMP  AL,0      ;¿es 0 el primer byte?
      JE   CODEXT    ;ir a leer el scan code
      JMP  ERROR     ;no es tecla de función; indicar error
CODEXT: MOV  AH,7
      INT  21H
      CMP  AL,3BH    ;¿es F1?
      JE   OPCION1
      CMP  AL,3CH    ;¿es F2?
      JE   OPCION2
      CMP  AL,3DH    ;¿es F3?
      JE   OPCION3
      JMP  ERROR     ;Tecla invalida; indicar error

```

Ejemplo 4: Mostrar un string en pantalla

```

MENSA: DB  'Ingrese su nombre: $'
...
      LEA  DX,MENSA   ;mostrar el mensaje
      MOV  AH,9
      INT  21H

```

Ejemplo 5: Leer un string (de hasta 50 caracteres) desde el teclado

```

NOMBRE DB  51,51 DUP(?)
...
      LEA  DX,NOMBRE   ;leer string
      MOV  AH,0AH
      INT  21H
      SUB  CH,CH       ;cargar la cantidad de caracteres
      MOV  CL,NOMBRE+1 ;leídos en CX
      ADD  DX,2        ;DX apunta al texto.

```

CAPITULO 17 DISCOS

El disco es el medio masivo de almacenamiento de datos mas ampliamente usado. En este capítulo nos referiremos a la estructura física y lógica de los discos, y de que manera podemos acceder a ellos.

17.1. ESTRUCTURA DE UN DISCO

Los discos (nombre general con el que identificaremos tanto al disco flexible o diskette como al disco rígido o winchester), son dispositivos magnéticos para almacenamiento de datos. Un disco consta de una serie de superficies o caras, recubiertas de un baño de óxido de hierro, donde se "graban" bits a través de impulsos electromagnéticos. Cada cara esta compuesta por circunferencias concéntricas llamadas pistas, numeradas secuencialmente desde afuera hacia adentro (la pista cero es la mas externa) y separadas entre si por una distancia fija. Todas las pistas tienen la misma capacidad de almacenamiento, con lo que las pistas mas cercanas al centro tendrán mayor densidad por ser de menor longitud. Cada pista se divide a su vez en sectores, siendo un sector la mínima unidad de E/S en disco. Según sea la cantidad de información que quepa en un sector, la cantidad de sectores por pista, la cantidad de pistas por cara y la cantidad de caras, tendremos entonces la capacidad de almacenamiento de un disco. Las especificaciones para los dos tipos de discos mas usados son:

Disco flexible o diskette

2 caras

40 pistas por cara

9 sectores por pista

512 bytes por sector

capacidad = 360 Kbytes

Disco duro o winchester

4 caras

305 pistas por cara

17 sectores por pista

512 bytes por sector
capacidad = 10.370 Kbytes (10 Mbytes)

Además de estos, existen otros discos con distintas capacidades de almacenamiento como ser diskettes de 1,2 Mbytes o discos duros de 20, 30, y 60 Mbytes.

Un concepto importante es el de cilindro: un cilindro corresponde al conjunto de pistas del mismo número sobre las distintas caras. Al igual que las pistas, los cilindros se numeran desde afuera hacia adentro siendo el cilindro cero (que abarca a las pistas cero de todas las caras) el mas externo.

Un sector contiene 3 partes bien diferenciadas:

- Cabecera
- Cuerpo
- Cola

La cabecera comienza con 4 bytes que tienen, respectivamente, el número de pista, el número de cara, el número de sector y el indicador de tamaño de sector (0 = 128 bytes, 1 = 256 bytes, 2 = 512 bytes, 3 = 1024 bytes). A continuación se ubican 2 de CRC (código de redundancia cíclica), los que contienen una suma de autocomprobación de los bytes anteriores; el CRC se calcula cuando se graba un sector, y cada vez que el mismo se lee, se recalcula y se compara con el valor leído (si no coinciden, se produce un error de CRC). El resto de la cabecera del sector contiene información para el controlador de disco. El cuerpo es la zona correspondiente a los datos propiamente dichos que se almacenan en el sector. La cola contiene 2 bytes de CRC correspondientes al cuerpo; se calcula en el momento de grabar el sector y, al igual que con el CRC de la cabecera, se recalcula cuando se lee el sector y se compara con el leído, produciéndose un error de CRC en el caso que no coincidan.

17.2. E/S SOBRE DISCO

La unidad de disco consta de un motor que produce que el disco gire a una determinada velocidad constante (en el caso de los discos flexibles, este motor no esta en constante funcionamiento sino que se pone en marcha cuando existe un requerimiento de acceso al disco). Cuando se produce una solicitud de E/S, comienza el movimiento de la cabeza lecto/grabadora para posicionarse sobre el comienzo del sector que debe acceder para leer o grabar. La cabeza se mueve desde afuera hacia adentro, es decir en el sentido creciente por número de pista. Para hacer un calculo del tiempo de lectura/grabación sobre disco, habrá que tener en cuenta lo siguiente:

- El tiempo que tarda la cabeza en posicionarse encima de la pista a la que debe acceder; se conoce como tiempo de posicionamiento y, en general, se lo considera como la mitad del tiempo máximo de posicionamiento (este se define como el tiempo que tarda la cabeza en desplazarse desde la primera hasta la última pista). Esta aproximación (promedio) se toma teniendo en cuenta que los accesos al disco son al azar, es decir que un acceso es independiente del anterior); sin embargo, para ser mas precisos en el calculo del posicionamiento, deberíamos adoptar alguna función probabilística mas compleja.
- El tiempo necesario para que el sector a acceder pase debajo de la cabeza lecto/grabadora una vez que est fue posicionada sobre la pista que corresponde; se denomina tiempo de latencia, y es totalmente dependiente de la velocidad de rotación del disco. En general, el tiempo de latencia se lo considera equivalente a la mitad del tiempo necesario para dar un giro completo.
- En el caso de los discos flexibles también habrá que tener en cuenta el tiempo que tarda el motor en alcanzar la velocidad de rotación máxima, que es de, aproximadamente, 0,5 segundos.

Para acceder a la información de un sector en disco son necesarios 4 datos: unidad, cara, pista de la cara y sector de la pista. El DOS maneja un sistema de identificación lógica o absoluta de los sectores, consistente en numerarlos desde afuera hacia adentro (es decir, por pista, cara y sector, en ese orden). En el caso de un disco flexible de 360 Kbytes de capacidad tendremos:

Sector lógico 0 : pista 0, cara 0, sector 1
Sector lógico 2 : pista 0, cara 0, sector 2
.....
Sector lógico 9 : pista 0, cara 1, sector 1
.....
Sector lógico 719 : pista 39, cara 1, sector 9

17.3. MAPA DE UN DISCO

Para que un disco sea utilizable es necesario inicializarlo, es decir prepararlo física y lógicamente para que pueda ser leído y grabado. Este proceso se denomina formateo, y consiste en la creación de una serie de áreas especiales para uso del sistema operativo. En DOS el formateo de un disco se efectúa a través del comando FORMAT, que genera 4 áreas especiales:

1. Area de bootstrap o registro de autoarranque
2. Area de tabla de aloación de archivos (FAT)
3. Area de directorio
4. Area de datos

17.3.1. Area de bootstrap o registro de autoarranque.

El área de bootstrap o registro de autoarranque es un pequeño programa en lenguaje de maquina, residente a partir del byte 0 de la pista 0, cara 0, sector 1, que produce el arranque del proceso de carga del sistema operativo; este proceso se conoce en general como IPL (Initial Program Loader), y lo primero que hace es verificar la existencia de los archivos de carga del DOS (archivos IBMBIO.COM e IBMDOS.COM).

Todos los discos tienen este registro, contengan o no efectivamente a los archivos del sistema operativo.

A partir del byte 4 (desplazamiento 3) del registro se encuentra la tabla de parametros del BIOS, que tiene la siguiente estructura:

DESP.	LONG.	DESCRIPCION
3	8	Identificación del sistema (número de versión)
11	2	Bytes por sector
13	1	Sectores por unidad de locación
14	2	Cantidad de sectores reservados al comienzo
16	1	Cantidad de copias de la FAT
17	2	Cantidad de entradas en el directorio raíz
19	2	Cantidad total de sectores en el disco
21	1	Identificación del formato
22	2	Cantidad de sectores por cada FAT
24	2	Cantidad de sectores por pista
26	2	Cantidad de caras
28	2	Cantidad de sectores especiales reservados

17.3.2. Area de tabla de locación de archivos (FAT).

El área de la tabla de locación de archivos contiene 2 copias de esta tabla. La FAT (File Allocation Table) es una tabla que representa un mapa del área de datos del disco; cada entrada de la FAT corresponde a un cluster o unidad de locación (una unidad de locación es la mínima cantidad de espacio en disco que puede ocupar un bloque de información). Cada entrada de la FAT es de 12 bits de longitud (es decir que hay dos entradas por cada 3 bytes), e indica si la unidad de locación está libre, defectuosa o asignado a un archivo (en el capítulo 18 veremos más en detalle como se asignan las unidades de locación a los archivos). Las dos primeras entradas de la FAT (es decir, los 3 primeros bytes), son especiales, y corresponden a las unidades de locación 0 y 1 (esas, en realidad, no existen como tales sino que corresponden al área de disco reservada para el sistema operativo). Estos 3 bytes contienen lo siguiente:

- byte 0: tipo de disco y formato
- byte 1: FFH
- byte 2: FFH

El primer byte es el más importante, pues trae información de relevancia; los posibles valores que toma y sus significados son los siguientes:

- FFH: disco flexible de 2 caras y 8 sectores por pista
- FEH: disco flexible de 1 cara y 8 sectores por pista
- FDH: disco flexible de 2 caras y 9 sectores por pista
- FCH: disco flexible de 1 cara y 9 sectores por pista

- F8H: disco rígido

El área de FAT ocupa 4 sectores en un disco flexible de doble cara y 9 sectores por pista; en un disco rígido, ocupa 16 sectores.

17.3.3. Área de directorio.

El directorio es una tabla que contiene el índice o catálogo de los archivos residentes sobre el disco.

Cada entrada del directorio ocupa 32 bytes de longitud, es decir que habrá 16 entradas por sector. En un disco flexible de doble cara y 9 sectores por pista el directorio ocupa 7 sectores (por lo tanto, hay capacidad para catalogar 112 archivos), mientras que en un disco rígido el directorio ocupa 32 sectores (la capacidad es, entonces, de 512 archivos). La estructura de cada entrada del directorio es la siguiente:

DESP.	LONG.	DESCRIPCION
0	8	Nombre del archivo
8	3	Extensión del archivo
11	1	Byte de atributo
12	10	Reservado
22	2	Hora de creación o de última modificación
24	2	Fecha de creación o de última modificación
26	2	Primera unidad de locación del archivo
28	4	Tamaño del archivo

El primer byte de cada entrada del directorio (es decir el primer byte del nombre de archivo) indica estado de dicha entrada. Los posibles valores y sus significados son los siguientes:

- 00H: Indica que la entrada no está utilizada, y que no existen más entradas utilizadas en el directorio.
- E5H: Indica entrada vacía; se utilizó pero está desocupada (el archivo fue borrado).
- 2EH: Indica entrada correspondiente a un subdirectorio; si el segundo byte es también 2EH, el campo de primera unidad de locación del archivo indica la del directorio padre (excepto cuando este es el directorio raíz, en cuyo caso el valor de dicho campo es 0000H).
- XXH: Cualquier otro valor es el código ASCII del primer carácter del nombre del archivo.

El nombre del archivo es un string de hasta 8 bytes de longitud; la extensión es un string de hasta 3 bytes de longitud. Mientras que el nombre es obligatorio (al menos un carácter, la extensión es optativa).

El byte de atributo es un conjunto de 8 banderas, de un bit cada una, que indican el atributo del archivo asignado a la entrada del directorio. El siguiente es el significado de cada bit:

- bit 7 (10000000B ---> 80H): Sin uso
- bit 6 (01000000B ---> 40H): Sin uso
- bit 5 (00100000B ---> 20H): Archivo modificado desde la última copia de resguardo (BACKUP).
- bit 4 (00010000B ---> 10H): Subdirectorio
- bit 3 (00001000B ---> 08H): Rótulo del volumen
- bit 2 (00000100B ---> 04H): Archivo del sistema
- bit 1 (00000010B ---> 02H): Archivo escondido
- bit 0 (00000001B ---> 01H): Archivo de lectura solamente

-El bit 5 (archivo modificado desde la última copia de resguardo) sirve para marcar aquellos archivos que han sido modificados.

-El bit 4 (subdirectorio) indica que la entrada corresponde a un subdirectorio (un subdirectorio es, al igual que el directorio, un índice de archivos; a diferencia del directorio raíz, un subdirectorio no está ubicado en una posición fija del disco sino que está en el área de datos, y no tiene un límite máximo de archivos. Una entrada de un subdirectorio tiene la misma estructura que una del directorio raíz).

-El bit 3 (rótulo del volumen) indica que la entrada corresponde al nombre del disco, el que se ubica en los bytes 1 a 11.

-El bit 2 (archivo del sistema) sirve para marcar los archivos propios del sistema operativo.

-El bit 1 (archivo escondido) sirve para "ocultar" el archivo, de modo que no puede ser visto a través del listado del directorio (comando DIR), ni copiado (COPY) ni borrado (ERASE, DEL); sin embargo puede ser visualizado (TYPE).

-El bit 0 (archivo de lectura solamente) sirve para proteger al archivo contra escritura; de este modo, el archivo sólo puede ser usado para cualquier operación que lo lea (COPY, TYPE, etc).
El atributo de un archivo puede estar dado por cualquier combinación de los bits 5, 2 y 0 del byte de atributo. Los bits 6 y 7 no se usan, y los bits 3 y 4 se utilizan por si mismos sin combinarse con ningún otro. La hora de creación o de última modificación tiene el siguiente formato:

- bits 15 a 11: Horas (0 a 23)
- bits 10 a 5 : Minutos (0 a 59)
- bits 4 a 0 : Segundos dividido 2 (0 a 29)

La fecha de creación o de última modificación tiene un formato similar:

- bits 15 a 9 : Número a sumarle a 1980(0 a 119). El rango del año es desde 1980 hasta 2099.
- bits 8 a 5 : Mes (1 a 12)
- bits 4 a 0 : Día (1 a 31)

La primera unidad de locación del archivo corresponde al número del primer cluster que ocupa el archivo en el área de datos. El tamaño del archivo es la cantidad de bytes que ocupa el archivo; el máximo tamaño de un archivo puede ser de 233-1 bytes. Los campos de hora, fecha, primera unidad de locación y tamaño se deben leer en sentido inverso, dado que están almacenados como sendos números enteros (de 2 bytes de longitud los 3 primeros y de 4 bytes el último).

Ejemplo : El siguiente vuelco hexadecimal corresponde a una entrada del directorio:

50525545424120204441542000000000000000000000A253760B9B0080080000

El contenido de estos bytes es el siguiente:

Nombre (5052554542412020) : PRUEBA

Extensión (444154) : DAT

Atributo (20) : Archivo

Hora (A253) : 10:29:04

Fecha (760B) : 22/11/1985

1ra. unidad de locación (9B00): 155

Tamaño (80080000) : 2.176 bytes

El siguiente es el procedimiento para obtener la hora y la fecha:

Hora: A253 --> 53A2 --> 0101001110100010 --> 01010 0111010 0010 -->

10 29 02 --> 10 horas, 29 minutos, 04 segundos

Fecha: 760B --> 0B76 --> 000010110110110 --> 0000101 1011 10110 -->

05 11 22 --> año 1985, mes 11, día 22

17.3.4. Área de datos. El área de datos de un disco es el lugar donde se almacenaran físicamente los archivos, y ocupa el resto del disco. Para un disco flexible de doble cara y 9 sectores por pista, el área de datos abarca 708 sectores, lo que significa 354 Kbytes efectivos para archivos; para un disco rígido, el área de datos abarca 20.691 sectores, que significan 10.345,5 Kbytes efectivos.

17.3.4.1. Subdirectorios.

Existen dos tipos de directorios: el directorio raíz y los subdirectorios. El contenido de ambos es el mismo, pero mientras el directorio raíz tiene un tamaño fijo y se almacena en una posición fija del disco, un subdirectorio puede tener cualquier tamaño y almacenarse en cualquier parte del área de datos del disco (es decir que se crea como un archivo cualquiera, pero con el atributo de subdirectorio). Cuando se crea un subdirectorio (comando MKDIR del DOS) se genera una entrada en el directorio padre con atributo 04H y tamaño cero (la longitud real de un subdirectorio se obtiene de la cantidad de unidades de locación que ocupe). En el subdirectorio se crean dos entradas con los nombres "." y "..", que sirven para mantener una estructura de árbol entre los distintos niveles de directorios. La primera entrada se refiere al propio subdirectorio, y la segunda se refiere al directorio padre. Los respectivos campos de primera unidad de locación indican la del propio subdirectorio y la del directorio padre (si esta es cero, significa que el directorio padre es el directorio raíz).

17.4. INTERRUPCIONES PARA EL ACCESO A DISCO

Existen, básicamente, dos maneras de trabajar para acceder al disco: a través del manejo de archivos o en forma absoluta. El primer modo (archivos), que será ampliamente analizado en el capítulo 18, es el soportado por el DOS; el modo de acceso absoluto esta soportado por el BIOS y también por algunas funciones del

DOS. A través del acceso absoluto a disco, podremos acceder a cualquier sector del mismo sin ningún tipo de limitación. Así, es posible entrar al área de FAT, al directorio, a archivos escondidos e, incluso, al registro de autoarranque (boot). Cuando se utilizan las interrupciones para acceder a disco se deben considerar los posibles errores de hardware; todas las funciones (tanto del DOS como del BIOS), retornan un valor de estado en el registro AH después de cada operación. Pero mientras que el DOS detecta y procesa todos los errores de hardware, el BIOS no hace nada. Es decir que si se usan las funciones del BIOS habrá que escribir en los programas párrafos destinados a la detección y recuperación de los errores.

La siguiente tabla muestra los distintos valores de estado, a que error corresponden y las posibles causas de error:

AH	ERROR	CAUSA PROBABLE
00H	No hay error	
02H	Marca de dirección no encontrada	Disco defectuoso
03H	No se puede escribir sobre el disco	Disco protegido
04H	Sector no encontrado	Disco defectuoso
08H	Error DMA	Adaptador defectuoso
10H	Error CRC	Disco defectuoso
20H	Error FDC	Adaptador defectuoso
40H	Error de posicionamiento (seek)	Unidad defectuosa
80H	Tiempo excedido (time-out)	Unidad defectuosa o no lista

Hay que tener en cuenta que al invocar una función de lectura puede aparecer un error de hardware que antes no existía; esto se debe a que, para operaciones de lectura, el BIOS no espera que el motor del disco alcance la máxima velocidad de rotación (aproximadamente 0,5 segundos). Por esta situación, es necesario repetir, como mínimo, 3 veces cualquier función de lectura con el BIOS antes de determinar que realmente existe un error (con el DOS, la lectura absoluta en disco se debe repetir no menos de 5 veces antes de determinar la existencia de un error). En las operaciones de escritura no ocurre lo mismo, dado que tanto el BIOS como el DOS esperan a que el motor alcance su máxima velocidad de giro.

Para el acceso a disco en forma absoluta existen las siguientes interrupciones:

- 13H BIOS (E/S a disco)
- 25H DOS (Lectura absoluta)
- 26H DOS (Grabación absoluta)

La interrupción 21H del DOS provee, en cambio, un conjunto de funciones de E/S basadas en el manejo de archivos. Esto será estudiado con detalle en el capítulo 18.

17.4.1. Interrupción 13H BIOS.

Esta interrupción brinda una serie de servicios de E/S absoluta sobre disco; las funciones se especifican a través de un código de función en el registro AH.

Las funciones correspondientes a la interrupción 13H del BIOS son las siguientes:

AH	FUNCION
----	---------

00H	Restaurar (reset) el controlador de disco
-----	---

Acción: Esta función permite inicializar el hardware de la interfaz del disco. Se utiliza cuando, al realizar una operación de E/S, aparece una condición de error; en este caso, se debe usar la función antes de volver a intentar la E/S.

Entrada: Ninguna

Salida: AH =código de estado

01H	Leer estado del disco
-----	-----------------------

Acción: Obtiene el estado de la última operación de E/S realizada.

Entrada: Ninguna

Salida: AH =código de estado

02H	Leer sectores
-----	---------------

Acción: Transfiere datos desde disco a un buffer en memoria (la transferencia de información se hace por sectores, cada uno de 512 bytes de tamaño).

Entrada: DL =Código de unidad (0 a 3)

DH =Número de cara (0 ó 1)

CH =Número de pista (0 a 39)

CL =Número de sector (1 a 9)

AL =Cantidad de sectores (1 a 9)

ES:BX =Dirección del buffer en memoria
 Salida: AL =Cantidad de sectores leídos
 AH =Código de estado
 CF =0 si no hubo error
 CF =1 si hubo error

03H Grabar sectores
 Acción: Transfiere datos desde un buffer de memoria a disco.
 Entrada: DL =Código de unidad (0 a 3)
 DH =Número de cara (0 ó 1)
 CH =Número de pista (0 a 39)
 CL =Número de sector (1 a 9)
 AL =Cantidad de sectores (1 a 9)
 ES:BX =Dirección del buffer en memoria
 Salida: AL =Cantidad de sectores grabados
 AH =Código de estado
 CF=0 si no hubo error
 CF=1 si hubo error

04 Verificar sectores
 Acción: Verifica el estado de los sectores indicados.
 Entrada: DL =Código de unidad (0 a 3)
 DH =Número de cara (0 ó 1)
 CH =Número de pista (0 a 39)
 CL =Número de sector (1 a 9)
 AL =Cantidad de sectores (1 a 9)
 Salida: AL =Cantidad de sectores grabados
 AH =Código de estado
 CF=0 si no hubo error
 CF=1 si hubo error

05 Formatear una pista
 Acción: Formatea (prepara) la pista indicada.
 Entrada: DL =Código de unidad (0 a 3)
 DH =Número de cara (0 ó 1)
 CH =Número de pista (0 a 39)
 AL =Cantidad de sectores (1 a 9)
 Salida: AH =Código de estado
 CF=0 si no hubo error
 CF=1 si hubo error

17.4.2. Interrupciones DOS de E/S absoluta.

Existen dos interrupciones del DOS que permiten efectuar E/S sobre disco en forma absoluta. Son las siguientes:

NRO	FUNCION
-----	---------

25H	Lectura absoluta
-----	------------------

Acción: Lee un conjunto de sectores a partir de un determinado sector lógico, y los transfiere a un buffer en memoria. Tras la ejecución, las banderas salvadas (por la instrucción INT) se mantienen en la pila, por lo que se las debe recuperar con la instrucción POPF; esto se debe hacer de esta manera dado que cierta información se carga sobre las banderas. Además, se debe resguardar la información de los registros (excepto los de segmento), pues también se pierde su contenido.

Entrada:	AL =Código de unidad (0 a 3)
	CX =Cantidad de sectores a leer
	DX =Número del sector lógico inicial
	DS:BX =Dirección del buffer en memoria
Salida:	AH =código de estado
	CF=0 si no hubo error

CF=1 si hubo error

AX =Identificación del error

(AH =Código de estado, AL =Código de error DOS)

26H Grabación absoluta

Acción: Transfiere desde un buffer de memoria, un conjunto de sectores a partir de un sector lógico determinado. Las especificaciones son las mismas que para la lectura absoluta.

Entrada: AL =Código de unidad (0 a 3)

CX =Cantidad de sectores a leer

DX =Número del sector lógico inicial

DS:BX =Dirección del buffer en memoria

Salida: AH =código de estado

CF=0 si no hubo error

CF=1 si hubo error

AX =Identificación del error

(AH =Código de estado, AL =Código de error DOS)

El disco es el medio masivo de almacenamiento de datos mas ampliamente usado. En este capítulo nos referiremos a la estructura física y lógica de los discos, y de que manera podemos acceder a ellos.

CAPITULO 18 ARCHIVOS

En el capítulo anterior hemos visto como es la estructura de los discos y de que manera podemos acceder a ellos en forma absoluta a través del BIOS y de algunas funciones del DOS. Sin embargo, la mayoría de las aplicaciones, acceden a los datos del disco en forma transparente (es decir, sin necesidad de especificar direcciones absolutas); esto es posible a través del manejo de archivos. En el presente capítulo trataremos en detalle el acceso a la información en disco mediante archivos. Para una mayor comprensión del tema veremos, en primer lugar, un conjunto de conceptos generales, para luego entrar de lleno en el manejo de archivos bajo MS-DOS.

18.1. CONCEPTOS GENERALES

En los siguientes párrafos efectuaremos un repaso de los conceptos principales referidos a los archivos.

18.1.1. Archivos, registros y campos.

Los datos se organizan y ven de forma distinta, dependiendo de que o quien los mire; así, para la computadora, los datos son bits y bytes. En cambio, para los usuarios, existen algunos límites lógicos para el manejo de los datos. A partir de esto, se pueden establecer y definir un conjunto de estructuras de datos para facilitar la manipulación de los mismos. Un archivo es una colección de datos relacionados entre si, identificado por un nombre único; un ejemplo de archivo puede ser el de todos los alumnos de la Universidad. Un archivo esta compuesto de unidades lógicas de datos llamadas registros; para el ejemplo citado, todos los datos de un alumno integran un registro del archivo de Alumnos. Un registro consta de un conjunto de datos únicos llamados campos, que representan información de un solo tipo; para el mismo ejemplo, el APELLIDO es un campo de cada uno de los registros del archivo de Alumnos. En general se puede establecer que un archivo es una colección lógica de registros, todos ellos con una misma configuración de campos.

18.1.1.1. Registro lógico y registro físico.

Podemos distinguir los registros entre registros lógicos y registros físicos o bloques. Los registros lógicos son aquellos que se refieren a la organización lógica de los datos (es decir, a su coherencia como tales); en cambio, los registros físicos son aquellos que se refieren a la organización física de los datos en los medios de almacenamiento. Un programa trabaja con registros lógicos, mientras que el sistema operativo trabaja con registros físicos. Los registros lógicos pueden ser de longitud fija (todos los registros lógicos de un archivo tienen la misma longitud en bytes), o de longitud variable (registros con distinta longitud en bytes). En general, un registro físico tiene una cantidad determinada de registros lógicos; además, un registro físico ocupa un número entero de sectores de disco.

18.1.1.2. Factor de bloqueo.

La relación entre registros físicos y registros lógicos esta dada por el factor de bloqueo. El factor de bloqueo es la cantidad de registros lógicos que caben en un registro físico o bloque. Un registro físico o bloque ocupa siempre una cantidad entera de sectores en disco. Sin embargo, no necesariamente se usa la totalidad del espacio con datos útiles, dado que existe una relación directa entre el tamaño de un bloque y el tamaño de un registro lógico. Por ejemplo, si un disco posee sectores de 1.024 bytes de tamaño, y alberga un archivo con registros lógicos de longitud fija de 100 bytes y factor de bloqueo=10, entonces para un bloque de tamaño =2 sectores, se almacenaran 20 registros lógicos por bloque (es decir, 2.000 bytes), y habrá un desperdicio de 48 bytes por bloque. El próximo bloque no comienza inmediatamente a continuación del anterior, sino que lo hace coincidiendo con el inicio del próximo sector (un bloque no puede estar "a caballo" de dos sectores).

18.1.2. Organización y acceso a archivos.

Toda aplicación que trabaje con datos en archivos debe definir como va a ser la organización de dichos datos y como va a ser el acceso a los mismos.

El hecho de tener que establecer estos aspectos esta relacionado con 2 factores principales: en primer lugar, la necesidad de administrar del mejor modo posible el recurso físico (medio de almacenamiento de datos); en segundo termino, la necesidad de trabajar con los datos en forma ordenada.

Con respecto a la administración del recurso físico, se debe considerar que el mismo no es ilimitado, y que el acceso a los datos consume tiempo; entonces, se trata de optimizar el espacio físico disponible y de minimizar el tiempo de acceso.

En cuanto al trabajo en forma ordenada, se debe buscar la posibilidad de disponer de los datos de la manera mas sencilla posible, tratando de hacer la menor cantidad posible de operaciones.

18.1.2.1. Organización.

La organización de un archivo se refiere a la disposición de los registros lógicos en el disco.

Existen 2 principales organizaciones de archivos: secuencial y al azar.

Los archivos con organización secuencial tienen sus registros lógicos almacenados secuencialmente, es decir uno a continuación del otro, y no existe ninguna manera de identificarlos individualmente. La organización secuencial es la mas sencilla de todas y la mas fácil de operar, sin embargo presenta grandes limitaciones en cuanto a sus posibilidades pues, por ejemplo, no permite la inserción de registros lógicos en el medio del archivo (sólo permite el agregado append de registros a partir del final del archivo), ni permite la baja física (delete) de registros.

Los archivos con organización al azar tienen sus registros lógicos almacenados aleatoriamente en el disco, es decir que, a priori, no siguen un orden determinado, pero pueden si ser identificados.

Existen, además, 2 tipos de organizaciones al azar: por número de registro lógico y por índice.

La primera, conocida como organización relativa o random, consiste en identificar, en forma implícita, a cada registro lógico con un número de registro relativo al comienzo del archivo. En este caso, aunque no exista un determinado registro lógico, si existe el espacio físico en disco correspondiente al mismo (se reserva el espacio). Un ejemplo de archivo organizado en forma relativa puede ser el de Socios de una Cooperativa, donde cada registro tiene información de un único socio y cada socio esta identificado con un número único, pudiendo reutilizarse los números de socios renunciando para nuevos socios. En este caso tendremos, en determinado momento, que habrá registros lógicos sin datos (de socios renunciando) entre registros lógicos con datos (socios activos), los que se irán completando a medida que ingresen nuevos socios. Desde el punto de vista de la ocupación en disco, los registros lógicos sin datos ocuparan, sin embargo, espacio en el mismo. La segunda organización al azar mencionada (por índice), se denomina indexada, y consiste en identificar, explícitamente, a cada registro lógico del archivo con una clave alfanumérica. La organización indexada se independiza de la ocupación en disco a costa de una mayor complejidad del sistema operativo.

A diferencia de la organización relativa, la organización indexada no reserva espacio físico para registros lógicos sin datos, sino que va ocupando espacio a medida que lo va necesitando. Esto es posible dado que los archivos indexados incluyen, además del arrea de datos, un arrea de claves donde se especifica la dirección física para cada una de las claves. Un ejemplo de archivo indexado puede ser el de Alumnos de la Universidad, donde cada registro corresponde a un alumno, y la clave esta compuesta por los campos APELLIDO, NOMBRE y CARRERA. De este modo, cuando un alumno egresa, el registro lógico correspondiente es dado de baja, liberándose el espacio físico que ocupaba en disco. Las Bases de Datos también son ejemplo de archivos indexados.

Para definir la organización de un archivo se debe tener en cuenta el uso que se le va a dar al mismo; por ejemplo, si el archivo debe admitir la posibilidad de insertar registros lógicos se deber optar por organización relativa o indexada, desechando la organización secuencial. Si, en cambio, el archivo se usada para almacenar información que sólo será tratada para listar, probablemente sea conveniente optar por la organización secuencial. Otro factor a tener en cuenta en la definición de la organización de un archivo es el espacio en disco que el mismo va a ocupar: los archivos relativos no liberan los registros lógicos no utilizados y los archivos indexados tienen, además del arrea de datos, un arrea de claves o índices que ocupa un espacio adicional. En general se puede establecer que, para una misma aplicación, un archivo relativo o un archivo indexado ocupan mas espacio que un archivo secuencial.

18.1.2.2. Acceso.

El acceso a un archivo se refiere al modo en que se accede a los registros lógicos del mismo.

Existen 2 tipos de acceso: secuencial y al azar.

El acceso secuencial significa acceder a los registros lógicos de un archivo en forma secuencial, es decir uno a continuación del otro sin saltarse ninguno. Es decir que para acceder al registro n es necesario acceder a los (n-1) registros lógicos anteriores. El acceso al azar es mucho mas flexible, ya que permite acceder a un registro en particular (a través de suministrar su identificación), sin importar en que parte del disco esta ubicado.

Como se puede deducir en una primera instancia, los modos de acceso están relacionados con el tipo de organización del archivos. Así, un archivo secuencial sólo podrá ser accedido en forma secuencial, mientras que un archivo con organización al azar podrá ser accedido tanto en forma secuencial como al azar.

18.1.3. Almacenamiento de archivos.

Los archivos se almacenan, fundamentalmente, en dispositivos magnéticos tales como discos y cintas. Existen también otros tipos de medios de almacenamiento de datos menos utilizados, tales como cintas de papel, tarjetas perforadas, microfilmaciones, etc.

Todo sistema operativo suministra un conjunto de funciones de interfaz entre los dispositivos físicos, donde residen los archivos, y las aplicaciones, que los utilizan. Esta parte del sistema operativo se denomina Administrador de Archivos y es la que se encarga de organizar físicamente las estructuras lógicas de datos. Existen muchas estrategias distintas de almacenar datos en los dispositivos, dependiendo esto del tipo de dispositivo y de la definición del sistema de computación (hacia donde está orientado su diseño).

Hay, básicamente, 2 maneras diferentes de almacenar archivos: en forma secuencial y en forma particionada. El almacenamiento secuencial consiste en tener un archivo en un sólo trozo, y se usa particularmente en los dispositivos secuenciales (cinta magnética, tarjetas perforadas, etc.). El almacenamiento particionado consiste en la posibilidad de tener un archivo ocupando varias porciones del dispositivo, y se usa en aquellos dispositivos que permiten un acceso aleatorio (discos magnéticos, discos ópticos, etc.).

La administración de archivos que se almacenan secuencialmente es bastante sencilla, dado que el sistema operativo debería llevar un registro (directorío) de archivos existentes donde sólo conste, para cada uno, la dirección inicial y la longitud. El borrado de un archivo consistiría solamente en eliminar la entrada correspondiente en el directorío. Una desventaja de este tipo de administración de archivos es que para crear un archivo nuevo es necesario suministrar la máxima longitud que el mismo puede alcanzar (si luego el archivo necesita extenderse más allá de la longitud definida, se debe crear un nuevo archivo de mayor tamaño). En general, esta administración de archivos es bastante ineficiente en el aprovechamiento del espacio físico, y por este motivo es que se dejó de utilizar en discos. La administración de archivos que se almacenan en forma particionada requiere del manejo de tablas que indiquen las distintas partes del disco que ocupa un archivo. Este método permite un mejor aprovechamiento del espacio físico, aunque a costa de una mayor complejidad del sistema operativo.

18.2. ARCHIVOS BAJO MS-DOS

El DOS suministra el soporte necesario para el manejo de archivos, gestionando el espacio físico necesario en disco y reportando errores.

Los distintos niveles de E/S en disco son los siguientes:

Aplicación : Solicita al DOS una operación de E/S sobre un archivo en disco.

DOS : Envía al BIOS el requerimiento de E/S direccionando a pistas y sectores.

BIOS : Emite los mandatos apropiados al adaptador de disco.

Adaptador : Genera las señales de hardware para activar la unidad de disco.

Unidad de disco: Produce la E/S física.

El DOS mantiene una lista de todos los archivos sobre el disco, llamada directorío. Como vimos en el capítulo anterior, el directorío está ubicado en una posición determinada sobre el disco. Usando la información del directorío, DOS convierte los requerimientos del usuario para acceder a datos en archivos en solicitudes de acceso a ubicaciones específicas en disco. Luego toma el control el BIOS para desencadenar todo el proceso de la E/S. La aplicación o programa de usuario, en el más alto nivel, puede manejar los datos de los archivos sin necesidad de saber de qué manera ellos están físicamente ubicados; cada pedido de E/S va pasando a los niveles más bajos, acercándose en cada paso a las especificaciones del hardware.

18.2.1. Registros y bloques.

El DOS trabaja con registros de longitud fija, es decir que todos los registros de un archivo tienen la misma cantidad de bytes. Los registros se agrupan en bloques, que pueden contener hasta 128 registros (0 a 127). Para un mismo archivo, el número de registros por bloque es constante (factor de bloque).

Para acceder a un registro, el DOS necesita dos parámetros: el número de bloque y el número de registro dentro del bloque. Otro modo de obtener estos parámetros es a través del número de registro relativo al comienzo del archivo, el factor de bloque y el tamaño de registro.

18.2.2. Áreas referenciadas por el MS-DOS.

Las funciones de E/S del DOS referencian dos áreas de datos en memoria: el bloque de control de archivo (File Control Block -- FCB) y el área de transferencia a disco (Disk Transfer Area -- DTA).

18.2.2.1. Bloque de control de archivo (FCB).

El FCB es un arrea que contiene información relativa a un archivo, y es usada por el DOS en cada operación que realiza para tomar los datos necesarios y para dejar algunos resultados. Por cada archivo abierto existe un FCB, que se ubica, físicamente, dentro del programa que utiliza el archivo. El FCB tiene 37 bytes de tamaño, y su estructura es la siguiente:

DESP.	LONG.	DESCRIPCION
0	1	Unidad (1 a 4, que se corresponde con A hasta D)
1	8	Nombre del archivo
9	3	Extensión del archivo
12	2	Bloque actual posicionado
14	2	Tamaño del registro lógico (en bytes)
16	4	Tamaño total del archivo (en bytes)
20	2	Fecha de la última modificación
22	10	Reservado para uso del DOS
32	1	Registro actual dentro del bloque (0 a 127)
33	4	Registro relativo al comienzo del archivo

El campo de tamaño del archivo aparece con los bytes en orden inverso.

Resulta útil describir el FCB a través de la directiva STRUC, para poder referenciar fácilmente a los campos. Esto se utiliza a menudo en los programas en assembler que efectúan E/S sobre archivos:

FCBS STRUC

UNIDAD	DB	?	;código de unidad
NOMBRE	DB	8 DUP(?)	;nombre
EXTEN	DB	3 DUP(?)	;extensión
BLOQUE	DW	?	;número de bloque actual
LONGREG	DW	?	;longitud de registro
TAMANIO	DW	2 DUP(?)	;tamaño del archivo
FECHA	DW	?	;fecha de última modificación
DT	?		;10 bytes reservados para DOS
REGACT	DB	?	;número del registro actual
REGREL	DB	2 DUP(?)	;número del registro relativo

FCBS ENDS

El uso, por parte del DOS, de la estructura de FCB, hace que sus funciones sean compatibles con el sistema operativo CP/M.

18.2.2.2. Area de transferencia a disco (DTA).

El DTA es un buffer al que accede el DOS en cada función de E/S en disco. Cuando se lee de disco, los datos leídos se almacenan en el DTA; de la misma manera, cuando se graba en disco, se extraen los datos del DTA. El DTA es único en cada programa, y se ubica, por default, a partir de la dirección 80H del PSP, con lo que su longitud es de 128 bytes (tamaño de PSP-desplazamiento inicial de DTA).

El DTA debe ser lo suficientemente grande para poder albergar el bloque de datos mas grande que utilice el programa en lectura o escritura y, en general, las operaciones de E/S involucran muchos mas que 128.

Entonces, antes de efectuar una operación de E/S, se le debe pasar al DOS la dirección de una DTA, la que puede ubicarse en cualquier parte de la memoria. Con esto se evita la destrucción de los datos del PSP que están a partir del desplazamiento 80H (datos de parámetros).

18.2.3. Acceso a los registros de un archivo.

Bajo DOS existen 3 modalidades de acceso a los registros de un archivo: secuencial, al azar y al azar por bloques.

18.2.3.1. Acceso secuencial.

Cuando se ejecutan las funciones de lectura/escritura secuencial, el DOS accede al registro especificado por los campos bloque actual y registro actual dentro del FCB del archivo, actualizando a continuación en forma automática estos campos para que apunten al próximo registro del archivo.

Para trabajar secuencialmente, se debe especificar en el FCB el tamaño de registro e inicializar con cero los campos de bloque actual y registro actual. De este modo se garantiza que el proceso comenzara desde el principio del archivo. Luego, la actualización de los campos, la hace DOS en forma automática.

18.2.3.2. Acceso al azar.

Este sistema permite acceder a cualquier registro del archivo. Para ello se utilizan las funciones del DOS de lectura/escritura al azar. Igual que para el acceso secuencial, se debe especificar el tamaño de registro lógico en el campo correspondiente del FCB. Para acceder al azar a un registro cualquiera, se debe indicar el número de registro relativo al comienzo del archivo en el campo correspondiente del FCB. El primer registro lógico del archivo se identifica con el número cero. Una vez efectuada esta especificación, el DOS calcula el número de bloque actual y el número de registro actual y luego efectúa la operación solicitada. Estos campos no se actualizan, por lo que quedan apuntando al mismo registro sobre el que se hizo la operación de E/S.

18.2.3.3. Acceso al azar por bloques.

Este sistema es similar al anterior, pero tiene la ventaja que se puede transferir mas de un registro a la vez. Para hacer esto, se debe especificar un número de registro relativo en el FCB y la cantidad de registros a transferir en el registro CX (contador).

18.2.4. Procedimiento estándar con un archivo bajo MS-DOS.

A manera de resumen, podemos indicar que los pasos para trabajar con un archivo bajo DOS son los siguientes:

- Cargar código de unidad y nombre de archivo sobre el FCB.
- Abrir archivo con DS:DX apuntando al FCB.
Con esta operación, el DOS localiza el archivo en el disco y carga información sobre el FCB (tamaño del archivo, fecha y el campo reservado).
- Leer y/o grabar el archivo mediante las funciones del DOS, utilizando cualquiera de los accesos vistos.
- Cerrar archivo con DS:DX apuntando al FCB.

18.3. FILE HANDLE

Así como el DOS es compatible con el sistema operativo CP/M a través de las funciones de E/S mediante el manejo del FCB, también tiene funciones de E/S que lo hacen compatible al sistema operativo UNIX. Estas funciones se introdujeron a partir de la versión 2.0 del MS-DOS, y se denominan funciones de manejadores de archivos o File Handle Functions.

El uso de las handle functions presenta algunas ventajas con respecto al FCB:

- El área de control de cada archivo es mantenida por MS-DOS en su propia zona de memoria; las FCB, en cambio, se ubican en la memoria del programa que usa los archivos.
- Soportan en forma completa y eficiente la estructura jerárquica de los directorios.
- Soportan archivos con registros de cualquier longitud, y pueden acceder (leer y escribir) bytes en cualquier lugar dentro del archivo.

Con las handle functions, cada archivo se identifica con un número de 16 bits llamado File Handle. El file handle es un apuntador a un área propia del MS-DOS que este utiliza para guardar información referente al archivo. A cada archivo que se abre, MS-DOS le asigna un file handle; cada invocación al MS-DOS para acceder a un archivo debe ir acompañada del file handle correspondiente.

18.3.1. Acceso a archivos con las file handle functions.

Cuando se desea crear o abrir un archivo se le debe pasar al MS-DOS el nombre del archivo en una cadena ASCIIZ (una cadena ASCIIZ es un string de caracteres ASCII seguido de un byte conteniendo el valor cero). La cadena ASCIIZ contiene la unidad, el path ("trayectoria" de directorios) y el nombre del archivo.

Ante el requerimiento, MS-DOS devuelve un file handle o un código de error.

El file handle debe ser conservado por el programa, ya que todas las operaciones sobre el archivo no se invocaran por nombre sino a través del file handle. La cantidad de archivos que pueden estar abiertos a la vez usando file handle es de 20. Sin embargo, MS-DOS limita la cantidad de archivos abiertos al número especificado en el parámetro FILES= del archivo de configuración CONFIG.SYS, cuyo valor default es de 8 pero puede llegar a un máximo de 255. Cuando un programa comienza su ejecución recibe 5 archivos ya abiertos, identificados con los file handle 0 a 4.

Estos archivos son los siguientes:

- 0: Consola para lectura (Teclado). CON
- 1: Consola para escritura (Pantalla). CON
- 2: Consola para errores (Pantalla). CON

3: Dispositivo auxiliar (Port Serial). AUX

4: Impresora (Port Paralelo). PRN

Los primeros tres se conocen como los dispositivos estándar de entrada (standar input), de salida (standar output) y de errores (standar error); este concepto es también utilizado por UNIX. De este modo, si, por ejemplo, se desea mostrar algo por pantalla, se pueden utilizar tanto las funciones específicas del BIOS o del DOS para salida por pantalla como las funciones del DOS para escribir sobre el archivo identificado con el file handle 1, que corresponde al dispositivo estándar de salida (es decir, la pantalla). A la funciones de abrir o crear archivo, MS-DOS devuelve el primer file handle disponible (la primera vez que se haga una de estas operaciones, el programa recibir el file handle 5, pues del 0 al 4 están ocupados por los archivos abiertos por default). Una vez que el archivo fue abierto, se puede empezar a leerlo y/o grabarlo; el sistema mantiene constantemente un puntero (Puntero de lectura/escritura, Input/Output Pointer), que apunta al siguiente byte a leer o grabar del archivo. Inicialmente, dicho puntero esta en cero, es decir que apunta al primer byte del archivo. Un archivo puede ser abierto sólo para lectura, sólo para escritura o para lectura y escritura. La operación de crear un archivo lo habilita sólo para escritura. Una operación importante que se puede hacer sobre un archivo abierto es de mover el puntero de lectura/escritura. Este puede moverse cualquier cantidad de bytes respecto a la posición actual, respecto al comienzo del archivo o respecto al fin del mismo; el sentido del movimiento puede ser tanto hacia adelante como hacia atrás. Esto permite acceder a la información del archivo en forma al azar.

18.4. ALMACENAMIENTO DE ARCHIVOS BAJO MS-DOS

Un disco formateado para trabajar bajo DOS se divide en unidades de alocución o clusters, siendo una unidad de alocución la mínima cantidad de espacio en disco que puede ser ocupada por datos. En el arrea de disco reservada para el sistema operativo, existe el arrea de FAT o tabla de alocución de archivos, que contiene información acerca del espacio en disco en el arrea de datos que ocupa cada archivo. Cada entrada de la FAT corresponde a cada unidad de alocución del arrea de datos del disco, y contiene un dato que indica si la unidad de alocución esta libre, si esta defectuosa (es decir, si incluye sectores malos, Bad Sectors), o si esta asignada a un archivo. En el caso de estar asignada a un archivo, el dato de la entrada indica si es la última unidad de alocución del archivo o el número de la siguiente unidad de alocución que ocupa el archivo. Entonces, ya podemos ir viendo un primer esquema de como es el almacenamiento de un archivo en disco:

- Un archivo estará, físicamente, ocupando unidades de alocución.
- Las unidades de alocución ocupadas por un archivo no necesariamente deben estar contiguas, dado que el uso de la FAT permite que, a través del esquema de encadenamientos de unidades de alocución, un archivo pueda ocupar diversas porciones del disco según haya unidades de alocución disponibles.

Este esquema es eficaz en el sentido de un mejor aprovechamiento del espacio del disco y de una mayor ductilidad para el manejo de archivos que van "creciendo" en tamaño; sin embargo, presenta la desventaja de no poder alocar mas de un archivo en la misma unidad de alocución, por lo que si una unidad de alocución no es totalmente usada por el archivo, el resto del espacio de la misma se desperdicia. El nivel de desperdicio esta dado por el tamaño de las unidades de alocución; para los discos flexibles de doble cara y 9 sectores por pista, las unidades de alocución ocupan 1.024 bytes (2 sectores), mientras que para el disco rígido de 10 Mbytes de capacidad, las unidades de alocución ocupan 2.048 bytes (4 sectores). Cada entrada de la FAT tiene 12 bits de tamaño, es decir que por cada 3 bytes hay dos entradas. Las dos primeras entradas (los 3 primeros bytes) son especiales y corresponden a las unidades de alocución teóricas 0 y 1, que no existen físicamente. La primera unidad de alocución real, identificada con el número cero, corresponde a la tercera entrada de la FAT (número 2), y físicamente esta ubicada en los sectores lógicos 12 y 13 (para disco flexible de doble cara y 9 sectores por pista). A partir de la tercera entrada en la FAT, el contenido de cada entrada puede ser:

- 000H: indica unidad de alocución disponible
- FFFH: Indica última unidad de alocución de un archivo
- FF7H: Indica unidad de alocución defectuosa
- XXXH: Indica el número de la próxima unidad de alocución de un archivo

La primera vez que se asigna espacio a un archivo, se recorre la FAT desde el principio, asignándosele la primera unidad de alocución disponible. Cuando se añade información a un archivo existente, se obtiene la primera unidad de alocución disponible y se encadena con la última entrada de la FAT que ocupaba el archivo. Cuando se borra un archivo, además de poner el código E5H en la primer posición de la

correspondiente entrada en el directorio, se ponen en cero todas las entradas de la FAT correspondientes a las unidades de alocución que están asignadas al archivo.

El DOS utiliza una copia de la FAT, la que esta inmediatamente a continuación de la FAT original, que se actualiza cada vez que se actualiza la original.

18.4.1. Algoritmo para interpretar cada entrada de la FAT.

La interpretación de cada entrada de la FAT significa hallar, a partir del número de unidad de alocución conocido, el contenido de su correspondiente entrada en la FAT, de modo de saber si apunta a otra unidad de alocución, si esta disponible, si corresponde a una unidad de alocución defectuosa o si es la ultima de un archivo. Para esto hay que considerar que las entradas tienen 12 bits y que el assembler puede manejar directamente datos de 8 y/o de 16 bits, por lo que se accede a cantidades de 16 bits y luego se debe descartar algún nibble. El siguiente es el algoritmo para hallar la entrada en la FAT correspondiente a la unidad de alocución u.a..

Para una mejor comprensión, veremos paralelamente un ejemplo:

a) Obtención del desplazamiento dentro de la FAT

$$u.a. \times 3$$

parte entera $\text{ÄÄÄÄÄÄÄÄ} = \text{desplazamiento dentro de la FAT}$ 2

Ejemplo:

$$u.a. = 155$$

$$155 \times 3$$

parte entera $\text{ÄÄÄÄÄÄÄÄ} = 232$

2

b) Obtención del contenido de la entrada

Se debe tomar la palabra que se halla a partir de la dirección calculada en a), e invertir los bytes. Luego, si u.a. es par, se debe desechar el dígito hexadecimal de la izquierda y si u.a. es impar se debe desechar el dígito hexadecimal de la derecha.

Ejemplo: A partir de la dirección 232 de la FAT esta la palabra CF09H. Al invertir los bytes queda 09CFH. Como u.a.=155 es impar, se debe desechar el dígito hexadecimal de la derecha. Por lo tanto, el contenido de la entrada de la FAT correspondiente a la unidad de alocución 155 es 09CH. Su significado es que el archivo continúa en la unidad de alocución 09CH, es decir 156.

18.4.2. Relación entre unidades de alocución y sectores.

Así como se obtuvo el contenido de la entrada en la FAT correspondiente a una unidad de alocución, también podemos calcular la dirección en disco (es decir, el número del primer sector lógico) donde comienza cada unidad de alocución. La expresión para efectuar este calculo es la siguiente:

$$s = (u.a. - 2) * T + I$$

donde:

S es el número del primer sector lógico en disco de la unidad de alocución.

u.a. es el número de unidad de alocución.

2 es la cantidad de entradas en la FAT no usadas para unidades de alocución.

T es el tamaño (en sectores) de una unidad de alocución.

I es el número del sector lógico inicial del arrea de datos.

Todos los parámetros dependen del tipo de disco, dado que los mismos varían de uno a otro. De este modo, la respectivas expresiones para un disco flexible de doble cara y 9 sectores por pista y para un disco rígido de 10 Mbytes ser n:

$$\text{disco flexible: } s = (u.a. - 2) * 2 + 12$$

$$\text{disco rígido : } s = (u.a. - 2) * 8 + 49$$

Ejemplo:

Calcular el número del primer sector lógico de la unidad de alocución 155 en un disco flexible de 2 caras y 9 sectores por pista:

$$\text{primer sector} = (155 - 2) * 2 + 12 = 318$$

La unidad de alocución 155 ocupa los sectores lógicos 318 y 318.

18.5. FUNCIONES DEL DOS PARA EL MANEJO DE ARCHIVOS

El DOS provee una importante cantidad de funciones para el manejo de archivos, todas a través de la interrupción 21H. Estas funciones podemos clasificarlas en 3 grupos:

1. Funciones generales
2. Funciones con FCB
3. Funciones con file handle

18.5.1. Funciones generales.

Este conjunto de funciones brindan una serie de servicios independientemente de si se va a trabajar con FCB o con file handle. Son las siguientes:

AH=0DH --- Restaurar unidad de disco

Elimina los buffers de todos los archivos. Los archivos que no hayan sido cerrados no quedaran bien registrados en el directorio.

AH=0EH --- Seleccionar unidad default

Permite seleccionar la unidad de disco por default. Una alternativa a esta función es la interrupción 11H del BIOS.

Entrada: DL =Número de unidad (0=A, 1=B, etc.)

Salida: AL =Cantidad de unidades lógicas en el sistema

AH=19H --- Obtener unidad default

Esta función es complementaria a la anterior; permite saber cual es la unidad de disco default.

Salida: AL =Número de unidad default (0=A, 1=B, etc.)

AH=1AH --- Establecer DTA

Asigna como DTA un area de memoria de 128 bytes de tamaño como mínimo.

Entrada: DS:DX =Dirección del DTA

AH=2FH --- Obtener DTA

Esta función es la complementaria a la anterior; permite conocer cual es la dirección de memoria del area asignada como DTA.

Salida: DS:DX =Dirección del DTA

18.5.2. Funciones con FCB.

Estas funciones permiten efectuar operaciones sobre archivos y directorios a través del manejo de FCB. Se pueden clasificar en:

1. De búsqueda en directorios
2. De archivos en general
3. De E/S de archivos

18.5.2.1. De búsqueda en directorios.

Estas funciones permiten encontrar archivos dentro del directorio a partir del nombre que figura en el FCB (el nombre del argumento de búsqueda puede contener los caracteres globales ? y *). Si existe una entrada del directorio cuyo nombre se iguala con el archivo buscado, el MS-DOS devuelve la entrada completa del archivo en el directorio en el DTA. Existen 2 funciones de búsqueda en directorios: buscar la primera entrada y buscar la siguiente entrada. Con la primera función, se inicializa el DTA y comienza la búsqueda en el directorio hasta encontrar el primer archivo; la segunda función busca a partir del último archivo encontrado hasta encontrar el próximo o indicar que no encontró nada.

AH=11H --- Buscar la primera entrada

Busca un archivo a partir de la primera entrada en el directorio. El nombre a buscar puede contener los caracteres globales ? y *.

Entrada: DS:DX =Dirección de FCB sin abrir

Salida: Si encontró un archivo

AL=00H

DTA(primer byte)=Unidad actual (1=A, 2=B, etc.)

DTA(byte 2 a 33)=Entrada completa del directorio

Si no encontró ningún archivo

AL=FFH

AH=12H --- Buscar la siguiente entrada

Es similar a la función anterior, pero la búsqueda la hace a partir de la última entrada encontrada.

Entrada: DS:DX =Dirección de FCB sin abrir

Salida: Si encontró un archivo

AL=00H

DTA(primer byte)=Unidad actual (1=A, 2=B, etc.)

DTA(byte 2 a 33)=Entrada completa del directorio

Si no encontró ningún archivo

AL=FFH

18.5.2.2. De archivos en general.

Estas funciones brindan una serie de facilidades para el mantenimiento de archivos. Son las siguientes:

AH=13H --- Borrar archivo (comandos DELETE y ERASE)

Elimina del directorio el archivo indicado en la FCB, y libera las unidades de alocución ocupadas por el mismo.

Entrada: DS:DX =Dirección FCB sin abrir

Salida: AL=00H si archivo encontrado.

AL=FFH si archivo no encontrado

AH=17H --- Renombrar archivo (comando RENAME)

Cambia el nombre de todos los archivos en sus correspondiente entradas del directorio. Cada aparición del primer nombre en el directorio activo se reemplaza por el segundo nombre (ambos nombres pueden incluir los caracteres globales ? y *).

Entrada: DS:DX =Dirección FCB FCB+17=segundo nombre

Salida: AL=00H si algún archivo encontrado

AL=FFH si ningún archivo encontrado o si se intentó renombrar con un nombre ya existente.

AH=23H --- Obtener tamaño del archivo

Devuelve el tamaño de un archivo, en registros, a partir del tamaño de registro.

Entrada: DS:DX =Dirección FCB sin abrir

FCB(longitud registro)=tamaño de registro en bytes

Salida: Si encontró archivo

AL=00H

FCB(registro relativo)=cantidad de registros

Si no encontró archivo

AL=FFH

18.5.2.3. De E/S de archivos.

Estas funciones manejan todas las operaciones de E/S sobre los archivos. Son las siguientes:

AH=0FH --- Abrir archivo existente (OPEN)

Abre un archivo existente, asociándole su correspondiente FCB.

Entrada: DS:DX =Dirección FCB sin abrir

Salida: AL=00H si encontró archivo

AL=FFH si no encontró archivo

Una vez abierto el archivo, el FCB se completa de la siguiente manera:

FCB(unidad)=Unidad actual (1=A, 2=B, etc.)

FCB(bloque actual)=0

FCB(longitud registro)=80H

FCB(tamaño archivo)=del directorio

FCB(fecha)=del directorio

Los campos FCB(registro actual) y FCB(registro relativo) deberán ser completados manualmente tras la apertura del archivo. Asimismo, se debe especificar el campo FCB(longitud registro) si es que el valor default de 80H es insuficiente.

AH=10H --- Cerrar archivo (CLOSE)

Esta función es complementaria a la anterior; cierra un archivo, desasignando el FCB que tenía asociado. La operación de cierre de archivo hace que se actualice la correspondiente entrada en el directorio.

Entrada: DS:DX =Dirección FCB abierta

Salida: AL=00H si encontró archivo
AL=FFH si no encontró archivo

AH=14H --- Lectura secuencial (READ)

Efectúa la lectura del registro apuntado por los campos bloque actual y registro actual del FCB, y lo almacena en el DTA. Tras la lectura, se incrementa la dirección del registro.

Entrada: DS:DX =Dirección FCB abierta

Salida: AL=00H operación sin error
AL=01H no hay datos en el registro (señal de EOF)
AL=02H no hay espacio en el segmento para el registro
AL=03H registro leído parcialmente

AH=15H --- Grabación secuencial (WRITE)

Efectúa la grabación desde el DTA del registro apuntado por los campos bloque actual y registro actual del FCB. Tras la grabación se incrementa la dirección del registro.

Entrada: DS:DX =Dirección FCB abierta

Salida: AL=00H operación sin error
AL=01H disco lleno
AL=02H no hay espacio en el segmento para el registro

AH=16H --- Crear o truncar archivo (CREATE)

Crea un nuevo archivo, asignándole una entrada en el directorio y un FCB. Si el archivo ya existe, entonces se abre pero con longitud 0.

Entrada: DS:DX=Dirección FCB abrir

Salida: AL=00H si encontró archivo
AL=FFH si no encontró archivo

AH=21H --- Lectura al azar (READ)

Es similar a la operación de lectura secuencial, pero con la diferencia de que el MS-DOS calcula previamente el valor de los campos bloque actual y registro actual del FCB a partir del valor del campo registro relativo del FCB. Otra diferencia con la lectura secuencial es que, tras la lectura, no se incrementa el apuntador de registro.

Entrada: DS:DX =Dirección FCB abierta

Salida: AL=00H operación sin error
AL=01H no hay mas datos disponibles (fin del disco)
AL=02H no hay espacio en el segmento para el registro
AL=03H registro leído parcialmente

AH=22H --- Grabación al azar (WRITE)

Es similar a la operación de lectura al azar: se graba desde el DTA en el registro apuntado por el campo registro relativo del FCB.

Entrada: DS:DX =Dirección FCB abierta

Salida: AL=00H operación sin error
AL=01H disco lleno
AL=02H no hay espacio en el segmento para el registro

AH=24H --- Especificar registro al azar

Esta operación es la previa a las de lectura o grabación al azar, pues hace que el MS-DOS calcule el número de registro relativo a partir de los datos de los campos bloque actual y registro actual del FCB. El valor calculado es almacenado en el campo registro relativo del FCB.

Entrada: DS:DX =Dirección FCB abierta

Salida: Si archivo encontrado
AL=00H
FCB(registro relativo)=valor

Si archivo no encontrado

AL=FFH

AH=27H --- Lectura al azar por bloque (READ)

Es idéntica a la operación de lectura al azar, excepto que se leen tantos registros como la cantidad indicada en CX.

Entrada: DS:DX =Dirección FCB abierta

CX =Cantidad de registros a leer

Salida: CX =Cantidad de registros leídos

AL =00H operación sin error

AL =01H fin de archivo

AL =02H no hay espacio en el segmento para los registros

AL =03H bloque leído parcialmente

AH=28H --- Grabación al azar por bloque (WRITE)

Es idéntica a la operación de grabación al azar, excepto que se graban tantos registros como la cantidad indicada en CX.

Entrada: DS:DX=Dirección FCB abierta

CX=Cantidad de registros a grabar

Salida: CX=Cantidad de registros grabados

AL=00H operación sin error

AL=01H disco lleno (no se grabó ningún registro)

18.5.3. Funciones con file handle.

Estas funciones permiten efectuar operaciones sobre archivos y directorios a través del manejo de file handle.

La mayoría de estas funciones devuelve un código de error en el registro AX que permite diagnosticar si hubo algún problema y cual fue. Los distintos códigos de error y su significado son los siguientes:

CODIGO SIGNIFICADO

01H	Número de función inválido
02H	Archivo no encontrado
03H	Trayectoria (path) no encontrada
04H	Demasiados archivos abiertos (no hay file handle disponible)
05H	Acceso denegado
06H	File handle inválido
07H	Bloques de control de memoria destruidos
08H	Memoria insuficiente
09H	Dirección de bloque de memoria inválida
0AH	Contexto inválido
0BH	Formato inválido
0CH	Código de acceso inválido (no hay archivos que hagan match)
0DH	Datos inválidos
0FH	Especificación de unidad inválida
10H	Intento de borrar directorio actual
11H	No es el mismo dispositivo
12H	No hay mas archivos
20H	Violación de partición
21H	Violación de permiso
22H	Cambio de disco inválido
23H	FCB no disponible
50H	Archivo ya existe
51H	Reservado
52H	No se puede hacer

Las funciones con file handle se pueden clasificar en:

1. De directorios
2. De búsqueda en directorios
3. De archivos en general

4. De E/S de archivos

18.5.3.1. De directorios.

Son las siguientes:

AH=39H --- Crear subdirectorio (comando MKDIR)

Permite crear un nuevo subdirectorio a partir de lo especificado en la cadena ASCIIIZ, asignándole los recursos correspondientes.

Entrada: DS:DX =Dirección cadena ASCIIIZ que contiene la unidad y trayectoria del nuevo subdirectorio

Salida: Si operación sin error, CF=0

Si operación con error, CF=1

AX =código de error (03,05)

AH=3AH --- Borrar subdirectorio (comando RMDIR)

Es la operación inversa a la anterior; elimina un subdirectorio ya existente, desafectando la entrada en el directorio.

Entrada: DS:DX =Dirección cadena ASCIIIZ que contiene la unidad y trayectoria del subdirectorio a borrar

Salida: Si operación sin error, CF=0

Si operación con error, CF=1

AX =código de error (03,05,10)

AH=3BH --- Cambiar directorio activo (comando CHDIR)

Cambia el directorio activo por el indicado en la cadena ASCIIIZ.

Entrada: DS:DX =Dirección cadena ASCIIIZ que contiene la unidad y trayectoria del nuevo directorio activo

Salida: Si operación sin error, CF=0

Si operación con error, CF=1

AX =código de error (03)

AH=47H --- Obtener directorio activo

Esta función devuelve en un arrea de memoria el nombre de la trayectoria correspondiente al actual subdirectorio activo para una unidad especificada.

Entrada: DL =Número de unidad (0=default, 1=A, 2=B, etc.)

DS:SI =Dirección de un buffer de 64 bytes donde se va a almacenar la trayectoria completa del actual subdirectorio activo.

Salida: Si operación sin error, CF=0

Si operación con error, CF=1

AX =código de error (0F)

En el buffer no figura la unidad ni tampoco la barra invertida (\) que indica el comienzo de la trayectoria. La trayectoria devuelta en el buffer finaliza con un byte con el valor 00H, con lo que queda armada una cadena ASCIIIZ.

18.5.3.2. De búsqueda en directorios.

Son similares a las funciones 11H y 12H con FCB. La diferencia es que el nombre de búsqueda esta incluido en una cadena ASCIIIZ y que se incluye un byte de atributo.

Son las siguientes:

AH=4EH --- Buscar primer nombre de archivo

Inicializa el DTA y comienza la búsqueda en el directorio hasta encontrar el primer archivo.

Entrada: CX =Atributo de los archivos a buscar

DS:DX =Dirección de la cadena ASCIIIZ que contiene el nombre a buscar

Salida: Si operación sin error, CF=0

Si operación con error, CF=1

AX =código de error (02,12)

El nombre de búsqueda debe incluir la especificación de unidad y la trayectoria.

Una vez encontrado un archivo, el DTA activo quedar con la siguiente estructura:

21 bytes - reservados para el MS-DOS

1 byte - atributo del archivo

2 bytes - hora del archivo

2 bytes - fecha del archivo

2 bytes - tamaño del archivo (palabra inferior)

2 bytes - tamaño del archivo (palabra superior)

13 bytes - nombre y extensión del archivo encontrado, seguido de un byte con 00H

AH=4FH --- Buscar siguiente nombre de archivo

Busca a partir del último archivo encontrado hasta hallar el próximo.

Entrada: DTA =Información del último archivo encontrado

Salida: Si operación sin error, CF=0

Si operación con error, CF=1

AX=código de error (12)

El DTA queda con la misma estructura que en la función anterior, pero para el nuevo archivo.

18.5.3.3. De archivos en general.

Estas funciones permiten efectuar una serie de operaciones generales sobre archivos, tales como crear, borrar, renombrar, etc. Son las siguientes:

AH=41H --- Borrar archivo (comandos DELETE y ERASE)

Elimina un archivo del disco, liberando su entrada en el directorio y desasignando las unidades de alocución que ocupa.

Entrada: DS:DX=Dirección cadena ASCIIZ conteniendo la unidad, trayectoria y nombre del archivo a borrar

Salida: Si operación sin error, CF=0

Si operación con error, CF=1

X=código de error (02,05)

AH=43H --- Obtener/fijar atributo de archivo

Permite leer y/o fijar el atributo de un archivo.

Entrada: DS:DX =Dirección cadena ASCIIZ conteniendo la unidad, trayectoria y nombre del archivo

AL =Código de función

0 - leer atributo

1 - fijar atributo

Si AL=1, CX =Atributo

Salida: Si operación sin error, CF=0

Si AL=0, CX =Atributo

Si operación con error, CF=1

AX=código de error (03,05)

AH=56H --- Renombrar archivo (comando RENAME)

Permite cambiar el nombre a un archivo.

Entrada: DS:DX =Dirección cadena ASCIIZ conteniendo la unidad, trayectoria y nombre del archivo

ES:DI =Dirección cadena ASCIIZ conteniendo el nuevo nombre

Salida: Si operación sin error, CF=0

Si operación con error, CF=1

AX =código de error (02,03,05,11)

AH=57H --- Obtener/fijar fecha y hora del archivo

Permite leer y/o fijar la fecha y hora de un archivo.

Entrada: BX =File handle

AL =Código de función

0 - leer

1 - fijar

Si AL=1, DX=Fecha

CX=Hora

Salida: Si operación sin error, CF=0

Si AL=0, DX=Fecha

CX=Hora

Si operación con error, CF=1

AX=Código de error (01,06)

18.5.3.4. De E/S de archivos.

Incluye las siguientes funciones:

AH=3CH --- Crear o truncar archivo (CREATE)

Crea un nuevo archivo o trunca (colocándole longitud 0) un archivo ya existente.

Entrada: DS:DX=Dirección cadena ASCIIZ conteniendo la unidad, trayectoria y nombre del archivo

CX=Atributo del archivo

Salida: Si operación sin error, CF=0

AX=File handle

Si operación con error, CF=1

AX=Código de error (03,04,05)

Si se indica atributo de directorio o de volumen, el MS-DOS lo ignora.

AH=3DH --- Abrir archivo (OPEN)

Abre un archivo existente para un determinado modo de acceso, asignándole el primer file handle libre.

Entrada: DS:DX=Dirección cadena ASCIIZ conteniendo la unidad, trayectoria y nombre del archivo

AL=Código de acceso

0 - sólo lectura

1 - sólo escritura

2 - lectura y escritura

Salida: Si operación sin error, CF=0

AX =File handle

Si operación con error, CF=1

AX =Código de error (02,03,04,05,12)

Una vez abierto el archivo, el puntero de lectura/escritura se posiciona en el primer byte del archivo, es decir que se inicializa en cero.

AH=3EH --- Cerrar archivo (CLOSE)

Cierra un archivo abierto, desafectando su file handle.

Entrada: BX =File handle

Salida: Si operación sin error, CF=0

Si operación con error, CF=1

AX=Código de error (06)

Si se cierra un archivo con BX=0, el MS-DOS interpreta que se hace CLOSE del teclado y este queda inhabilitado.

AH=3FH --- Leer archivo (READ)

Lee una cantidad determinada de bytes de un archivo abierto a partir de la actual posición del puntero de lectura/escritura y los almacena en un buffer en memoria.

Entrada: BX =File handle

CX =Cantidad de bytes a leer

DS:DX =Dirección del buffer donde se va a almacenar la información leída

Salida: Si operación sin error, CF=0

AX=Cantidad de bytes leídos

Si operación con error, CF=1

AX=Código de error (05,06)

Si al retornar sin error AX contiene un valor menor a lo solicitado ($AX < CX$), significa que se encontró el fin de archivo (EOF).

AH=40H --- Grabar archivo (WRITE)

Graba una cantidad determinada de bytes de un buffer en memoria en un archivo abierto a partir de la actual posición del puntero de lectura/escritura.

Entrada: BX =File handle

CX =Cantidad de bytes a grabar

DS:DX =Dirección del buffer donde est la información

Salida: Si operación sin error, CF=0

AX =Cantidad de bytes grabados

Si operación con error, CF=1

AX =Código de error (05,06)

Si al retornar sin error AX contiene un valor menor a lo solicitado ($AX < CX$), significa que grabó una cantidad de bytes y encontró el disco lleno.

AH=42H --- Mover puntero de lectura/escritura

Esta función desplaza el puntero de lectura/escritura para dejarlo apuntando al primer byte de la próxima operación de E/S.

Entrada: BX =File handle

CX =Cantidad de bytes a desplazarse (parte alta)

DX =Cantidad de bytes a desplazarse (parte baja)

AL =M, todo de desplazamiento

0 - desde el comienzo del archivo

1 - desde la posición actual

2 - desde el fin del archivo

Salida: Si operación sin error, CF=0

DX =Nueva posición del puntero (parte alta)

AX =Nueva posición del puntero (parte baja)

Si operación con error, CF=1

AX=Código de error (01,06)

El método de desplazamiento indica desde donde se comienza la cuenta del número de bytes a desplazar.

La cantidad de bytes a desplazar es un entero de 32 bits, con signo. El signo indica hacia que dirección mover el puntero (signo positivo, hacia el final; signo negativo, hacia el comienzo). Cualquiera que sea el método de desplazamiento utilizado, el valor de retorno en AX y DX refleja la posición actual del puntero respecto al comienzo del archivo. Si se utiliza el método 2 (desde el final del archivo) con CX=0 y DX=0, se obtiene como resultado en AX y DX el tamaño del archivo.

AH=5BH --- Crear archivo (CREATE)

Es similar a la función 3CH, pero con la diferencia que produce un error si es un archivo existente.

Entrada: DS:DX =Dirección cadena ASCIIZ conteniendo la unidad, trayectoria y nombre del archivo

CX =Atributo del archivo

Salida: Si operación sin error, CF=0

AX =File handle

Si operación con error, CF=1

AX =Código de error (03,04,05,50)

APENDICE A ALMACENAMIENTO DE DATOS NUMERICOS

En este apéndice se incluyen algunas características acerca de los datos numéricos que se pueden manejar con el microprocesador INTEL 8088-8086.

A.1. ENTEROS

El 8088-8086 opera con números enteros, con y sin signo, de 8 ó 16 bits.

Enteros sin signo.

Los enteros sin signo de 8 bits ocupan un byte, y su rango es:

00 <= N => FF (hexadecimal)
00000000 <= N => 11111111 (binario)
0 <= N => 29-1 (exp. base 2)
0 <= N => 255 (decimal)

Los enteros sin signo de 16 bits ocupan una palabra, y su rango es:

0000 <= N => FFFF (hexadecimal)
0000000000000000 <= N => 1111111111111111 (binario)
0 <= N => 217-1 (exp. base 2)
0 <= N => 65535 (decimal)

Enteros con signo

Para representar números enteros con signo, el 8088-8086 utiliza notación complemento a 2 (el bit mas alto indica el signo). Los enteros con signo de 8 bits tienen el siguiente rango:

80 <= N => 7F (hexadecimal)
10000000 <= N => 01111111 (binario)
-28 <= N => 28-1 (exp. base 2)
-128 <= N => 127 (decimal)

Los enteros con signo de 16 bits tienen el siguiente rango:

8000 <= N => 7FFF (hexadecimal)
1000000000000000 <= N => 0111111111111111 (binario)
-215 <= N => 215-1 (exp. base 2)
-32768 <= N => 32767 (decimal)

A.2. PUNTO FLOTANTE

El 8088-8086 no puede, en realidad, operar directamente (a nivel hardware) con números en punto flotante (o números reales); para que lo pueda hacer, es necesario que cada lenguaje de programación brinde una interface de software apropiada. En este apéndice, se describen los números en punto flotante tal como los opera el lenguaje BASIC. Los números en punto flotante pueden ser de simple y doble precisión, según la cantidad de dígitos disponibles para la mantisa: cuantos mas dígitos tenga la mantisa, mayor precisión tendrá el número. Los números en punto flotante tienen 3 campos: exponente, signo y mantisa.

El exponente ocupa los primeros 8 bits (byte mas alto), y siempre es una cantidad positiva. Para poder representar números con exponente negativo, este se escribe en notación "con exceso" de 128, de modo que el verdadero exponente de un número es el que resulta de restarle 128 al número representado en el campo de exponente. El signo ocupa el bit que esta a continuación del exponente. La mantisa esta a continuación del bit de signo, y esta normalizada. Esto significa que para interpretar la mantisa se debe agregar un 1 en el bit mas alto (al tener la mantisa normalizada, se considera que el primer dígito es siempre una 1 binario aunque no se lo explicita en la representación). El tamaño de la mantisa varía según la precisión: es de 22 bits para simple precisión y de 51 bits para doble precisión. Un número de punto flotante en simple precisión ocupa una doble palabra (4 bytes) de memoria, y tiene la siguiente estructura :

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXPONENTE								S	MANTISA NORMALIZADA																						

Bits 0 a 22 Mantisa normalizada. Para ganar precisión se omite de representar el primer dígito decimal, pues al estar la mantisa normalizada este siempre existe. Entonces, al calcular el número se debe considerar este dígito implícito.

Bit 23 Signo de la mantisa. Si es 0, la mantisa es positiva; si es 1, la mantisa es negativa.

Bits 24 a 31 Exponente con exceso a 128. El campo del exponente es un entero positivo; para conocer el valor del exponente se debe restar 128 al valor representado. El rango para los positivos será entonces:

El siguiente es un resumen de los comandos del EDLIN, con su formato y acción:

B.1. Editar líneas (Edit Line)

Formato: [línea]

Acción: Visualiza una línea para editarla.

B.2. Copiar líneas (Copy Lines)

Formato: [línea - inicial],[línea - final],línea - destino C

Acción: Copia el rango de líneas indicado entre la línea inicial y la línea final, justo a partir de la línea destino.

B.3. Borrar líneas (Delete Lines)

Formato: [línea - inicial],[línea - final] D

Acción: Borra el rango de líneas indicado entre la línea inicial y la línea final. Si no se especifican números de línea, se borra la línea actual.

B.4. Finalizar edición (End Edit)

Formato: E

Acción: Salva el programa sobre el disco, incluyendo todos los cambios efectuados, y retorna el control al DOS.

B.5. Insertar líneas (Insert Lines)

Formato: [línea] I

Acción: Inserta todas las líneas que se introduzcan desde el teclado, a partir del número de línea indicado. Para salir del modo de inserción, se deben presionar, simultáneamente, las teclas CTRL y BREAK.

B.6. Listar líneas (List Lines)

Formato: [línea - inicial],[línea - final] L

Acción: Despliega por pantalla el rango de líneas indicado entre la línea inicial y la línea final.

B.7. Mover líneas (Move Lines)

Formato: [línea inicial],[línea final],línea destino M

Acción: Mueve el rango de líneas indicado entre la línea inicial y la línea final, justo a partir de la línea destino.

B.8. Abandonar edición (Quit Edit)

Formato: Q

Acción: Retorna al DOS sin salvar los cambios hechos durante la edición.

B.9. Reemplazar texto (Replace Text)

Formato: [li],[lf][?]R viejo - string[<F6>nuevo-string]

Acción: Reemplaza el viejo string por el nuevo string, a través de todo el rango de líneas indicado por li (línea inicial) y lf (línea final). El signo de interrogación es opcional, y permite la confirmación de cada reemplazo. Si no se especifica el nuevo string, el resultado es la eliminación del viejo string (en realidad, el viejo string es reemplazado por un carácter nulo).

B.10. Buscar texto (Search Text)

Formato: [li],[lf]S string

Acción: Efectúa la búsqueda del string especificado, a través del rango de líneas indicado entre li y lf.

COMANDOS DEL DEBUG

1. PARAMETROS

Las siguientes son las especificaciones para los parámetros de los comandos del DEBUG.

dirección: Sirve para indicar una dirección de memoria. Una dirección consta de dos partes, el segmento y el desplazamiento. El segmento puede ser la denominación de un registro o un valor hexadecimal; el desplazamiento debe ser un valor hexadecimal. Las dos partes de una dirección van separadas por un ":" (dos puntos). Para algunos comandos no es necesario indicar el segmento; en esos casos, el segmento es asumido por default. Ejemplos de direcciones son:

CS:0100
4BA:0100
100

byte: Es un valor hexadecimal de uno o dos caracteres.

unidad: Es un valor de uno o dos dígitos para identificar desde que unidad se van a cargar los datos, o en que unidad se van a grabar. Por ejemplo, 0 equivale a unidad A, 1 a unidad B, etc.

archivo: Es un string con la especificación de un archivo, es decir, unidad, trayectoria, nombre y extensión. Por ejemplo: A:\SUBDIR1\DATOS\MAESTRO.DAT

lista: Es una lista de valores, que puede estar integrada por bytes o strings. Los elementos de la lista deben estar separados por, al menos, un blanco. Por ejemplo:

F3 'XYZ' 8D 4 "abcd"

dirección puerta: Es un valor hexadecimal, de 1 a 4 caracteres, para especificar una dirección de puerta de acceso de datos.

rango: Es una serie de dos direcciones, que especifican la dirección más baja y más alta, respectivamente, de un área de memoria. Un rango se puede indicar de dos formas: mediante dos direcciones, donde la segunda va separada de la primera por un blanco, o mediante una dirección seguida de la letra L, que indica longitud, y un valor hexadecimal. Ejemplos de rangos son:

CS:100 110
CS:100 L 11

registro: Es una o dos letras para identificar un registro. Los nombres de registros válidos son: AX, BX, CX, DX, SP, BP, SI, DI, DS, ES, SS, CS, IP, PC, F.

sector: Es un valor hexadecimal, de 1 a 3 caracteres, para indicar un número de sector de disco. Los comandos que utilizan este parámetro siempre necesitan dos especificaciones de sector, donde el primero indica el número de sector relativo de comienzo y el segundo la cantidad de sectores que deben cargarse o grabarse.

string: Es una serie de caracteres alfanuméricos, encerrados entre comillas simples o dobles.

valor: Es un número hexadecimal, de 1 a 4 caracteres.

2. COMANDOS

El siguiente es un sumario de los comandos del DEBUG y su acción:

2.1. Ensamblar (Assemble)

Formato: **A**[dirección]

Acción: Convierte en lenguaje assembler el contenido de la memoria, a partir de "dirección". El registro asumido por default es CS. Por ejemplo: A CS:100.

2.2. Comparar (Compare)

Formato: **C** rango dirección

Acción: Compara el contenido de dos bloques de memoria. El primer bloque está dado por "rango", y el segundo bloque está dado por "dirección". La longitud de la comparación viene dada por "rango". El registro asumido por default es DS.

Por ejemplo : C 100 L20 200.

2.3. Visualizar memoria (Dump)

Formato: **D**[dirección]

o

D[rango]

Acción: Despliega por pantalla el contenido de la memoria a partir de "dirección" o a partir de la dirección inicial de "rango". Si no se especifica "dirección", el comando Dump asume por default que la dirección de comienzo es la que sigue a la última dirección visualizada. El registro utilizado por default es DS. Por ejemplo: D DS:100 DS:200

2.4. Modificar memoria (Enter)

Formato: **E** dirección [lista]

Acción: Sustituye el contenido de uno o más bytes, comenzando en "dirección", por los valores indicados en "lista".

Si no se indica "lista", este comando permite visualizar y modificar bytes de forma secuencial, hasta finalizar con ENTER. El registro asumido por default es DS. Por ejemplo: E CS:100.

2.5. Llenar memoria (Fill)

Formato: **F** rango lista

Acción: Completa las posiciones de memoria que pertenezcan a "rango", con los valores indicados en "lista". El registro asumido por default es DS. Por ejemplo: F 4BA:100 L5 F3 "XYZ" 8D.

2.6. Ejecutar (Go)

Formato: **G**[=dirección][dirección[dirección...]]

Acción: Ejecuta lo indicado en la memoria a partir de la dirección indicada como primer parámetro (éste, si se incluye, debe estar precedido obligatoriamente por el signo =), hasta la próxima dirección indicada (punto de ruptura), donde visualiza el contenido de la memoria y de los registros.

Se pueden indicar hasta diez puntos de ruptura distintos. Si no se indica el primer parámetro, la ejecución comienza a partir de la próxima instrucción apuntada por el IP (Instruction Pointer). El registro usado es CS. Por ejemplo: G 102 1EF 208.

2.7. Aritmética hexadecimal (Hexarithmetic)

Formato: **H** valor valor

Acción: Efectúa la suma de los dos valores y resta el segundo del primero. Por ejemplo: H 0F 8.

2.8. Entrada (Input)

Formato: **I** direc. puerta

Acción: Lee y visualiza (en hexadecimal), un byte procedente de la puerta de acceso de datos indicada. Por ejemplo: I 2F8.

2.9. Cargar (Load)

Formato: **L** [dirección[unidad sector sector]]

Acción: Carga en la memoria un archivo o sectores de disco, a partir de "dirección". Si sólo se indica "dirección", se cargará en memoria el archivo que haya sido previamente indicado en el comando Name, o el archivo con el que se haya invocado al DEBUG. El registro asumido por default es CS. Por ejemplo: L 4BA:100 1 0F 6D.

2.10. Mover (Move)

Formato: **M** rango dirección

Acción: Traslada el contenido de las posiciones de memoria especificadas por "rango", a las posiciones de memoria a partir de "dirección". El registro asumido por default es DS. Por ejemplo M CS:100 110 DS:500

2.11. Nombre (Name)

Formato: **N** archivo

Acción: Prepara una serie de tablas internas del DEBUG, para que puedan ser utilizadas por los comandos Load y Write.

Por ejemplo: N miprog.exe.

2.12. Salida (Output)

Formato: **O** direc. puerta byte

Acción: Envía "byte" a la puerta de acceso especificada. Por ejemplo: O 2F8 4F.

2.13. Continuar (Proceed)

Formato: **P**[=dirección][valor]

Acción: Detiene en la instrucción siguiente la ejecución de una llamada a subrutina, una instrucción de ciclo, una interrupción o una instrucción con prefijo de repetición.

2.14. Abandonar DEBUG (Quit)

Formato: **Q**

Acción: Termina el programa DEBUG y retorna el control al DOS.

2.15. Registros (Register)

Formato: **R**[registro]

Acción: Visualiza el contenido hexadecimal de uno o todos los registros y banderas, permitiendo la posibilidad de modificarlos. Si no se especifica "registro", este comando exhibe la totalidad de los registros.

Por ejemplo: R BP.

2.16. Buscar (Search)

Formato: **S** rango lista

Acción: Busca los caracteres de "lista" en "rango". El registro asumido por default es DS.

Por ejemplo: S CS:100 L 1 41 AB E.

2.17. Trazado (Trace)

Formato: **T**[=dirección][valor]

Acción: Ejecuta una o más instrucciones, empezando por la que está en CS:IP, o, si está especificado, en "dirección" (si existe, debe ser precedido obligatoriamente por el signo =). Si se especifica "valor", se ejecutarán tantas instrucciones como el valor indicado. Se visualiza el contenido de todos los registros y banderas después de ejecutar cada una de las instrucciones. Si en "dirección" no se especifica segmento, se asume por default el segmento CS.

Por ejemplo: T DS:100 10.

2.18. Desensamblar (Unassemble)

Formato: **U**[dirección]

o

U[rango]

Acción: Traduce el contenido de memoria, a partir de "dirección" (o a partir de la dirección inicial de "rango"), en sentencias assembler, y visualiza sus direcciones y valores hexadecimales junto con las sentencias assembler.

El registro asumido por default es CS.

Por ejemplo: U 04AB:0100 L 7.

2.19. Grabar (Write)

Formato: **W** [dirección[unidad sector sector]]

Acción: Graba en disco el contenido de "dirección". Si se indican todos los parámetros, se grabará el contenido de la memoria a partir de la dirección especificada, en el disco de la unidad especificada, a partir del sector indicado en primer término, un número de sectores según lo indicado en segundo término. Si no se especifican parámetros, o sólo se especifica "dirección", se grabará el archivo que fue invocado con el comando Name. El registro asumido por default es CS.

Por ejemplo: W 1

FD 1 100 A.

APENDICE D OPERADORES

D.1. OPERADORES ARITMETICOS

Operador	Formato	Función
+	valor 1 + valor 2	suma valor 1 y valor 2
-	valor 1 - valor 2	resta valor 1 y valor 2
*	valor 1 * valor 2	multiplica valor 1 y valor 2
/	valor 1 / valor 2	divide valor 1 y valor 2, devuelve el cociente
MOD	valor 1 MOD valor 2	divide valor 1 y valor 2, devuelve el resto
SHL	valor 1 SHL expresión	desplaza valor 1 a la izquierda, la cantidad indicada por expresión
SHR	valor 1 SHR expresión	desplaza valor 1 a la derecha la cantidad indicada por expresión

D.2. OPERADORES LOGICOS

Operador	Formato	Función
AND	valor 1 AND valor 2	efectúa el AND lógico entre valor 1 y valor 2
OR	valor 1 AND valor 2	efectúa el OR lógico entre valor 1 y valor 2
XOR	valor 1 AND valor 2	efectúa el OR EXCLUSIVO lógico entre valor 1 y valor 2
NOT	NOT valor	invierte cada uno de los bits de valor,obtiene el complemento a 2

D.3. OPERADORES RELACIONALES

Operador	Formato	Función
EQ	operando 1 EQ operando 2	Da verdadero si ambos operandos son idénticos
EN	operando 1 EQ operando 2	Da verdadero si ambos operandos son distintos
LT	operando 1 EQ operando 2	Da verdadero si operando 1 es menor que operando 2
GT	operando 1 EQ operando 2	Da verdadero si operando 1 es mayor que operando 2
LE	operando 1 EQ operando 2	Da verdadero si operando 1 es menor o igual que operando 2
GE	operando 1 EQ operando 2	Da verdadero si operando 1 es mayor o igual que operando 2

D.4. OPERADORES DE RETORNO DE VALORES

Operador	Formato	Función
\$	\$	Devuelve el actual valor del contador de posiciones de memoria
SEG	SEG variable SEG rotulo	Devuelve el valor del segmento de variables o rotulo
OFFSET	OFFSET variable OFFSET rotulo	Devuelve el valor del desplazamiento de variable o rotulo
LENGTH	LENGTH variable	Devuelve la longitud en unidades (bytes o words) de variable, que debe ser definida como un arreglo.
TYPE	TYPE variable TYPE rotulo	Para una variable,TYPE devuelve 1,2 o 4 según sea BYTE, WORD, o DOUBLEWORD. Para un rotulo devuelve -1 o -2 según sea NEAR o FAR
SIZE	SIZE variable SIZE rotulo	Devuelve el producto de LENGTH veces TYPE, es decir la cantidad de bytes que ocupa una variable.
MASK	MASK campo-record	Devuelve la configuración en bits del récord (al que pertenece campo-record), con valor 1 en los bits del campo indicado y valor 0 en los restantes bits del record

D.5. OPERADORES DE ATRIBUTOS

Operador	Formato	Función
PTR	tipo PTR expresion	Redefine el atributo de tipo(byte o word) o el atributo de distancia (NEAR o FAR), de un operando de dirección de memoria. tipo es el nuevo atributo y expresión es el identificador cuyo atributo es cambiado
DS: ES: SS: CS:	REG-SEG:expr. direccion REG-SEG:rotulo REG-SEG:variable	Reasigna el segmento de un rotulo, una variable o una expresión de dirección
SHORT	JMP SHORT rotulo	Indica al compilador que la bifurcación a rotulo no es más lejana que 127 bytes de la próxima instrucción
THIS	THIS atributo-distancia THIS atributo-tipo	Crea un operando de dirección de memoria de un atributo de distancia (NEAR o FAR) o de un atributo tipo (BYTE o WORD), en un valor de desplazamiento igual al actual valor del contador de posiciones, y en un valor de segmento igual al segmento en el cual está escrito el operador
HIGH	HIGH valor HIGH expresión	Devuelve el byte de más alto orden de un valor o expresión de 16 bits
LOW	LOW valor LOW expresión	Devuelve el byte de más bajo orden de un valor o expresión de 16 bits

D.6. OPERADORES DE MACROS

Operador	Formato	Función
&	texto & texto	Concatena textos o símbolos. Permite la creación de rótulos u operandos
::	:: comentario	Hace que el comentario no se incluya en la expansión de una macro
!	! carácter	Toma al carácter como valor literal y no como símbolo
%	% expresión	Convierte la expresión, que generalmente es un símbolo, en un número a la base de numeración activa.

APENDICE E **CODIGOS DE LAS TECLAS**

E.1. CODIGOS Y CARACTERES DEL TECLADO (KEYBOARD)

La siguiente tabla indica los scan codes de cada tecla del teclado y los caracteres generados por cada tecla en minúscula, en mayúscula (Shift) y en combinación con las teclas Ctrl y Alt:

Tecla Scan

Nro (Dec)	Code (Hex)	Minúscula	Mayúscula	Ctrl	Alt
1	29	`	~		
2	02	1	!		(1)
3	03	2	@	(1)	(1)
4	04	3	#		(1)
5	05	4	\$		(1)
6	06	5	%		(1)
7	07	6	^	RS	(1)
8	08	7	&		(1)
9	09	8	*		(1)
10	0A	9	(		(1)
11	0B	0	)		(1)
12	0C	-	_	US	(1)
13	0D	=	+		(1)
14	2B	\		FS	
15	0E	BS	BS	DEL	
16	0F	TAB	(1)		
17	10	q	Q	DC1	(1)
18	11	w	W	ETB	(1)
19	12	e	E	ENQ	(1)
20	13	r	R	DC2	(1)
21	14	t	T	DC4	(1)
22	15	y	Y	EM	(1)
23	16	u	U	NAK	(1)
24	17	i	I	HT	(1)
25	18	o	O	SI	(1)
26	19	p	P	DLE	(1)
27	1A	[	{	ESC	(1)
28	1B	]	}	GS	
30	1D	Ctrl			
31	1E	a	A	SOH	(1)
32	1F	s	S	DC3	(1)
33	20	d	D	EOT	(1)
34	21	f	F	ACK	(1)
35	22	g	G	BEL	(1)
36	23	h	H	BS	(1)
37	24	j	J	LF	(1)
38	25	k	K	VT	(1)
39	26	l	L	FF	(1)
40	27	;	:		
41	28	'	"		
43	1C	CR	CR	LF	
44	2A	Shift			
46	2C	z	Z	SUB	(1)
Nro (Dec)	Code (Hex)	Minúscula	Mayúscula	Ctrl	Alt
47	2D	x	X	CAN	(1)
48	2E	c	C	ETX	(1)
49	2F	v	V	SYN	(1)

50	30	b	B	STX	(1)
51	31	n	N	SO	(1)
52	32	m	M	CR	(1)
53	33	,(coma)	<		
54	34	.(punto)	>		
55	35	/	?		
57	36	Shift			
58	38	Alt			
61	39	SP	SP	SP	SP
64	3A	Caps Lock			
65	3C	F2 (1)	(1)	(1)	(1)
66	3E	F4 (1)	(1)	(1)	(1)
67	40	F6 (1)	(1)	(1)	(1)
68	42	F8 (1)	(1)	(1)	(1)
69	44	F10 (1)	(1)	(1)	(1)
70	3B	F1 (1)	(1)	(1)	(1)
71	3D	F3 (1)	(1)	(1)	(1)
72	3F	F5 (1)	(1)	(1)	(1)
73	41	F7 (1)	(1)	(1)	(1)
74	43	F9 (1)	(1)	(1)	(1)
90	01	ESC	ESC	ESC	
95	45	Num Lock			(2)
100	46	Scroll Lock			(2)
105	54	Sys Req (2)			
106	37	*	PrtSc (2)		(1)

(1) Indica la existencia de un código extendido (ver sección E.3.).

(2) Corresponde a una combinación de teclas especiales que desencadena una determinada acción (ver sección E.4.).

E.2. CODIGOS Y CARACTERES DEL TECLADO NUMERICO (KEYPAD)

La siguiente tabla indica los scan codes de cada tecla del teclado numérico y los caracteres generados por cada tecla en modo normal (minúscula o mayúscula), y en combinación con las teclas Num Lock, Ctrl y Alt:

Tecla Scan

Nro (Dec)	Code (Hex)	Normal	Num Lock	Ctrl	Alt
91	47	Home (1)		7	Clear Screen
92	4B	Left (1)	4		Reverse Word (1)
93	4F	End (1)	1		Erase to EOL (1)
96	48	Up (1)	8		
97	4C		5		
98	50	Down (1)		2	
99	52	Ins (1) (2)		0	
101	49	PgUp (1)		9	Top of Text and Home
102	4D	Right (1)		6	Advance Word (1)
103	51	PgDn (1)		3	Erase to EOS (1)
104	53	Del (1) (2)		.	(2) (2)
107	4A	-	-		
108	4E	+	+		

(1) Indica la existencia de un código extendido (ver sección E.3.).

(2) Corresponde a una combinación de teclas especiales que desencadena una determinada acción (ver sección E.4.).

E.3. CODIGOS EXTENDIDOS

Cada vez que se pulsa una tecla, se devuelven dos códigos: el código ASCII del carácter en el registro AL y el scan code en el registro AH. En los casos de teclas especiales o combinaciones de teclas (caracteres no ASCII), en AL se devuelve un cero y en AH un código extendido.

Los códigos extendidos corresponden, entonces, al segundo valor que devuelve una tecla especial cuando es pulsada o cuando se pulsa alguna combinación de teclas.

La siguiente tabla indica los códigos extendidos y a que teclas especiales o combinaciones de teclas corresponden.

Código	Tecla(s)
03	Ctrl 2
0F	Shift Tab (back Tab)
10-19	Alt Q hasta Alt P (fila superior de letras)
1E-26	Alt A hasta Alt L (fila media de letras)
2C-32	Alt Z hasta Alt M (fila inferior de letras)
3B-44	F1 hasta F10 (sin mayúsculas)
47	Home
48	Up
49	PgUp
4B	Left
4D	Right
4F	End
50	Down
51	PgDn
52	Ins
53	Del
54-5D	Shift F1 hasta Shift F10
5E-67	Ctrl F1 hasta Ctrl F10
68-71	Alt F1 hasta Alt F10
72	Ctrl PrtSc
73	Ctrl Left
74	Ctrl Right
75	Ctrl End
76	Ctrl PgDn
77	Ctrl Home
78-83	Alt 1 hasta Alt = (fila de números)
84	Ctrl PgUp

E.4. COMBINACIONES DE TECLAS ESPECIALES

Las siguientes combinaciones de teclas producen efectos especiales:

Alt-Ctrl-Del: Carga del sistema.

Ctrl-Break: Se invoca a la interrupción BIOS 1BH, que produce un "break" del teclado.

Ctrl-Num Lock: Se suspende la rutina del teclado hasta que se vuelva a pulsar cualquier tecla (excepto Num Lock). Sirve para suspender temporariamente cualquier operación, para reanudarla después.

Shift-PrtSc: Invoca a la interrupción BIOS 05H que produce la salida por impresora del contenido de la pantalla (hardcopy). El hardcopy se hace carácter a carácter o pixel a pixel según sea el modo de funcionamiento de la pantalla.

Sys Req: Invoca la interrupción 15H (Extensiones de E/S del sistema)

E.5. TABLA DE CARACTERES ASCII

	0	1	2	3	4	5	6	7
0	NUL	DEL	SP	0	@	P		p

1	SOH	DC1	!	1	A	Q	a	q
2	STX	DC2	"	2	B	R	b	r
3	ETX	DC3	#	3	C	S	c	s
4	EOT	DC4	\$	4	D	T	d	t
5	ENQ	NAK	%	5	E	U	e	u
6	ACK	SYN	&	6	F	V	f	v
7	BEL	ETB		7	G	W	g	w
8	BS	CAN	(	8	H	X	h	x
9	HT	EM	)	9	I	Y	i	y
A	LF	SUB	*	:	J	Z	j	z
B	VT	ESC	+	;	K	[	k	{
C	FF	FS	,	<	L	\	l	
D	CR	GS	-	=	M	]	m	}
E	SO	RS	.	>	N	^	n	~
F	SI	US	/	?	O	_	o	DEL

NUL - Nulo (Null)

SOH - Comienzo de cabecera (Start of Heading)

STX - Comienzo de texto (Start of Text)

ETX - Final de texto (End of Text)

EOT - Fin de transmisión (End of Transmission)

ENQ - Requerimiento (Enquiry)

ACK - Reconocimiento (Acknowledge)

BEL - Campanilla (Bell)

BS - Retroceso (Back Space)

HT - Tabulación horizontal (Horizontal Tabulation)

LF - Avance de línea (Line Feed)

VT - Tabulación vertical (Vertical Tabulation)

FF - Avance de página (Form Feed)

CR - Retorno de carro (Carriage Return)

SO - Shift Out

SI - Shift In

DLE - Escape de enlace de datos (Data Link Escape)

DCn - Control del dispositivo n (Device Control)

NAK - Reconocimiento negativo (Negative Acknowledge)

SYN - Espera de sincronismo (Synchronous Idle)

ETB - Fin de bloque de transmisión (End of Transmission Block)

CAN - Cancelar (Cancel)

EM - Fin del medio (End of Medium)

SUB - Sustituir (Substitute)

ESC - Escapar (Escape)

FS - Separador de archivo (File Separator)

GS - Separador de grupo (Group Separator)

RS - Separador de registro (Record Separator)

US - Separador de unidad (Unit Separator)

SP - Espacio (Space)

DEL - Borrar (Delete)

Corregido y pasado a word por **Fernando H. CEJAS-1997**